ASR1606

Datasheet

Communications Processor

ASR CONFIDENTIAL

Doc. No. ASR1606
CONFIDENTIAL
Document Classification: Proprietary Information

Document Conventions

	**Note:** Provides related information or information of special importance.
	**Caution:** Indicates potential damage to hardware or software, or loss of data.
	**Warning:** Indicates a risk of personal injury.

Document Status

Doc Status: Preliminary	Technical Publication:1.0

ASR CONFIDENTIAL

Disclaimer
No part of this document may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying and recording, for any purpose, without the express written permission of ASR. ASR retains the right to make changes to this document at any time, without notice. ASR makes no warranty of any kind, expressed or implied, with regard to any information contained in this document, including, but not limited to, the implied warranties of merchantability or fitness for any particular purpose. Further, ASR does not warrant the accuracy or completeness of the information, text, graphics, or other items contained within this document.
ASR products are not designed for use in life-support equipment or applications that would cause a life-threatening situation if any such products failed. Do not use ASR products in these types of equipment or applications.
At all times hereunder, the recipient of any such information agrees that they shall be deemed to have manually signed this document in connection with their receipt of any such information.
Copyright © 2015–2022. ASR Microelectronics (Shanghai) Co., Ltd. All rights reserved .
Patent(s) Pending—Products identified in this document may be covered by one or more ASR patents and/or patent applications.

Document Purpose

This document constitutes software related specifications and thermal specifications, mechanical data, package signal locations, targeted electrical specifications, functional bus waveforms, and board design considerations for the ASR1606, including functional overviews for the applications processor and cellular modem subsystems.

Number Representation

In this document, the numbers used to represent bit states are given in:

- binary numbers with a prefix of 0b For example, 0b1 or 0b11

  or

- hexadecimal with a prefix of 0x For example 0x6B

Single-bit items have either of two states:

- **Clear** — the item contains the value 0b0.
- **Set** — the item contains the value 0b1.

ASR CONFIDENTIAL

- General
  - 22nm process, BGA package with 0.4mm pitch.
  - Operation temperature:
    -40~85°C(Industrial Grade)
    -20~70°C (Commercial Grade)

- Application Processor
  - ARM Cortex-R5 with 614MHz clock
    - 32K I-Cache
    - 32K D-Cache
    - 64KB ATCM and 64KB BTCM
  - 64KB ROM and 64KB on-chip SRAM for application usage

- Memory
  - Embedded 8bit Octal-SPI pSRAM, support DDR mode. Up to 400Mbyte/s bandwidth.
  - Embedded flash with XIP (eXecute In Place) and QSPI mode support, up to 102MHz。

- Peripheral Controllers
  - Keypad controller:
    - Support 8x8 keypad with hardware scanner and internal pull-up resistors.
    - Support multiple key press for gaming
    - Support direct key with build-in de-bouncing
  - GPIO (×85)
    - Pull-up/pull-down programmable
    - 1.8V/3.3V IO
  - UART (×4)
    - All support flow-cotrol except UART3
    - With hardware flow-control, up to 3.6Mbps
    - Two UART support Full 8-wire connection
  - USB 2.0(×1)
  - EMMC/SD Controller
    - Compatible with EMMC v4.5(up to HS200)
    - Support 1bit/4bit bus-width
  - SDIO Device Controller
    - for high bandwidth host CPU connection
  - I2C (×5)
    - Modes: 100KHz/400KHz/1MHz/3.4MHz
  - PWM (×4)
  - Dual SIM/USIM card controller
  - General SSP interface(×3)
    - Support both master and slave mode
    - Can be configured to SPI, PCM, I2S and Microwire format.
- Security System
  - True Random Number Generator
  - 512bits OTP for security key
  - Secure Boot/Strap/Bonding
- Wireless connection interfaces
  - Bluetooth5.0, FM, GNSS
- Auxiliary analog inputs
  - Auxiliary ADC with 5 inputs
- Integrated Resistive touch controller
  - 4 wire inputs
- Audio Codec
  - Integrated High quality audio codec and audio front-end
- Debug System
  - JTAG for CPU & DSP sub-system
  - UARTs and USB
- Boot System
  - Initial AP boot from UART/USB/SPI NOR
  - Strap pin & OTP for boot control
  - 64KB BootROM
- Modem baseband
  - FDD/TDD LTE CAT1
  - Support VoLTE
- Integrated RF transceiver
  - Cover frequency bands from 450MHz to 2.7GHz
  - Support up to 8 Rx ports and 4 Tx ports
  - Support both DCXO and TCXO
- 32Kless
- Ultra low power
  - VoLTE 70mA

Copyright © 2022 ASR

Table of Contents

COVER 1
DOCUMENT PURPOSE 3
NUMBER REPRESENTATION 3
PRODUCT OVERVIEW 4
NAMING CONVENTIONS 14

1. INTRODUCTION 16

1.1 SOC DEVICE FEATURES 16

2. FUNCTIONAL DESCRIPTION OVERVIEW 20

2.1 ARCHITECTURE OVERVIEW 20
2.2 CHIP CONFIGURATION 22
2.3 MEMORY MAP 22
2.4 INTERRUPT CONNECTIVITY AND ASSIGNMENTS 25
2.5 DMA CONNECTIVITY AND ASSIGNMENTS 36

3. PIN MUXING AND MULTI-FUNCTION PINS 43

3.1 FUNCTION ASSIGNMENTS FOR MULTIPLEXED I/O PINS 43
3.2 PINOUT 46
3.3 MULTIPLEXED SIGNAL FUNCTIONS 54
3.4 PAD-RING I/O DOMAINS POWER CONTROL 57
3.5 PIN CONTROL UNIT 63
3.6 FUNCTION ASSIGNMENTS FOR MULTI-FUNCTION I/O PINS 74
3.7 REGISTER DESCRIPTIONS 76

4. ARM CORTEX-R5 MODEM SUBSYSTEM 77

4.1 OVERVIEW 77
4.2 FUNCTIONAL DESCRIPTION 78
4.3 MEMORY MAP 80
4.4 INTERRUPT 80

5. SQU 81

5.1 OVERVIEW 81
5.2 FUNCTIONALITY 81

6. PSRAM MEMORY CONTROLLER 83

6.1 INTRODUCTION 83
6.2 FEATURE LIST 83
6.3 OVERVIEW 84
6.4 FUNCTIONAL DESCRIPTION 85

7. MODEM FUNCTIONAL DESCRIPTION 89

7.1 OVERVIEW OF LTE PLATFORM 89
7.2 INTRODUCTION TO LTE MODEM 89

8. COMMUNICATION SUBSYSTEM PERIPHERALS FUNCTIONAL DESCRIPTION 92

8.1 COMMUNICATION IPC 92
8.2 COMMUNICATION SUBSYSTEM TIMERS 92
8.3 UNIVERSAL SUBSCRIBER ID MODULE (USIM) CONTROLLER 95
8.4 REGISTER DESCRIPTIONS 127

9. POWER MANAGEMENT AND CLOCKS 128

ASR CONFIDENTIAL

9.1 POWER MANAGEMENT UNITS 128
9.2 POWER DOMAINS 134
9.3 POWER STATES 136
9.4 CLOCKS 140
9.5 RESET 149

10. JTAG 153

10.1 FEATURES 153
10.2 OPERATION 153
10.3 BOARD JTAG DESIGN NOTE 160
10.4 REGISTER DESCRIPTIONS 160

11. DMA CONTROLLER 161

11.1 OVERVIEW 161
11.2 FEATURES 161
11.3 OPERATION 163
11.4 REGISTER DESCRIPTIONS 199

12. USB 2.0 DEVICE 200

12.1 OVERVIEW 200
12.2 FEATURES 200
12.3 BLOCK DIAGRAMS 201
12.4 USB 2.0 OPERATION 202
12.5 REGISTER DESCRIPTIONS 207
12.6 USB2 TRANSMIT FIFO USAGE NOTE 207

13. REAL-TIME CLOCK (RTC) 208

13.1 OVERVIEW 208
13.2 INTERRUPTS 208
13.3 TRIM PROCEDURE 210
13.4 REGISTER DESCRIPTIONS 213

14. TIMERS 214

14.1 OPERATION 214
14.2 WATCHDOG TIMER FUNCTIONALITY 215
14.3 TIMER INTERRUPTS 217
14.4 REGISTER DESCRIPTIONS 217

15. HIGH-SPEED TWSI BUS INTERFACE UNIT (I2C) 218

15.1 OVERVIEW 218
15.2 FEATURES 219
15.3 SIGNAL DESCRIPTIONS 220
15.4 OPERATION 220
15.5 REGISTER DESCRIPTIONS 271

16. SSP SERIAL PORTS 272

16.1 OVERVIEW 273
16.2 FEATURES 274
16.3 SIGNAL DESCRIPTIONS 275
16.4 OPERATION 276
16.5 REGISTER DESCRIPTIONS 316

17. SDIO 317

ASR CONFIDENTIAL

17.1 Overview ... 317
17.2 Feature ... 317
17.3 Signal Descriptions ... 317
17.4 Operation ... 319

18. UART CONTROLLERS ... 332

18.1 Key Features ... 332
18.2 Overview ... 333
18.3 Signal Descriptions ... 335
18.4 Operation ... 337
18.5 Register Descriptions ... 356

19. GENERAL-PURPOSE I/O UNIT ... 357

19.1 Overview ... 357
19.2 Function Description ... 357
19.3 Software Guide ... 359
19.4 Register Descriptions ... 361

20. AUDIO CODEC ... 362

20.1 Overview ... 362
20.2 Features ... 362
20.3 Block Diagrams ... 362
20.4 Audio Front-End Performance ... 363
20.5 Control & Interface ... 364
20.6 Audio Codec Operation ... 364
20.7 Register Descriptions ... 365

21. SD HOST CONTROLLER ... 366

21.1 Features ... 366
21.2 Signal Descriptions ... 368
21.3 SD/MMC Bus Protocol Description ... 369
21.4 Special Bus Transactions ... 371
21.5 Card Detection ... 379
21.6 SPI Mode ... 379
21.7 MMC Mode ... 379
21.8 CE-ATA Mode ... 379
21.9 Register Descriptions ... 379

22. RESISTIVE TOUCH SCREEN CONTROLLER ... 380

22.1 Introduction ... 380
22.2 Feature List ... 380
22.3 Overview ... 381
22.4 Function description ... 381

23. RF TRANSCEIVER ... 385

23.1 Functional Description ... 385
23.2 Electrical Specifications ... 393

24. MECHANICAL DRAWINGS ... 398

25. PART ORDER NUMBERING/PACKAGE MARKING ... 399

25.1 Part Order Numbering ... 399
25.2 Package Marking ... 399

A ACRONYMS AND ABBREVIATIONS ... 401

ARC CONFIDENTIAL

B REVISION HISTORY 405

ASR CONFIDENTIAL

List of Tables

TABLE 1: RELATED DOCUMENTS 15
TABLE 2:PSRAM AXI PORT CONNECTIONS 21
TABLE 3: ICU HARDWARE INTERRUPT 27
TABLE 4 CEVA-BX2 INTERRUPTS 30
TABLE 5:MIRQ INTERRUPT SOURCE 34
TABLE 6:AIRQ INTERRUPT SOURCE 35
TABLE 7:AP DMA PERIPHERALS 38
TABLE 8:CP DMA PERIPHERALS 42
TABLE 9:ASR1606 PIN LIST 47
TABLE 10 IO PARAMETER 53
TABLE 11:SIGNAL TYPES 55
TABLE 12:MULTIPLEXED SIGNAL DESCRIPTION 55
TABLE 13:POWER SUPPLIES DESCRIPTIONS 60
TABLE 14:PIN FUNCTION EXAMPLE 68
TABLE 15: <DRIVE> FIELD SETTINGS 72
TABLE 16:PREDEFINED INSTRUCTIONS 86
TABLE 17:LTE MODEM MODULES 91
TABLE 18: USIM CONTROLLER SIGNALS SUMMARY 97
TABLE 19:USIM SMARTCARD PINOUT 99
TABLE 20:DCSR SETUP TO IGNORE EOR 126
TABLE 21:AMUCRM MASK BITS 131
TABLE 22:ASR1606 POWER STATES 137
TABLE 23:APPLICATION APB PERIPHERALS 146
TABLE 24:COMMUNICATION APB PERIPHERALS 147
TABLE 25:PRIMARY CLOCK DOMAINS CLOCK SHUTDOWN CONDITIONS 148
TABLE 26:SYSTEM RESET DOMAINS 151
TABLE 27:JTAG INSTRUCTION DESCRIPTION 155
TABLE 28:DMA SUPPORT MATRIX 162
TABLE 29:CHANNEL PRIORITY 166
TABLE 30:CHANNEL STATES BASED ON SOFTWARE CONFIGURATION 168
TABLE 31:CONFIGURATION FOR PERIPHERAL BUS PERIPHERAL RELATED DATA TRANSFERS 185
TABLE 32:CONFIGURATION FOR MEMORY-TO-MEMORY DATA TRANSFERS 185
TABLE 33:SYSTEM BUS PERIPHERAL RELATED DATA TRANSFER CONFIGURATION 186
TABLE 34:CONFIGURATION FOR COMPANION CHIP (CC) RELATED DATA TRANSFERS 186
TABLE 35:DMA QUICK REFERENCE FOR ON-CHIP PERIPHERALS 188
TABLE 36:DMA QUICK REFERENCE FOR ON-CHIP PERIPHERALS 220
TABLE 37:TWSI BUS DEFINITIONS 221
TABLE 38:MODES OF OPERATION 226
TABLE 39:START AND STOP BIT DEFINITIONS 229
TABLE 40:MASTER TRANSACTIONS 244
TABLE 41:SLAVE TRANSACTIONS 261
TABLE 42:GENERAL CALL ADDRESS SECOND BYTE DEFINITIONS 270
TABLE 43:ASR1606 EXTERNAL SIGNAL DESCRIPTIONS 275
TABLE 44:PROGRAMMABLE PROTOCOL PARAMETERS 300
TABLE 45:SSP_SSCR1[TFT] AND SSP_SSCR1[RFT] VALUES WITH DMA BURST SIZES 311
TABLE 46 SDIO INTERFACE SIGNAL DESCRIPTION 318
TABLE 47 SDIO ELECTRICAL FUNCTION DEFINITION - 1-BIT 321
TABLE 48 SDIO ELECTRICAL FUNCTION DEFINITION - SPI 322
TABLE 49 SDIO ELECTRICAL FUNCTION DEFINITION - 4-BIT 322
TABLE 50 SD MODE, SDIO COMMANDS 324
TABLE 51 SPI MODE, SDIO COMMANDS 324
TABLE 52 SDIO REGISTERS 325
TABLE 53 SDIO TIMING DATA1 331
TABLE 54:SUPPORTED BAUD RATES 334

CONFIDENTIAL

TABLE 55:UART SIGNAL DESCRIPTIONS 335
TABLE 56:RECOMMENDED BAUD RATES 355
TABLE 57:GPIO INTERRUPTS 359
TABLE 58: ANALOG MICROPHONE INPUT TO ADC PERFORMANCE 363
TABLE 59: GENERAL AUDIO RX PATH FEATURE SUMMARY 363
TABLE 60:SD HOST SIGNAL DESCRIPTIONS 368
TABLE 61: TSC CONTROL REGISTER 381
TABLE 62: TSC STATUS REGISTER 382
TABLE 63: X/Y VOLTAGE DATA REGISTER0~7(XYDR0~7) 383
TABLE 64: LTE OPERATING BANDS 385
TABLE 65: RF PORT MAPPING AND BAND CONFIGURATIONS 387
TABLE 66: LOGIC LEVELS 391
TABLE 67: R F POWER SUPPLY PIN DETAILS 392
TABLE 68: ABSOLUTE MAXIMUM RATINGS[1] 393
TABLE 69: RECOMMENDED OPERATING CONDITIONS[1] 393
TABLE 70: ELECTRICAL CHARACTERISTICS 394
TABLE 71: ELECTRICAL CHARACTERISTICS 395
TABLE 72: ELECTRICAL CHARACTERISTICS (THREE-WIRE SERIAL BUS INTERFACE) 395
TABLE 73: ELECTRICAL CHARACTERISTICS (REFERENCE CLOCKS - ALL MODES) 396
TABLE 74: ELECTRICAL CHARACTERISTICS (APT DAC) 397
TABLE 75:ASR1606 PART ORDER OPTIONS 399
TABLE 76:ASR1606 WORK TEMPERATURE 399
TABLE 77:REVISION HISTORY 405

ASR CONFIDENTIAL

List of Figures

FIGURE 1:ASR1606 ARCHITECTURE BLOCK DIAGRAM 20
FIGURE 2:ARM CORTEX-R5 SUBSYSTEM MEMORY MAP 24
FIGURE 3:INTERRUPT CONNECTIVITY 26
FIGURE 4 DMA SUPPORT MATRIX 37
FIGURE 5 DMAC BLOCK DIAGRAM 38
FIGURE 6: COMPLETE PINOUT 238-BALL 8.2X7.4 MM BGA PACKAGE 46
FIGURE 7:HARDWARE OR SOFTWARE-CONFIGURABLE POWER DOMAIN CONTROL BLOCK DIAGRAM 59
FIGURE 8:MULTI-FUNCTION PIN - INPUT PATH 66
FIGURE 9:MULTI-FUNCTION PIN - OUTPUT PATH 67
FIGURE 10:MFPR STRUCTURE 70
FIGURE 11:FUNCTION ASSIGNMENTS FOR MULTI-FUNTION I/O PINS 74
FIGURE 12:ARM CORTEX-R5 MODEM CPU SUBSYSTEM BLOCK DIAGRAM 78
FIGURE 13: SRAM CONTROLLER BLOCK DIAGRAM 81
FIGURE 14:LTE SYSTEM BLOCK DIAGRAM 89
FIGURE 15:LTE MODEM BLOCK DIAGRAM 90
FIGURE 16:TIMERS BLOCK DIAGRAM 93
FIGURE 17:INTERFACING THE USIM CONTROLLER TO A USIM SMARTCARD 98
FIGURE 18:BYTE SENT VERSUS TIME 100
FIGURE 19:T=0 PROTOCOL COMMUNICATION METHOD 101
FIGURE 20:COMPLETE BLOCK STRUCTURE FOR T=1 PROTOCOL 103
FIGURE 21:T=0 CHARACTER TRANSMISSION FORMAT 106
FIGURE 22:FRAMING ERROR 107
FIGURE 23:CHARACTER WAITING TIME, USIM CONTROLLER 108
FIGURE 24:BLOCK GUARD TIME, USIM CONTROLLER 111
FIGURE 25:BLOCK WAITING TIME, USIM CONTROLLER 113
FIGURE 26:SMARTCARD CLOCK STOP 115
FIGURE 27: BAUD RATE SAMPLING PULSES WHEN NUMBER OF SAMPLES PER BIT IS 6 117
FIGURE 28:SPACING BETWEEN SAMPLES WHEN BAUD DIVISOR IS 2 117
FIGURE 29:SMARTCARD ACTIVATION 121
FIGURE 30:SMARTCARD DE-ACTIVATION 123
FIGURE 31:ASR1606 POWER MANAGEMENT BLOCKS 129
FIGURE 32:POWER ON SEQUENCE TIMING 130
FIGURE 33:PMU WAKEUP CONNECTIVITY 132
FIGURE 34: ASR1606 POWER DOMAINS 135
FIGURE 35:TOP-LEVEL BLOCK DIAGRAM 140
FIGURE 36:CLOCK SOURCES FOR ASR1606 141
FIGURE 37:ARM CORTEX-R5 SUBSYSTEM CLOCK 144
FIGURE 38:POR AND WDT RESET SYNCHRONIZERS 150
FIGURE 39:TAP CONTROLLER STATE DIAGRAM 158
FIGURE 40:SECONDARY JTAG CONTROLLER CONNECTION 159
FIGURE 41:DMAC BLOCK DIAGRAM 163
FIGURE 42:DREQ TIMING REQUIREMENTS 164
FIGURE 43:DESCRIPTOR-FETCH TRANSFER CHANNEL STATE DIAGRAM 171
FIGURE 44:FLOW CHART FOR DESCRIPTOR BRANCHING 172
FIGURE 45:NO-DESCRIPTOR-FETCH TRANSFER CHANNEL STATE DIAGRAM 175
FIGURE 46:DESCRIPTOR CHAIN FOR SOFTWARE IMPLEMENTATION OF FULL AND EMPTY BITS 199
FIGURE 47:USB 2.0 BLOCK DIAGRAM 201
FIGURE 48:USB 2.0 PHY BLOCK DIAGRAM 202
FIGURE 49:TIMERS BLOCK DIAGRAM 215
FIGURE 50:TWSI BUS CONFIGURATION EXAMPLE 222
FIGURE 51:TWSI BUS INTERFACE UNIT BLOCK DIAGRAM 223
FIGURE 52:TX AND RX FIFOS SHOWN IN FIFO MODE 225
FIGURE 53:SDA AND SCL SIGNALS DURING START AND STOP CONDITIONS 228

CONFIDENTIAL

FIGURE 54:START AND STOP CONDITIONS 231
FIGURE 55:DATA FORMAT OF FIRST BYTE IN MASTER TRANSACTION 234
FIGURE 56:ACKNOWLEDGE PULSE ON THE TWSI BUS 235
FIGURE 57:CLOCK SYNCHRONIZATION DURING ARBITRATION 238
FIGURE 58:ARBITRATION PROCEDURE FOR TWO MASTERS 239
FIGURE 59:HIGH SPEED MODE DATA TRANSFER 243
FIGURE 60:MASTER RECEIVER READ FROM SLAVE TRANSMITTER 246
FIGURE 61:MASTER-RECEIVER READ FROM SLAVE-TRANSMITTER / REPEATED START/MASTER-TRANSMITTER WRITE TO SLAVE-RECEIVER 246
FIGURE 62:A COMPLETE DATA TRANSFER 246
FIGURE 63:TX FIFO ENTRY FORMAT 257
FIGURE 64:RX FIFO ENTRY FORMAT 257
FIGURE 65:TRANSMIT AND RECEIVE FIFOS 257
FIGURE 66:CASE 1: WRITE 10 BYTES 259
FIGURE 67:CASE 2: READ 8 BYTES FOLLOWED BY WRITE 2 BYTES 259
FIGURE 68:MASTER TRANSMITTER WRITE TO SLAVE RECEIVER 263
FIGURE 69:MASTER RECEIVER READ FROM SLAVE-TRANSMITTER 263
FIGURE 70:MASTER-RECEIVER READ TO SLAVE-TRANSMITTER, REPEATED START, MASTER-TRANSMITTER WRITE TO SLAVE RECEIVER 263
FIGURE 71:GENERAL CALL ADDRESS 269
FIGURE 72:TEXAS INSTRUMENTS* SYNCHRONOUS SERIAL FRAME PROTOCOL (SINGLE TRANSFERS) 287
FIGURE 73:TEXAS INSTRUMENTS* SYNCHRONOUS SERIAL FRAME PROTOCOL (MULTIPLE TRANSFERS) 288
FIGURE 74:MOTOROLA* SPI FRAME PROTOCOL (SINGLE TRANSFERS) 290
FIGURE 75:MOTOROLA* SPI FRAME PROTOCOL (MULTIPLE TRANSFERS) 291
FIGURE 76:MOTOROLA* SPI FRAME PROTOCOLS FOR SPO AND SPH PROGRAMMING (SPH SET) 293
FIGURE 77:MOTOROLA* SPI FRAME PROTOCOLS FOR SPO AND SPH PROGRAMMING 293
FIGURE 78:NATIONAL SEMICONDUCTOR* MICROWIRE FRAME PROTOCOL (SINGLE TRANSFER) 295
FIGURE 79:NATIONAL SEMICONDUCTOR* MICROWIRE FRAME PROTOCOL (MULTIPLE TRANSFERS) 296
FIGURE 80:PROGRAMMABLE SERIAL PROTOCOL FORMAT 299
FIGURE 81:PROGRAMMABLE PROTOCOL FORMAT (CONSECUTIVE TRANSFERS) 300
FIGURE 82:TI SSP WITH SSP_SSCR1[TTE] = 1 AND SSP_SSCR1[TTELP]] = 0 302
FIGURE 83:TI SSP WITH SSP_SSCR1[TTE] = 1 AND SSP_SSCR1[TTELP] = 1 303
FIGURE 84:MOTOROLA* SPI WITH <TXD THREE-STATE ENABLE> = 1 AND <TXD THREE-STATE ENABLE ON LAST PHASE> = 0 303
FIGURE 85:NATIONAL MICROWIRE WITH SSP_SSCR1[TTE] = 1 AND SSP_SSCR1[TTELP] = 0 304
FIGURE 86:PSP FORMAT WITH SSP_SSCR1[TTE] = 1, SSP_SSCR1[TTELP] = 0, AND SSP_ SSCR1[SFRMDIR] = 1 304
FIGURE 87:PSP FORMAT WITH SSP_SSCR1[TTE] = 1, AND EITHER SSP_SSCR1[TTELP] = 1, OR SSP_ SSCR1[SFRMDIR] = 0 305
FIGURE 88:NETWORK MODE (EXAMPLE USING 4 TIME SLOTS) 308
FIGURE 89:SSP BAUD RATE GENERATION 315
FIGURE 90 SDIO PHYSICAL CONNECTION - 4BIT MODE1 319
FIGURE 91 SDIO PHYSICAL CONNECTION - 1BIT MODE1 320
FIGURE 92 SDIO PHYSICAL CONNECTION – SPI MODE1 321
FIGURE 93 MULTI-PORT DATAFLOW 328
FIGURE 94 SDIO PROTOCOL TIMING DIAGRAM - NORMAL MODE 330
FIGURE 95 SDIO PROTOCOL TIMING DIAGRAM – HIGH SPEED MODE 330
FIGURE 96:EXAMPLE UART DATA FRAME 337
FIGURE 97:EXAMPLE NRZ BIT ENCODING — 0B0100_1011 338
FIGURE 98:IR TRANSMIT AND RECEIVE EXAMPLE 351
FIGURE 99:XMODE EXAMPLE 352
FIGURE 100:GENERAL-PURPOSE I/O BLOCK DIAGRAM 358
FIGURE 101: AUDIO CODEC BLOCK DIAGRAM 363
FIGURE 102: AUDIO CODEC INTERFACE DIAGRAM 364
FIGURE 103:SD HOST BLOCK DIAGRAM 368
FIGURE 104:“NO RESPONSE” AND “NO DATA” OPERATION 370
FIGURE 105:MULTIPLE BLOCK READ OPERATION 370

ASR CONFIDENTIAL

FIGURE 106:MULTIPLE BLOCK WRITE WITH CARD BUSY OPERATION 370
FIGURE 107:READ WAIT CONTROLLED BY STOPPING CLOCK 371
FIGURE 108:COMMAND TOKEN FORMAT 372
FIGURE 109:RESPONSE TOKEN FORMAT 372
FIGURE 110:DATA PACKET FORMAT, STANDARD BUS (ONLY SDDATA[0] USED) 373
FIGURE 111:DATA PACKET FORMAT, WIDE BUS (ALL FOUR DATA LINES USED) 373
FIGURE 112:HOST INITIALIZATION FLOW CHART–SDIO RESPONSE TO NON-I/O AWARE INITIALIZATION 375
FIGURE 113:HOST INITIALIZATION FLOW CHART–CARD INITIALIZATION FLOW IN SD MODE 376
FIGURE 114:HOST INITIALIZATION FLOW CHART–CARD INITIALIZATION IN SPI MODE 377
FIGURE 115 TSC BLOCK DIAGRAM 381
FIGURE 116 SW STATE MACHINE DESIGN 384
FIGURE 117: R F BLOCK DIAGRAM 388
FIGURE 118:238-PIN 8.2X7.4MM BGA PACKAGE 398
FIGURE 119:COMMERCIAL PACKAGE MARKING AND PIN 1 LOCATION 399

ASR CONFIDENTIAL

About this Datasheet

This section describes:

- Document Purpose
- Number Representation
- Naming Conventions

Naming Conventions

All signal names appear in uppercase. Active low signals are identified with a lower case n: SDRAM_CLKn
Individual signals within a signal name and bits within a register bit field are enclosed in square brackets:
EXTERNAL_ADDRESS[31:0] CS[1]n
REGISTER_BITFIELD[3:0] REGISTER_BIT[0]

ASR CONFIDENTIAL

Related Documentation

Table 1 lists supplemental information sources for the ASR1606. To obtain these or other related documents, contact ASR or sales representatives.

Table 1: Related Documents

Document	Doc Number
ASR Documentation	
ASR1606 Pinout	ASR1606

ASR CONFIDENTIAL

1. Introduction

ASR1606 is a highly cost-efficient System on Chip (SoC) device which integrates application processing subsystem, communication subsystem, audio codec, and embedded pSRAM + flash to enable the most compact single-chip LTE single-mode data module, POC, POS, and other IoT solutions. The communication subsystem integrates LTE CAT1 modem baseband and RF transceiver to cover 450MHz~2.7GHz bands for worldwide roaming. The application subsystem runs on a Cortex-R5 processor with integrated interface for 300K pixel camera and SPI LCD.

In addition, an extensive set of interfaces and connectivity peripherals are included to interface to cameras, displays, sensors, GPS, Bluetooth, etc.

This chapter provides an overview of the platform. It includes the following sections:

- Section 1.1, Soc device features

1.1 SOC device features

1.1.1 Platform Features

- General
  - 22nm process, BGA package with 0.4mm pitch.
  - Operation temperature: -20~70 (C grade), -40~85 (I grade).
- Application Processor
  - ARM Cortex-R5 up to 614MHz clock.
    - 32K I-Cache
    - 32K D-Cache
    - 64KB ATCM and 64KB BTCM
  - 64KB ROM and 64KB on-chip SRAM for application usage.
- Memory
  - Embedded 8bit Octal-SPI pSRAM, support DDR mode. Up to 400Mbyte/s bandwidth.
  - Embedded flash with XIP (eXecute In Place) and QSPI mode support, up to 102MHz.
- Peripheral Controllers
  - Keypad controller:
    - Support 8x8 keypad with hardware scanner and internal pull-up resistors.
    - Support multiple key press for gaming
    - Support direct key with build-in de-bouncing

ASR CONFIDENTIAL

  - GPIO (×85)
    - Pull-up/pull-down programmable
    - 1.8V/3.3V IO
  - UART (×4)
    - All support flow-cotrol except UART3
    - With hardware flow-control, up to 3.6Mbps
    - Two UART support Full 8-wire connection
  - USB 2.0(×1)
  - EMMC/SD Controller
    - Compatible with EMMC v4.5(up to HS200)
    - Support 1bit/4bit bus-width
  - SDIO Device Controller
    - for high bandwidth host CPU connection
  - I2C (×5)
    - Modes: 100KHz/400KHz/1MHz/3.4MHz
  - General SSP interface(×3)
    - Support both master and slave mode
    - Can be configured to SPI, PCM, I2S and Microwire format.
  - PWM (×4)
  - Dual SIM/USIM card controller
  - Integrated Resistive touch controller
    - 4 wire inputs
- Wireless connection interfaces
  - Bluetooth 5.0, FM,GNSS
- Security System
  - True Random Number Generator
  - 512bits OTP for security key
  - Secure Boot/Strap/Bonding
- Auxiliary analog inputs
  - Auxiliary ADC with 5 inputs
- Debug System
  - JTAG for CPU & DSP sub-system
  - UARTs and USB
- Boot System
  - Initial AP boot from UART/USB/SPI NOR
  - OTP for boot control
  - 64KB BootROM

ASR CONFIDENTIAL

1.1.2 Multimedia Features

- Camera & ISP
  - Camera support up to 300K pixel sensor
  - Support 1/2 bit SPI interface
- Audio Codec
  - Integrated High quality audio codec and audio front-end
    - ADC: 90dB SNR@20~20kHz
    - DAC: 95dB SNR@20~20kHz
  - single MIC input
  - Audio content sampling rates: 8kHz to 48kHz
  - Quad vocoders for adaptive multi-rate (AMR)
  - Noise suppression and echo cancellation
  - Digital side tone generator with programmable gain
  - Voice power amplifier with programmable gain

ASR CONFIDENTIAL

1.1.3 Modem features

- Modem baseband
  - FDD/TDD LTE CAT1/M Bis
  - Support VoLTE
- Integrated RF transceiver
  - Cover frequency range from 450MHz to 2.7GHz
  - Support up to 8 Rx ports and 4 Tx ports
  - Support both DCXO and TCXO
- 32KHz crystal is not needed in SCS mode
- Ultra low power
  - VoLTE 70mA

ASR CONFIDENTIAL

2. Functional Description Overview

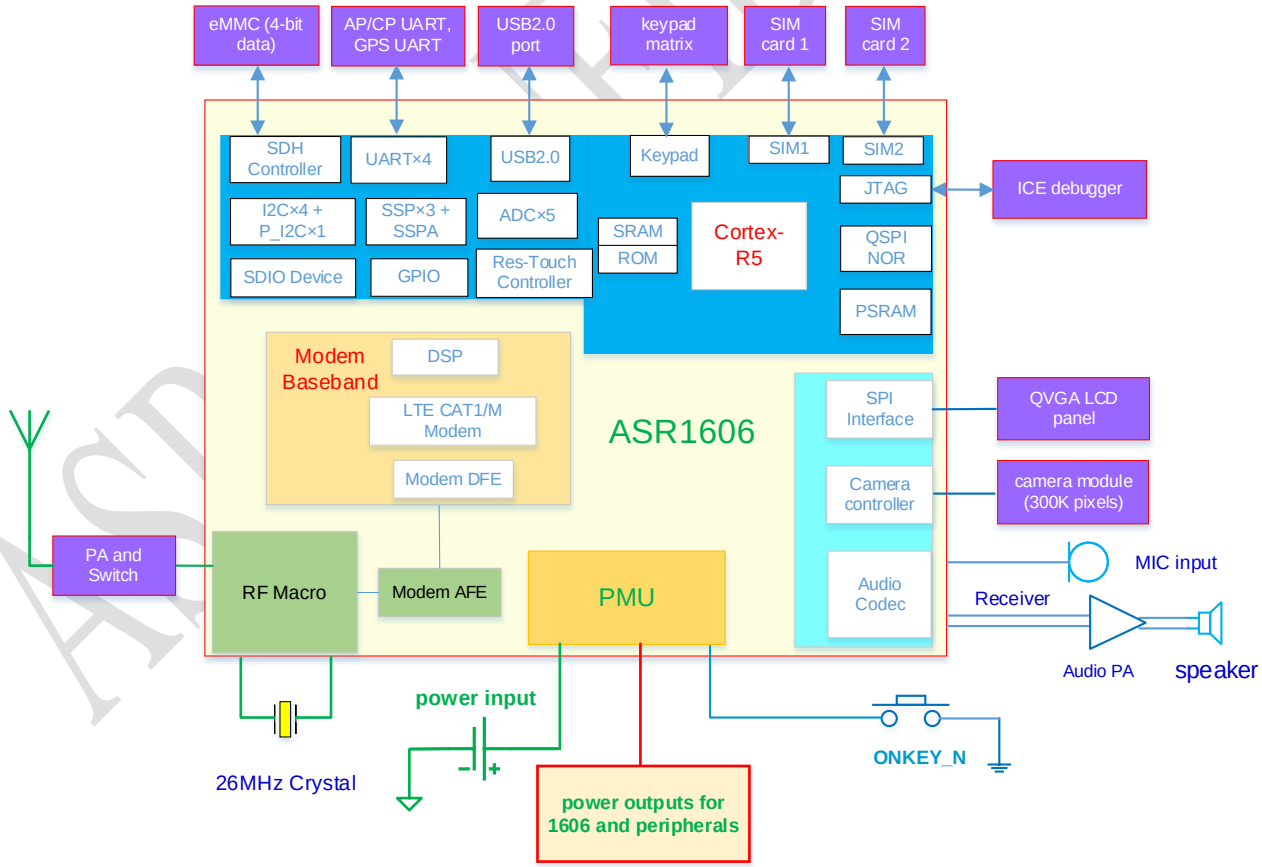
- Section 2.1, Architecture Overview
- Section 2.2, Chip Configuration
- Section 2.3, Memory Map
- Section 2.4, Interrupt Connectivity and Assignments
- Section 2.5, DMA Connectivity and Assignments

2.1 Architecture Overview

2.1.1 Block Diagram

This section provides an overview of the ASR1606 platform. The main components of the ASR1606 architecture are shown in Figure 1.

Figure 1:ASR1606 Architecture Block Diagram



ASR CONFIDENTIAL

2.1.2 Main Buses Connectivity and Arbitration

2.1.2.1 PSRAM Controller Arbitration

The PSRAM Controller supports 4 AXI slave ports. Arbitration exists among them. Refer to Table 2 for details on the port connections to the PSRAM Controller.

Table 2:PSRAM AXI Port Connections

P0	P1	P2	P3
Multi-Media	ARM Cortex-R5 and DTC	AXI Fabric1 s0 port	DSP and Modem

The PSRAM Controller provides 2 arbitration schemes:

- priority-based arbitration
- weighed round robin arbitration

Priority-based arbitration is used to control the latency of important accesses while weighed round robin arbitration is used to control the bandwidth distribution. There is no hardware constraint for using both schemes at the same time but doing so can introduce certain complexity.

ASR CONFIDENTIAL

2.2 Chip Configuration

The ASR1606 platform provides several configuration options, which are controlled using the following hardware mechanisms:

- Section 2.2.1, Dedicated Control Pins

2.2.1 Dedicated Control Pins

One dedicated control pin determines the following chip configuration settings:

- Section 2.2.1.1, JTAG Select

Both control pins have default values. External pullup resistors must be mounted on the board to use values other than the default.

2.2.1.1 JTAG Select

The ASR1606 platform primary JTAG interface (PRI_TDI, PRI_TMS, PRI_TCK, and PRI_TDO) can be used for other functions as described in the pin-muxing documentation. This feature enables using these four pads as functional pads also in the final mobile phone design where JTAG is not required.

2.3 Memory Map

This section provides an overview of the processor physical address map for memory and memory-mapped registers. It also summarizes the registers within the independent coprocessors in the processor core. Refer to individual unit chapters of the peripherals volume for the mapping of individual registers.

All internal control and configuration registers are mapped into physical memory space on 32-bit address boundaries. Use 32-bit word access loads and stores to access internal registers. Internal register space must be mapped as non-cacheable. In general, many buffers and FIFOs can be accessed in byte, half-word, and word sizes. Byte and half-word accesses to internal registers are not permitted and yield unpredictable results. Refer to the individual peripheral chapters for details.

Register space with no register mapping is defined as *reserved space*. Reading or writing reserved space causes unpredictable results.

The processor does not use all register bit locations. The unused bit locations are marked as reserved and are allocated for future use. Write reserved bit locations with 0b0. Ignore the values of these bits during Read operations because they are unpredictable.

ASR CONFIDENTIAL

The physical memory map includes external memory, internal memory, and the memory-mapped internal registers of the peripheral controllers. Services-unit registers are accessed indirectly through the peripheral applications subsystem. Thus, the peripheral-register address map includes all services-unit registers for power management, clock management, the TWSI interface (which controls external supply regulators), and the real-time clock. Software can use the memory management unit in the core to map portions of this physical address map to the virtual address map.

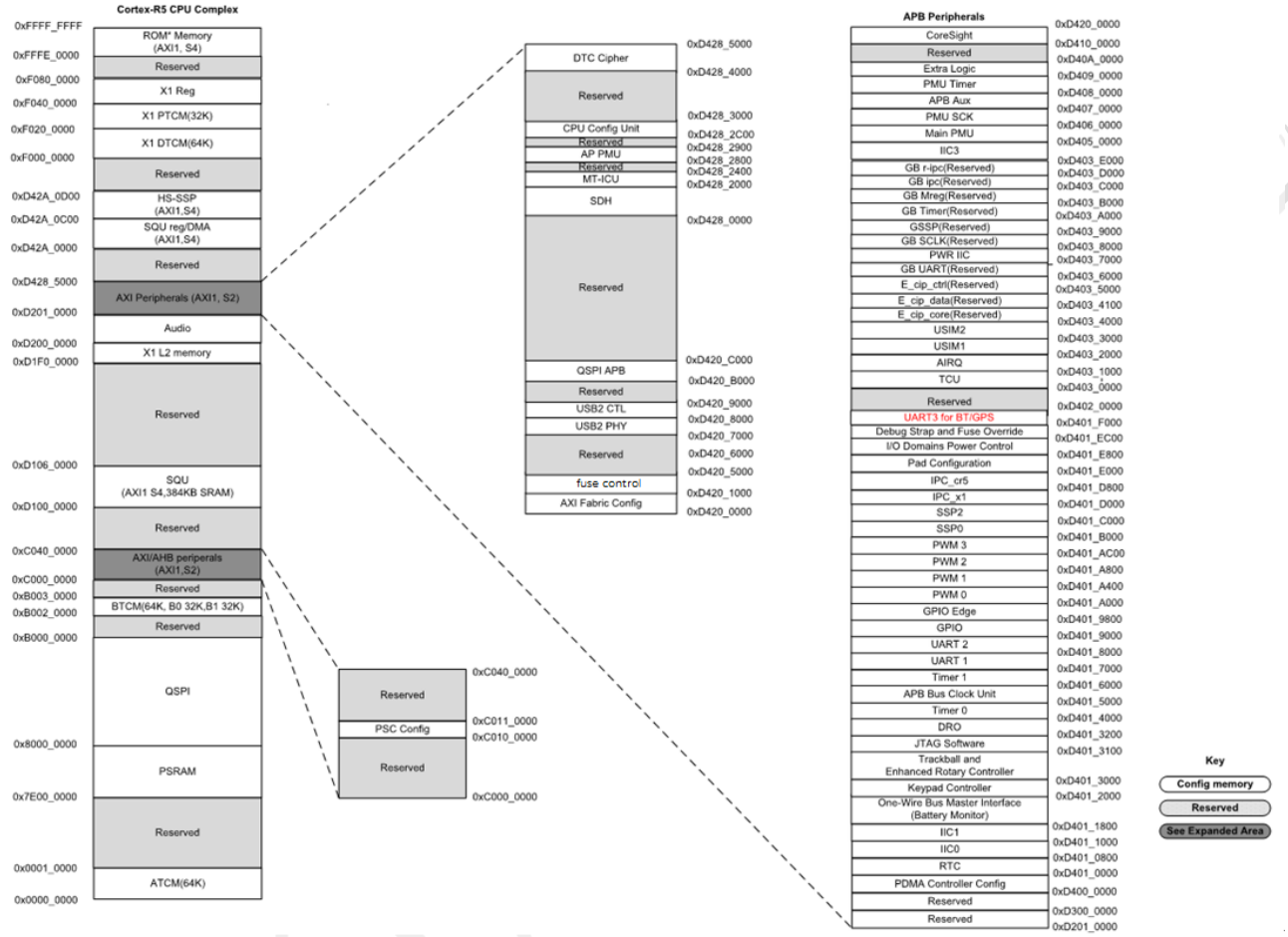
Note

Accessing reserved portions of the memory map results in a data-abort exception. Accessing reserved portions of a particular peripheral address space does not cause a data-abort exception, but the data returned is undefined.

Refer to Figure 2 for the ASR1606 memory map definitions.

ASR CONFIDENTIAL

Figure 2: ARM Cortex-R5 Subsystem Memory Map



ASR Confidential

2.4 Interrupt Connectivity and Assignments

2.4.1 Overview

The ASR1606 platform contains the following Interrupt controllers:

ICU–Connected to the ARM Cortex-R5 Subsystem.

CEVA-BX2 internal ICU

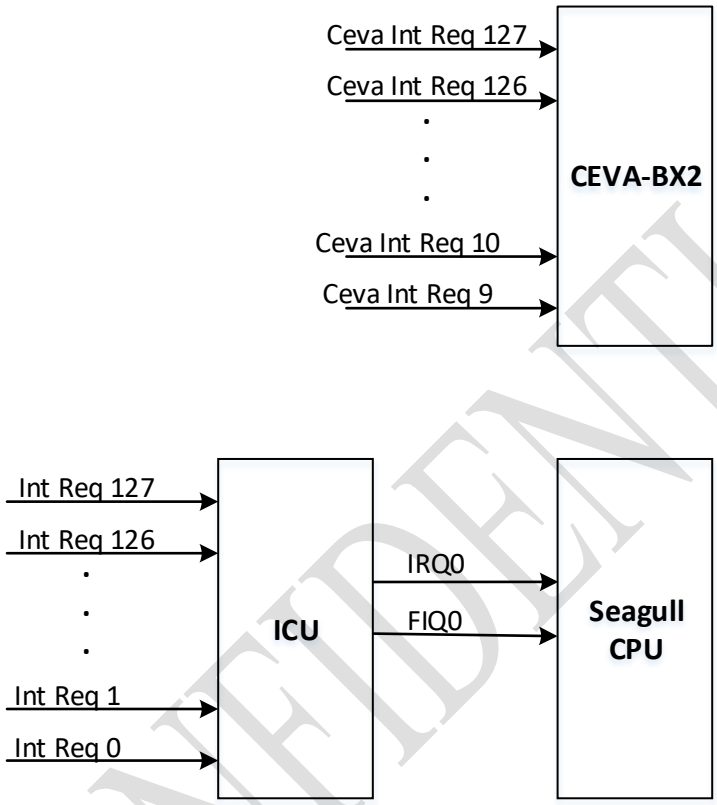
EIC–Connected to the Modem DSP core (ASR1606 uses EIC inputs IVG7-IVG15)

- Modem DSP Interrupt Request (MIRQ)–Connected to the EIC (input IVG14)
- Inputs IVG13 and IVG15 are reserved for software Refer to Figure 3 for an illustration.

Typically, every input of an Interrupt Controller is assigned to one interrupt source. Sometimes several sources are ORed or muxed together and connected to one input.

ASR CONFIDENTIAL

Figure 3:Interrupt Connectivity



ASR CONFIDENTIAL

2.4.2 ICU Interrupt Connectivity

As shown in Figure 3, the ICU has two types of interrupts:

Interrupt Request (IRQ) for standard Interrupt handling

Fast Interrupt Request (FIQ) for fast, low latency interrupt handling

It takes 128 asynchronous level interrupt inputs. Every interrupt can be assigned to the FIQ or IRQ. All the 128 interrupts are routed to ARM Cortex-R5 processor.

Table 3 lists the inputs to the ICU. The interrupt priority is programmable, so the order of the interrupts in the table is significant only among the interrupts that are assigned the same priority.

Table 3: ICU Hardware Interrupt

Interrupt number	Interrupt source
[0]	m1_arm946e_s_int
[1]	m1_ssp2_int_req
[2]	m1_ssp1_int_req
[3]	m1_ssp_int_req
[4]	~m1_PMIC_INT_DI
[5]	m1_rtc_hzclk_int_ndr
[6]	m1_rtc_slp_alarm_ndr
[7]	m1_i2c0_int
[8]	RSVD
[9]	m1_kp_int
[10]	m1_usim_1_int_irq
[11]	m1_usim_2_int_irq
[12]	m1_onewire_int
[13]	m1_timer_1_irq
[14]	m1_timer_2_irq|m1_timer_3_irq
[15]	m1_i2c1_int
[16]	m1_ipc_ceva2cp_int0
[17]	m1_ipc_ceva2cp_int1
[18]	m1_ipc_ceva2cp_int2
[19]	m1_ipc_ceva2cp_int3
[20]	m1_ipc2pmu_ceva_wakeup
[21]	m1_ipc_cp2ceva_int0
[22]	m1_ipc_cp2ceva_int1

ASR CONFIDENTIAL

[23]	m1_ipc_cp2ceva_int2
[24]	m1_ipc_cp2ceva_int3
[25]	m1_ipc2pmu_cp_wakeup
[26]	mc_irq
[27]	m1_ffuart1_int
[28]	m1_dro_int
[29]	m1_timer1_1_irq
[30]	m1_timer1_2_irq|m1_timer1_3_irq
[31]	m1_cp_timer_1_irq
[32]	m1_cp_timer_2_irq|m1_cp_timer_3_irq
[33]	tsen_int (Temperature sensor)
[34]	m1_ffuart4_int
[35]	m1_wdt_irq
[36]	m1_pm_xsc_wakeup_int
[37]	pmu_irq2
[38]	pmu_irq
[39]	(mmc1_int|mmc2_int|mmc3_int)
[40]	aeu_int
[41]	RSVD
[42]	tsc_int_rdy
[43]	pl310 l2ccintr_cp
[44]	usb_int | usb_vbus_id_wakeup
[45]	qspi_int
[46]	tsc_int_up
[47]	dma_int0 (AP MAIN DMA)
[48]	dma_int1 (AP MAIN DMA)
[49]	m1_gpio_int_ap
[50]	nCTIIRQ_cp(CR5)
[51]	RSVD
[52]	m1_ripc_int1
[53]	usb_battery_wakeup
[54]	m1_i2c_int
[55]	m1_gpio_int_cp
[56]	m1_ripc_int0
[57]	~npmuirq_cp(CR5)
[58]	ipe_irq
[59]	m1_uart_int (uart2)
[60]	gpio_edge_det_wakeup
[61]	tsc_int_dn

ASR CONFIDENTIAL

[62]	fabric0_timeout
[63]	tsen_auxadc_int
[64]	pmic_wakeup_int
[65]	airq_ext1_int
[66]	airq_ext2_int
[67]	pmu_int
[68]	dtc_ch0_int
[69]	dtc_ch1_int
[70]	dtc_error_int
[71]	m1_dtc_trigger_int
[72]	gpio_int
[73]	d2_modem_entry_int
[74]	gpb_timer1_int
[75]	gpb_timer2_int
[76]	gpb_timer3_int
[77]	VNS Modem int0
[78]	VNS Modem int1
[79]	cevax_cverbit_r (cevabx2)
[80]	cevax_seq_eotbit_r (cevabx2)
[81]	gsm_timeout1_int_level
[82]	sg_error_resp_int
[83]	ccr5_reg_nmi_int
[84]	cevax_ddma_dbg_match_r (cevabx2)
[85]	rf_err_int_to_bx2
[86]	rf_seq_int_to_bx2
[87]	apb_mon_int
[88]	cevax_snoop_sn_int (cevabx2)
[89]	cevax_ocm_debug_r (cevabx2)
[90]	m1_ripc_int2
[91]	m1_lte_end_sleep_int
[92]	bb_lte_wakeup_sticky
[93]	m1_i2c2_int
[94]	m1_lte_tcu_or_rtu_latch_int
[95]	m1_ffuart3_int
[96]	vns_modem_wakeup_int
[97]	m1_airq_int
[98]	m1_i2c3_int
[99]	scs_rtc_reg_sync_int
[100]	aud_plug_int

ASR CONFIDENTIAL

[101] [102]	aud_ocp_int hook_key_int
[103]	arb_timeout1_int_level
[104]	arb_timeout2_int_level
[105]	dfe_int
[106]	sspa_int
[107]	adma_ch0_int
[108]	adma_ch1_int
[109]	cp_timeout1_int_level
[110]	cp_timeout2_int_level
[111]	dma8_int
[112]	m1_lte_rx_time_signals[13]
[113]	m1_lte_tx_time_signals[13]
[114]	m1_lte_irat_time_signals[13]
[115]	sdh2icu_wakeup_int1|sdh2icu_wakeup_int2|sdh2icu_wakeup_int3
[116]	keypad_wakeup
[117]	sdu_int2icu
[118]	classg_shortpwr_int
[119]	RSVD
[120]	RSVD
[121]	RSVD
[122]	RSVD
[123]	RSVD
[124]	RSVD
[125]	RSVD
[126]	RSVD
[127]	RSVD

Table 4 CEVA-BX2 Interrupts

Interrupt number	Interrupt source
[8~0]	Internal interrupts
[9]	m1_arm946e_s_int
[10]	m1_ssp2_int_req
[11]	m1_ssp1_int_req
[12]	m1_ssp_int_req
[13]	~m1_PMIC_INT_DI
[14]	m1_rtc_hzclk_int_ndr

ASR CONFIDENTIAL

[15]	m1_rtc_slp_alarm_ndr
[16]	m1_i2c0_int
[17]	RSVD
[18]	m1_kp_int
[19]	m1_usim_1_int_irq
[20]	m1_usim_2_int_irq
[21]	m1_onewire_int
[22]	m1_timer_1_irq
[23]	m1_timer_2_irq|m1_timer_3_irq
[24]	m1_i2c1_int
[25]	m1_ipc_ceva2cp_int0
[26]	m1_ipc_ceva2cp_int1
[27]	m1_ipc_ceva2cp_int2
[28]	m1_ipc_ceva2cp_int3
[29]	m1_ipc2pmu_ceva_wakeup
[30]	m1_ipc_cp2ceva_int0
[31]	m1_ipc_cp2ceva_int1
[32]	m1_ipc_cp2ceva_int2
[33]	m1_ipc_cp2ceva_int3
[34]	m1_ipc2pmu_cp_wakeup
[35]	mc_irq
[36]	m1_ffuart1_int
[37]	m1_dro_int
[38]	m1_timer1_1_irq
[39]	m1_timer1_2_irq|m1_timer1_3_irq
[40]	m1_cp_timer_1_irq
[41]	m1_cp_timer_2_irq|m1_cp_timer_3_irq
[42]	tsen_int (Temperature sensor)
[43]	m1_ffuart4_int
[44]	m1_wdt_irq
[45]	m1_pm_xsc_wakeup_int
[46]	pmu_irq2
[47]	pmu_irq
[48]	(mmc1_int|mmc2_int|mmc3_int)
[49]	aeu_int
[50]	RSVD
[51]	tsc_int_rdy
[52]	pl310 l2ccintr_cp
[53]	usb_int | usb_vbus_id_wakeup

ASR CONFIDENTIAL

[54]	qspi_int
[55]	tsc_int_up
[56]	dma_int0 (AP MAIN DMA)
[57]	dma_int1 (AP MAIN DMA)
[58]	m1_gpio_int_ap
[59]	nCTIIRQ_cp(CR5)
[60]	RSVD
[61]	m1_ripc_int1
[62]	usb_battery_wakeup
[63]	m1_i2c_int
[64]	m1_gpio_int_cp
[65]	m1_ripc_int0
[66]	~npmuirq_cp(CR5)
[67]	ipe_irq
[68]	m1_uart_int (uart2)
[69]	gpio_edge_det_wakeup
[70]	tsc_int_dn
[71]	fabric0_timeout
[72]	tsen_auxadc_int
[73]	pmic_wakeup_int
[74]	airq_ext1_int
[75]	airq_ext2_int
[76]	pmu_int
[77]	dtc_ch0_int
[78]	dtc_ch1_int
[79]	dtc_error_int
[80]	m1_dtc_trigger_int
[81]	gpio_int
[82]	d2_modem_entry_int
[83]	gpb_timer1_int
[84]	gpb_timer2_int
[85]	gpb_timer3_int
[86]	VNS Modem int0
[87]	VNS Modem int1
[88]	cevax_cverbit_r (cevabx2)
[89]	cevax_seq_eotbit_r (cevabx2)
[90]	gsm_timeout1_int_level
[91]	sg_error_resp_int
[92]	ccr5_reg_nmi_int
[93]	cevax_ddma_dbg_match_r (cevabx2)

ASR CONFIDENTIAL

[94]	rf_err_int_to_bx2
[95]	rf_seq_int_to_bx2
[96]	apb_mon_int
[97]	cevax_snoop_sn_int (cevabx2)
[98]	cevax_ocm_debug_r (cevabx2)
[99]	m1_ripc_int2
[100]	m1_lte_end_sleep_int
[101]	bb_lte_wakeup_sticky
[102]	m1_i2c2_int
[103]	m1_lte_tcu_or_rtu_latch_int
[104]	m1_ffuart3_int
[105]	vns_modem_wakeup_int
[106]	m1_airq_int
[107]	m1_i2c3_int
[108]	scs_rtc_reg_sync_int
[109]	aud_plug_int
[110]	aud_ocp_int
[111]	hook_key_int
[112]	arb_timeout1_int_level
[113]	arb_timeout2_int_level
[114]	dfe_int
[115]	sspa_int
[116]	adma_ch0_int
[117]	adma_ch1_int
[118]	cp_timeout1_int_level
[119]	cp_timeout2_int_level
[120]	dma8_int
[121]	m1_lte_rx_time_signals[13]
[122]	m1_lte_tx_time_signals[13]
[123]	m1_lte_irat_time_signals[13]
[124]	sdh2icu_wakeup_int1|sdh2icu_wakeup_int2|sdh2icu_wakeup_int3
[125]	keypad_wakeup
[126]	sdu_int2icu
[127]	classg_shortpwr_int

ASR CONFIDENTIAL

Table 5:MIRQ interrupt source

Int Num	Int source	Module	Edge/Level	Connection
0	cp_dma_channel_int[0]	CP DMA	Level	BX2 INT1
1	cp_dma_channel_int[1]			
2	cp_dma_channel_int[2]			
3	cp_dma_channel_int[3]			
4	cp_dma_channel_int[4]			
5	cp_dma_channel_int[5]			
6	cp_dma_channel_int[6]			
7	cp_dma_channel_int[7]			
8	ml_tcu_latch_int	MTU	Edge	
9	reserved			
10	reserved			
11	reserved			
12	reserved			
13	equ_int	EQ	Edge	
14	vit_int	Viterbi	Edge	
15	e_cip_int	E-cipher	Edge	
16	gpoa[0]	TCU	SW Configuration	
17	gpoa[1]			
18	gpoa[2]			
19	gpoa[3]			
20	gpoa[4]			
21	gpoa[5]			
22	gpoa[6]			
23	gpoa[7]			
24	gpoa[8]			
25	gpoa[9]			
26	gpoa[10]			
27	gpoa[11]			
28	gpoa[12]			
29	gpoa[13]			
30	gpoa[14]			
31	gpoa[15]			
32	gpob[0]			
33	gpob[1]			
34	gpob[2]			

ASR CONFIDENTIAL

35	gpob[3]			
36	gpob[4]			
37	gpob[5]			
38	gpob[6]			
39	gpob[7]			
40	gpob[8]			
41	gpob[9]			
42	gpob[10]			
43	gpob[11]			
44	gpob[12]			
45	gpob[13]			
46	gpob[14]			
47	gpob[15]			

Table 6:AIRQ interrupt source

Int Num	Int source	Module	Edge/Level	Connection
0	cp_dma_channel_int[0]	CP DMA	Level	CR5 #97
1	cp_dma_channel_int[1]			
2	cp_dma_channel_int[2]			
3	cp_dma_channel_int[3]			
4	cp_dma_channel_int[4]			
5	cp_dma_channel_int[5]			
6	cp_dma_channel_int[6]			
7	cp_dma_channel_int[7]			
8	ml_sck_snapshot_int	Slow clock	Edge	
9	ml_sck_early_wakeup_int			
10	ml_sck_wakeup_int		SW Configuration	
11	ml_tcu_latch_int	MTU	Edge	
12	ml_lte_rx_time_signals[13]	MTU		
13	ml_lte_tx_time_signals[13]	MTU		
14	ml_lte_irat_time_signals[13]	MTU		
15	reserved			
16	tcu_exception_int	TCU	Level	
17	e_cip_int	E-cipher	Edge	
31~18	reserved			
32	gpoa[0]	TCU	SW Configuration	
33	gpoa[1]			
34	gpoa[2]			
35	gpoa[3]			

ASR CONFIDENTIAL

36	gpoa[4]			
37	gpoa[5]			
38	gpoa[6]			
39	gpoa[7]			
40	gpoa[8]			
41	gpoa[9]			
42	gpoa[10]			
43	gpoa[11]			
44	gpoa[12]			
45	gpoa[13]			
46	gpoa[14]			
47	gpoa[15]			
48	gpob[0]			
49	gpob[1]			
50	gpob[2]			
51	gpob[3]			
52	gpob[4]			
53	gpob[5]			
54	gpob[6]			
55	gpob[7]			
56	gpob[8]			
57	gpob[9]			
58	gpob[10]			
59	gpob[11]			
60	gpob[12]			
61	gpob[13]			
62	gpob[14]			
63	gpob[15]			

2.5 *DMA Connectivity and Assignments*

There are two DMAs in ASR1606, they are AP DMA and CP DMA. Both AP DMA and CP DMA have the same features, the only different is that AP DMA has 32 DMA channels, CP DMA only has 8 DMA Channels.

The Direct-Memory Access Controller(DMAC) transfers data to and from memory in response to request generated by peripheral devices or companion chips. The peripheral devices and companion chips do not directly supply address and commands to the Memory Controller. Instead, the states required to manage a data stream are maintained in DMA Channels, in the DMAC. Every DMA request from a peripheral device generates a memory-bus transaction. The processor can access the peripheral bus through the bridge portion the DMA thus bypassing the system DMA. Therefore, this document refer to DMA as the "DMA Bridge".

ASR CONFIDENTIAL

2.5.1 Features

The DMAC provides the following features.

- Supports memory-to-memory, peripheral-to-memory, and memory-to-peripheral transfers in Flowthrough mode.
- Supports Flowthrough mode for transfers between flash and PSRAM.
- Employs a priority mechanism to process active channels(4 channels with outstanding DMA requests, at any given time)
- All each of the channels to operate for Descriptor-fetch or no-Descriptor-fetch transfers.
- Retrieves trailing bytes in Receive peripheral-device buffers
- Support programmable data-burst size(8,16 or 32 bytes) and programmable peripheral device data widths(1 byte, half-word, or word)
- Support up to 8192 bytes of data transfer per Descriptor. Larger transfers can be performed by chaining multiple Descriptors.
- Support flow-control bits to process requests from peripheral devices.

Requests are not processed unless the flow-control bit is set.

Figure 4 DMA Support Matrix

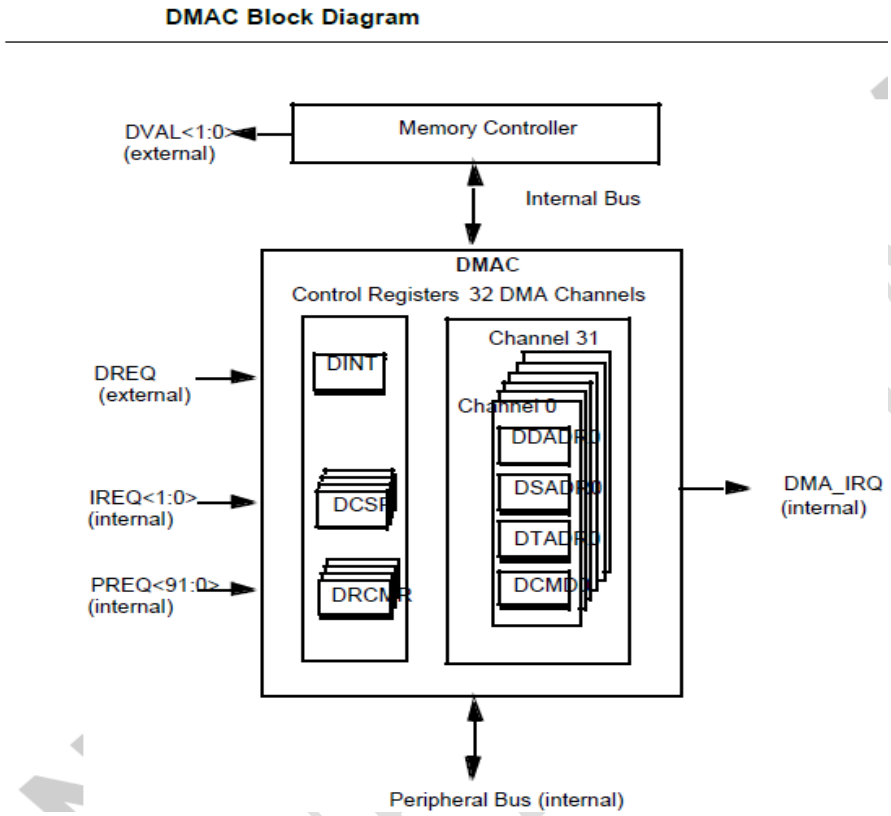
DMA Support Matrix

	Internal Memory	External Memory	Internal Peripheral	External Peripheral	Companion Chip
Internal Memory	flowthrough				
External Memory	flowthrough	flowthrough			
Internal Peripheral	flowthrough	flowthrough	—		
External Peripheral	flowthrough	flowthrough	—	—	

Figure 5 shows the AP DMA (32 DMA Channels) Block Diagram. CP DMA(8 DMA Channels) is just the same except only has 8 DMA channels.

ASR Revised CONFIDENTIAL

Figure 5 DMAC Block Diagram



The following table shows the dma request numbers for AP DMA peripherals.

Table 7:AP DMA peripherals

DRCMR Reg #	Description	Comments	dma_ signals	TOP signals	status
0		Spare for future use			
1	CLK_REQ from padring		dreq1		
2		Spare for future use	apb_req_r[0]		
3		Spare for future use	apb_req_r[1]		
4	dma_req for uart2 rx	UART2(0xD4018000)	apb_req_r[2]	m1_uart_rx_req	
5	dma_req for uart2 tx	UART2(0xD4018000)	apb_req_r[3]	m1_uart_tx_req	
6		Spare for future use	apb_req_r[4]		
7		Spare for future use	apb_req_r[5]		
8	dma_req for usim1 rx	USIM1(0xD4032000)	apb_req_r[6]	m1_usim_1_rx_dma_req	
9	dma_req for usim1 tx	USIM1(0xD4032000)	apb_req_r[7]	m1_usim_1_tx_dma_req	
10	dma_req for usim2 rx	USIM2(0xD4033000)	apb_req_r[8]	m1_usim_2_rx_dma_req	

Copyright © 2022 ASR

11	dma_req for usim2 tx	USIM2(0xD4033000)	apb_req_r[9]	m1_usim_2_tx_dma_req	
12		Spare for future use	apb_req_r[10]		
13		Spare for future use	apb_req_r[11]		
14		Spare for future use	apb_req_r[12]		
15		Spare for future use	apb_req_r[13]		
16		Spare for future use	apb_req_r[14]		
17		Spare for future use	apb_req_r[15]		
18		Spare for future use	apb_req_r[16]		
19		Spare for future use	apb_req_r[17]		
20		Spare for future use	apb_req_r[18]		
21	dma_req for uart1 rx	UART1(0xD4017000)	apb_req_r[19]	m1_ffuart1_rxreq	
22	dma_req for uart1 tx	UART1(0xD4017000)	apb_req_r[20]	m1_ffuart1_txreq	
23	dma_req for uart3 rx	UART3(0xD401F000)	apb_req_r[21]	m1_ffuart3_rxreq	
24	dma_req for uart3 tx	UART3(0xD401F000)	apb_req_r[22]	m1_ffuart3_txreq	
25	dma_req for i2c0 tx	I2C0(0xD4011000)	apb_req_r[23]	m1_i2c0_tx_dmareq	
26	dma_req for i2c0 rx	I2C0(0xD4011000)	apb_req_r[24]	m1_i2c0_rx_dmareq	
27	dma_req for i2c1 tx	I2C1(0xD4010800)	apb_req_r[25]	m1_i2c1_tx_dmareq	
28	dma_req for i2c1 rx	I2C1(0xD4010800)	apb_req_r[26]	m1_i2c1_rx_dmareq	
29		Spare for future use	apb_req_r[27]		
30	dma_req for pwr i2c tx	PWR_I2C(0xD4037000)	apb_req_r[28]	m1_gsm_i2c_tx_dmareq	
31	dma_req for pwr i2c rx	PWR_I2C(0xD4037000)	apb_req_r[29]	m1_gsm_i2c_rx_dmareq	
32	dma_req for i2c2 tx	I2C2(0xD403E000)	apb_req_r[30]	m1_i2c2_tx_dmareq	
33	dma_req for i2c2 rx	I2C2(0xD403E000)	apb_req_r[31]	m1_i2c2_rx_dmareq	
34	dma_req for i2c3 tx	I2C3(0xD403F000)	apb_req_r[32]	m1_i2c3_tx_dmareq	
35	dma_req for i2c3 rx	I2C3(0xD403F000)	apb_req_r[33]	m1_i2c3_rx_dmareq	
36	dma_req for uart4 rx	UART4(0xD401F800)	apb_req_r[34]	m1_ffuart4_rxreq	
37	dma_req for uart4 tx	UART4(0xD401F800)	apb_req_r[35]	m1_ffuart4_txreq	
38		Spare for future use	apb_req_r[36]		
39		Spare for future use	apb_req_r[37]		
40		Spare for future use	apb_req_r[38]		
41		Spare for future use	apb_req_r[39]		
42		Spare for future use	apb_req_r[40]		
43		Spare for future use	apb_req_r[41]		
44		Spare for future use	apb_req_r[42]		
45		Spare for future use	apb_req_r[43]		

Copyright © 2022 ASR

46		Spare for future use	apb_req_r[44]		
47		Spare for future use	apb_req_r[45]		
48		Spare for future use	apb_req_r[46]		
49		Spare for future use	apb_req_r[47]		
50		Spare for future use	apb_req_r[48]		
51		Spare for future use	apb_req_r[49]		
52	dma_req for ssp0 rxfifo	SSP0 (0xD401B000)	apb_req_r[50]	m1_ssp_rfs_dma_req	
53	dma_req for ssp0 txfifo	SSP0 (0xD401B000)	apb_req_r[51]	m1_ssp_tfs_dma_req	
54	dma_req for ssp1 rxfifo	SSP0 (0xD401B800)	apb_req_r[50]	m1_ssp1_rfs_dma_req	
55	dma_req for ssp1 txfifo	SSP0 (0xD401B800)	apb_req_r[51]	m1_ssp1_tfs_dma_req	
56		Spare for future use	apb_req_r[54]		
57		Spare for future use	apb_req_r[55]		
58		Spare for future use	apb_req_r[56]		
59		Spare for future use	apb_req_r[57]		
60	dma_req for ssp2 rxfifo	SSP2 (0xD401C000)	apb_req_r[58]	m1_ssp2_rfs_dma_req	
61	dma_req for ssp2 txfifo	SSP2 (0xD401C000)	apb_req_r[59]	m1_ssp2_tfs_dma_req	
62		Spare for future use	apb_req_r[60]		
63		Spare for future use	apb_req_r[61]		
64		Spare for future use	apb_req_r[62]		
65		Spare for future use	apb_req_r[63]		
66		Spare for future use	apb_req_r[64]		
67		Spare for future use	apb_req_r[65]		
68	dma_req for aeu read channel	AEU (0xD4201000)	ireq[2]	mux_aeu_dtc_ireq	
69	dma_req for aeu write channel	AEU (0xD4201000)	ireq[3]	aeu_dma_w_request	
70		Spare for future use			
71		Spare for future use	apb_req_r[66]		
72		Spare for future use	apb_req_r[67]		
73		Spare for future use	apb_req_r[68]		
74		Spare for future use			
75		Spare for future use	apb_req_r[69]		
76		Spare for future use	apb_req_r[70]		
77		Spare for future use	apb_req_r[71]		
78		Spare for future use	apb_req_r[72]		
79		Spare for future use	apb_req_r[73]		
80		Spare for future use	apb_req_r[74]		
81		Spare for future use	apb_req_r[75]		
82		Spare for future use	apb_req_r[76]		
83		Spare for future use	apb_req_r[77]		
84		Spare for future use	apb_req_r[78]		
85		Spare for future use	apb_req_r[79]		
86		Spare for future use	apb_req_r[80]		
87		Spare for future use	apb_req_r[81]		
88		Spare for future use	apb_req_r[82]		
89		Spare for future use	apb_req_r[83]		

Copyright © 2022 ASR

90		Spare for future use	apb_req_r[84]		
91		Spare for future use	apb_req_r[85]		
92		Spare for future use	apb_req_r[86]		
93		Spare for future use	apb_req_r[87]		
94		Spare for future use	apb_req_r[88]		
95		Spare for future use	apb_req_r[89]		
96		Spare for future use	apb_req_r[90]		
97	dma_req for qspi rxfifo	QSPI (0x80000000)	ireq[0]	qspi_rx_req	
98		Spare for future use			
99	dma_req for qspi txfifo	QSPI (0x80000000)	ireq[1]	qspi_tx_req	

DMA Request to Channel Map Registers 0-63 and 64-99

ASR CONFIDENTIAL

The following table shows the dma request numbers for CP DMA peripherals.

Table 8:CP DMA peripherals

DMA	DRCMR Reg #	Description	Comments	dma_ signals	CPSS TOP signals	status
DREQ 0	0		Spare for future use			
DREQ 1	1		Spare for future use			
PREQ 0	2	Request for RF RxFIFO dma req	RF (0xD4800000)	apb_req_r[0]	rxf_frmrdy_req	
PREQ 1	3	Request for ASPC E-Cipher dma req	GPB E-Cipher(0xD4430000)	apb_req_r[1]	e_cip_dma_req	
PREQ 2	4		Spare for future use	apb_req_r[2]		
PREQ 3	5		Spare for future use	apb_req_r[3]		
PREQ 4	6		Spare for future use	apb_req_r[4]		
PREQ 5	7		Spare for future use	apb_req_r[5]		
PREQ 6	8		Spare for future use	apb_req_r[6]		
PREQ 7	9		Spare for future use	apb_req_r[7]		

ASR CONFIDENTIAL

3. Pin Muxing and Multi-Function Pins

The ASR1606 has both dedicated (single-function) and multi-function pins. The Pin Control Unit (PCU) manages the configuration of the multi-function pins, including the selection of alternate peripheral functions. This chapter details the following:

- Section 3.1, Function Assignments for Multiplexed I/O Pins
- Section 3.2, Pinout
- Section 3.3, Multiplexed Signal Functions
- Section 3.4, Pad-Ring I/O Domains Power Control
- Section 3.5, Pin Control Unit
- Section 3.6, Function Assignments for Multi-function I/O Pins
- Section 3.7, Register Descriptions

The PCU contains registers (one register per multi-function pin) that configure the reset and low-power mode behavior of each multi-function pin, especially if the pin holds its last driven output value or is forced into one of these states:

- Output driven high
- Output driven low
- Input high impedance
- Input pulled high
- Input pulled low

These registers default to an appropriate state at reset or power up, but are software-configured thereafter.

The PCU also contains registers to configure the output drive strength of individual pins when configured as outputs. Many, but not all, multi-function pins support configuration as a software-managed GPIO channel that is programmed as an input or output, which can serve as an interrupt source. Many pins can also generate wakeup events to bring one of the subsystems out of low-power mode.

3.1 Function Assignments for Multiplexed I/O Pins

Most of the ASR1606 I/O pins are multiplexed. These pins are also known as *multi-function pins* because each of them is assigned to more than one function. Each pin has a dedicated Multi-Function Pin Register (MFPR) that controls its functionality and behavior. The main purpose of the MFPRs is to select one of several functions assigned to the multi-function pin.

The remainder of the ASR1606 I/O pins are dedicated pins that are assigned to

ASR CONFIDENTIAL

only one function. These pins do not have MFPRs associated with them.

Note

Using the MFPRs, select the function to be assigned to the multi-function pin before enabling or configuring the internal peripheral that is associated with that function.

Failure to perform this step in this order can lead to switching-related transitions that can be interpreted by the peripheral as valid data or strobe.

Signal descriptions for Functions 1 through 7 of the multi-function pins are provided in each block's function introduction.

The MFPR is described in Section 3.6, Function Assignments for Multi-function I/O Pins.

ASR CONFIDENTIAL

The mapping between the MFPRs and the multi-function pins they control is shown in Table 9.

Note Function 0 is the pin default function after reset.

Table 9 shows the functions of the multiplexed pins in the ASR1606.

ASR CONFIDENTIAL

3.2 Pinout

This section provides the pinout for the ASR1606 device. It is available in below packages:

- 238-pin 8.2X7.4mm , 0.4mm pitch BGA Package

As described in Section 3.1, Function Assignments for Multiplexed I/O Pins, signals may have dedicated pins or may be available only as an alternate function of a multi-function pin.

Figure 6: Complete Pinout 238-ball 8.2x7.4 mm BGA Package

BGA238	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
A	NC-A1	LDO8		LDO7		VINLDO	LDO1		GPIO_49	QSPI_DAT2		QSPI_DAT1	QSPI_CS1		GPIO_121		GPIO_26	VSS
B	VINREF	RSTN	AGND	VDDIO	LDO6	VINLDO	LDO3	GPIO_122	GPIO_50	GPIO_36	QSPI_DAT3	QSPI_DAT0	QSPI_CLK	PWR_SCL/SCL	PWR_SDA/SDA	GPIO_124	GPIO_25	GPIO_22
C	VREF1V2	EXTON1N	AGND	VSS	LDO2	LDO4	VSS		GPIO_52	GPIO_35		GPIO_31	VCC18_QSPI			GPIO_23	GPIO_24	GPIO_21
D		ONKEYN	ALARM	VSS	VSS	VSS	VSS	GPIO_54	GPIO_51	GPIO_33	GPIO_34	USIM2_UIO	USIM2_UCLK				EXT_32K_IN	
E	VX1	VX1		VFB1		VSS	GPIO_70	GPIO_53	GPIO_28	GPIO_27		USIM_UCLK	USIM2_URSTN			VCXO_EN/SLPN	GPIO_29	GPIO_20
F	PVIN1	PGND	PGND	VAON	LDO5	GPIO_62		MN_CLK_REQ		GPIO_37		USIM_UIO	USIM_URSTN			VSS	GPIO_30	
G		GPIO_64	GPIO_63		GPIO_60/PWM	GPIO_61								PSRAM_VSS				GPIO_32
H	USB_N	USB_P	AVSS_USB2	GPIO_65	GPIO_66							PSRAM_VSS	PSRAM_VDD	PSRAM_VDD		VSS	GPIO_05	GPIO_04
J	GPIO_00	VCXO_REQ	VCXO_OUT	AVDD18_USB2		AVSS09_USB2				PMIC_INT/INTN							GPIO_06	
K		GPIO_03	GPIO_68/DVC2			AVDD08_USB2	VCC33_USIM	MPLL_TST_AD	QSPI_CS2	VSS	AVSS09_PSRAM		AVDD18_PSRAM		VQPS		GPIO_07	
L	GPIO_02	GPIO_01	GPIO_69	DVC_0/DVC1		AVDD33_USB2	VCC33_USIM2	VCC33_UART	VCC18_GPIO1	VSS	AVDD09_PSRAM	VCC_M1	PRI_TMS	PRI_TRST_N	PRI_TDI		GPIO_11	GPIO_10
M	AUXADC_IN1	AUXADC_IN2	GPIO_08	GPIO_09	GPIO_18			VCC18_USIM	VSS	VCC_M1	VCC_M1	VCC_M1	VCC18_PMIC2	PRI_TCK		GPIO_15	GPIO_73	GPIO_72
N		AUXADC_IN3	GPIO_19		GPIO_16	BG_OUT		VCC18_TD2	VSS	VSS	VCC_M1	VBUS_ON		PRI_TDO	GPIO_13	GPIO_12	GPIO_71	
P	AUD_EARP	AUD_EARN		AUXADC_IN4	AUXADC_IN5	GPIO_17			VSS					VCC18_PMIC1		GPIO_14	GPIO_74	GPIO_80
R	AUD_MICP	AUD_MICN	AUD_REFGND_DAC	AUD_AVSS18	AVSS_PLLM	AVDD18_PLLM						VSS	RESET_IN_N	GND_TCXO			GPIO_79	GPIO_78
T		AUD_AUREF10_PAD	AUD_AUREFGND	AUD_MICBIAS	AUD_AVDD18	VDD18_ADC	GND_ADC					VAPT		GND_TCXO	OCLK1	TCXO_IN	XTAL_IN	XTAL_OUT
U		MPLL_TST_CK	AVDD09_PLLM		VSSU_PLLM	GND_RX_BB	GND_RX_BB	VDD18_RX_BBLO	GND_PLLRX_LO	GND_PLLTX_LO	VDD18_PLLTX	GND_TXGUARD	GND_TX_ANA	GND_RF_DIG		VDDTCXO_OUT	VDD18_TCXOIN	CLK_SEL
V	RTN_AUXADCIN2	RTP_AUXADCIN1		VDD18_RX_LNA	GND_RX_LNA		GND_LNA_LMB_ISL							VDD18_TX_ANA	VDD18_RF_DIG		DCS_MODE	ALARM_WAKEUP
W	LNA1_MHB		LNA3_MHB		LNA5_MHB		GND_LNA_MHB_ISL		GND_PLLRX	GND_PLLTX		TXLB1	GND_TXL_MHB	GND_TXL_MHB				
Y	GND_LNA_LMB_ISL	LNA2_MHB		LNA4_MHB		LNA7_LMB	LNA6_MHB	LNA8_LMB				TXLB2	TXMHB2	TXMHB1			NC-Y17	NC-Y18

ASR CONFIDENTIAL

Table 9:ASR1606 Pin List

Ball #	Name	Type	Power Domain	Functional Description
A1	NC-A1			Not connected
A2	LDO8	P	3.3v	Output of PMIC LDO8
A4	LDO7	P	3.3v	Output of PMIC LDO7
A6	VINLDO	P	5v	INPUT of PMIC LDO1~8
A7	LDO1	P	3.3v	Output of PMIC LDO1
A9	GPIO_49	I/O,PU	1.8v	GPIO 49
A10	QSPI_DAT2	I/O,PD	1.8v	QSPI flash DAT 2
A12	QSPI_DAT1	I/O,PD	1.8v	QSPI flash DAT 1
A13	QSPI_CS1	I/O,PU	1.8v	QSPI flash Chip Select #1
A15	GPIO_121	I/O,PD	1.8v	GPIO 121
A17	GPIO_26	I/O,PU	1.8v	GPIO 26
A18	VSS	G		Digital Core ground
B1	VINREF	P	5v	Filter supply for reference
B2	RSTN	DO	1.8v	Output of system reset
B3	AGND	G		Analog and reference ground
B4	VDDIO	P	1.8v	1.8v Supply for I/O
B5	LDO6	P	3.3v	Output of PMIC LDO6
B6	VINLDO	P	5v	INPUT of PMIC LDO1~8
B7	LDO3	P	3.3v	Output of PMIC LDO3
B8	GPIO_122	I/O,PD	1.8v	GPIO 122
B9	GPIO_50	I/O,PU	1.8v	GPIO 50
B10	GPIO_36	I/O,PU	1.8v	GPIO 36
B11	QSPI_DAT3	I/O,PD	1.8v	QSPI flash DAT 3
B12	QSPI_DAT0	I/O,PD	1.8v	QSPI flash DAT 0
B13	QSPI_CLK	I/O,PD	1.8v	QSPI flash clock
B14	PWR_SCL/SCL	I/O,PU	1.8v	PMIC I2C SCL
B15	PWR_SDA/SDA	I/O,PU	1.8v	PMIC I2C SDA
B16	GPIO_124	I/O,PD	1.8v	GPIO 124
B17	GPIO_25	I/O,PU	1.8v	GPIO 25
B18	GPIO_22	I/O,PD	1.8v	GPIO 22
C1	VREF1V2	AO	1.2v	1.2v reference
C2	EXTON1N	DI	5v	EnableN signal
C3	AGND	G		Analog and reference ground
C4	VSS	G		Digital Core ground
C5	LDO2	P	3.3v	Output of PMIC LDO2
C6	LDO4	P	3.3v	Output of PMIC LDO4
C7	VSS	G		Digital Core ground
C9	GPIO_52	I/O,PU	1.8v/3.3v	GPIO 52

ASR CONFIDENTIAL

C10	GPIO_35	I/O,PU	1.8v	GPIO 35
C12	GPIO_31	I/O,PU	1.8v/3.3v	GPIO 31
C13	VCC18_QSPI	P		QSPI 1.8v power supply
C16	GPIO_23	I/O,PU	1.8v	GPIO 23
C17	GPIO_24	I/O,PD	1.8v	GPIO 24
C18	GPIO_21	I/O,PD	1.8v	GPIO 21
D2	ONKEYN	DI	5v	External onkey
D3	ALARM	DI	5v	RTC alarm input
D4	VSS	G		Digital Core ground
D5	VSS	G		Digital Core ground
D6	VSS	G		Digital Core ground
D7	VSS	G		Digital Core ground
D8	GPIO_54	I/O,PU	1.8v/3.3v	GPIO 54
D9	GPIO_51	I/O,PU	1.8v/3.3v	GPIO 51
D10	GPIO_33	I/O,PD	1.8v	GPIO 33
D11	GPIO_34	I/O,PD	1.8v	GPIO 34
D12	USIM2_UIO	I/O,PU	1.8v/3.3v	USIM #2 UIO
D13	USIM2_UCLK	I/O,PU	1.8v/3.3v	USIM #2 UCLK
D17	EXT_32K_IN	I	1.8v	32K clock input
E1	VX1	P	5v	Switching output of buck1
E2	VX1	P	5v	Switching output of buck1
E4	VFB1	AIO	3.75v	Buck1 feedback pin
E6	VSS	G		Digital Core ground
E7	GPIO_70	I/O,PD	1.8v	GPIO 70
E8	GPIO_53	I/O,PU	1.8v/3.3v	GPIO 53
E9	GPIO_28	I/O,PD	1.8v	GPIO 28
E10	GPIO_27	I/O,PD	1.8v/3.3v	GPIO 27
E12	USIM_UCLK	I/O,PU	1.8v/3.3v	USIM #1 UCLK
E13	USIM2_URSTN	I/O,PU	1.8v/3.3v	USIM #2 RSTN
E16	VCXO_EN/SLPN	O	1.8v	VCXO enable
E17	GPIO_29	I/O,PU	1.8v	GPIO 29
E18	GPIO_20	I/O,PD	1.8v	GPIO 20
F1	PVIN1	P	5v	Supply input for buck1
F2	PGND	P		Power ground for buck1
F3	PGND	P		Power ground for buck1
F4	VAON	P	5v	VRTC regulator output
F5	LDO5	P	3.3v	Output of PMIC LDO5
F6	GPIO_62	I/O,PD	1.8v	GPIO 62
F8	MN_CLK_REQ	I/O, PD	1.8v	GPIO123
F10	GPIO_37	I/O,PD	1.8v	GPIO 37
F12	USIM_UIO	I/O,PU	1.8v/3.3v	USIM #1 UIO

ASR CONFIDENTIAL

F13	USIM_URSTN	I/O,PU	1.8v/3.3v	USIM #1 RSTN
F16	VSS	G		Digital Core ground
F17	GPIO_30	I/O,PU	1.8v	GPIO 30
G2	GPIO_64	I/O,PD	1.8v	GPIO 64
G3	GPIO_63	I/O,PD	1.8v	GPIO 63
G5	GPIO_60/FPWM	I/O,PD	1.8v	GPIO 60
G6	GPIO_61	I/O,PD	1.8v	GPIO 61
G14	PSRAM_VSS	G		PSRAM Device ground
G18	GPIO_32	I/O,PU	1.8v	GPIO 32
H1	USB_N	AIO	1.8v	USB2 data N
H2	USB_P	AIO	1.8v	USB2 data P
H3	AVSS_USB2	G		USB ground
H4	GPIO_65	I/O,PD	1.8v	GPIO 65
H5	GPIO_66	I/O,PD	1.8v	GPIO 66
H12	PSRAM_VSS	G		PSRAM Device ground
H13	PSRAM_VDD	P	1.8v	PSRAM Device power
H14	PSRAM_VDD	P	1.8v	PSRAM Device power
H16	VSS	G		Digital Core ground
H17	GPIO_05	I/O,PD	1.8v	GPIO 05
H18	GPIO_04	I/O,PD	1.8v	GPIO 04
J1	GPIO_00	I/O,PD	1.8v	GPIO 00
J2	VCXO_REQ	I/O,PD	1.8v	OCLK1 request
J3	VCXO_OUT	I/O,PD	1.8v	VCXO output
J4	AVDD18_USB2	P	1.8v	USB 1.8v power
J6	AVSS09_USB2	G		USB PHY digital ground
J10	PMIC_INT/INTN	I	1.8v	PMIC interrupt (active low)
J17	GPIO_06	I/O,PD	1.8v	GPIO 06
K2	GPIO_03	I/O,PD	1.8v	GPIO 03
K3	GPIO_68/DVC2	I/O,PD	1.8v	GPIO 68
K6	AVDD08_USB2	P	0.9v	USB PHY digital power
K7	VCC33_USIM	P	3.3v/1.8v	UIM 3.3v/1.8v power supply
K8	MPLL_TST_AD	AIO	1.8v	Analog Test Point
K9	QSPI_CS2	I/O, PD	1.8v	QSPI flash chip select #2
K10	VSS	G		Digital Core ground
K11	AVSS09_PSRAM	G		PSRAM PHY Digital ground
K13	AVDD18_PSRAM	P	1.8v	PSRAM PHY analog power
K15	VQPS	P	1.8v	1.8v fuse programming power
K17	GPIO_07	I/O,PU	1.8v	GPIO 07
L1	GPIO_02	I/O,PD	1.8v	GPIO 02
L2	GPIO_01	I/O,PD	1.8v	GPIO 01
L3	GPIO_69	I/O,PD	1.8v	GPIO 69

ASR CONFIDENTIAL

L4	DVC_0/DVC1	DI	1.8v	Digital DVC control for buck
L6	AVDD33_USB2	P	3.3v	USB 3.3v power
L7	VCC33_USIM2	P	3.3v/1.8v	Usim2 3.3v/1.8v power supply
L8	VCC33_UART	P	3.3v/1.8v	CP_UART 3.3v/1.8v power supply
L9	VCC18_GPIO1	P	1.8v	GPIO1 I/O domain power
L10	VSS	G		Digital Core ground
L11	AVDD09_PSRAM	P	0.9v	PSRAM PHY digital power
L12	VCC_M1	P	0.9v	Digital Core power
L13	PRI_TMS	I/O, PU	1.8v	Primary JTAG TMS
L14	PRI_TRST_N	I	1.8v	Primary JTAG port reset_n
L15	PRI_TDI	I/O, PU	1.8v	Primary JTAG TDI input
L17	GPIO_11	I/O,PU	1.8v	GPIO 11
L18	GPIO_10	I/O,PU	1.8v	GPIO 10
M1	AUXADC_IN1	AIO		AUXADC channel1 input/IPC output current 0
M2	AUXADC_IN2	AIO		AUXADC channel2 input/IPC output current 1
M3	GPIO_08	I/O,PD	1.8v	GPIO 08
M4	GPIO_09	I/O,PU	1.8v	GPIO 09
M5	GPIO_18	I/O,PD	1.8v	GPIO 18
M8	VCC18_USIM	P	1.8v	Connect to 1.8v power
M9	VSS	G		Digital Core ground
M10	VCC_M1	P	0.9v	Digital Core power
M11	VCC_M1	P	0.9v	Digital Core power
M12	VCC_M1	P	0.9v	Digital Core power
M13	VCC18_PMIC2	P	1.8v	PMIC2 I/O domain power
M14	PRI_TCK	I/O, PD	1.8v	Primary JTAG TCK
M16	GPIO_15	I/O,PD	1.8v	GPIO 15
M17	GPIO_73	I/O,PU	1.8v	GPIO 73
M18	GPIO_72	I/O,PU	1.8v	GPIO 72
N2	AUXADC_IN3	AIO		AUXADC channel3 input/IPC output current 2
N3	GPIO_19	I/O,PD	1.8v	GPIO 19
N5	GPIO_16	I/O,PD	1.8v	GPIO 16
N6	BG_OUT	AO	0.6v	Analog bandgap voltage output
N8	VCC18_TD2	P	1.8v	TD2 I/O domain power
N9	VSS	G		Digital Core ground
N10	VSS	G		Digital Core ground
N11	VCC_M1	P	0.9v	Digital Core power
N12	VBUS_ON	I	1.8v	VBUS detection
N14	PRI_TDO	I/O, PU	1.8v	Primary JTAG TDO output

ASR CONFIDENTIAL

N15	GPIO_13	I/O,PD	1.8v	GPIO 13
N16	GPIO_12	I/O,PD	1.8v	GPIO 12
N17	GPIO_71	I/O,PU	1.8v	GPIO 71
P1	AUD_EARP	AO	1.8v	Receiver Positive output
P2	AUD_EARN	AO	1.8v	Receiver Negative output
P4	AUXADC_IN4	AIO		AUXADC channel4 input
P5	AUXADC_IN5	AIO		AUXADC channel5 input
P6	GPIO_17	I/O,PD	1.8v	GPIO 17
P9	VSS	G		Digital Core ground
P14	VCC18_PMIC1	P	1.8v	PMIC1 I/O domain power
P16	GPIO_14	I/O,PD	1.8v	GPIO 14
P17	GPIO_74	I/O,PU	1.8v	GPIO 74
P18	GPIO_80	I/O,PD	1.8v	GPIO 80
R1	AUD_MICP	AI	1.8v	Audio MIC Positive input
R2	AUD_MICN	AI	1.8v	Audio MIC Negative input
R3	AUD_REFGND_DAC	G		Audio DAC reference ground
R4	AUD_AVSS18	G		Audio ground
R5	AVSS_PLLM	G		PLL analog ground
R6	AVDD18_PLLM	P	1.8v	PLL power
R12	VSS	G		Digital Core ground
R13	RESET_IN_N	I	1.8v	SoC reset input
R14	GND_TCXO	G		TCXO ground
R17	GPIO_79	I/O,PD	1.8v	GPIO 79
R18	GPIO_78	I/O,PD	1.8v	GPIO 78
T2	AUD_AUREF10_PAD	AIO	1.8v	Audio reference
T3	AUD_AUREFGND	G		Audio reference ground
T4	AUD_MICBIAS	AIO	1.8v	Audio MICBIAS
T5	AUD_AVDD18	P	1.8v	Audio power
T6	VDD18_ADC	P	1.8v	AUXADC power
T7	GND_ADC	G		AUXADC ground
T12	VAPT	AIO	1.8v	APT voltage for RF PA
T14	GND_TCXO	G		TCXO ground
T15	OCLK1	AIO	1.8v	Reference clock buffer output
T16	TCXO_IN	AIO	1.8v	TCXO crystal input
T17	XTAL_IN	AIO	1.8v	DCXO crystal input
T18	XTAL_OUT	AIO	1.8v	DCXO crystal output
U2	MPLL_TST_CK	AIO	1.8v	PLL test point
U3	AVDD09_PLLM	P	0.9v	PLL digital power
U5	VSSU_PLLM	G		PLL digital ground
U6	GND_RX_BB	G		RF RX digital ground
U7	GND_RX_BB	G		RF RX digital ground
U8	VDD18_RX_BBLO	P	1.8v	RF RX power supply

ASR CONFIDENTIAL

U9	GND_PLLRX_LO	G		RF RX PLL ground
U10	GND_PLLTX_LO	G		RF TX PLL ground
U11	VDD18_PLLTX	P	1.8v	RF power supply
U12	GND_TXGUARD	G		RF TX ground
U13	GND_TX_ANA	G		RF TX analog ground
U14	GND_RF_DIG	G		RF digital ground
U16	VDDTCXO_OUT	P	1.8v	TCXO buffer power
U17	VDD18_TCXOIN	P	1.8v	TCXO input power
U18	CLK_SEL	AIO	1.8v	CLK select for VCXO source
V1	RTN_AUXADCIN2	AIO	1.8v	AUX ADC input 2
V2	RTP_AUXADCIN1	AIO	1.8v	AUX ADC input 1
V4	VDD18_RX_LNA	P	1.8v	RF RX LNA power
V5	GND_RX_LNA	G		RF RX LNA ground
V7	GND_LNA_LMB_ISL	G		RF RX LNA ground
V14	VDD18_TX_ANA	P	1.8v	RF TX power supply
V15	VDD18_RF_DIG	P	1.8v	RF digital power supply
V17	DCS_MODE	I	1.8v	Control single crystal mode or dual crystal mode
V18	ALARM_WAKEUP	O	1.8v	Alarm wakeup output to PMIC in single crystal mode
W1	LNA1_MHB	AIO	1.8v	RF MHB LNA1
W3	LNA3_MHB	AIO	1.8v	RF MHB LNA3
W5	LNA5_MHB	AIO	1.8v	RF MHB LNA5
W7	GND_LNA_MHB_ISL	G		RF RX LNA ground
W9	GND_PLLRX	G		RF RX PLL ground
W10	GND_PLLTX	G		RF TX PLL ground
W12	TXLB1	AIO	1.8v	RF TX Low band 1
W13	GND_TXLMHB	G		RF TX ground
W14	GND_TXLMHB	G		RF TX ground
Y1	GND_LNA_LMB_ISL	G		RF RX LNA ground
Y2	LNA2_MHB	AIO	1.8v	RF MHB LNA2
Y4	LNA4_MHB	AIO	1.8v	RF MHB LNA4
Y6	LNA7_LMB	AIO	1.8v	RF LMB LNA7
Y7	LNA6_MHB	AIO	1.8v	RF MHB LNA6
Y8	LNA8_LMB	AIO	1.8v	RF LMB LNA8
Y12	TXLB2	AIO	1.8v	RF TX Low band 2
Y13	TXMHB2	AIO	1.8v	RF TX MHB 2
Y14	TXMHB1	AIO	1.8v	RF TX MHB 1
Y17	NC-Y17			Not connected
Y18	NC-Y18			Not connected

ASR CONFIDENTIAL

Table 10 IO parameter

For 1.8V IO					
Power Domain	**Symbol**	**Description**	**Min**	**Typ**	**Max**
1.8V Input	Vih	High Level input	VCC*0.7	1.8V	VCC+0.2
	Vil	Low Level input	-0.3	0V	0.3*VCC
	Rpu	Pull up resister	55K	79K	121K
	Rpd	Pull down resister	51K	87K	169K
	Iil	Input leakage current Pad in input mode			10uA
1.8V Ouput	Voh	High Level output	VCC-0.2V		
	Vol	Low Level output			0.2V
	Iol DCS[1:0]= 00 01 10 11	Low level output current when Vpad=0.2V	13mA 25mA 37mA 49mA		
	Ioh DCS[1:0]= 00 01 10 11	High level output current when Vpad=VCC-0.2V	11mA 21mA 32mA 42mA		
For 3.3V/1.8V IO					
Power Domain	**Symbol**	**Description**	**Min**	**Typ**	**Max**
3.3V Input	Vih	High Level input	2		VCC+0.3
	Vil	Low Level input	-0.3	0V	0.8
	Rpu	Pull up resister	26K	47K	72K
	Rpd	Pull down resister	27K	54K	267K
	Iil	Input leakage current			10uA
3.3V Ouput	Voh	High Level output	2.4V		
	Vol	Low Level output			0.4V

ASR CONFIDENTIAL

	Iol DS[2:0]= 000 001 010 011 100 101 110 111	Low level output current when Vpad=0.4V	 7mA 10mA 14mA 18mA 21mA 24mA 28mA 31mA		
	Ioh DS[2:0]= 000 001 010 011 100 101 110 111	High level output current when Vpad=VCC-0.5V	 7mA 10mA 13mA 16mA 19mA 23mA 26mA 29mA		

3.3 *Multiplexed Signal Functions*

This section provides a detailed description of the function 0 through 7 signals assigned to the I/O pins in the ASR1606 processor. The functions are arranged in groups according to their interface. Most ASR1606 I/O pins are multi-function pins so that they can be configured for one of several available functions using the MFPRs. Some functions can be configured to be present on several different pins.

Table 10 contains detailed information about the ASR1606 signals assigned to the I/O pins.

Note

Table 12 describes multiplexed functions only. All functions that are assigned to a pin as its primary function are described in Section 3.1, Function Assignments for Multiplexed I/O Pins.

The signals in this section are organized by their function (power supplies, and so on) and are listed in alphabetical order by interface first and then by signal name.

ASR CONFIDENTIAL

Table 11:Signal Types

Pin Type	Description
A, I	Analog input
A, I/O	Analog input/output
A,O	Analog output
I/O	Digital input/output
n	Active low

Table 12:Multiplexed Signal Description

JTAG, Primary		
Signal		**Description**
Name	**Type**	
CLK_REQ	I	Request to send out the M/N clock of Main PMU
PRI_TCK	I	**Primary JTAG Interface 1 Test Clock** For all transfers on the JTAG test interface.
PRI_TDI	I	**Primary JTAG Interface 1 Test Data Input** Used to send data from the JTAG controller to the ASR1606 processor. This pin has an internal pullup resistor.
PRI_TDO	O	**Primary JTAG Interface 1 Test Data Output** Used to return data from the ASR1606 processor to the JTAG controller.
PRI_TMS	I	**Primary JTAG Interface 1 Test Mode Select** Selects the test mode required from the JTAG controller. This pin has an internal pullup resistor.
PRI_TRSTn	I	**Primary JTAG Interface 1 Test Reset** IEEE 1194.1 test reset
SLAVE_RESET_OUT	O	Reset from chip to reset external IC
VCXO_OUT	O	26 MHz VCXO output clock
VCXO_REQ	I	REquest to send out VCXO
JTAG, Secondary for ARM Cortex-R5/CEVA-BX2 DSP		
SEC2_TCK	I	**Secondary JTAG Interface 2 Test Clock** For all transfers on the JTAG test interface.
SEC2_TDI	I	**Secondary JTAG Interface 2 Test Data Input** Used to send data from the JTAG controller to the ASR1606 processor. This pin has an internal pullup resistor.
SEC2_TDO	O	**Secondary JTAG Interface 2 Test Data Output** Used to return data from the ASR1606 processor to the JTAG controller.
SEC2_TMS	I	**Secondary JTAG Interface 2 Test Mode Select** Selects the test mode required from the JTAG controller. This pin has an internal pullup resistor.
SEC2_TRSTn	I	**Secondary JTAG Interface 2 Test Reset** IEEE 1194.1 test reset
Keypad Controller		
KP_DKIN[3:0]	I	Keypad Direct Key Inputs [3:0]
KP_MKIN[7:0]	I	Keypad Matrix Key Inputs [7:0]
KP_MKOUT[7:0]	O	Keypad Matrix Key Outputs [7:0]
Miscellaneous		
ANALOG_TP		PLL test pin

ASR CONFIDENTIAL

EXT_32K_IN	I	32 kHz clock
MN_CLK_OUT	O	**Fractional (M/N) Divided Clock** Main PMU general purpose M/N fractional clock divider clock output. CLK_REQ must be set as Function 0 and pulled high for the 13 MHz clock to be output on GPIO[124] (MN_CLK_OUT).
PMIC_INTn	O	PMIC interrupt
RESET_INn	I	Function reset signal
Signal		**Description**
Name	**Type**	
SLEEP_IND	I	PMIC sleep setting
VCXO_EN	O	VCTCXO enable
SSP, where x signifies SSP 0, SSP 1, or SSP 2		
SSPx_FRM	I/O	**Synchronous Serial Port Frame 0/1/2** The serial frame sync can be configured as an output (master mode operation) or an input (slave mode operation).
SSPx_RXD	I	Synchronous Serial Port Receive Data **0/1/2** Serial data latched using the bit clock
SSPx_SCLK	I/O	Synchronous Serial Port Clock **0/1/2** The serial bit clock can be configured as an output (master mode operation) or an input (slave mode operation).
SSPx_TXD	O	Synchronous Serial Port Transmit Data **0/1/2** Serial data driven out synchronously with the bit clock
I2S_SCLK	O	I2S Audio Clock
TWSI		
PWR_SDA	I/O	TWSI serial data/address signal
PWR_SCL	I/O	TWSI serial clock line signal
TWSI, Common		
CI2C_SCL	I/O, OD	TWSI Clock
CI2C_SDA	I/O, OD	TWSI Data
UARTx, where x signifies UART 1		
UART1_CTSn	I	UART 1 Clear-to-Send
UART1_RTSn	O	UART 1 Request-to-Send
UART1_RXD	I	UART 1 Receive Data
UART1_TXD	O	UART 1 Transmit Data
USB		
USB_N	I/O	USB D±
USB_P	I/O	

ASR CONFIDENTIAL

Signal		Description
Name	**Type**	
VBUS_ON	I	USB VBUS present indicator
USIM, where x signifies USIM1 or 2		
USIMx_UCLK	O	USIM clock
USIMx_UIO	I/O, OD	USIM data
USIMx_URSTn	O	USIM reset
Unconnected		
NC	NC	No Connect Pin is floating and is not connected internally to any active circuitry and does not have any electrical continuity to any other pin

3.4 Pad-Ring I/O Domains Power Control

There are three power domain types in the ASR1606 device:

- Fixed domains—These are domains that have no configurable options
- Software-configurable power domains—These are domains that can work at more than one voltage level and are configured by software
- Hardware and software-configurable power domains—These are domains that can work at more than one voltage level and are configured either by hardware or software

The configuration for power domains control is done through dedicated control registers, available per configurable power domain as described in Table 13 and in the *ASR1606 Registers*.

3.4.1 Fixed Power Domains

Fixed power domains are functional in a fixed configuration. No software or hardware configuration is available for these domains.

Registers associated with the fixed power domains should not be used, but are reserved for future use.

3.4.2 Software Configurable Power Domains

This group of power domains can work at either 3.0V or 1.8V.

After power up, these domains are configured by default to operate at 3.3V. Software is required to configure each domain according to the usage model for the specific design.

Caution

Use extreme caution when configuring the domains through software!

Incorrect configuration could cause irreversible damage to the ASR1606, which may cause it to become unusable (for example, by configuring a 3.3V domain to 1.8V).

ASR CONFIDENTIAL

Note

The power domain voltage level is design specific and should be configured only once at power up. Once configured, the voltage level cannot be changed again.

ASR CONFIDENTIAL

3.4.3 Hardware and Software-Configurable Power Domains

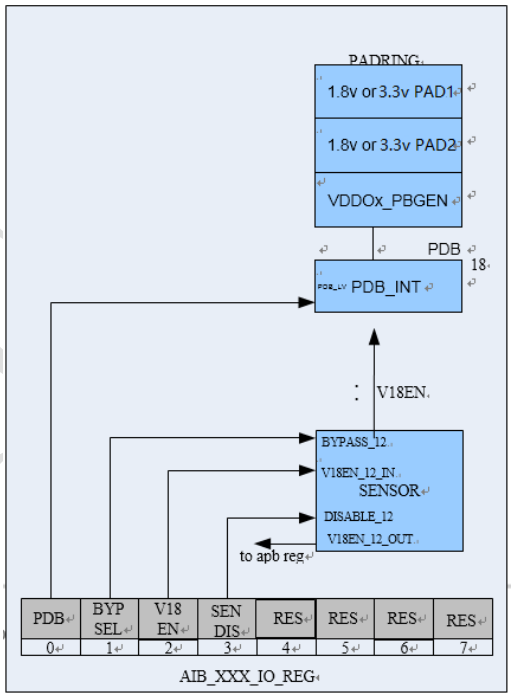
In the ASR1606, this group of power domains actually includes a single domain, which is used for the DFI. This domain is capable of operating at either 3.3V or 1.8V.

Hardware configuration is required for the DFI since it is used as the boot address of the processor. In such cases, the boot code must be executed prior to the power domains software configuration; hence, software cannot be used to configure this domain.

This operational voltage of the domain is configured in hardware by default, using an internal sense circuitry that senses the voltage applied to the domain power pins.

Figure 7 shows a high-level block diagram for a hardware and software-configurable power domain, showing the relation between the configuration register and the domain pads. Other domains are generated derivative of this type; hence the same diagram applies to domains that can be configured by software only, excluding the hardware sensor circuitry.

Figure 7:Hardware or Software-Configurable Power Domain Control Block Diagram



ASR CONFIDENTIAL

3.4.4 Power Supply Pin List

Table 13 lists detailed information about the available power supplies.

Table 13:Power Supplies Descriptions

Pin Name	Domain Name	Domain Voltage	Voltage Control	Control Register	Signal Description
AUD_AVDD18	AUDIO	1.8V	N/A - Fixed domain	N/A	Positive supply for AUDIO: connect to an external 1.8V power supply.
AUD_AVSS18	AUDIO	0V	N/A - Fixed domain	N/A	Ground reference for the AUD_AVDD18 supply
AVDD08_USB2	USB	0.9V	N/A - Fixed domain	N/A	Positive supply for USB PHY digital power: connect to an external 0.9V power supply.
AVDD09_PLLM	Analog	0.9V	N/A - Fixed domain	N/A	Positive supply for analog module digital power: connect to an external 0.9V power supply.
AVDD09_PSRAM	Psram	0.9V	N/A - Fixed domain	N/A	Positive supply for PSRAM PHY digital power: connect to an external 0.9V power supply.
AVDD18_PLLM	Analog	1.8V	N/A - Fixed domain	N/A	Positive supply for analog module analog power: connect to an external 1.8V power supply.
AVDD18_PSRAM	Psram	1.8V	N/A - Fixed domain	N/A	Positive supply for PSRAM PHY analog power: connect to an external 1.8V power supply.
AVDD18_USB2	USB	1.8V	N/A - Fixed domain	N/A	Positive supply for USB transceiver: connect to an external 1.8V power supply.
AVDD33_USB2	Usb	3.3V	N/A - Fixed domain	N/A	Positive supply for USB transceiver: connect to an external 3.3V power supply.
AVSS_PLLM	analog	0V	N/A - Fixed domain	N/A	analog ground reference for AVDD18_PLLM
AVSS_USB2	USB	0V	N/A - Fixed domain	N/A	ground reference for AVDD18_USB2

ASR CONFIDENTIAL

AVSS09_PSRAM	psram	0V	N/A - Fixed domain	N/A	Ground reference for AVDD09_PSRAM
AVSS09_USB2	USB	0V	N/A - Fixed domain	N/A	Ground reference for AVDD08_USB2
VCC_M1	MAIN	0.8-1.0	By PMIC	N/A	Positive supply for internal logic: connect to a low-voltage, adjustable (0.8V - 1.0V nominal) system power supply.This supply feeds most of the digital domains and all the internal SRAM domains of ASR1606 device.
VCC18_GPIO1	VCC18_GPIO1	GPIO1	1.8V	N/A -Fixed domain	N/A
VCC18_PMIC1	PMIC1	1.8V	N/A - Fixed domain	N/A	Positive supply PMIC domain: connect to an external 1.8V power supply.
VCC18_PMIC2	PMIC2	1.8V	N/A - Fixed domain	N/A	Positive supply PMIC domain: connect to an external 1.8V power supply.
VCC18_TD2	GPIO2	1.8V	SW	N/A	Positive supply GPIO2 domain: connect to an external 1.8V power supply.
VCC18_USIM	USIM	1.8V	SW	N/A	connect to an external 1.8v power supply
VCC33_UART	UART	3.3v/1.8v	SW	N/A	Positive supply UART domain: connect to an external 1.8/3.0V power supply.
VCC33_USIM	USIM	1.8/3.3V	SW	AIB_USIM_IO_REG	Positive supply USIM domain: connect to an external 1.8/3.0V power supply.
VCC33_USIM2	USIM2	1.8/3.3V	SW	AIB_USIM2_IO_REG	Positive supply USIM2 domain: connect to an external 1.8/3.0V power supply.
VQPS	Analog	1.8V	N/A - Fixed domain	N/A	1.8V efuse power supply:This generates the power for efuse.

ASR CONFIDENTIAL

VSS	MAIN	0v	By PMIC	N/A	Ground reference for the VCC_MAIN supply
VSSU_PLLM	analog	0v	N/A - Fixed domain	N/A	Digital ground reference for AVDD09_PLLM

ASR CONFIDENTIAL

Copyright © 2022 ASR

3.5 Pin Control Unit

3.5.1 Features

This section lists the general features of the Pin Control Unit (PCU):

- Controls the state of pins during reset and low-power modes
- Supports holding the last driven or register-defined state for all outputs during reset and low-power modes
- Manages selection of GPIO and other alternate peripheral functions
- Supports software configuration of output drive strength
- Supports external interrupt generation and wakeup-event detection

3.5.2 Overview

There are 3 categories of pins. Only multi-function pins—and none-power and dedicated I/O pins—are controlled by the PCU.

The basic operation of each multi-function pin is controlled by the MFPRs. Each register controls a single multi-function pin. To program multiple pins (for example, to select all the pins for a multi-pin function, such as a UART), several registers must be programmed in sequence. Register operations may be completed in a slightly different order than the order of the requests; therefore, completion must be verified by performing a read operation.



Note Software must not assume that pins are programmed in exactly the same order as the requests were issued.

ASR CONFIDENTIAL

Each MFPR is assumed to be 32-bits wide and access must be word-aligned.

Each multi-function pin has a physical address in the overall processor address map. The MFPR addresses are provided in Figure 11. To allow easy additions/updates of functions to a multi-function pin, ASR recommends that software keep the configuration of each multi-function pin isolated and easy to re-define.

A single multi-function pin can have up to 8 alternate functions (shown in Figure 11. as functions 0 to 7).

A multi-function pin always has an input path, but that value can be ignored, so as long as power is applied to the multi-function pin, it must have a firm logic value present on the board. This output can be sourced from an external device or driven by the device itself.

A multi-function pin can be programmed to take 1 of these output values:

- Resistive pull up (nominal 50 KOhm)
- Resistive pull down (nominal 50 KOhm)
- Driven high
- Driven low
- High-impedance

As each pin can be selected dynamically, this allows functions such as drive 0 and pull high.

3.5.3 Multi-function Pin Details

The multi-function pin has a relatively simple basic function. A number of logical functions are connected by a multiplexer to a single pin, and 1 is selected by the <AF_SEL> field in the MFPRs. Each logical function consists of some or all of the following:

- Data-out signal
- Output-enable signal
- Input signal
- Pull-up control
- Pull-down control

An alternate function can be an output, input, or bi-directional function.

Any function that is not selected is logically disconnected from the pins for both input and output. In that case, the output signals are ignored, and the input signals are forced to a default state as they are fed to the internal device.

In all cases, the direction of a pin is controlled by hardware. The configuration of the alternate function connects the pin to its driving logic and that logic is responsible for determining whether the pin is an input or output. In the case of a GPIO function, the direction is controlled by the GPIO function using the

ASR CONFIDENTIAL

direction register of the GPIO controller.

Multiple pins can be allocated to the same function, the output data and enable signals being duplicated and the input signals being ORed together, or NANDed (inverted AND) if active low, to form a single function. Software must avoid the case in which multiple pins are assigned to the same function, or, if this is desired, manage any complications.

Figure 7 and Figure 8 are simplified illustrations of the input and output paths of a single multi-function I/O pin.

Refer to Figure 7 for illustrations of the following details:

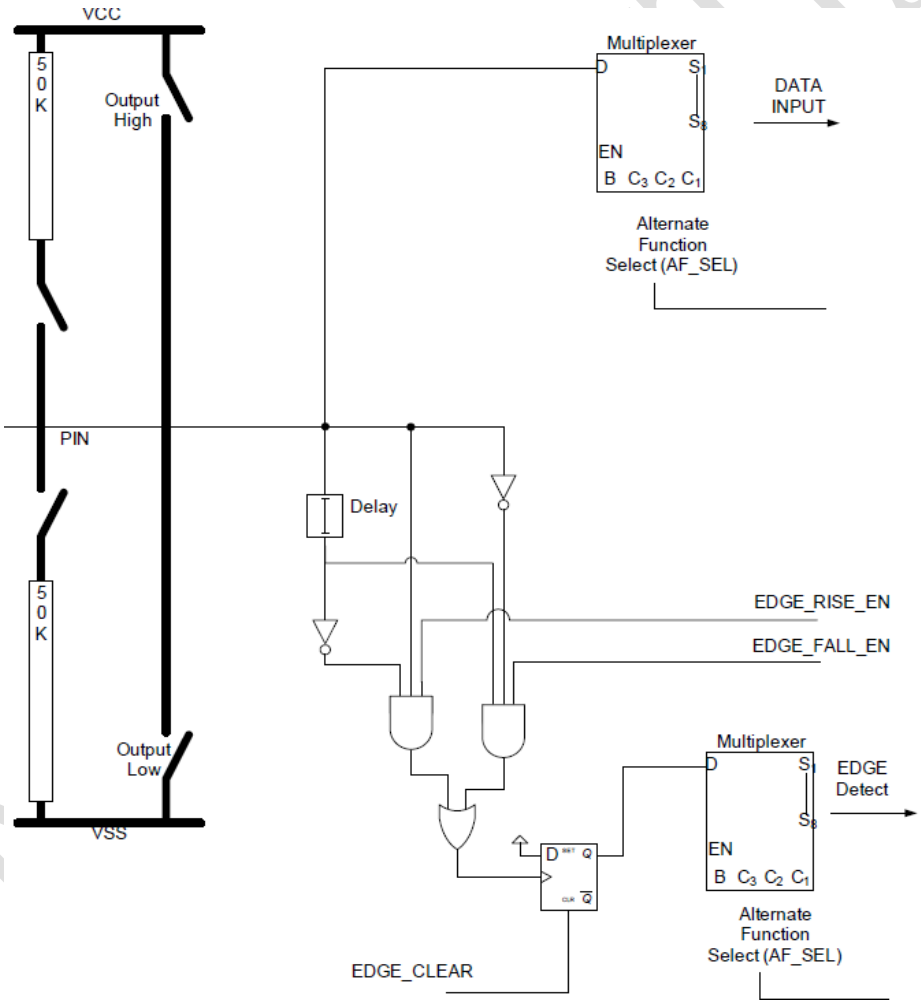
The data signal coming from the I/O pin is connected to a multiplexer and then distributed to one out of several internal units that is selected by the <AF_SEL> field. The data coming from the I/O pin is sometimes (not for all multi-function I/O pins) connected to a wakeup-detection logic used to wake up the Application Subsystem, Communication Subsystem, or both.

ASR CONFIDENTIAL

The data signal coming from the I/O pin is connected to a multiplexer and then distributed to one out of several internal units that is selected by the <AF_SEL> field. The data coming from the I/O pin is sometimes (not for all multi-function I/O pins) connected to a wakeup-detection logic used to wake up the Application Subsystem, Communication Subsystem, or both.

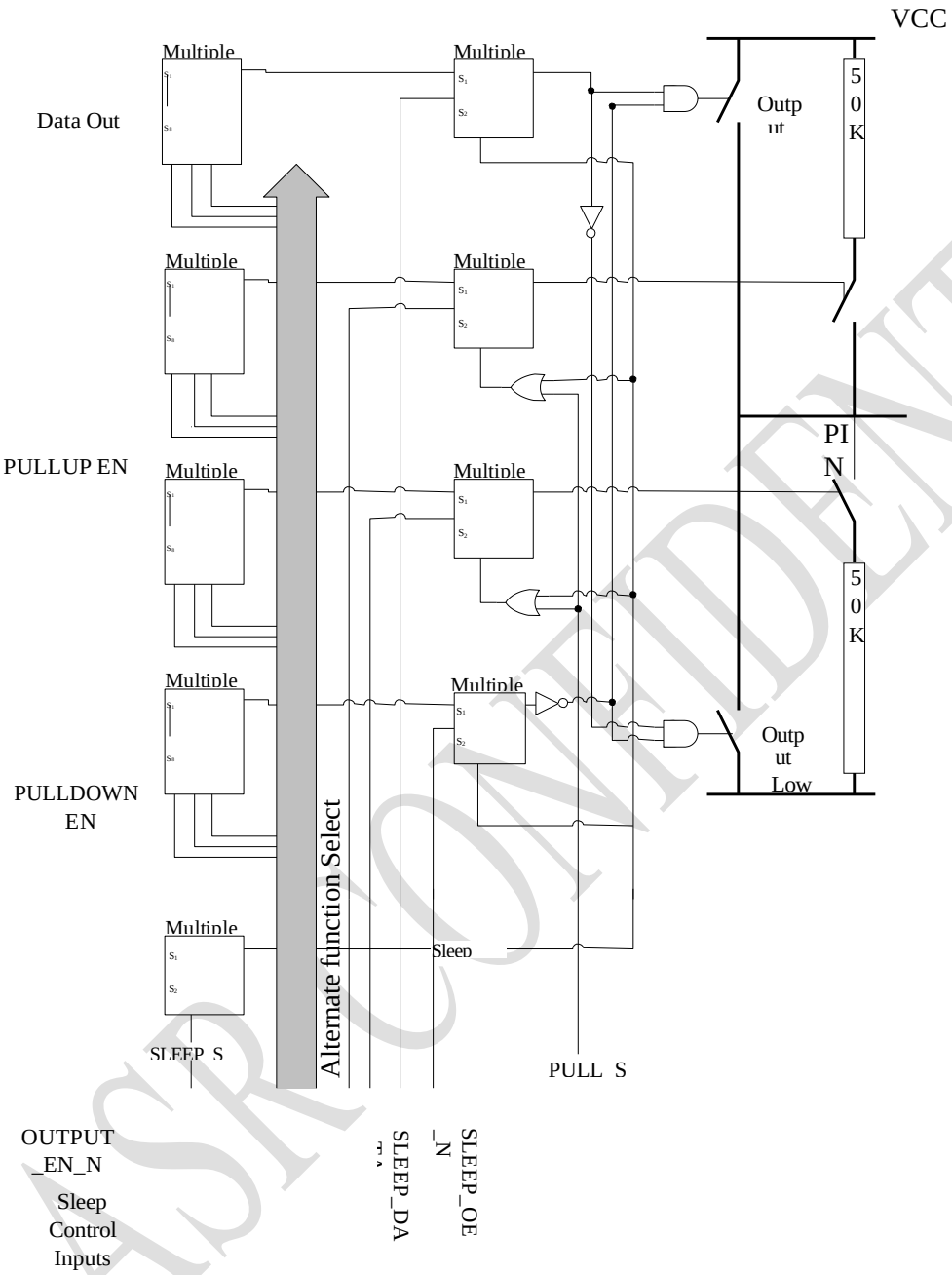
Each multi-function, I/O pin has edge detection.

Figure 8:Multi-Function Pin - Input Path



ASR CONFIDENTIAL

Figure 9:Multi-function Pin - Output Path



ASR CONFIDENTIAL

3.5.3.1 Examples

For example, if a single pin is defined to have 2 alternate functions:

- SSP data-out signal
- UART data input

The pin functions are defined in Table 14.

Table 14:Pin Function Example

Description	SSP Function Definitions	UART Function Definitions
Capability	Capable of driving 0, 1, or three-state	Capable of defining a three-state with no data input
Data Input or Output?	No data input	No data output
Function	0	1
	Data-out and the output-enable connections connected to the SSP, but the data input connections are not used.	Output-enable connection set to a 0 (forcing the output to be High-Z) and the data output unconnected; The data input is connected to the UART

- When Function 0 is selected, the output pin follows the SSP signals. The data enable output from the UART is ignored, and the data input is forced to be a 0 (the standard default case).
- When Function 1 is selected, the SSP outputs are ignored, the output is High-Z, and the input is connected to the UART.

This configuration presents an issue of potentially floating inputs when a device does not always drive the input.

Software should not use more than 1 option at a time, especially when the pin is input. For example, GPIO[23] exists on 2 separate pins (MMC1_CD and GPIO[23]), and UART2_TXD, UART2_RXD, UART2_CTSn, and UART2_RTSn are also available on several different pins.

3.5.4 Checking for Completion of a Multi-function Pin Operation

A significant amount of operations can be queued between software and the actual write of the MFPR. It is important that the software be able to find out when the last operation completed and took effect. This is done by performing a read operation to the address of the last performed operation. No further operation can be performed to the PCU until this read response has returned, which can take a significant amount of time (ns).

3.5.5 Register Access for MFPR Functionality

Refer to ASR1606 Registers for details on the power domain control registers and the MFPR registers.

The MFPRs are read/write registers and each register can be read or written

ASR CONFIDENTIAL

to by the same address.
Perform reads carefully since read operation access time is high due to not being pipelined.

3.5.5.1 Access to Non-existent Registers or Pins

Access to non-existent registers or pins result in the following:

- Write access to a register within the MFPR region that does not exist–Not aborted and simply results in no effect

ASR CONFIDENTIAL

- Read access to a register within the MFPR region that does not exist– Proceeds identically to read accesses to existing registers but returned data is unpredictable

Note

With normal register accesses, the replying register confirms the access.

With register accesses to non-existent registers, to ensure the register access actually completes, hardware provides a time-out period. The time-out period is a non-programmable timer that is large enough to ensure that any possible register would have confirmed within the given time. From the software viewpoint, this timeout operation is transparent.

3.5.5.2 Pin Control Unit Address Map

The address region for the MFPRs is 0xD401E0B4 ~ 0xD401E32C

Figure 10 shows the address offset allocation per multi-function pin.

Note

Using the MFPRs, select the function to be assigned to the multi-function pin before enabling or configuring the internal peripheral that is associated with that function.

Failure to perform this step in this order can lead to switching-related transitions that can be interpreted by the peripheral as valid data or strobe.

3.5.5.3 Register Structure

As explained in Section 3.3, Multiplexed Signal Functions, basic operation of each multi-function pin is controlled by MFPRs. Each register controls a single multi-function pin.

Each register is assumed to be 32 bits wide, and access must be word-aligned. The MFPR structure is as shown in Figure 10.

Figure 10:MFPR Structure

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Reserved																PULL_SEL	PULLUP_EN	PULLDN_EN	DRIVE		DRIVE	ST[1:0]		SLE	EDGE_CLEAR	EDGE_FALL_EN	EDGE_RISE_EN	SPU	AF_SEL		
Default	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	1	?	?	?	?	?	0	0	0	1	0	0	0	0	0	0

ASR CONFIDENTIAL

Through the MFPRs, software uses multi-function pins in the following ways:

- Controls the state of pins during reset and low-power modes
- Supports holding the last driven or register-defined state for all outputs during reset and low-power modes
- Manages selection of alternate peripheral functions
- Supports software configuration of output drive strength
- Supports external interrupt generation and wakeup-event detection

ASR CONFIDENTIAL

3.5.6 Drive Configuration

Table 15 shows settings for the <DRIVE> field in the MFPRs. The assumption is a load of 10 pF.

Table 15: <DRIVE> Field Settings

1.8v I/O driven strength	Iol DCS[1:0]= 00 01 10 11	Low level output current when Vpad=0.2V	 7.6mA 15.2mA 22.6mA 29.7mA	 12.8mA 25.3mA 37.4mA 49.0mA	 18.0mA 35.5mA 52.2mA 67.9mA
	Ioh DCS[1:0]= 00 01 10 11	High level output current when Vpad=VCC-0.2V	 4.8mA 9.5mA 14.3mA 18.9mA	 10.8mA 21.5mA 32.1mA 42.4mA	 18.9mA 37.4mA 55.9mA 73.9mA
1.8V/3.3v I/O drive strength					
	Iol DCS[2:0]= 000 001 010 011 100 101 110 111	Low level output current when Vpad=0.2V	 3.1mA 4.7mA 6.2mA 7.7mA 9.3mA 10.8mA 12.3mA 13.8mA	 5.5mA 8.2mA 10.8mA 13.4mA 16.1mA 18.7mA 21.2mA 23.7mA	 8.6mA 12.7mA 16.9mA 20.8mA 24.9mA 28.8mA 32.6mA 36.3mA
	Ioh DCS[2:0]= 000 001 010 011 100 101 110 111	High level output current when Vpad=VCC-0.2V	 5.0mA 7.5mA 10.1mA 12.6mA 15.1mA 17.6mA 20.1mA 22.6mA	 7.5mA 11.2mA 14.9mA 18.6mA 22.3mA 26.0mA 29.8mA 33.4mA	 10.5mA 15.7mA 21.0mA 26.2mA 31.4mA 36.5mA 41.8mA 46.9mA

3.5.7 Edge-Detect Operation

Some multi-function I/O pins are connected to edge-detection logic. Only when such a pin is configured to a specific function from a list of predefined functions is this logic not masked. The edge-detection logic is capable of detecting a rising or falling edge on the input path of the pin. When enabled, the pins are always monitored, such that a transition triggers the edge-detection logic.

The detection is controlled by three of the Multi-function Pin Control Register x fields as follows:

- <EDGE_CLEAR>: Enables and disables the function
- <EDGE_RISE_EN>: If enabled, allows detection of rising-edge transactions
- <EDGE_FALL_EN>: If enabled, allows detection of falling-edge transactions

These bits are available only for multi-function pins that are connected to edge-detection logic and can only be accessed by application core software.

ASR CONFIDENTIAL

The output of the edge-
detection logic is mainly used to provide a wakeup event in the Application Subsystem D1/D2 power states or the Communication Subsystem D2 power state and is NOT connected to the edge-detection function of the GPIO controller. Some of these outputs are also used as interrupts assigned to some of the Communication Subsystem interrupt controllers.

The edge-detection logic detects an event based on the edge, and not on a clock sampling a pulse, although a small minimum pulse width is required. For rising-edge detection, an event is detected when the signal was initially 0 and then a few ns later, changes to 1, so a pulse of only a few ns is detected.

Both rising and falling edge detection can be enabled, thereby producing an event whenever the signal changes state (even briefly). An event is a level signal, asserted for as long as the EDGE_CLEAR bit is not set to 1.

For an event to be captured in the edge-detection logic, all of the following conditions must be met in the order they are specified:

- The multi-function I/O pin must have at least one function that is capable of waking up the Application or Communication Subsystem. The multi-function I/O pin should be configured to select this function.
- Either EDGE_RISE_EN, EDGE_FALL_EN, or both fields in the relevant MFPR must be set to 1.
- The <EDGE_CLEAR> field in the relevant MFPR must be set to 0 to enable the edge-detection logic in a separate AIB transaction.

ASR CONFIDENTIAL

- The value that is driven on the ASR1606 I/O pin by the external device should change from 0 to 1, or from 1 to 0. The new value must be kept for at least 10 ns. The transition type (from 0 to 1 or from 1 to 0) must match the type that is configured in the EDGE_RISE_EN and EDGE_ FALL_EN bits.

3.5.8 Pulling State Control

Pulling state of a multi-function pin is able to be controlled by the <PULL_SEL>, <PULLUP_EN>, and <PULLDN_EN> fields in the MFPRs.

During reset, default value of <PULL_SEL> is 1, which helps define a default pulling state.

Furthermore, software is able to change the pulling state of the pin after reset, regardless of its alternate function. When <PULL_SEL> is set to 1, <PULLUP_EN> and <PULLDN_EN> define the pulling state of the pin.

3.6 *Function Assignments for Multi-function I/O Pins*

Figure 11:Function Assignments for Multi-funtion I/O Pins

Power Domain	Pad Name	Pad Edge wakeup	default pull	Function 0	Function 1	Function 2	Function 3	Function 4	Function 5	Function 6	Function 7
	Antena SW				1.8V / 2.8V						
USIM2	USIM2_UCLK	NO	UP	USIM2_UCLK	GPIO[45]	DSP_GPIO0		LTE_RX_TIME[11]	RF_GPIO[7]	AP_I2C_4_SCL	AP_CW[8]
1.8/3.3V	USIM2_UIO	NO	UP	USIM2_UIO	GPIO[46]	DSP_GPIO1		LTE_TX_TIME[11]	RF_GPIO[8]	AP_I2C_4_SDA	AP_CW[9]
	USIM2_URSTn	NO	UP	USIM2_URSTn	GPIO[47]	DSP_GPIO2		LTE_IRAT_TIME[11]	RF_GPIO[9]	NDR_CW[10]	AP_CW[10]
	PMIC2 domain				1.8V						
PMIC2	MEM_RST			MEM_RST							
1.8v	EXT_32K_IN	NO		EXT_32K_IN							
	PWR_SCL	NO		PWR_SCL							
	PWR_SDA	NO		PWR_SDA							
	VCXO_EN	NO		VCXO_EN							
	GPIO_04	Yes	DOWN	GPIO[4]	*KP_MKIN[2]*	RF_GPIO[13]		UART4_RXD	NDR_CW[4]	AP_CW[4]	SSP1_SCLK
	GPIO_05	Yes	DOWN	GPIO[5]	*KP_MKOUT[2]*	RF_GPIO[14]		UART4_TXD	NDR_CW[5]	AP_CW[5]	SSP1_FRM
	GPIO_06	Yes	DOWN	GPIO[6]	*KP_MKIN[3]*	RF_GPIO[15]	PWM3	UART4_CTS_N	NDR_CW[6]	AP_CW[6]	SSP1_RXD
	GPIO_07	Yes	UP	GPIO[7]	*KP_MKOUT[3]*		PWM4	UART4_RTS_N	NDR_CW[7]	AP_CW[7]	SSP1_TXD
	GPIO_10	Yes	UP	GPIO[10]	I2C_2_SCL	*KP_MKIN[5]*	PWM3		NDR_CW[10]	AP_CW[10]	
	GPIO_11	Yes	UP	GPIO[11]	I2C_2_SDA	*KP_MKOUT[5]*	PWM4		NDR_CW[11]	AP_CW[11]	
	GPIO_12	Yes	DOWN	GPIO[12]	SSP2_SCLK	*KP_MKIN[6]*	*KP_DKIN[4]*		NDR_CW[12]	AP_CW[12]	AP_I2C_3_SCL
	GPIO_13	Yes	DOWN	GPIO[13]	SSP2_FRM	*KP_MKOUT[6]*	*KP_DKIN[5]*		NDR_CW[13]	AP_CW[13]	AP_I2C_3_SDA
	GPIO_14	Yes	DOWN	GPIO[14]	SSP2_RXD	*KP_MKIN[7]*	*KP_DKIN[6]*		NDR_CW[14]	AP_CW[14]	UART3_RXD
	GPIO_15	Yes	DOWN	GPIO[15]	SSP2_TXD	*KP_MKOUT[7]*	*KP_DKIN[7]*		NDR_CW[15]	AP_CW[15]	UART3_TXD
	GPIO_20	Yes	DOWN	GPIO[20]	SDU_DAT[3]	GPS_STROBE	MMC1_DAT[3]	SEC2_TMS	SEC1_TMS	32KHz CLK OUT	Smart BAT
	GPIO_21	Yes	DOWN	GPIO[21]	SDU_DAT[2]	ISIM_VCLK_OUT	MMC1_DAT[2]	SEC2_TCK	SEC1_TCK	rf_jtag_tms	AP_UART1_CTS_N
	GPIO_22	Yes	DOWN	GPIO[22]	SDU_DAT[1]	ISIM_SPI_SCK	MMC1_DAT[1]	SEC2_TDO	SEC1_TDO	rf_jtag_tck	AP_UART1_RTS_N
	GPIO_23	Yes	UP	GPIO[23]	SDU_HOST_INT	ISIM_SPI_SDI[1]	GPS_STROBE	SEC2_TRSTn	SEC1_TRSTn	32KHz CLK OUT	AP_UART1_RI_N

Copyright © 2022 ASR

	GPIO_24	Yes	DOWN	GPIO[24]	SDU_DAT[0]	ISIM_SPI_SDI[0]	MMC1_DAT[0]	SEC2_TDI	SEC1_TDI	rf_jtag_do	AP_UART1_DCD_N
	GPIO_25	Yes	UP	GPIO[25]	SDU_CMD	AP_I2C_4_SCL	MMC1_CMD	SEC2_PSU_RTCK	NDR_CW[0]	rf_jtag_trst_n	AP_UART_DTR
	GPIO_26	Yes	UP	GPIO[26]	SDU_CLK	AP_I2C_4_SDA	MMC1_CLK		NDR_CW[1]	rf_jtag_tdi	AP_UART_DSR
	GPIO_29	Yes	UP	GPIO[29]	AP_UART1_RXD	PLL1 lock			NDR_CW[4]	AP_CW[4]	
	GPIO_30	Yes	UP	GPIO[30]	AP_UART1_TXD	PLL2 lock			NDR_CW[5]	AP_CW[5]	
	GPIO_32	Yes	UP	GPIO[32]	CP_UART_DCD_N	PWM2	LCD_IO[2]/SMPN_DB[2]		NDR_CW[7]	AP_CW[7]	AP_UART1_RTS_N
	GPIO_71	Yes	UP	GPIO[71]	UART4_RXD	SSP1_SCLK	AP_I2C_3_SCL	*KP_DKIN[0]*	*KP_MKIN[0]*		
	GPIO_72	Yes	UP	GPIO[72]	UART4_TXD	SSP1_FRM	AP_I2C_3_SDA	*KP_DKIN[1]*	*KP_MKOUT[0]*		
	GPIO_73	Yes	UP	GPIO[73]	UART4_CTS_N	SSP1_RXD	UART3_RXD	*KP_DKIN[2]*	*KP_MKIN[1]*		
	GPIO_74	Yes	UP	GPIO[74]	UART4_RTS_N	SSP1_TXD	UART3_TXD	*KP_DKIN[3]*	*KP_MKOUT[1]*		
	GPIO_121	Yes	DOWN	GPIO[121]	GPIO[99]	32KHz CLK OUT					
	GPIO_124	Yes	DOWN	GPIO[124]	MN_CLK_OUT	GPS_STROBE	32KHz CLK OUT	CP_UART_DSR			
	TD2 domain				1.8V						
TD2 domain	GPIO_00	Yes	DOWN	GPIO[0]	*KP_MKIN[0]*	RF_GPIO[9]	TSC_XL	UART4_CTS_N	NDR_CW[0]	AP_CW[0]	
1.8v	GPIO_01	Yes	DOWN	GPIO[1]	*KP_MKOUT[0]*	RF_GPIO[10]	TSC_XR	UART4_RTS_N	NDR_CW[1]	AP_CW[1]	
	GPIO_02	Yes	DOWN	GPIO[2]	*KP_MKIN[1]*	RF_GPIO[11]		UART4_RXD	NDR_CW[2]	AP_CW[2]	
	GPIO_03	Yes	DOWN	GPIO[3]	*KP_MKOUT[1]*	RF_GPIO[12]		UART4_TXD	NDR_CW[3]	AP_CW[3]	
	GPIO_08	Yes	DOWN	GPIO[8]	PWM1	*KP_MKIN[4]*	PWM3		NDR_CW[8]	AP_CW[8]	
	GPIO_09	Yes	UP	GPIO[9]	PWM2	*KP_MKOUT[4]*	PWM4		NDR_CW[9]	AP_CW[9]	
	GPIO_16	Yes	DOWN	GPIO[16]	SSPA_BITCLK	SSP0_CLK	*KP_DKIN[0]*			mn_div_clk1	
	GPIO_17	Yes	DOWN	GPIO[17]	SSPA_SYNC	SSP0_FRM	*KP_DKIN[1]*			VCXO_REQ2	
	GPIO_18	Yes	DOWN	GPIO[18]	SSPA_DATA_OUT	SSP0_TXD	*KP_DKIN[2]*	SSP0_RXD		VCXO_MM_OUT	
	GPIO_19	Yes	DOWN	GPIO[19]	SSPA_SDATA_IN	SSP0_RXD	*KP_DKIN[3]*	SSP0_TXD		mn_div_clk2	
	GPIO_68	Yes	DOWN	GPIO[68]	DVL[1]		mn_div_clk2		NDR_CW[9]	AP_CW[9]	TRACK_DATA[6]
	GPIO_69	Yes	DOWN	GPIO[69]	DVL[2]	TSC_YD	DSP_GPIO3	LTE_RX_TIME[0]	NDR_CW[10]	AP_CW[10]	TRACK_DATA[5]
	GPIO_70	Yes	DOWN	GPIO[70]		TSC_YU	DSP_GPIO4	LTE_TX_TIME[0]	NDR_CW[11]	AP_CW[11]	TRACK_DATA[4]
	DVC[0]	Yes	DOWN	DVL[0]	GPIO[67]		mn_div_clk1		NDR_CW[8]	AP_CW[8]	TRACK_DATA[7]
	MN_CLK_REQ	Yes	DOWN	MN_CLK_REQ	GPIO[123]	SMART_BAT	EXT_DMA_REQ[1]		VCXO_REQ1	GPIO[41]	
	VCXO_REQ	Yes	DOWN	VCXO_REQ	GPIO[125]	PLL_BYPASS	EXT2_INT_REQ		GPIO[94]	GPIO[42]	GPIO[86]
	VCXO_OUT	NO	DOWN	VCXO_OUT	PWM1	GPIO[126]	PWM4		GPIO[95]	GPIO[43]	GPIO[87]
	USIM connectivity:				OFF/1.8V/3V						
USIM	USIM_UCLK	NO	UP	USIM_UCLK	GPIO[81]						
0/1.8V/3V	USIM_UIO	NO	UP	USIM_UIO	GPIO[82]						
	USIM_URSTn	NO	UP	USIM_URSTn	GPIO[83]						
	CP UART domain				OFF/1.8V /3V						
UART	GPIO_51	Yes	UP	GPIO[51]	CP_UART_RXD			MMC1_DAT[3]	NDR_CW[14]	AP_CW[14]	
1.8v/3.3v	GPIO_52	Yes	UP	GPIO[52]	CP_UART_TXD			MMC1_DAT[2]	NDR_CW[15]	AP_CW[15]	
	GPIO_53	Yes	UP	GPIO[53]	CP_UART_CTS_N	UART3_RXD	VCXO_REQ1	MMC1_DAT[1]	EXT_AIRQ1		
	GPIO_54	Yes	UP	GPIO[54]	CP_UART_RTS_N	UART3_TXD	VCXO_OUT1	MMC1_DAT[0]	EXT_AIRQ2		
	GPIO_31	Yes	UP	GPIO[31]	CP_UART_RI_N	PWM1	LCD_IO[1]/SMPN_DB[1]	MMC1_CMD	NDR_CW[6]	AP_CW[6]	AP_UART1_CTS_N
	GPIO_27	Yes	DOWN	GPIO[27]	CP_UART_DTR	PLL1 monitor		MMC1_CLK	NDR_CW[2]		
	PMIC1 domain				OFF/1.8V /3V						
PMIC1 domain	VBUS_ON	NO		VBUS_ON							
1.8v	PMIC_INTn	NO (can wakeup)		PMIC_INTn							
	PRI_TRSTn	NO		PRI_TRSTn							
	PRI_TDI	Yes	UP	PRI_TDI	GPIO[117]	PWM1				GPIO[37]	GPIO[96]
	PRI_TMS	Yes	UP	PRI_TMS	GPIO[118]	PWM2				GPIO[38]	GPIO[97]

Copyright © 2022 ASR

	PRI_TCK	Yes	DOWN	PRI_TCK	GPIO[119]					GPIO[39]	GPIO[98]
	PRI_TDO	Yes	UP	PRI_TDO	GPIO[120]		GPIO[85]			GPIO[40]	GPIO[84]
	QSPI_CS2	Yes	UP	QSPI_CS2	GPIO[77]	RF_GPIO[9]	DSP_GPIO7	MN_CLK_OUT		LTE_RX_TIME[2]	TRACKCLKIN
	GPIO_78	Yes	DOWN	GPIO[78]		RF_GPIO[10]	DSP_GPIO8			LTE_TX_TIME[2]	
	GPIO_79	Yes	DOWN	GPIO[79]		RF_GPIO[7]	DSP_GPIO5		NDR_CW[8]	AP_CW[8]	
	GPIO_80	Yes	DOWN	GPIO[80]		RF_GPIO[8]	DSP_GPIO6		NDR_CW[9]	AP_CW[9]	
	GPIO_60	Yes	DOWN	GPIO[60]	RF_GPIO[0]	TCU_GPOA[11]	RF_GPIO[0]		*KP_DKIN[0]*	NDR_CW[9]	
	GPIO_61	Yes	DOWN	GPIO[61]	RF_GPIO[1]	RF_CONT[12]	RF_GPIO[1]		*KP_DKIN[1]*	NDR_CW[10]	SSP2_RXD
	GPIO_62	Yes	DOWN	GPIO[62]	RF_GPIO[2]	RF_CONT[11]	RF_GPIO[2]		*KP_DKIN[2]*	NDR_CW[11]	SSP2_TXD
	GPIO_63	Yes	DOWN	GPIO[63]	RF_GPIO[3]	RF_CONT[10]	RF_GPIO[3]		*KP_DKIN[3]*	NDR_CW[12]	SSP2_FRM
	GPIO_64	Yes	DOWN	GPIO[64]	RF_GPIO[4]	RF_CONT[5]	RF_GPIO[4]	GPS_STROBE	LTE_RX_TIME[11]	NDR_CW[13]	SSP2_SCLK
	GPIO_65	Yes	DOWN	GPIO[65]	RF_GPIO[5]	RF_CONT[4]	RF_GPIO[5]	mn_div_clk1	LTE_TX_TIME[11]	NDR_CW[14]	
	GPIO_66	Yes	DOWN	GPIO[66]	RF_GPIO[6]	RF_CONT[1]	RF_GPIO[6]	mn_div_clk2	LTE_IRAT_TIME[11]	NDR_CW[15]	
	VCC_1.8V				1.8V						
GPIO1 domain 1.8v	QSPI_DAT3	NO	DOWN	QSPI_DAT[3]	GPIO[71]		GPIO[48]		NDR_CW[12]	AP_CW[12]	TRACK_DATA[3]
	QSPI_DAT2	NO	DOWN	QSPI_DAT[2]	GPIO[72]		GPIO[55]		NDR_CW[13]	AP_CW[13]	TRACK_DATA[2]
	QSPI_DAT1	NO	DOWN	QSPI_DAT[1]	GPIO[73]	SSP2_RXD	GPIO[56]		NDR_CW[14]	AP_CW[14]	TRACK_DATA[1]
	QSPI_DAT0	NO	DOWN	QSPI_DAT[0]	GPIO[74]	SSP2_TXD	GPIO[57]		NDR_CW[15]	AP_CW[15]	TRACK_DATA[0]
	QSPI_CLK	NO	DOWN	QSPI_CLK	GPIO[75]	SSP2_SCLK	GPIO[58]				TRACECTL
	QSPI_CS1	NO	UP	QSPI_CS1	GPIO[76]	SSP2_FRM	GPIO[59]				TRACKCLK
	GPIO_28	Yes	DOWN	GPIO[28]	CP_UART_DSR	PLL2 monitor	32KHz CLK OUT	MN_CLK_OUT	GPS_STROBE		
	GPIO_33	Yes	DOWN	GPIO[33]	SSP0_CLK	SSP2_SCLK	LED_CTRL1		NDR_CW[8]	AP_CW[8]	
	GPIO_34	Yes	DOWN	GPIO[34]	SSP0_FRM	SSP2_FRM			NDR_CW[9]	AP_CW[9]	
	GPIO_35	Yes	UP	GPIO[35]	SSP0_RXD	SSP2_RXD	UART3_RXD		NDR_CW[10]	AP_CW[10]	
	GPIO_36	Yes	UP	GPIO[36]	SSP0_TXD	SSP2_TXD	UART3_TXD		NDR_CW[11]	AP_CW[11]	
	GPIO_37	Yes	DOWN	GPIO[37]							
	GPIO_49	Yes	UP	GPIO[49]	*CI2C_SCL*				NDR_CW[12]	AP_CW[12]	
	GPIO_50	Yes	UP	GPIO[50]	*CI2C_SDA*				NDR_CW[13]	AP_CW[13]	
	GPIO_122	Yes	DOWN	GPIO[122]	SLAVE_RESET_OUT	32KHz CLK OUT	LTE_RX_TIME[1]				

3.7 *Register Descriptions*

Refer to *ASR1606 Registers* for details on the MFPR registers.

ASR CONFIDENTIAL

4. ARM Cortex-R5 Modem Subsystem

- Section 4.1, Overview
- Section 4.2, Function Description
- Section 4.3, Memory Map
- Section 4.4,Interrupt

4.1 *Overview*

The ARM Cortex-R5 modem CPU subsystem has the following features.

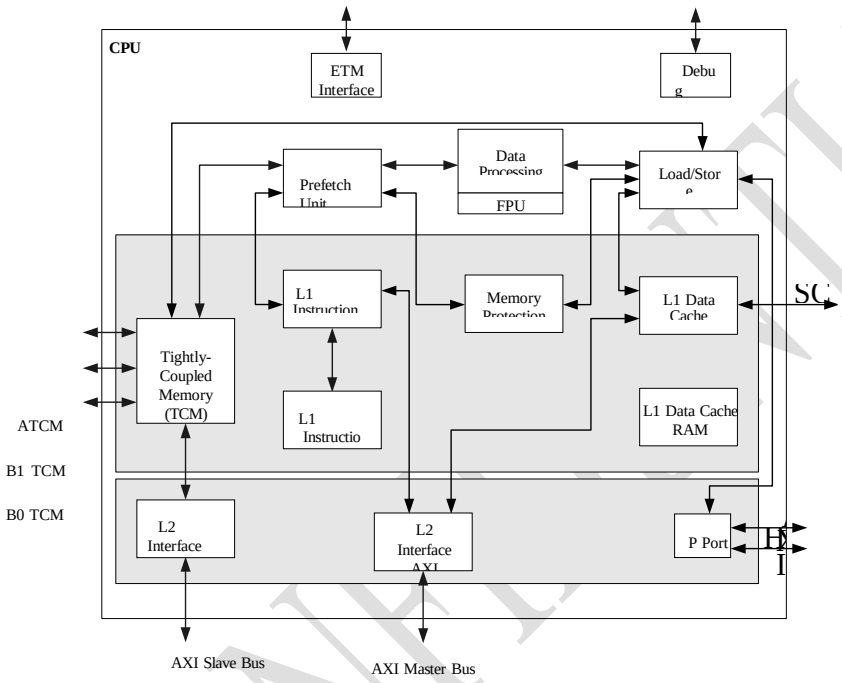
- Integer unit implementing ARMv7-R instruction set
- Dynamic branch prediction with global history buffer and a 4-entry return stack
- ARMv7-R architecture Memory Protection Unit (MPU)
- L1 memory system with:
  - 32 KB Instruction Cache (I-Cache)
  - 32 KB Data Cache (D-Cache)
  - 64 KB ATCM
  - 64 KB BTCM

  No L2 Cache in Cortex R5.
- AXI3 Bus interface with:
  - Two 64-bit master AXI3 interfaces for accessing:
    - PSRAM
    - Communication subsystem fabric
  - 64-bit slave AXI3 interface to TCM memories and L1 Cache RAM blocks
- Debug components:
  - Cross Trigger Interface (CTI)
  - Embedded Trace Macrocell (ETM)
  - Debug APB

ASR CONFIDENTIAL

Figure 12 provides a high-level block diagram of the ARM Cortex-R5 modem CPU subsystem block diagram.

Figure 12:ARM Cortex-R5 Modem CPU Subsystem Block Diagram



4.2 *Functional Description*

The ARM Cortex-R5 modem is a mid-range CPU for use in deeply-embedded, real-time systems. It implements the ARMv7-R architecture and includes Thumb-2 technology for optimum code density and processing throughput. The pipeline has a single Arithmetic Logic Unit (ALU) but implements limited dual-issuing of instructions for efficient use of other resources, such as the register file.

Figure 12 shows a high-level ARM Cortex-R5 modem core block diagram. The PreFetch Unit (PFU) performs the following actions:

- fetches instructions from the memory system
- predicts branches
- passes instructions to the Data Processing Unit (DPU)

ASR CONFIDENTIAL

Copyright © 2022 ASR

The DPU executes all instructions and uses the Load/Store Unit (LSU) for data memory transfers. The PFU and LSU interface to the L1 memory system that contains the L1 Instruction and Data Caches and the TCM interfaces.

The ARM Cortex-R5 modem L1 memory system consists of:

- 64 KB ATCM
- 64 KB BTCM
- 32 KB Instruction Cache (I-Cache)
- 32 KB Data Cache (D-Cache)

The ARM Cortex-R5 modem subsystem uses the 88PL310 as the L2 controller AXI interface. The 88PL310 provides 2 AXI master ports to access the Memory Controller and Communication subsystem fabric, respectively. Ceva BX2 and CPDMA are able to access TCMs through the AXI slave port. Be note, here only use L2 controller AXI interfaces, ASR1606 ARM Cortex-R5 does not have L2 Cache.

ASR CONFIDENTIAL

4.3 *Memory Map*

The ARM Cortex-R5 memory map is shown in Chapter 2.3, Figure 2.

4.4 *Interrupt*

The ARM Cortex-R5 modem core gets interrupts from ICU controller, sources as shown in Table3: ICU Hardware Interrupt.

Copyright © 2022 ASR

5. SQU

The SQU includes two AXI Slave Interfaces. This chapter details the following:

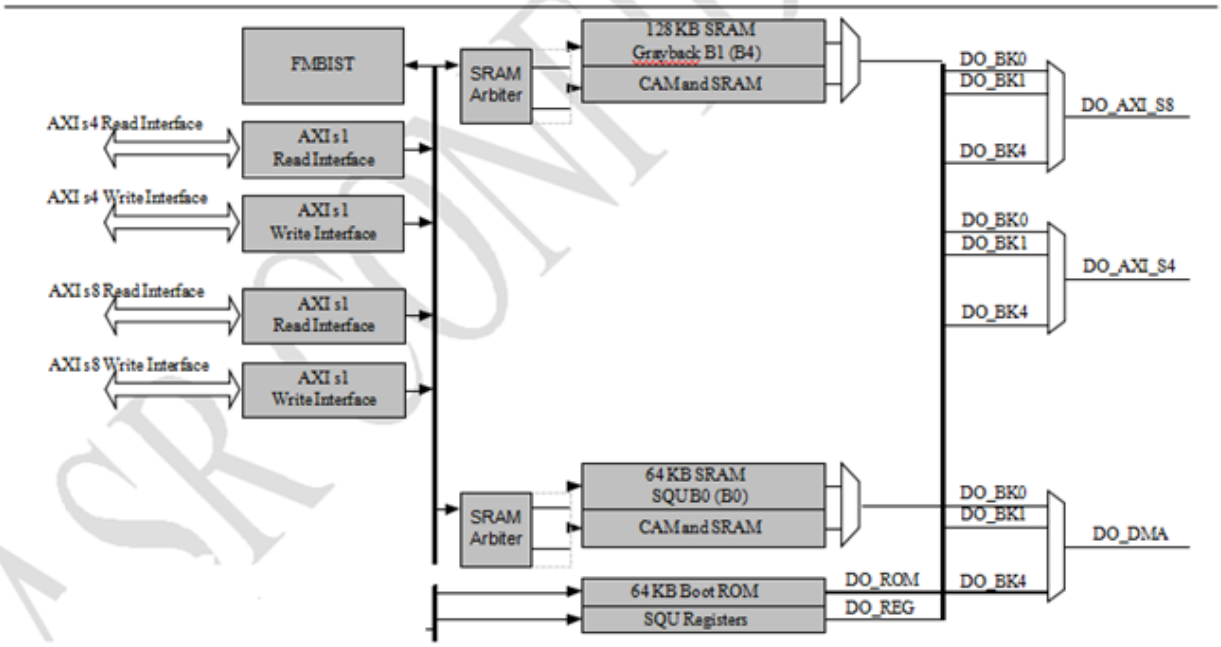
- Section 5.1, Overview
- Section 5.2, Functionality

5.1 Overview

The SQU provides on-chip SRAM/ROM access. It has two AXI slave ports to service SRAM/ROM access from all AXI masters. Only AXI_S4 can access Boot ROM and SQU SRAM.

With its unique architecture, the SQU provides low-latency access through cycle-based arbitration and high throughput on as many as five concurrent accesses for a maximum configuration of five SRAM banks(In ASR1606, only support one bank 32KB SRAM).

Figure 13: SRAM Controller Block Diagram



5.2 Functionality

5.2.1 Basic Overview

The following blocks interface between AXI protocol and the internal SRAM banks:

- AXI_S4_RD_IF
- AXI_S4_WR_IF
- AXI_S8_RD_IF
- AXI_S8_WR_IF

ASR CONFIDENTIAL

The SRAM_ARB block arbitrates access requests using either a fair round-robin arbitration scheme or fixed priority arbitration scheme based upon SQU register setting.

The SRAM_ARB does cycle-based arbitration. It evaluates requests every cycle and gives access to the master based upon the arbitration result. In this way, the scheme is not transaction-based arbitration and helps achieve the best low latency for SRAM accesses.

ASR CONFIDENTIAL

Copyright © 2022 ASR

6. PSRAM Memory Controller

This section describes the following:

- Section 6.1, Introduction
- Section 6.2, Feature List
- Section 6.3, Overview
- Section 6.4, Function description

6.1 Introduction

PSC is PSRAM Controller which connect between AXI and PSRAM device. There are cache buffer and reorder buffer inside to increase device bus utilization and performance.

The PSRAM Controller is designed for ARM AXI bus protocols. It is fully scalable and has 4 generic ports.

It support programmable sequence table for communicate with variable device from different vendor.

6.2 Feature List

- AXI: 4 ACE-Lite ports
  MMAP register for program PSC base addr and accepted address range
  5 level AXI QoS supported
  AXI ports arbitration based on QoS or Weight Round Robin.
  exclusive monitor
  supports asynchronous and synchronous clocks between AXI masters and PSRAM controller.
  write bandwidth limitation control
- Cache for buffer device data
  16KB full associate cache, 1 write port and 1 read port, 16Byte width each
  configurable cacheline size(64B/128B/256B)
  memory type: write back/write through/non-cacheable, memory type configured in 16 address range registers
  full write and part read optimization for WB type linefill for write.

WB Write back	Cacheline state: inv/valid clean/valid dirty 1. for both read and write, if cachemiss, linefill from device 2. if read, read data from cache 3. if write, write data to cache 4. if cache is full when linefill, replace a line selected by PS LRU(PSeudo Least Recent Use) if it is clean line, just inv it if it is dirty line, evict data to device
WT	Cacheline state: inv/valid clean, No dirty state

ASR CONFIDENTIAL

Write through	For read, 1. if cachemiss, linefill from device 2. if cachehit, read data from cache For write, 1. if cachemiss, no linefill, just write to device 2. if cachehit, write data to both cache and device
NC Non cacheable	No cache access, read and write to device directly

cachelock supported, 8 cachelock range
flush and invalidate cacheline supported, all or programmable address range
supports asynchronous or synchronous clocks between Cache and Device.

- Communication with Device
  instructions are defined to control device bus
  sequence tables base on instructions are configurable to read/write/MRR/MRW device.
  AXI access will trigger read or write sequence
  register trigger sequence supported
  dual Chip Select supported
- Priority Control, on AXI QoS
  Priority in Queue
  a) 2 priority category in WCB, RCB based on AXI QoS
  b) 5 priority category and 4 level QoS in LFQ based on request type and AXI QoS
  c) 5 level qos in ROB based on AXI QoS
  aging counter in WCB/RCB and LFQ.
- MISC
  Table is implement to support Low Power and Dynamic Frequency Change
  register write and register read TX/RX buffer supported
  interrupt for rx_fifo_full and performance overflow
  Performance counter.
  d) axi monitor on 4 AXI ports
  e) controller clock domain counter
  f) phy clock domain counter

6.3 Overview

The PSRAM Controller has an AXI that consists of 5 AXI channels. The write and read address channels, write data, read data, and write respond channels have a programmable queue depth. However, the queue depth of the write data is always 16 fold of the write address queue depth. The interface each can be independently strapped to be synchronous or asynchronous mode. Asynchronous mode introduces 2 receiver cycles latency each direction.

ASR CONFIDENTIAL

Write requests are stored in the write command buffer (wcb) and their data is packed and stored in the write data buffer (wdb). The PSRAM Controller write buffer can store up to 16 AXI bursts.

Read requests are first stored in the read command buffer (rcb). It also store up to 16 AXI burst.

Read and write request will first compare with cache tags, if cache hit, it get data from cache or write to cache immediately. If cache miss, the request will go to line fill queue(lfq) to communicate with device. It support up to 8 cachemiss.

Write data comes from the write buffer to Cache or device. Read data comes from Cache or device to the re-ordering buffers (ROBs) where data is re-ordered back to the original AXI order. Note that only read data of the same AXI ID needs to be in order while read data of different AXI IDs can be returned back to the AXI masters without any delay.

6.4 Functional Description

6.4.1 PSRAM Initialization and Sequence Table Setting

After system reset, please follow the PSRAM device requirement to set power and reset pad.

All communications with PSRAM device like read, write, mode register access and so on are accomplished through the Sequence Tables which are constructed by predefined instructions.

there are 16 Sequence table(64 32bit register) totally, each table entry has 8 instructions.

There are 2 ways to start the execution of a sequence.

1. from AXI read and write, it only trigger read and write to device. the entry index is set in register.
2. user trigger a sequence directly by register, it is used for debug and Mode Register Write, Mode Register Read and so on.

Predefined instructions are listed below.

ASR CONFIDENTIAL

Table 16:Predefined instructions

	delay	phy					instruction				
		ce	ck_en	wr_en	cmd/ data	rd_en	ddr 15	CS 14	instr 14:10	pads 9:8	operand 7:0
INSTR_DUMMY	0p	0	1	0	NA	0	y	D	8;	bit1: dis CK bit0: dis CEN	delay
INSTR_DUMMY_CDN	0p	0	1	0	NA	0	y	D	d;	bit1: dis CK bit0: dis CEN	delay
INSTR_CMD	1	0	1	1	0/1	0	y	D	1;	0	to bus
INSTR_ADDR	0p/8	0	1	1	0/1	0	y	D	2;	y	bits
INSTR_CADDR	0p/8	0	1	1	0/1	0	y	D	3;	y	bits
INSTR_RADDR	0p/8	0	1	1	0/1	0	y	D	4;	y	bits
INSTR_WRITE	0p|cch	0	1	1	data	0	y	D	5;	y	cache sram size for reg req
INSTR_WRITE_MR	1	0	1	1	0/1	0	y	D	c	y	selection from mr_data
INSTR_READ	0p|cch	0	1	0	NA	1	y	N	6;	y	size for reg req
INSTR_READ_MR	1	0	1	0	NA	1	y	N	7;	y	byte size for reg req
INSTR_END	1	1	0	0	NA	0		N	0;	0	
INSTR_WAIT_0	{Pads, op}	1	0	0	NA	0		N	9;	delay	delay
INSTR_WAIT_1	{Pads, op}	1	0	0	NA	0		N	a	delay	delay on 26M clk
INSTR_WAIT_2	{Pads, op}	1	0	0	NA	0		N	b	delay	delay on 32K clk

*pads=y, means set this field to 'h3

*for CS in bit 14

D: single CS and dual CS supported

N: only single CS supported

D=0, single CS, depends on ADDR

D=1, dual CS

if cs_loc==0, CS is not enable in reg, all treated as D=0 and will on cs_default

For example, the sequence table for read to APM device can be set as

INSTR_CMD	Read command
INSTR_ADDR	Read address
INSTR_DUMMY	Read latency
INSTR_READ	Read data from device
INSTR_END	Sequence complete

ASR CONFIDENTIAL

6.4.2 Data Request Arbitration

there are arbitration in 4 place, among 4 AXI ports, WCB, RCB and LFQ.

1. arbitration among 4 AXI port
   a) AXI arbitration use QoS round robin or weight round robin, this is select by register setting.
   b) write bandwidth limitation
      write bandwidth can be limited by register. User could configure a time window and data size transferred in the window for bandwidth. write request would be blocked in AXI arbitration if write bandwidth is fulfilled until the time window expire.
      below features could be enable or disable by register:
      if 4 master all exceed the bandwidth limitation, it restart the window immediately.
      if no access from any master, the master could be treated as fulfilled.
2. arbitration among WCB/RCB
   a) WCB/RCB QoS region
   5 level QoS from AXI port will map to 2 level (high and low) in WCB/RCB. The high or low QoS threshold is controlled by register.
   high level will get arbitration first, if there is no high level request, low level req will get arbitration.
   PingPong arbiter is used for arbitration among same level requests.
   aging counter is implement to prevent low priority request from starving.
   b) WCB/RCB grant write/read ratio
   When LFQ has 2 or more entries, WCB and RCB can be granted at same T, but when only 1 entry left in LFQ, there is competition between WCB and RCB. There is register to control the WCB/RCB grant ratio in this scenario.
3. arbitration among LFQ
   a) full write and partial read: WB type only
      for WB write, it require 2 access to device, linefill and eviction, full write and part read is designed to optimize the linefill by reduce linefill data size.
      full write feature, if write full cacheline, no need to linefill-read from device.
      part read feature, if part of cacheline will be written, this part is not read from device.
      part read only valid when full write feature is enable.
   b) LFQ: QoS
      4 priority level(QoS) can be set for 5 priority category:
      4. critical read(axi qos[2]=1)
      3. full write for WB
      2. higher priority from AXI QOS
      1. read access for WB, read and write access for WT and NC
      0. write access for WB
      the priority of each category can be configured by register.
      Hardware will find the top level priority and select the oldest to send to device.
      critical read is added for really latency sensitive master, it do not have aging counter, it should be set as the highest qos in lfq. note, it can't exceed pri read in RCB but it can exceed any in LFQ.

ASR CONFIDENTIAL

c) LFQ: aging counter
there are aging counters for each entry in lfq, threshold could be configured for each category, if counter exceed threshold, it will be set qos as thr_qos.
counter added when a lfq entry is granted and not itself.

d) LFQ: QoS automatic adjustment, can be enable or disable by register
automatically increase lfq cat_1 qos to cat_2 qos if WCB is not full and RCB is full
automatically increase lfq cat_0 qos to lfq_thr_qos if WCB is full and RCB is not full

6.4.3 Addressing and Memory Size

PSRAM Controller has MMAP register to set the start address and address range of the PSRAM device.
Stating address is AXI address[31:23], it must align with area length. Area_len is chip density.
PSRAM Controller can support single CS or dual CS. Dual CS is also controlled in MMAP register. It can support 4/8/16/32MB for each CS. NOTE, the area_len for each CS must be the same and their address ranges are in contiguous address space.

6.4.4 Frequency Control

PSRAM Controller support 2 Frequency point. For each frequency it has some dedicate registers to separate the settings.
PSRAM Controller has Dynamic Frequency Change Table to accomplish the frequency change sequence automatically with the support of SOC pmu.

6.4.5 Power Saving

clock gating and phy power saving are implemented and controlled by register.

ASR CONFIDENTIAL

7. Modem Functional Description

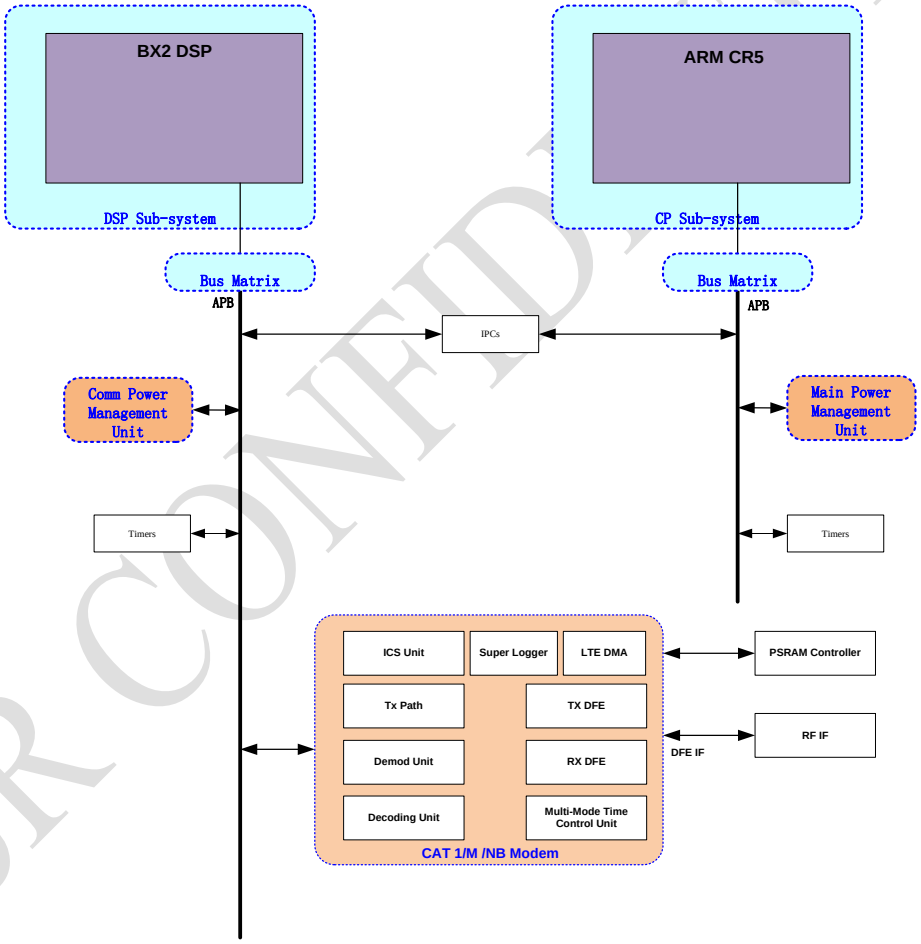
This section describes the following:

- Section 7.1, Overview of LTE platform
- Section 7.2, Introduction to LTE Modem

7.1 Overview of LTE platform

Figure 14 shows the block diagram of the LTE system.

Figure 14:LTE System Block Diagram



7.2 Introduction to LTE Modem

7.2.1 Overview

The main features of the LTE modem are as follows:

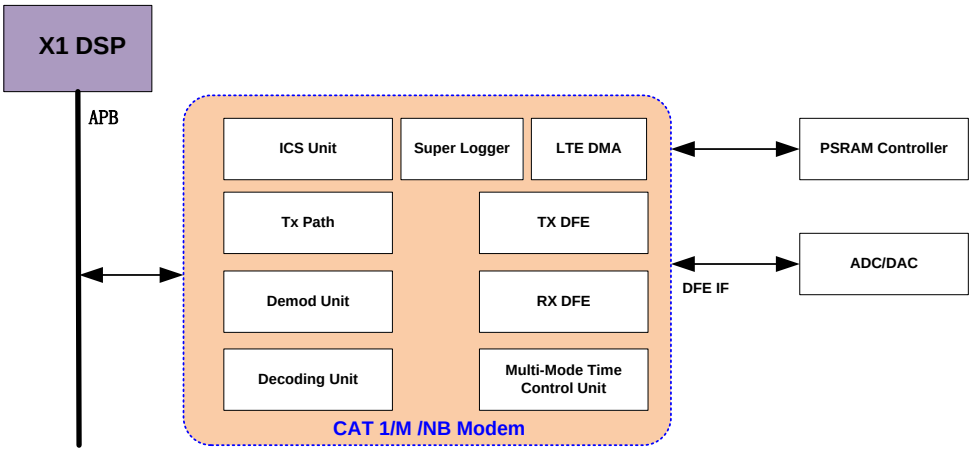
- Supports both FDD and TDD

ASR CONFIDENTIAL

- Supports Category 1:
  - Downlink maximum throughput 10Mbps
  - Uplink maximum throughput 5Mbps
  - Supports transmission modes: TM1~TM8
  - Supports MBSFN (15kHz only)
- Supports Category M1:
  - Downlink maximum throughput 1Mbps
  - Uplink maximum throughput 1Mbps
  - Supports transmission modes: TM1, TM2, TM6, TM9
- Supports the following bandwidths:
  - 1.4 MHz
  - 3 MHz
  - 5 MHz
  - 10 MHz
  - 15 MHz
  - 20 MHz
- Supports one Rx antenna (one layer)
- Supports SPS, DRX, uplink TTI bundling

The LTE modem architecture is shown in Figure 15.

Figure 15:LTE Modem Block Diagram



The LTE modem subsystem can be divided into the following parts:

- ICS modules
- Rx path modules (demodulation and decoding)
- Tx path modules (encoding and tx modulation)
- Control modules (timing controller unit)
- LTE DMA engine

ASR CONFIDENTIAL

- Super Logger

7.2.2 Module Description

Table 17 describes each module within the LTE modem (as shown in Figure 15).

Table 17:LTE Modem Modules

Module	Description / Functionality
TxDFE	Digital front end for TX, sample rate adjustment
RxDFE	Digital front end for RX, sample rate adjustment and data dispatch
Timing controller unit	Generates the timing reference/control signals to all modules
LTE DMA	LTE modem DMA engine
Super Logger	Logger for debug
CP Sync	CP correlation to find the coarse timing
ICS	Carrier sorting, SSS/PSS/BCH detection
Coarse AxC	Coarse frequency/timing adjustment

Module	Description / Functionality
Channel Estimation	Estimates frequency domain channel and noise
Measurement	Measures signal strength for serving cell and neighboring cell
Demod	Demodulates the RX data, generate soft symbol for decoder
CSI Calculation	Calculates channel state information for feedback
AxC	Automatic timing/frequency/gain control
De-RM and HARQ	De-Rate Matching and HARQ control
S2SB	Calculates soft bits from symbols
TC decoder	Turbo decoder
CC decoder	Viterbi decoder
CRC	CB/TB CRC check
TX encoding	Encodes the UL data
symbol mapping	Mapping bits to symbols
Tx modulation	OFDM modulation, RS insertion
Tx frontend	TX digital front end, upsampling

ASR CONFIDENTIAL

8. Communication Subsystem Peripherals Functional Description

- Section 8.1, Communication IPC
- Section 8.2, Communication Subsystem Timers 2
- Section 8.3, Universal Subscriber ID Module (USIM) Controller
- Section 8.4, Register Descriptions

8.1 Communication IPC

The Communication IPC unit connects the ARM Cortex-R5 and BX2 DSP.

8.2 Communication Subsystem Timers

The Communication Subsystem Timers has a triple timer plus a Watchdog timer. The three 32-bit general-purpose timers are based on a selectable functional clock (13, 6.5, and 3.25 MHz). This section provides information about the timer operation, related register addresses, and bit definitions.

Operation

The operating system timers and watchdog timer module consist of three 32-bit Timer Clock Control Registers (TCCRn) up counters and one 16-bit WDT up counter. The module has two clock inputs:

- Selectable fast clock (13, 6.5, and 3.25 MHz)
- Slow clock (32.768 kHz)

The fast clock is selectable through the <Functional Clock Select> field in the Clock/Reset Control Register for Timers (refer to the *ASR1606 Registers* for more information).

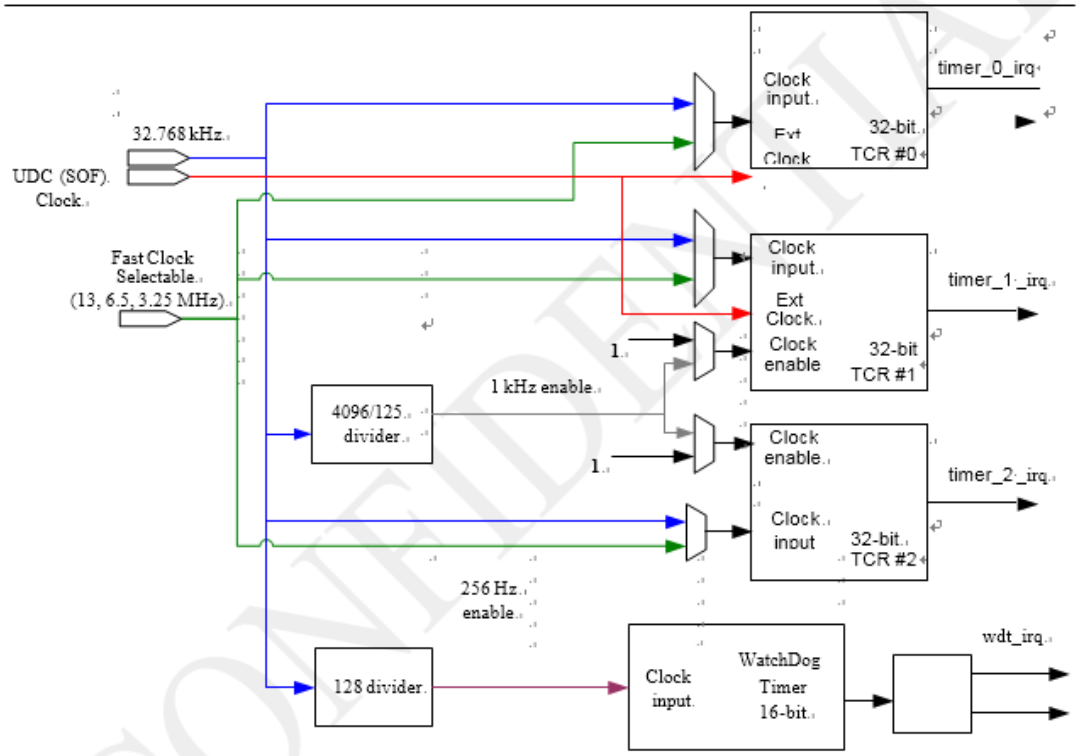
- Timer 1 operates at fast clock, slow clock of 32.768 kHz.
- Timer 2 operates at fast clock of 1 MHz (VCTCYCXO/13), slow clock of 32.768 kHz.

Each of the three Timer Count Registers (TCRn, where n is 1, 2, or 3) has three 32-bit Timer Match Registers (TMR_Tn_Mm) associated with it. When the value in the Operating System Count Register (TCRn) matches the value within any of the Match Registers, and the interrupt-enable bit is set, the corresponding bit in the Timer Status Register (TSR) is set. These bits are also routed to the Interrupt Controller where they are programmed to generate an interrupt. In addition, the 16-bit Watchdog timer operates using a timer module-derived clock of 256 Hz.

ASR CONFIDENTIAL

Reprogramming any Timer Control register (TCCR, TPLVRn, TMR_Tn_Mm, TPLCRn, TCMRn, or TILRn) while the timer is operating is not guaranteed to be valid. To safely reprogram any of these registers, disable the timer, reprogram it, and re-enable it. The functionality of one enabled timer is not affected by the programming of another timer.

Figure 16:Timers Block Diagram



Watchdog Timer (WDT) Functionality

When the value in the Watchdog timer matches the value in TWMR, and the TWMER[WME] bit is set, the wdt_rst_src# signal is asserted, creating a Watchdog-timer reset event in the system.

The Watchdog Reset mode is invoked when the software fails to prevent the Watchdog timeout from occurring properly. Watchdog resets are generated when the software is not executing properly and has possibly destroyed data. During Watchdog Reset mode, most internal registers in the system are reset to their defined default values. The exception is the real-time clock.

To avoid WDT reset, the software must restart the WDT before it matches the match value. Set the TWCR[WCR] field to restart the WDT.

ASR CONFIDENTIAL



Note

The TWSR register (and some other internal WDT flops), which determines whether a match event has occurred, is reset upon the assertion of power-on reset or external-master reset. All other registers are reset upon the assertion of power-on reset, external-master reset, or Watchdog-timer reset.

ASR CONFIDENTIAL

When a WDT reset event occurs, the WDT generates a 4 msec-wide pulse on the wdt_rst_src# output. As a consequence, the system is held in a state of reset for this period of time. The system enters a reset sequence when this reset signal is de-asserted.

Writing to all WDT registers is protected by two Access registers: TWFAR and TWSAR. Follow these steps to enable Writes to any WDT register:

1. Write the proper key value to the TWFAR register
2. Write the proper key value to the TWSAR register
3. Perform the Write operation on the preferred WDT register. Once this Write operation is complete, the WDT Registers are relocked. This sequence must be repeated to perform another Write operation.

8.3 Universal Subscriber ID Module (USIM) Controller

This section describes the Universal Subscriber Identity Module (USIM) Controller and registers that are supported by the ASR1606.

There is one instance of the USIM Controller in the ASR1606. Its signal names are preceded by "USIM_".

8.3.1 Overview

The USIM Controller is a primary device and communication interface for a CAT1 mobile handset. The USIM Controller supports communication with SmartCards as specified in the standard *ISO 7816-3* and technical specification *3G TS 31.101* of the 3rd Generation Partnership Project.

SmartCards are used in many applications and the CAT1 network. A USIM SmartCard is only one of many applications. SmartCards usually consist of CPU, Flash memory, and a serial-communication interface device similar to the one described in this section. More sophisticated SmartCards contain a PLL for frequency enhancement. Encryption accelerators also can exist in a SmartCard since many of their applications are security oriented. In all SmartCard applications, the physical layer and data link layer are identical. Familiarity with the standards cited in this section enables a better understanding of this section.

Software controls the session between the USIM Controller and the SmartCard by updating the USIM Controller registers. Choosing protocol type and parameters, receiving or sending a byte to/from the SmartCard, activating/de-activating the SmartCard, setting transmitter/receiver baud rates, and more, are accomplished via read/write operations to the USIM Controller registers. Transforming the byte convention (inverse to direct and vice-versa, according to the session convention) is performed within the USIM Controller. Therefore, software does not have to perform format inversion before a character is received. The USIM Controller provides functionality to support the above standards, but it is software that ensures the standards are met.

ASR CONFIDENTIAL

8.3.2 Features

- Compatible with any USIM SmartCard that is compliant with standard ISO 7816-3 and 3G TS 31.101 and operates in voltages of 1.8V or 3V
- Supports control lines for two-level voltage supply (1.8V and 3V)
- Supports USIM SmartCard reset pin control (using reset pin control and power supply control, warm/cold reset can be software initiated)
- Supports T=0 and T=1 protocols
- Programmable SmartCard clock frequency
- Supports any combination of the following clock-rate conversion factor F and bit-rate adjustment factor D:
  - F = {372, 512, 558}
  - D = {1, 2, 4, 8, 12, 16, 20, 32}
- Auto-error signal in T=0 Receive mode
- Auto-character repeat in T=0 Transmit mode
- Transforms inverted format to regular format, and vice-versa
- Programmable block-guard time period
- Programmable extra-guard time period
- Programmable character-waiting time period
- Programmable block-waiting time period
- Programmable timeout period
- Programmable CPU interrupt request on an error-signal detection
- Programmable CPU interrupt request when a SmartCard is connected

8.3.3 Signal Descriptions

This section describes the signal pins that are used by the USIM Controller. Table 18 lists all of the ASR1606 pins that connect between the USIM Controller and the USIM SmartCard. Figure 17 shows the connections.

ASR CONFIDENTIAL

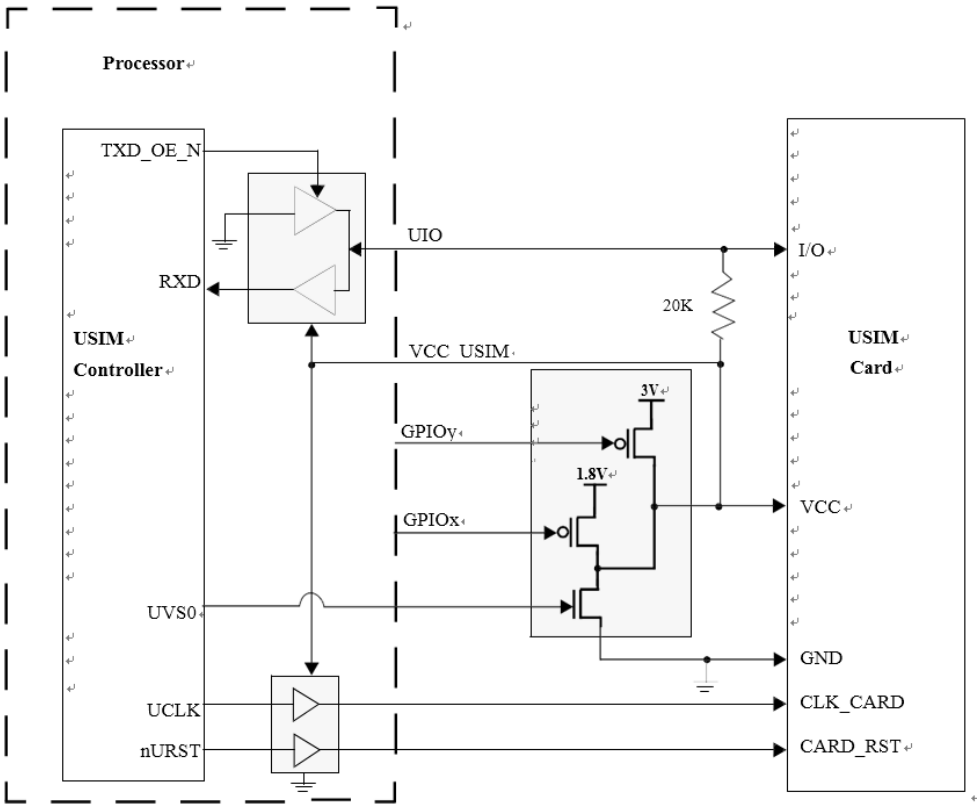
Table 18: USIM Controller Signals Summary

Unit Signal Name	Direction	Description
UIO	Bidirectional	USIM I/O Data - Receive and transmit data signal The bi-directional signal should be connected directly to the USIM SmartCard. When asserted, the UIO signal is forced to Vlow. When de-asserted, the UIO signal is pulled up by a 20K external resister. The USIM SmartCard can act as a pulldown on the UIO signal. **NOTE:** When the USIM SmartCard and the ASR1606 operate at different voltage levels, the I/O voltage level of the SmartCard must be a voltage level that the ASR1606 supports, that is, 1.8V or 3.0V.
UCLK	Output	SmartCard clock UCLK should be connected directly to the SmartCard clock pin. The SmartCard cannot use any other clock source.
URSTn	Output	SmartCard reset URSTn should be connected directly to the SmartCard reset pin: 0 = Resets the SmartCard 1 = Normal operation

The UPND bit engages/disengages a NMOS switch that pulls these pins to ground. Default value is 1 (NMOS in UCLK and URSTn is on).

ASR CONFIDENTIAL

Figure 17:Interfacing the USIM Controller to a USIM SmartCard



8.3.4 Operation

8.3.4.1 USIM SmartCard Interface Description

A SmartCard has three signal pins and two power supplies as listed in Table 19

ASR CONFIDENTIAL

Table 19:USIM SmartCard Pinout

SmartCard Pin	Function	Pin Direction	Device Pin
I/O	Data input/output (bidirectional) pin with a pull up transistor. See *ISO 7816-3* for pin specifications.	USIM SmartCard <-> ASR1606	UIO
SmartCard_RST	USIM SmartCard reset	ASR1606- > USIM SmartCard	URSTn

SmartCard Pin	Function	Pin Direction	Device Pin
CLK_SmartCard	Clock input Frequencies are between 1-5 MHz. See *ISO 7816-3* for pin specifications. SmartCards manufactured before April 2000 may have a frequency limitation of 4 MHz. Clock stops on low or high phase are supported.	ASR1606 -> USIM SmartCard	UCLK
VCC	SmartCard power supply Supplies 0V, 1.8V, 3V with max currents 0 mA, 30 mA, 50 mA, respectively according to SmartCard class (B,C). See *ISO 7816-3* for pin specifications.	Power supply -> SmartCard	VCC_USIM
GND	Mutual USIM SmartCard, SIM interface and VCC ground reference voltage	Power supply -> SmartCard	VSS_USIM

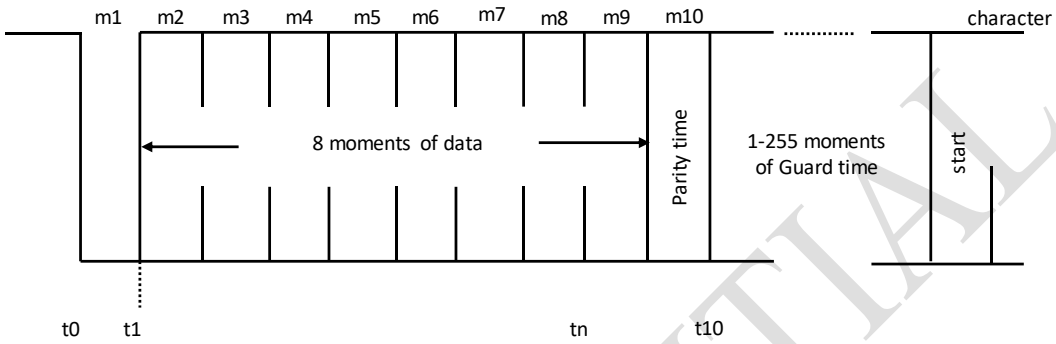
8.3.4.2 Coding Conventions

The USIM Controller performs serial-to-parallel conversion with optional format inversion on data characters received from the USIM SmartCard, and parallel-to-serial conversion with optional format inversion on data characters transmitted to the USIM SmartCard. Every byte sent or received consists of a start bit followed by an 8-bit data string, parity bit, and guard time. The period of time devoted for transmission or reception of a single bit is called either an *Elementary Time Unit (etu)* or a *moment*.

The start bit is mandatory. At least one guard bit after the parity is mandatory. Extra guard time is optional. The USIM Controller allows software to add up to 255 extra guard time moments. Refer to the USIM Extra Guard Time Register in the *ASR1606 Registers*.

ASR CONFIDENTIAL

Figure 18:Byte Sent Versus Time



Data bits may appear in two possible conventions as selected by the <Transmit/Receive Bit Order> and <Bit inversion> fields in the USIM Line Control Register (see the *ASR1606 Registers*):

ASR CONFIDENTIAL

- **Direct Convention**—Data bits sent on moments m2-m9 (see Figure 18) are transmitted LSB first-MSB last (that is, the least significant bit is transmitted on m2 and the most significant bit on m9). During moments m2-m10 (data and parity bits), transmission of V_{high} is interpreted as a logic 1, while transmission of V_{low} is interpreted as a logic 0.
- **Inverse Convention**—Data bits sent on moments m2-m9 (see Figure 18) are transmitted MSB first-LSB last (that is, the most significant bit is transmitted on m2 and the least significant bit on m9). During moments m2-m10 (data and parity bits), transmission of V_{low} is interpreted as a logic 1, while transmission of V_{high} is interpreted as a logic 0.

The first byte sent from the SmartCard to the USIM Controller, after reset de-assertion, determines the convention to be used throughout the entire session. A SmartCard that encodes and decodes data using the direct convention sends 0x3B. A SmartCard that uses the inverse convention sends 0x3F.

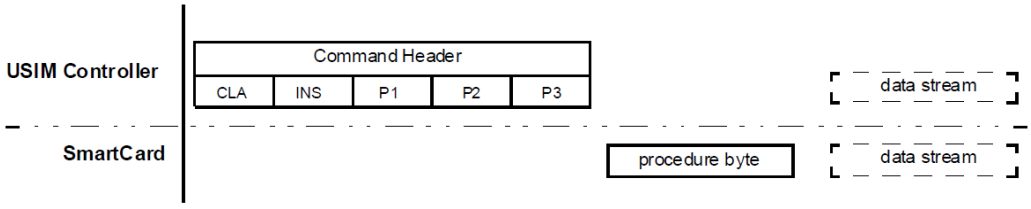
One of the following scenarios is possible when receiving this first character:

1. Both the USIM Controller and SmartCard are set to the same convention and the first character is received correctly (that is, received data was either 0x3F or 0x3B with no parity error).
2. USIM Controller and SmartCard are set to opposite conventions (that is, the USIM Controller asserts a parity error since it expects the opposite parity bit convention).

Protocols

Both T=0 and T=1 protocols, as defined in the standards ISO 7816-3 and 3G TS 31.101, are supported by the USIM Controller. The protocols for transmit and receive can be selected via the TX_T1 and RX_T1 bits, respectively, as described in the USIM Line Control Register in the *ASR1606 Registers*. The USIM Controller does not analyze or generate the data content or block structure that is transmitted or received using the protocols. It is the responsibility of software to generate and analyze the command headers and block structure to decode data meaning. The protocols begin after either the answer-to-reset (ATR) or a successful protocol and parameter selection (PPS) exchange. See the specifications for a more detailed explanation of the protocols.

Figure 19:T=0 Protocol Communication Method



ASR CONFIDENTIAL

Note

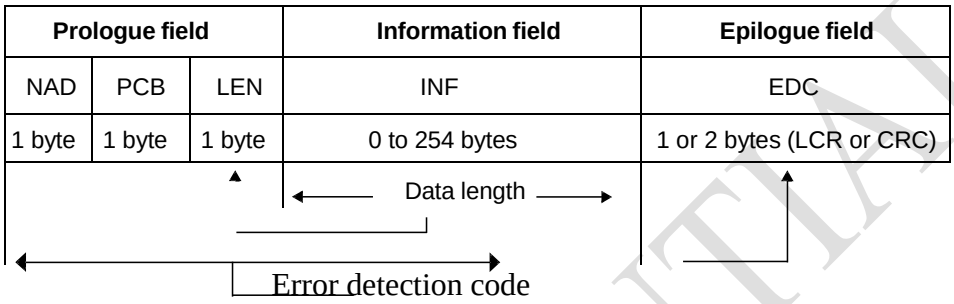
The data stream is transmitted either from the USIM Controller to the SmartCard or from the SmartCard to the USIM Controller, depending on the command type (read or write)

In the T=0 protocol, the USIM Controller sends a command to the SmartCard by software writing a five-byte command header to the TXFIFO. The header tells the SmartCard what to do. The software-controlled command processing continues with the transfer of a variable number of data bytes in one direction under the control of procedure bytes sent by the SmartCard. See Figure 20. The T=0 protocol allows for character retransmission when errors are detected. The protocol also requires two extra guard-time moments to follow the parity bit; these are inserted automatically by the USIM Controller.

ASR CONFIDENTIAL

Refer to the *ISO 7816-3* standard for the definition of the terms used in Figure 20.

Figure 20:Complete Block Structure for T=1 Protocol



In the T=1 protocol, data is sent via a sequence of bytes known as a block. By analyzing block content, software detects at any given time the proper direction of the data flow (Transmit or Receive). For example, software detects the length of the block sent from the SmartCard to the USIM Controller by analyzing the third byte of the received block (marked as a LEN byte). Each character has a special meaning according to its position in the block. There is no character retransmission in the T=1 protocol. Errors must be handled by software. The T=1 protocol requires one extra guard-time moment following the parity bit; this is automatically inserted by the USIM Controller. The USIM Controller supports DMA transfers in for long T=1 protocol blocks.

Receive and Transmit activity for each mode can be monitored using one of three techniques: interrupt, polled, or DMA. See Section FIFO Operation for more details.

Protocol switching can occur during the ATR, immediately after the ATR, or after a successful PPS. Unpredictable results may occur when changing the protocol under other scenarios.

Errors

The USIM Error Control Register and the USIM Interrupt Enable Register (as described in the
ASR1606 Registers) allow software to control how errors are handled.

In T=0 protocol, the following errors are signaled to the SmartCard when the USIM Controller is in receiver mode:

- The parity check fails, that is, exclusive OR-ing of 8 data bits and a parity bit yields the same value as the eps bit (see bit no.1)
- The RXFIFO is full and another byte is being received.

In T=0 protocol, the USIM Controller senses for an error signaled by the SmartCard

ASR CONFIDENTIAL

on the UIO pin.

T=0 Error The appearance of an error signal during transmission in T=0 mode is referred to as a T=0 error. While transmitting a character, the USIM Controller senses for an error signal, V_{low}, during the second mandatory guard time moment, on the UIO pin. If an error signal is sensed, the USIM Controller's transmitter resends the previous byte. When the number of "resends" meets the T=0 error trigger level, <T=0 Error Trigger Level (Threshold)> field in the USIM Error Control Register, transmission stops, an interrupt request is generated if enabled, and the <T=0 Error> field in the USIM Line Status Register is asserted. Software can then determine whether to proceed with the next byte or re-transmit the same byte via the control bits <Clear T=0 Error> and <Repeat Character Transmission> fields in the USIM Error Control Register, respectively. By writing a byte to the DUT Transmit Holding Register, thus allowing the T=0 retransmission to complete allows the T=0 error to clear. So the error cannot be cleared without allowing the retry to proceed to completion. In T=1 mode, there is no repetition of data, so this interrupt is not applicable.

ASR CONFIDENTIAL

Parity Error In T=0 mode, the USIM Controller automatically signals the SmartCard in the event of a parity error by placing V_{low} on the UIO pin during guard time, see Figure 21. The erroneous byte does not enter the RXFIFO. Retransmission by the SmartCard of the previous byte is expected, so no data is lost. When the number of repetitions meet the parity-error trigger level, <Parity Error Trigger Level> field in the USIM Error Control Register, an interrupt request is generated (if enabled) and the <Parity Error> field in the USIM Line Status Register is asserted. A parity error generates an error signal to the SmartCard regardless of the parity-error trigger value.

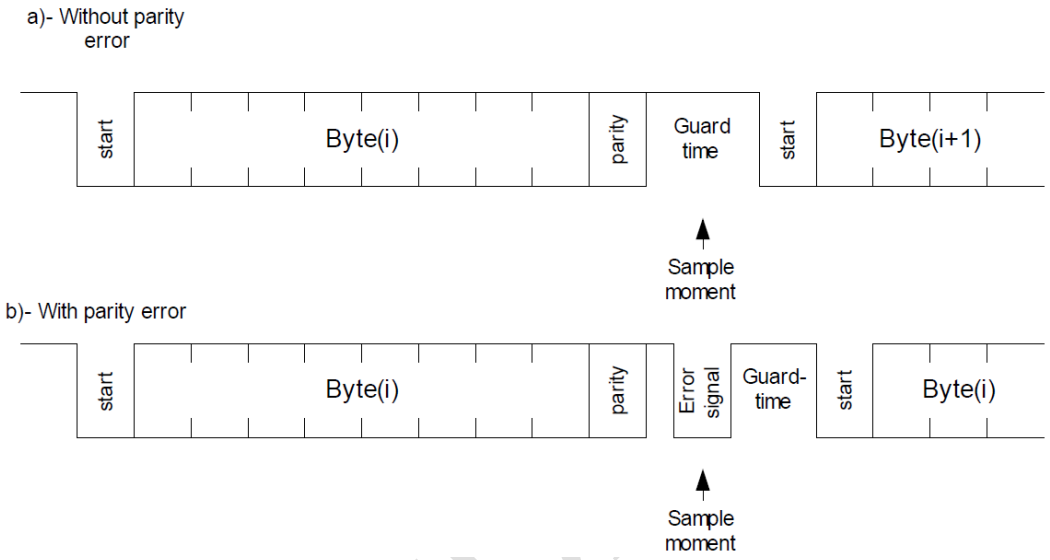
Note

Reception does not stop after a parity-error interrupt request is generated. If the parity error continues to repeat, the USIM Controller continues to signal the SmartCard on each occurrence. Software can either reset the SmartCard or receive the character with a parity error by switching to T=1 mode and then reverting back to T=0 mode. Software can check the <Parity Error> field in the USIM Receive Buffer Register to validate the character was received with parity via the T=1 mode.

In the T=1 protocol, error signaling or sensing is not performed. When a parity error occurs, the erroneous byte enters the RXFIFO, the <Parity Error> field in the USIM Line Status Register is asserted, and a parity-error interrupt request is generated if enabled. The parity error-trigger level has no effect in T=1 mode. The erroneous bytes are indicated in the RXFIFO by the assertion of <Parity Error> field in the USIM Receive Buffer Register (see the *ASR1606 Registers*). The total number of bytes with parity errors in the RXFIFO can be determined by the <Parity Error Number> field in the USIM FIFO Status Register.

ASR CONFIDENTIAL

Figure 21:T=0 Character Transmission Format



Note

The ISO spec requires multiple samples of UIO for each moment. Refer to Determining the Baud Rate and the USIM Factor Latch Register for more information on sampling requirements.

ASR CONFIDENTIAL

Overrun Error The RXFIFO can store up to 256 bytes. If the RXFIFO is not emptied before the 257th byte received from the SmartCard, the <Receiver Data Overrun Error> field in the USIM Line Status Register is asserted and an overrun interrupt request is generated (if enabled), and the 257th byte does not enter the RXFIFO. In T=0 mode, the USIM Controller automatically signals the SmartCard in the event of a parity error by placing $V_{low\ on\ the\ UIO\ pin\ during\ guard\ time}$ (see Figure 21).

Framing Error After a byte is received, the USIM Controller verifies that the $_{UIO}$ pin is kept at V_{high} during the last moment of mandatory guard time. If the UIO pin drops to V_{low}, the <Framing Error> field in the USIM Line Status Register asserts, and an interrupt request is generated if enabled (see Figure 22).

Figure 22:Framing Error



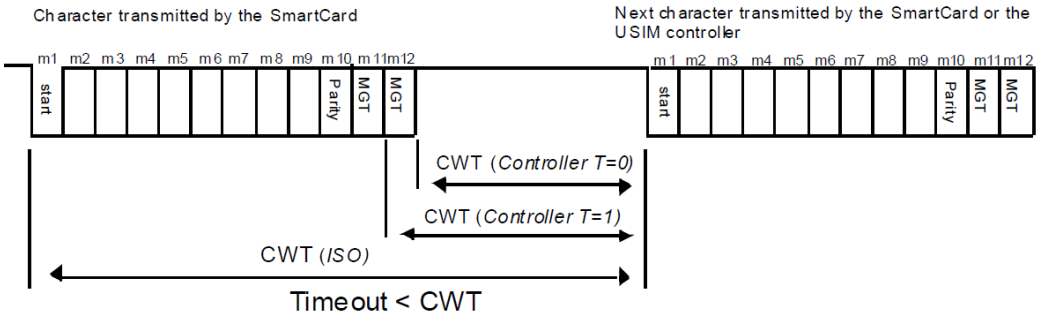
Waiting Times

Waiting times are protocol-defined parameters from standard *ISO 7816-3* and specification
3G TS 31.101. Waiting times can be selected in the USIM Character Waiting Time Register and the USIM Block Waiting Time Register (see the *ASR1606 Registers)*. To avoid unpredictable results, software must set the waiting times before changing the baud rate (see Programmable Baud Rate Generator).

Character Waiting Time (CWT) In the T=0 and T=1 protocols, the number of moments separating consecutive characters in the same block must not exceed the character waiting time (CWT). The ISO standard defines CWT as work waiting time (WWT) for the T=0 protocol. In T=0 mode, CWT is measured 12 etu after the start bit. In T=1 mode, the CWT is measured 11 etu after the start bit. See the USIM Character Waiting Time Register and Figure 23.

ASR CONFIDENTIAL

Figure 23:Character Waiting Time, USIM Controller



For T= 0: Mandatory Guard Time (MGT) requires at least 2 guard bits
For T= 1: Mandatory Guard Time (MGT) requires at least 1 guard bit

ASR CONFIDENTIAL

Note

The USIM Controller measures the CWT differently from the way it is defined in the ISO standard, but the effect is the same. Figure 23 shows the difference. Use the following formulas to convert the ISO standard CWT to CWT as used in the ASR1606 USIM Controller:

Equation 1: For T=0 CWT=CWT(ISO)-12

Equation 2: For T=1 CWT = CWT(ISO) – 11

As shown in Figure 23, CWT does not measure the time between two characters transmitted by the USIM Controller but instead measures the time after the USIM Controller receives a character transmitted by the SmartCard. Software ensures that data is delivered continuously and that the upper software layers (data, application, and more) are standard compliant.

A timeout interval can be specified by software writing to the USIM Time-Out Register (see the *ASR1606 Registers*). Measurement of the timeout interval begins after reception of a character is complete.Measurement stops with the renewal of transmit or receive activity before the timeout interval has expired or when the timeout interval expires. Upon expiration of the timeout interval, the <Character Waiting Time> field in the USIM Line Status Register asserts and a CWT interrupt request is generated, if enabled.

If the CWT is less than the block guard time described in Block Guard Time (BGT), a CWT violation occur each time the USIM Controller transmits data after the SmartCard has finished transmitting data. This should not be an issue for the T=0 protocol since the ISO specification defines a WWT/CWT of 9600 etu. For the T=1 protocol, this may be an issue since the 3G specification defines 12 etu<CWT<43 etu. Ignore CWT violations when the USIM Controller is transmitting data.

When using the CWT interrupt request while receiving data, software can identify either the end of a block or loss of communication synchronization (that is, software expects more characters to come after the SmartCard finishes transmitting). By using a CWT interrupt request while transmitting data, software can monitor performance by verifying that the transmission was not stopped for more than the CWT etu.

ASR CONFIDENTIAL

Block Guard Time (BGT) The T=0 and T=1 protocols require different guard times between bytes transmitted in opposite directions. This guard time is called *block guard time (BGT)*. Software specifies the block guard time to between 0-255 moments by writing to the USIM Block Guard Time Register (see the *ASR1606 Registers)*. The USIM Controller automatically ensures that BGT has elapsed before transmission is initiated. The USIM Controller is ready to receive a character immediately after transmission even when the SmartCard does not obey BGT. In T=0 mode BGT is measured 12 etu after the start bit. In T=1 mode the BGT is measured 11 etu after the start bit. See the USIM Block Guard Time Register in the *ASR1606 Registers* and Figure 23.

Note

The USIM Controller measures the BGT differently from the way it is defined in the ISO standard, but the result is the same. Figure 23 shows the difference. Use the following formulas to convert the ISO standard BGT to BGT as used in the ASR1606 USIM Controller:

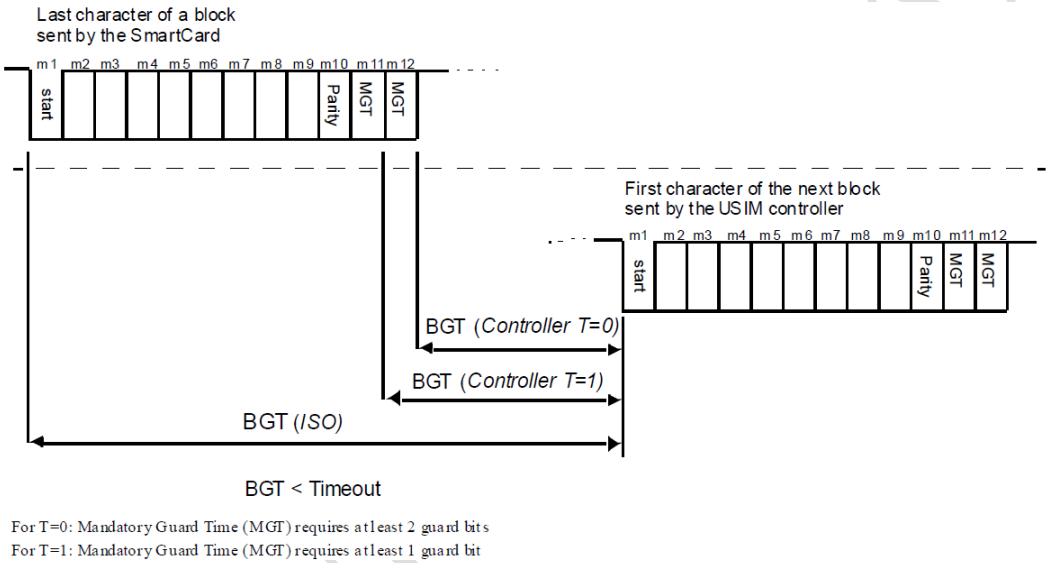
Equation 3: For T=0 BGT = BGT(ISO) – 12

Equation 4: For T=1 BGT = BGT(ISO) – 11

ASR CONFIDENTIAL

When receiving a character, the USIM Controller does not initiate any transmission until the character is fully received and the block guard time has elapsed (even if the TXFIFO is full). As soon as the BGT has elapsed, the USIM Controller starts transmitting. Transmission continues until the TXFIFO is emptied or the FCR[TX_HOLD] bit is asserted.

Figure 24:Block Guard Time, USIM Controller



Block Waiting Time (BWT) When the SmartCard is expected to reply, it should do so no later than the *block waiting time* (BWT) interval. In T=0 mode, BWT is measured 12 etu after the start bit. In T=1 mode, BWT is measured 11 etu after the start bit. See the USIM Block Waiting Time Register in the *ASR1606 Registers* and Figure 25.

ASR CONFIDENTIAL

Note

The USIM Controller measures the BWT differently from the way it is defined in the ISO standard, but the result is the same. Figure 25 shows the difference. Use the following formulas to convert the ISO standard BWT to BWT as used in the ASR1606 USIM Controller:

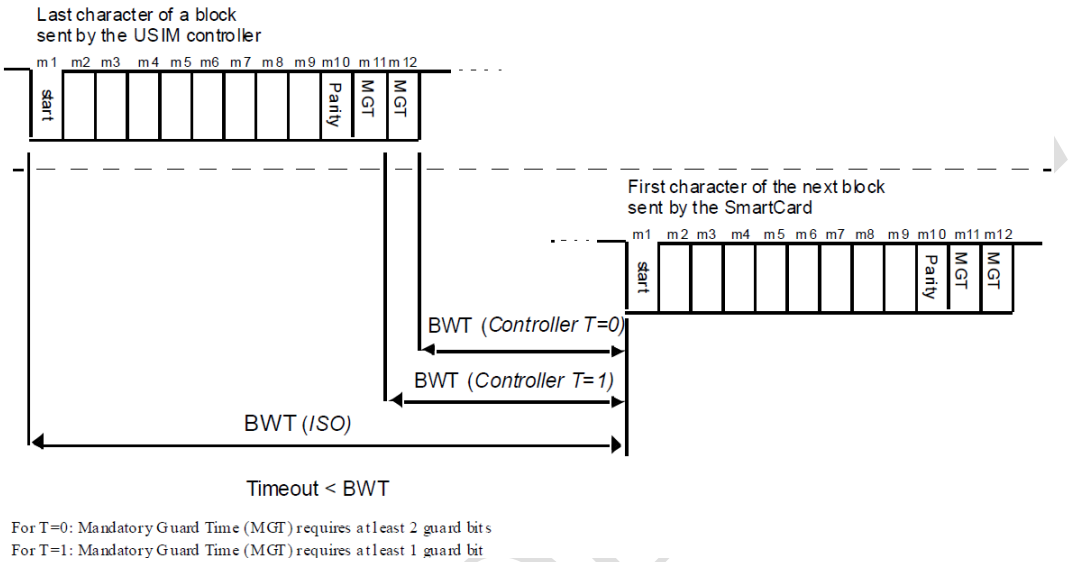
Equation 5: For T=0 BWT = BWT(ISO) – 12

Equation 6: For T=1 BWT = BWT(ISO) – 11

A timeout interval can be specified by software writing to the USIM Time-Out Register (see the *ASR1606 Registers*). Measurement of the timeout interval begins after a SmartCard reset or after the end of a transmission. Measurement stops with the renewal of Transmit or Receive activity before the timeout interval has elapsed or when the timeout interval has elapsed. Upon expiration of the timeout interval, the <Block Waiting Time> field in the USIM Line Status Register asserts and a BWT interrupt request is generated, if enabled. BWT is useful in detecting an unresponsive SmartCard.

ASR CONFIDENTIAL

Figure 25:Block Waiting Time, USIM Controller



For T=0: Mandatory Guard Time (MGT) requires at least 2 guard bits
For T=1: Mandatory Guard Time (MGT) requires at least 1 guard bit

8.3.4.3 Clock Control

The USIM Controller generates and controls the clock (UCLK) that it supplies to the SmartCard based on the values in the USIM Clock Register in the *ASR1606 Registers*. The frequency of the internal clock (CLK_USIM) that drives the USIM Controller internal logic is 48 MHz as noted in Equation 7. UCLK is defined in Equation 8.

Equation 7: USIM Internal Clock

$$CLK_USIM = 48\ MHz$$

Equation 8: USIM SmartCard Clock

$$UCLK = \frac{CLK_USIM}{CLKR[DIVISOR] \times 2}$$

The *ISO 7816-3* and *3G TS 31.101* specifications limit UCLK to a range of 1-4 MHz. UCLK is set to 4 MHZ after a system reset. ASR recommends that the SmartCard clock frequency be changed only immediately after the answer-to-reset (ATR) or immediately after a successful PPS. See the ISO and 3G specifications for more information on the ATR and PPS.

Changing the SmartCard clock affects the etu period. Software can write a new value to the USIM Clock Register only when the <Clock Change Request> field in it is de-asserted. The USIM Clock Register does not update when the <Clock

ASR CONFIDENTIAL

Change Request> field is asserted. The <Clock Change Request> field ensures that the USIM Controller has completed all pending transmissions or receptions, and the block guard time, before UCLK is changed.

Clock Stop

It is possible to stop UCLK. The voltage level of UCLK while it is stopped can be selected via the
<Stop Level> field in the USIM Clock Register. Asserting the <Stop SmartCard Clock> field in the USIM Clock Register stops UCLK only after the USIM Controller completes all pending transmission, reception, and block-guard time. When de-asserting the <Stop SmartCard Clock> field, UCLK is re-activated.

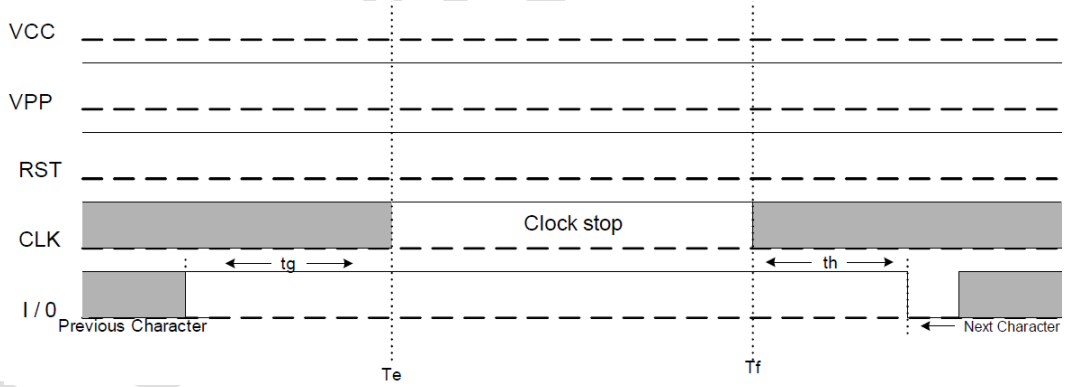
ASR CONFIDENTIAL

Note *ISO 7816-3* defines guard-time periods before stopping the clock and after re-activating it, as specified in Figure 26

For power savings, the USIM Controller internal clock, CLK_USIM, can be disabled by setting the
<Stop USIM Controller Internal Clock> field in the USIM Clock Register. After system reset, CLK_ USIM is enabled by default.

Note The USIM Controller registers can still be read or written when CLK_USIM is disabled.

Figure 26:SmartCard Clock Stop



ASR CONFIDENTIAL

Note

tg and th have minimum values:

- tg > 1860 UCLK cycles
- th > 700 UCLK cycles
- tg and th define a time interval while Te and Tf define a point in time.

Programmable Baud Rate Generator

The USIM Controller contains a programmable baud rate generator. The same baud rate is used for transmitting and receiving data. Although the baud rate can be configured at any given time, the actual change occurs only when: the TXFIFO is empty, no Receive or Transmit activities are in process, and the block guard time has elapsed. The Divisor Latch Register should be written when changing the baud rate even if the value is the same (see Handling a PPS).

Determining the Baud Rate When setting a new baud rate, software must specify three parameters in the following order:

1. **RATIO**—Ratio of the internal clock (CLK_USIM) versus the SmartCard clock (UCLK), see Equation 9. UCLK is defined in the USIM Clock Register in the *ASR1606 Registers*.

ASR CONFIDENTIAL

2. **FACTOR**—Number of samples/bit minus one (the ISO standard specifies a minimum of six samples/bit), see Figure 27. This parameter is loaded in the USIM Factor Latch Register in the *ASR1606 Registers*.
3. **DIVISOR**—Number CLK_USIM cycles between samples, see Figure 29. This parameter is loaded in the USIM Divisor Latch Register in the *ASR1606 Registers*.

Figure 27: Baud Rate Sampling Pulses When Number of Samples per Bit is 6

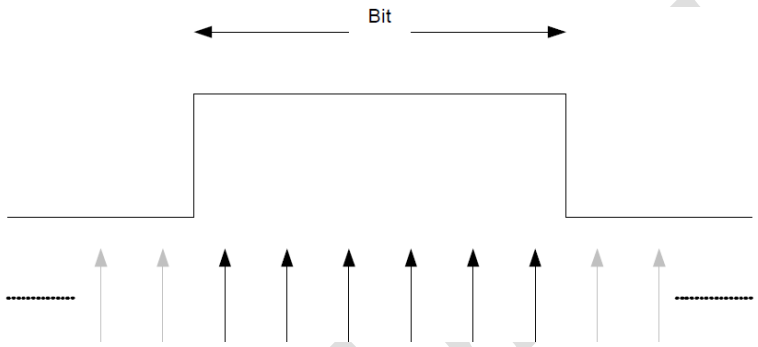
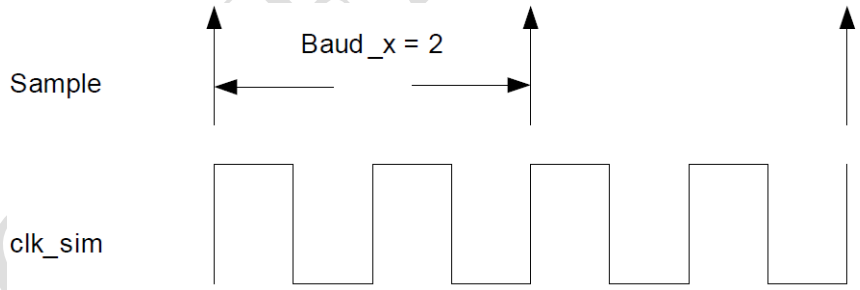


Figure 28:Spacing Between Samples When Baud Divisor is 2



The baud rate, defined in Equation 11, is determined by using the above parameters with the following equations. The F and D parameters in Equation 10 are delivered by the SmartCard in the answer-to-reset (ATR) or negotiated in the PPS defined in the ISO standard.

Equation 9:
$$RATIO = \frac{CLK_USIM}{UCLK \times 2}$$

Equation 10:
$$\frac{DIVISOR \times (FACTOR + 1)}{RATIO \times 2} = \frac{F}{D}$$

ASR CONFIDENTIAL

Equation11:

$$\text{BaudRate} = \frac{\text{CLK_USIM (in Hz)}}{(\text{DIVISOR}) \times (\text{FACTOR} + 1)} = \frac{\text{UCLK} \times \text{RATIO} \times 2}{(\text{DIVISOR}) \times (\text{FACTOR} + 1)}$$

Handling a PPS After a successful PPS, the USIM Controller is ready to start transmitting with the new protocol parameters. When the USIM Divisor Latch Register is written, the USIM Controller restarts the counting of all waiting times (BGT, CWT, EGT, and BWT). So, software must set the waiting times before writing to the USIM Divisor Latch Register. The USIM Factor Latch Register should also be written before the USIM Divisor Latch Register is written. To restart the baud rate, software must write to the USIM Divisor Latch Register even if the value has not changed. The USIM Controller starts transmitting only after the new BGT has elapsed.

Examples of Setting the Baud Rate The following two examples cover situations where F divided by D produces an integer number and where F divided by D does not produce an integer number.

Example 1: Setting a New Baud Rate—F/D is an Integer Number

Consider the case where the F and D parameters that were delivered by the SmartCard in the TA(1) byte of the ATR are 512 and 32, respectively. The USIM Controller is expected to start the session in the new baud rate specified by the SmartCard. Since F divided by D is an integer number, F/D=16, any value can be set to *RATIO* that results in a SmartCard clock that is approved by the ISO standard. For this example, set *RATIO* = 6, resulting in UCLK = 4 MHz clock to the SmartCard. Now, software must select a *DIVISOR* and a *FACTOR* that obey Equation 10:

$$\frac{\text{DIVISOR} \times (\text{FACTOR} + 1)}{6 \times 2} = 16$$

Possible combinations are *DIVISOR* = 12 and *FACTOR = 15 or DIVISOR* = 24 and *FACTOR = 7*. Other combinations are allowed by the ISO standard as long as the number of samples per bit (*FACTOR)* is not less than 5.

With UCLK = 4 MHz, the baud rate is

$$\frac{4\ \text{MHz}}{16} = 250\text{kHz}$$

ASR CONFIDENTIAL

Example 2: Setting a New Baud Rate—F/D is Not an Integer Number

Consider the case where F and D parameters that were delivered by the SmartCard in the TA(1) byte of the ATR are 372 and 20, respectively.

Because, $\frac{F}{D} = 18\frac{3}{5}$ set *RATIO = 10.*

(*RATIO = 5* is also possible, but not all SmartCards operate at a clock frequency that is greater than 4 MHz). Now, software must set *DIVISOR* and *FACTOR* to obey

Equation 10:

$$\frac{\text{DIVISOR} \times (\text{FACTOR} + 1)}{10 \times 2} = \frac{372}{20}$$

A possible combination is *DIVISOR* = 31 and FACTOR=11

The baud rate is $\frac{48\ \text{MHz}}{31 \times 12} = 192.032\ \text{kHz}$.

SmartCard Management

The USIM SmartCard Control Register (see the *ASR1606 Registers*) allows the USIM Controller to:

- Specify the USIM SmartCard voltage supply
- Assert reset (URSTn) to the SmartCard
- Specify what it places on the UIO pin when it is transmitting to the SmartCard
- Force UIO to V_{low}.

Software uses this register to manage startup and shutdown of the SmartCard. In particular, it is used while executing the following routines:

ASR CONFIDENTIAL

SmartCard activation (“cold reset”)
SmartCard warm reset
SmartCard de-activation

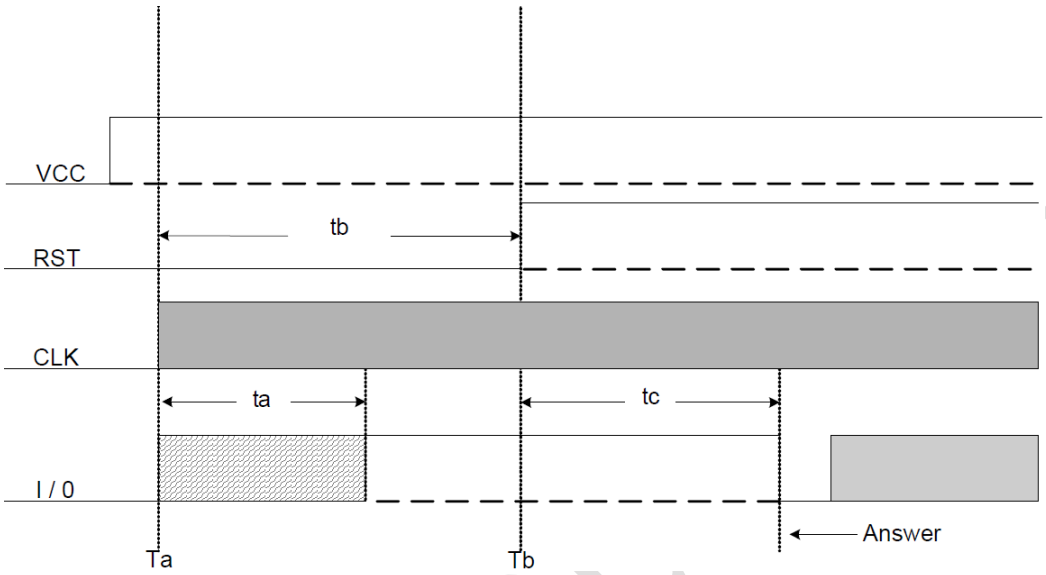
SmartCard Activation (“Cold Reset”)

Perform the following operations in sequence to activate the USIM SmartCard (see Figure 29):

1. Reset the <Stop USIM Controller Internal Clock> field in the USIM Clock Register (after a system reset, this bit is reset to its reset value of 1 by default and so this step can be skipped)
2. Clear The <SmartCard Reset> field in the USIM SmartCard Control Register.
3. Turn on the VCC voltage by setting the <SmartCard Voltage> field in the USIM SmartCard Control Register. On first activation, set the lowest voltage level (set to 0x2)
4. Enable the SmartCard I/O pin to return to V_{high} by clearing the <Force TXD> field in the USIM SmartCard Control Register.
5. Activate the SmartCard clock by clearing the stop bit, <Stop SmartCard Clock> field in the USIM Clock Register.
6. Verify that <SmartCard Reset> field was asserted for at least 400 UCLK cycles. When UCLK is set to the lowest frequency allowed by the ISO standard, 400 cycles take 0.4 ms (400 µs). See tb in Figure 29.
7. De-assert URSTn by setting the <SmartCard Reset> field. The SmartCard answers to reset (ATR) within 400-40000 SmartCard clock cycles. See tc in Figure 29. If an ATR is not returned after 40000 SmartCard clock cycles, then repeat steps 1-7 with VCC voltage level increased from 1.8V to 3.0V in step 3 (set the <SmartCard Voltage> field to 0x1).

ASR CONFIDENTIAL

Figure 29:SmartCard Activation



Warm Reset

Warm reset is performed to reset the SmartCard after activation. A SmartCard reset does not reset the USIM Controller RXFIFO and TXFIFO. The RXFIFO and TXFIFO are reset by writing 1 to the <Reset RXFIFO> and <Reset TXFIFO> fields in the USIM FIFO Control Register, respectively. Perform the following operations in sequence to execute a warm reset:

1. Clear the <SmartCard Reset> field in the USIM SmartCard Control Register.

ASR CONFIDENTIAL

2. Verify that <SmartCard Reset> field was cleared for at least 400 UCLK cycles. When UCLK is set to the lowest frequency allowed by the ISO standard, 400 cycles take 0.4 ms (400 µs).
3. De-assert URSTn by setting the <SmartCard Reset> field.

SmartCard De-activation

Perform the following operations in sequence to de-activate the USIM SmartCard:

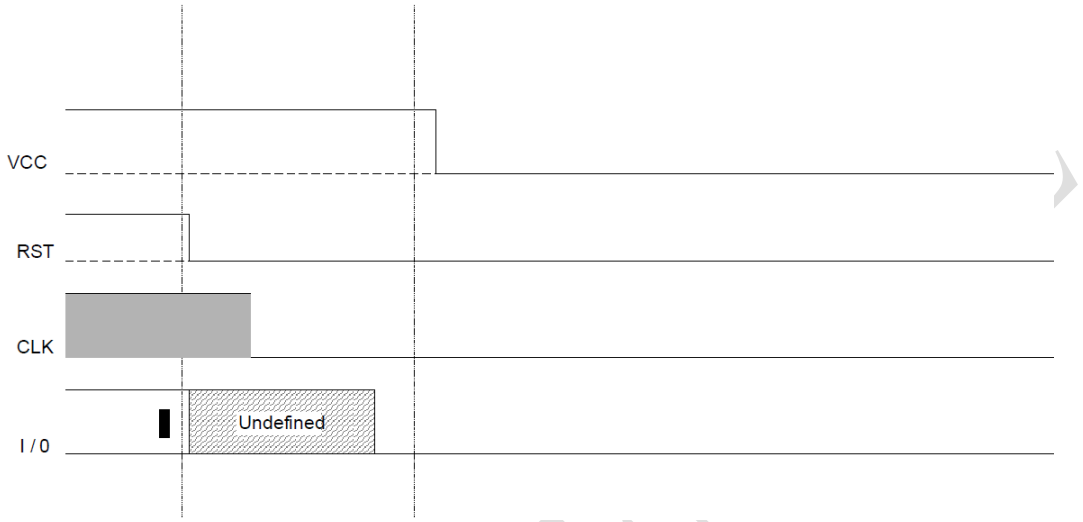
1. Clear the <SmartCard Reset> field in the USIM SmartCard Control Register.
2. Stop the clock on V_{low} by clearing the <Stop Level> field in the USIM Clock Register, and setting the <Stop SmartCard Clock> field in the USIM Clock Register.
3. Force the UIO pin to ground level by setting <Force TXD> field in the USIM SmartCard Control Register.
4. Turn the VCC voltage to ground level by setting the <SmartCard Voltage> field in the field in the USIM SmartCard Control Register to 0x0 for at least 200 UCLK cycles after the UIO pin is forced low. The UIO pin can be monitored via the <Reflects Sampled Data from UIO Pin> field in the USIM Line Status Register. An operating system timer can be used to determine the amount of time that elapsed since <Reflects Sampled Data from UIO Pin> field went low.
5. Set the <Stop USIM Controller Internal Clock> field in the USIM Clock Register for power savings (see the *ASR1606 Registers*).

Note

When set, the <Clock Change Request> field in the USIM Clock Register indicates that a SmartCard clock change is occurring. Any write to the USIM Clock Register is ignored while the <Clock Change Request> field is set.

ASR CONFIDENTIAL

Figure 30:SmartCard De-activation



8.3.4.4 FIFO Operation

The USIM Controller has two FIFOs: an RXFIFO (256-byte deep) and an TXFIFO (16-byte deep). Each FIFO is accessed via the USIM Controller RBR and THR Register, respectively. The USIM FIFO Control Register allows software to:

- Reset the FIFOs
- Hold transmission
- Mask parity errors in the RXFIFO
- Set the trigger levels for each FIFO

The FIFOs can be serviced via interrupt-request mode, polling mode, or DMA mode.

Interrupt Request Mode

In Interrupt Request mode, software tracks and handles FIFO activities via interrupt requests. The interrupt requests are enabled via the USIM Interrupt Enable Register in the *ASR1606 Registers*. The source that caused the interrupt request can be determined by reading the USIM Interrupt Identification Register in the *ASR1606 Registers*.

Receiver Related Interrupts During data reception, LSR[RX_WORKING] is set and the following can occur when enabled:

- **Receiver Data Ready Interrupt**—This interrupt indicates the number of bytes in the RXFIFO is equal to or greater than the trigger level specified in the <Receiver Interrupt Trigger Level (Threshold)> field in the USIM FIFO

ASR CONFIDENTIAL

Control Register. The interrupt request is cleared when the number of bytes in the RXFIFO drops below the <Receiver Interrupt Trigger Level (Threshold)> field value. Empty the RXFIFO by reading the USIM Receive Buffer Register.

- **Character Waiting Time Interrupt**—This interrupt is an ISO protocol parameter that indicates that the maximum delay between the leading edges of two consecutive characters has been violated. See Section, Character Waiting Time (CWT) and the ISO standard for full details. The interrupt request is cleared by setting <Character Waiting Time Interrupt> field in the USIM Interrupt Identification Register.
- **Receiver Time Out Interrupt**—This interrupt helps software to detect when trailing bytes are left in the RXFIFO for some time with no additional RX activity. It occurs when at least one character remains in the RXFIFO and the timeout period programmed into the USIM Time-Out Register in the *ASR1606 Registers*, has expired. The interrupt request is cleared by:
  - Reading the RBR Register
  - Resetting the RXFIFO
  - Receiving another character before the timeout interval has elapsed.
- **Framing Error Interrupt**—This interrupt occurs when the UIO pin is asserted low during guard time. See Section, Framing Error for full details. The interrupt request is cleared by writing a 1 to the IIR[FRAMERR] bit.
- **Parity Error Interrupt in T=0 mode**—This interrupt occurs when the received data contains a parity error. See Section, Parity Error for full details. The interrupt request is cleared by writing a 1 to the IIR[PERR] bit.
- **Parity Error Interrupt in T=1 mode**—This interrupt occurs when the received data contains a parity error. See Section, Parity Error for full details. The interrupt request is cleared by writing a 1 to the IIR[PERR] bit.
- **Receiver Data Overrun Interrupt**—This interrupt occurs when the RXFIFO is full and cannot accept any more data. See the Note on page 116 for full details. The interrupt request is cleared by writing a 1 to the IIR[OVRN] bit.

Transmitter Related Interrupts During transmission, the LSR[TX_WORKING] bit is set and the following can occur when enabled:

- **Transmitter Data Refill Interrupt**—This interrupt indicates that the number of bytes in the TXFIFO is less than the trigger level specified in the FCR[TX_TL] field. The interrupt request is cleared when the number of bytes in the TXFIFO is increased to a point where the TXFIFO is at or above its trigger level.
- **Block Waiting Time Interrupt**—This interrupt is an ISO protocol parameter that indicates that the maximum delay between the leading edge of the last character of a block sent by the USIM

  Controller and the leading edge of the first character of the next block sent by the SmartCard has been violated. See Section, Block Waiting Time (BWT)

ASR CONFIDENTIAL

and the ISO standard for full details. The interrupt request is cleared by setting IIR[BWT].

- **T=0 Error Interrupt**—In T=0 mode, this interrupt indicates that the SmartCard has signaled the USIM Controller transmitter to repeat the transmission. See Section, T=0 Error for full details. The interrupt request is cleared via the T0_REPEAT or T0_CLR bits in the Error Control Register.

Other Interrupts . **SmartCard Detection Interrupt**—This interrupt indicates that a SmartCard has been inserted into its connector. The interrupt remains asserted until the it is cleared. The interrupt only asserts upon detection of a new SmartCard. The interrupt request is cleared by writing a 1 to the <SmartCard Detection> field in the USIM Interrupt Identification Register.

Polled Mode

In Polled mode, interrupt and DMA requests are disabled. FIFOs are accessed by software reading and writing the USIM Receive Buffer Register and USIM Transmit Holding Register, respectively. Software checks the receiver and transmitter status by reading the USIM Line Status Register and USIM FIFO Status Register. The USIM FIFO Status Register in the *ASR1606 Registers*, holds information regarding FIFO status. The USIM Extra Guard Time Register in the *ASR1606 Registers*, contains status information on data transfers. Since the receiver and the transmitter are controlled separately, either one or both can operate in Polled mode.

DMA Request Mode

In DMA Request mode, data is entered and removed from the FIFOs via DMA requests. DMA requests are enabled in the USIM Interrupt Enable Register in the *ASR1606 Registers*. The USIM Controller has two DMA requests: one for TXFIFO data service and one for RXFIFO data service.

Note

The FIFO trigger levels in the FIFO Control Register must be set to 8 bytes to avoid unpredictable results when working in DMA mode. Other trigger levels are not supported in this mode.

ASR CONFIDENTIAL

Transmit Data Service When the DMA_TX request is enabled in the USIM Interrupt Enable Register and the number of bytes in the TXFIFO is below eight, the DMA transmit request is generated. The DMA controller then writes data to the TXFIFO. The DMA controller must be configured to 8-byte burst size, with data width of 1 byte. Each write by the DMA to the USIM Transmit Holding Register adds a single byte to the TXFIFO. The number of bytes to be transmitted is programmed in the DMA controller.

Receive Data Service The DMA Receive request is generated in two situations:

- When the DMA_RX request is enabled in the USIM Interrupt Enable Register and the number of bytes in the RXFIFO is equal to or greater than eight
- A time-out situation occurs when:
  - The <Enable Dma Receiver Requests In Event Of Time-out Situation> field is enabled in the USIM Interrupt Enable Register and
  - The RXFIFO is not empty but the trigger level was not reached and
  - The RBR was not accessed before the timeout period (defined in the USIM Time-Out Register in the *ASR1606 Registers*) expired.

A DMA Receive request is generated to remove trailing bytes from the RXFIFO.

When one of the above situations occurs, the DMA controller reads data from the RXFIFO. For each DMA request, the DMA Controller can read up to 8 bytes of data. The number of bytes to be read is programmed in the DMA Controller.

Software ensures that any remaining data in the RXFIFO is handled properly when the DMA reaches the end of its chain. This should not be an issue since software detects how much data is being received via the ISO 7816-3 protocol. The USIM Controller indicates an end-of-receive (EOR) status whenever the TOR or CWTR timeouts have expired and the DMA is reading the last byte in the RXFIFO.

To avoid having the DMA stop servicing the current chain when an EOR occurs and the end of a block has not yet been reached (recommended), software sets up the DMA Channel Control/Status Register, DCSR, as noted in Table 20.

Table 20:DCSR Setup to Ignore EOR

DCSR Bit Name	Bit Value	Comment
StopIrqEn	1	Interrupt when the descriptor is finished
EORIrqEn	0	No DMA interrupt after an EOR
EORJmpEn	1	DMA services another channel if an EOR occurs
EORStopEn	0	Keep waiting for more bytes until the descriptor is finished

For more information, refer to the DMA Channel Control/Status Registers 0-31 in

ASR CONFIDENTIAL

the DMA Controller Registers section in the *ASR1606 Registers*.

Do not use DMA for transferring trailing bytes from the Receive FIFO. Always use PIO to transfer the trailing bytes. The <Enable DMA Receiver Requests In Event Of Time-out Situation> field must be cleared to disable the generation of EOR for the DMA.

Special Condition:

In T=1 DMA mode, parity errors enter the RXFIFO. A DMA Receive request results in reading only the first 8 bits from the RBR, the parity error bit is not read. To determine if a block contains a parity error in this mode, software follows one of two possible approaches shown below. Each approach requires the DMA interrupt to be asserted. For more details on the DMA interrupt, refer to Section Servicing Internal Peripherals.

- With the parity-error interrupt disabled, the DMA reads the entire block regardless of parity errors. After the block is read, the DMA interrupt occurs and software checks LSR[PERR] bit to determine if the block contains any parity errors.
- When a parity error occurs, the parity-error software interrupt-service routine waits for the DMA interrupt. Upon receiving the DMA interrupt, indicating that the whole block has been read, software then requests retransmission of the block.

8.4 Register Descriptions

Refer to the *ASR1606 Registers* for details on the USIM registers.

ASR CONFIDENTIAL

9. Power Management and Clocks

This section describes the following:

- Section 9.1, Power Management Units
- Section 9.2, Power Domains
- Section 9.3, Power States
- Section 9.4, Clocks
- Section 9.5, Reset

9.1 Power Management Units

Control of the power modes and system clocks are distributed in the ASR1606. There are 3 Power Management Units (PMUs) as listed below:

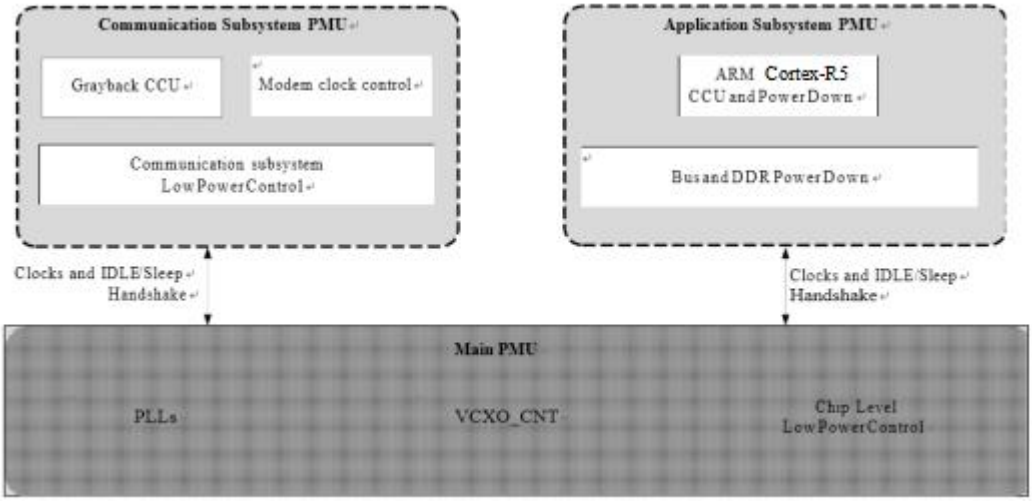
- **Main PMU (MPMU)** - Manages the PLLs, chip level power states, system wake up, main clock distribution and gating, and software controlled system reset
- **Application Subsystem PMU (APMU)** - Manages subsystem power states, clocking and reset of major Application Subsystem units, and initiating frequency change requests
- **Communication PMU (CPMU)** - Manages the power states of major Communication Subsystem units. Also contains the Communication Subsystem Clock Control Unit registers.

The main power and clock management components of the processor are shown in Figure 31.

ASR CONFIDENTIAL

Copyright © 2022 ASR

Figure 31:ASR1606 Power Management Blocks



9.1.1 Main PMU (MPMU)

The Main PMU is responsible for chip-level power management. Once the Application Subsystem PMU and CPMU enter the lowest power state, the Main PMU takes control of the chip-level low- power states.

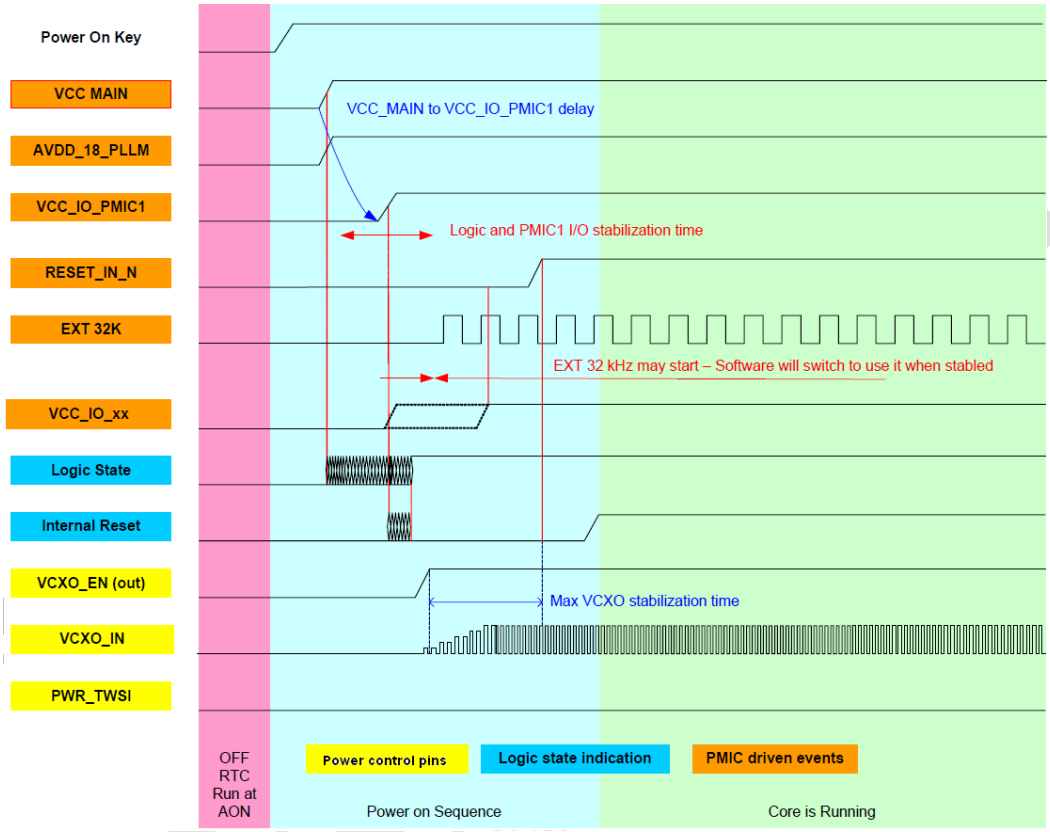
9.1.1.1 Power-up Sequence

The power-up sequence of ASR1606 processor is as follows:

1. Power on reset is asserted by PMIC.
2. PMIC turn on the power supply to core logic first and then to the external IO.
3. The PMIC turn on VCXO_EN.
4. An external reset is driven by PMIC for at least 7ms for VCXO/DCXO and clock limiter stabilization. After the clock is stable, the external reset is de-asserted by PMIC and VCTCXO is stable for the ASR1606 processor, and the processor also uses this clock to generate a pseudo 32 kHz clock.
5. PLL1 is power on with the default factors.
6. The PMU inside ASR1606 processor waits until PLL1 is locked.
7. The PMU provides clock to the application subsystem. The Ceva BX2 and APB clocks to all APB peripherals are off and are turned on by software.
8. The PMU negates the system reset and, upon fuse definition, releases reset of the ARM Cortex-R5 subsystem CPU.
9. Seagull begins to run code from the boot ROM; it releases the reset signal of Ceva BX2 and baseband logic and enables the other system clocks, as required.

ASR CONFIDENTIAL

Figure 32:Power on Sequence Timing



9.1.1.2 Wake-Up Logic

The PMU contains a decoder to detect sleep wake-up events, the decoder has eight level-sensitive, active-high, maskable inputs. Each can trigger a wake up from the Sleep state.

PMU wakeup events are shown in Figure 33. For each one of the inputs in Figure 33, there is a mask bit.

AWUCRM(in MPMU) and CWUCRM(in CPMU) are the mask registers for the wakeup events to resume PLL clock and wake up communication subsystem. To mask a specific event, the associated bit in both AWUCRM and CWUCRM registers must be cleared.

There are two types of events involved in these registers: pm_xsc_clkres and commtop_wake. Pm_xsc_clkres resumes the PLL clock and wakes up the chip level and AP subsystem from sleep state.

The commtop_wake wakes up the communication subsystem from Idle or Powerdown state. If it is 0, combined with the voting register, communication subsystem may enter Idle or Powerdown mode.

Bits [27:26] mask the communication subsystem wakeup event. The other bits

Copyright © 2022 ASR

are for pm_xsc_clkres masking (note: there is some redundancy).The bit descriptions are defined in Table 21.

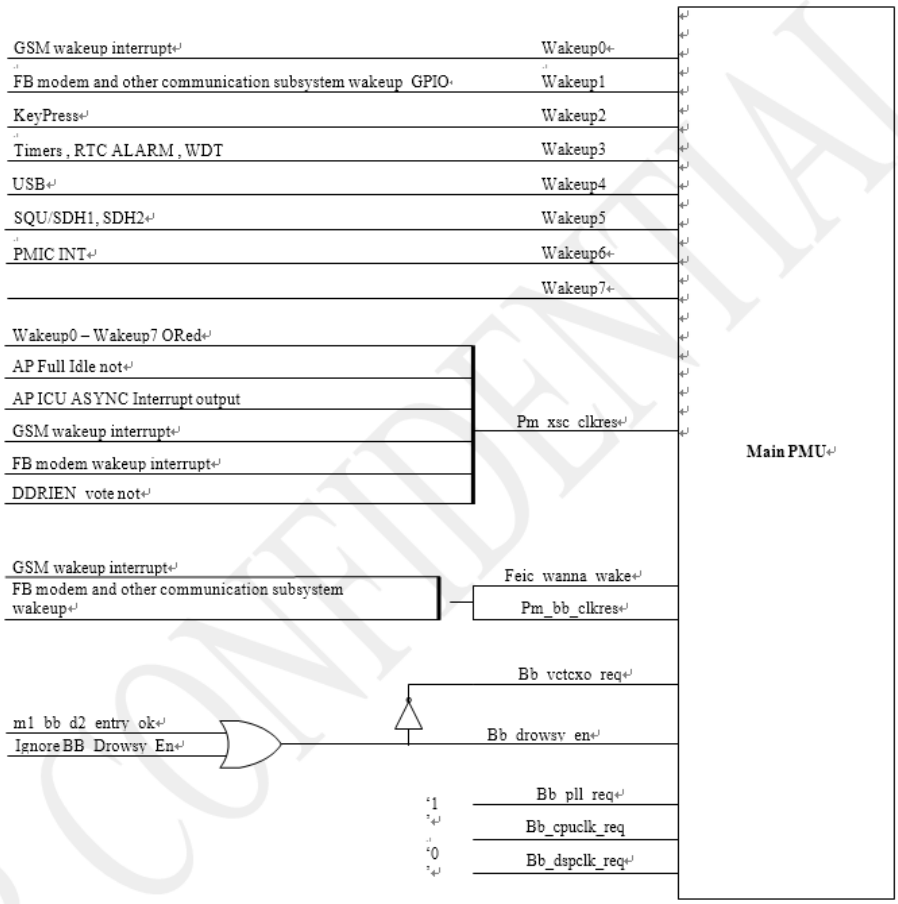
Table 21:AMUCRM Mask Bits

30:31	Reserved	
31	audio_plug and audio_plug wakeup DSP mask	ADU_DET_WK_DSP
30	audio adma/dfe wakeup mask	ADU_INT_WK
29	GSM WakeupWMX	GSM_WAKEUPWMX
28	Venus WakeupX	Venus_WAKEUPX
27	GSM WakeupWM	GSM_WAKEUPWM
26	Venus WakeupWM	Venus_WAKEUPWM
25	AP ASYNC INT	AP_ASYNC_INT
24	AP Full idle	AP_FULL_IDLE
23	SDH1	SDH1
22	SDH2	SDH2
21	KeyPress	KEYPRESS
20	audio_plug and audio_ocp	audio_plug_ocp_wakeup
19	hook_key	HOOK_KEY
20	audio_plug and hook key	audio_plug_ocp_wakeup
19	audi_ocp	HOOK_KEY
18	WDT	WDT
17	RTC ALARM	RTC_ALARM
16	CP Timer 3	CP_TIMER_3
15	CP Timer 2	CP_TIMER_2
14	CP Timer 1	CP_TIMER_1
13	AP2 Timer 3	AP2_TIMER_3
12	AP2 Timer 2	AP2_TIMER_2
11	AP2 Timer 1	AP2_TIMER_1
10	AP1 Timer 3	AP1_TIMER_3
9	AP1 Timer 2	AP1_TIMER_2
8	AP1 Timer 1	AP1_TIMER_1
7	Wakeup7	WAKEUP7
6	Wakeup6	WAKEUP6
5	Wakeup5	WAKEUP5
4	Wakeup4	WAKEUP4
3	Wakeup3	WAKEUP3
2	Wakeup2	WAKEUP2
1	Wakeup1	WAKEUP1
0	Wakeup0	WAKEUP0

If not properly configured, the wakeup events could be high out of reset (for example, the modem) and prevent the chip from entering Sleep mode. The AWUCRS read-only register checks which wakeup event is active.

ASR CONFIDENTIAL

Figure 33:PMU Wakeup Connectivity



ASR CONFIDENTIAL

9.1.2 Application Subsystem PMU (APMU)

The Application Subsystem PMU consists of the following major parts:

- **ARM Cortex-R5 Processor clocking and low-power mode state machine**–The state machine controls clock generation for the ARM Cortex-R5 processor and its interfaces. It controls the entry and exit conditions and the sequence into and out of the ARM Cortex-R5 processor low-power modes.
- **AXI Fabric and PSRAM clocking and low power mode state machine**–The state machine controls clock generation for the buses and PSRAM. Once the ARM Cortex-R5 state machine is in low-power mode, it controls the entry and exit sequence into and out of the bus and PSRAM low-power modes.

The Application Subsystem PMU controls the clock gating and Power mode on the ARM Cortex-R5 and AXI/PSRAM. Once the Application Subsystem PMU enters the lowest Power mode, the Main PMU takes control and may place the ASR1606 in chip-level Sleep mode according to the Main PMU controls.

The Application Subsystem PMU controls the following:

- ARM Cortex-R5 processor clock generators and low-power mode entry and exit
- PSRAM and AXI clock generators and low-power mode entry and exit
- Frequency and frequency change of the ARM Cortex-R5 processor and AXI/PSRAM

9.1.3 Communication Subsystem PMU (CPMU)

The Communication subsystem PMU is responsible for the following:

- Ceva-BX2 DSP subsystem clock generators and low-power mode entry

and exit, communication peripherals clock generation.

ASR CONFIDENTIAL

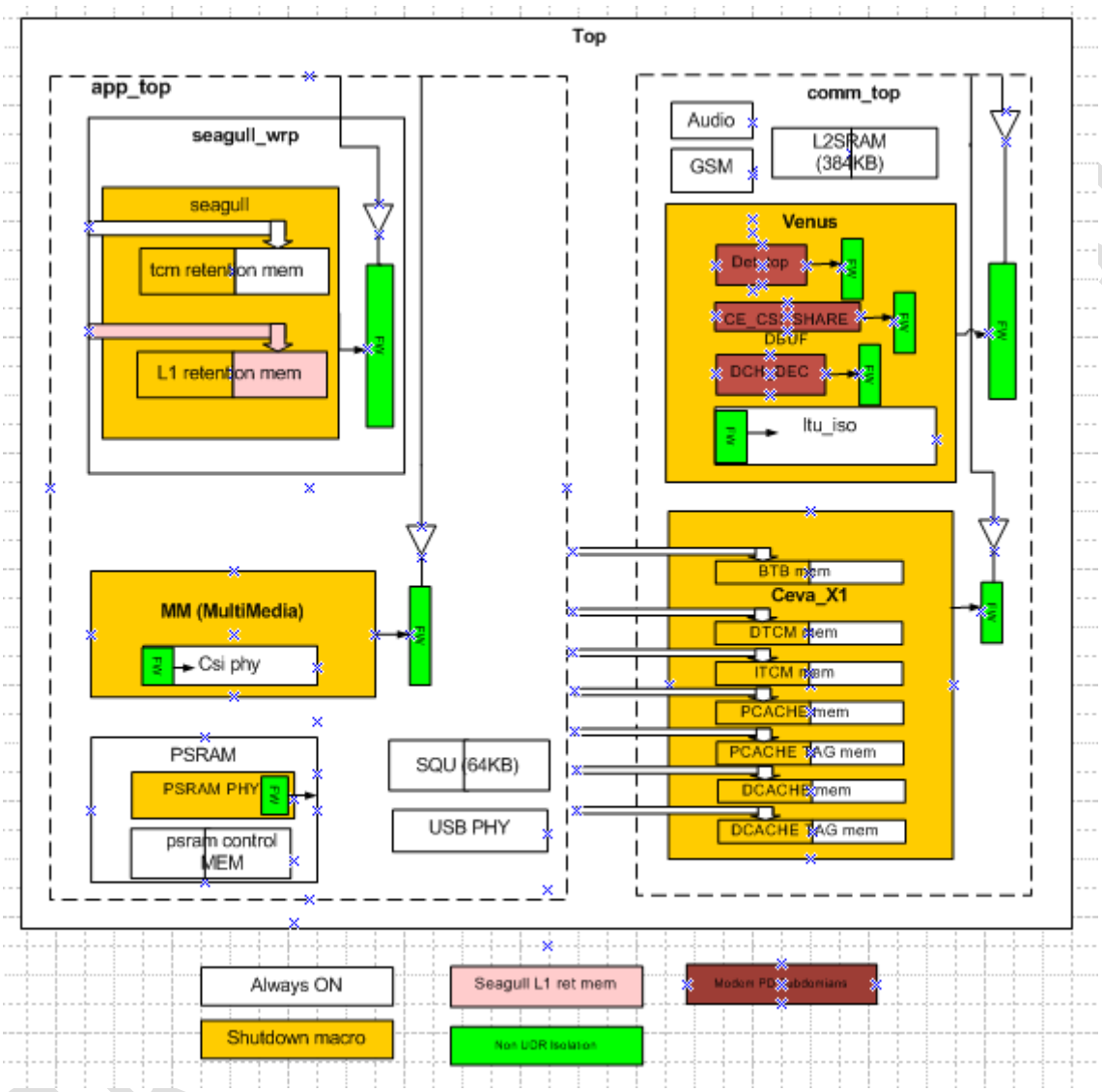
9.2 Power Domains

The ASR1606 internal logic modules are separated into power domains to allow unused or idled circuits to turn off to save power. Figure 34 contains a block diagram of the power domains. The power domains are listed below:

- **Always On (AON):** The AON domain refers to the digital core power plane that sources from the external power management IC without switching capability. This domain powers logic that never powers down, for example wake-up logic, DTC and PMU. This domain is also used to source power to the other switched power domains.
- **ARM Cortex-R5 Subsystem:** Switched domain containing the ARM ARM Cortex-R5 core logic.
  This domain may be switched off when the core is idle.
- **Venus Modem:** Switched domain containing the modem logic. This domain may be switched off when the modem is idle or forced off by software when the logic is not needed.
- **Ceva-BX2:** Switched domain containing the modem logic. This domain may be switched off when the modem is idle or forced off by software when the logic is not needed.
- **Multi-Media:** Switched domain containing the modem logic. This domain may be switched off when the modem is idle or forced off by software when the logic is not needed.

ASR CONFIDENTIAL

Figure 34: ASR1606 Power Domains



Most domains are controlled by state machines that are configurable to allow detection of logic idle and automatically power down the domains. The state machines also turn the domains back on after a wake up event and non-idle indication.

Software Control

Software power domain control of the Venus Modem modules are available using the Communication Subsystem Power Control Register in the Communication Subsystem PMU.

Hardware Control

ASR

Several state machines distributed between the Main PMU, Application Subsystem PMU, Ceva BX2 DSP PMU, and Communication Subsystem PMU control the power domain states automatically based on settings configured in registers. The state machines monitor the activity of the logic contained within the power domains. If enabled, the state machines power down the domains only when the logic is idle. Other dependencies, such as a module needing access to PSRAM memory, are taken into account before turning off certain power domains.

Software configures the behavior of the state machines using several registers distributed throughout the PMU and CCU register sets. Refer to the register definitions for further detail.

9.3 Power States

The ASR1606 is designed to operate in many different power states to allow for minimal power consumption is a variety of situations. A combination of software and hardware controlled mechanisms are used to transition between each power state.

ASR CONFIDENTIAL

The device is split into several clock and power domains as described in previous sections. Each clock domain can be made idle by gating the source clock. Each power domain can be powered down once all the clock domains contained within it are made idle, assuming there is a power switch for the domain. Power states are defined in terms of idle clocks and power domain state.

The frequency of each clock domain is not considered when defining power states. The frequencies are considered only when defining the Supported Product Points. Power states are independent of Product Point as clocks are either gated or not gated and power domains are either on or off.

9.3.1 Allowable States

This section describes the core and chip-level power states available in the ASR1606 and the high-level requirements for entering and exiting each state.

Generally, the chip level power state must remain active until all the cores have entered idle or Powerdown state, allowing the subsystems to enter idle or power down state. Once all internal modules are idle or powered down, the chip may enter an idle or deep Sleep mode as configured.

Most idle and power down states are controlled by logic state machines contained in the PMU modules that perform the necessary request/acknowledge handshakes. Software configures the allowable modes through register settings.

Table 22:ASR1606 Power States

State	Cores	Peripherals	PLLs	Oscillator	Notes
Core Level Power States–Chip-level is active or idle.					
Active state	On	On	On	On	• Dynamic clock frequency change for both cores • Independent clock frequency control for both cores by switching to async state for one core
Internal Idle state	Core internal clock disabled	Inactive	On	On	Core and bus idle independently controlled
External Idle state	Core clock generator disabled	Inactive	On	On	
Powerdown state	One or more core powers are off for part of the device Only logic required to prevent boot on wake is powered on	Partially powered off	Off or on	Off or on	ARM Cortex-R5, and Ceva-BX2 processors turned off
Chip-Level Power States–All core-level power states are idle or powered down.					

ASR CONFIDENTIAL

Sleep state with VCXO on	Inactive	Inactive (except detection logic running with slow clock 32 kHz)	Off	Off	
Sleep state with VCXO off	Inactive and voltage scaled down to retain power states	Inactive (except detection logic running with slow clock 32 kHz)	Off	Off	
Active State	All cores are active	Active	On	On	
State	Cores	Peripherals	PLLs	Oscillator	Notes
Idle or Partial Idle State	At least once core is in Idle or Powerdown state	May be active	On	On	
Power off	Entire ASR1606 is powered off				

ASR CONFIDENTIAL

Core Level Power States

The list below describes the power states supported by the cores in the ASR1606:

- ARM Cortex-R5 and Ceva-BX2 DSP may be at one of the following low power states:
  - Active state–Low/high frequency (processor is active and can be configured at difference frequencies)
  - Internal Idle state–External clock generators are active.
  - Idle state–External clock generators are off. Internal core clocks are off.
  - Power down state
- AXI bus and PSRAM may be at one of the following low power states:
  - Active state–Low/high frequency
  - Idle state–External clock generators are off, and PSRAM is at low power state.

The AXI Bus and PSRAM Idle state can be entered only when ARM Cortex-R5 is in low power state and ARM Cortex-R5, and Ceva-BX2 DSP all vote to allow it.

Chip Level Power States

The ASR1606 may operate in any of the following full chip power states:

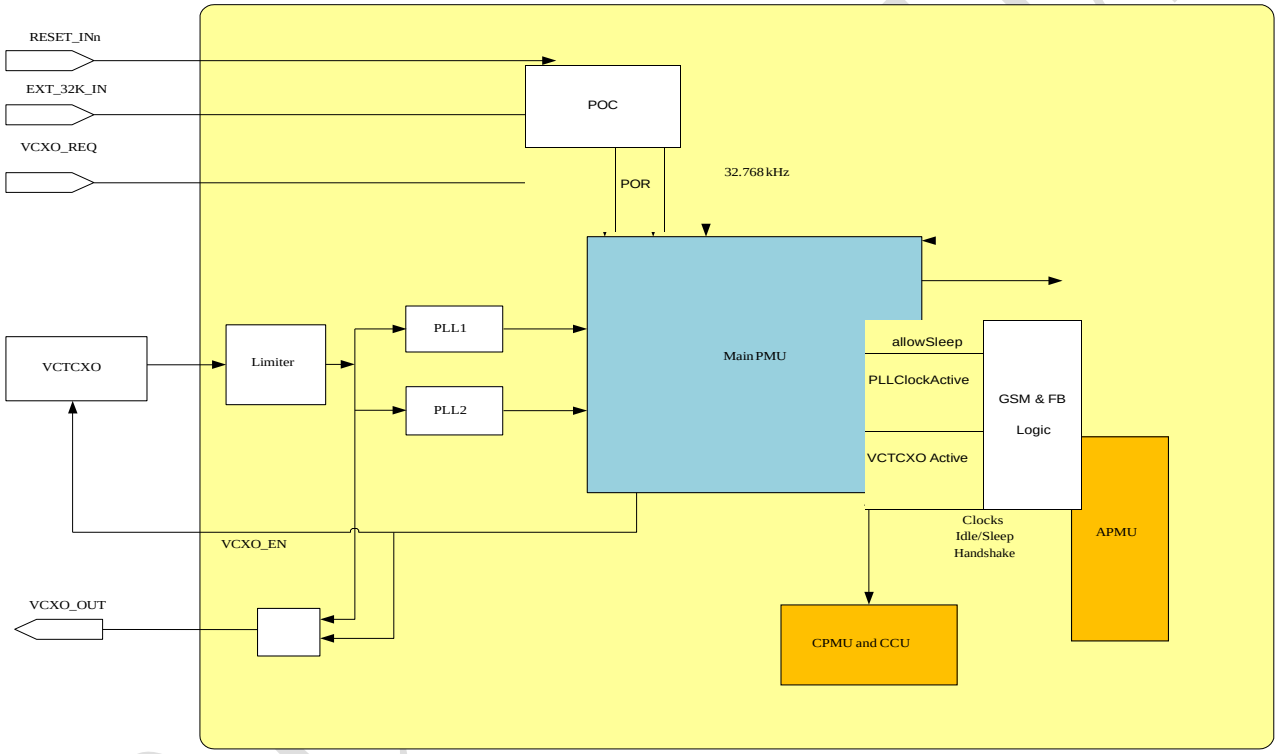
- System active state–The ARM Cortex-R5 and Ceva-BX2 CPU, all AXI agents, baseband logic, and APB clocks are running.
- System is partially idle or in powered down state–One or more of the cores are idle or in Powerdown state, but some bus or module clocks are still running.
- System Idle state–All modules in the ASR1606 are in Idle or Powerdown state except for those running on the 32 kHz clock.
- Sleep state–All system PLLs are off, and VCTCXO may be off. Some logic is running on the 32 kHz clock. The logic VCC supply is down to its minimum logic state-retention voltage.

ASR CONFIDENTIAL

9.4 Clocks

This section describes the source clocks that are input to the device and the derived clocks needed to operate the different clock domains inside the chip. Figure 35 contains a diagram of the input clocks and internal modules that use the clock signals.

Figure 35:Top-Level Block Diagram

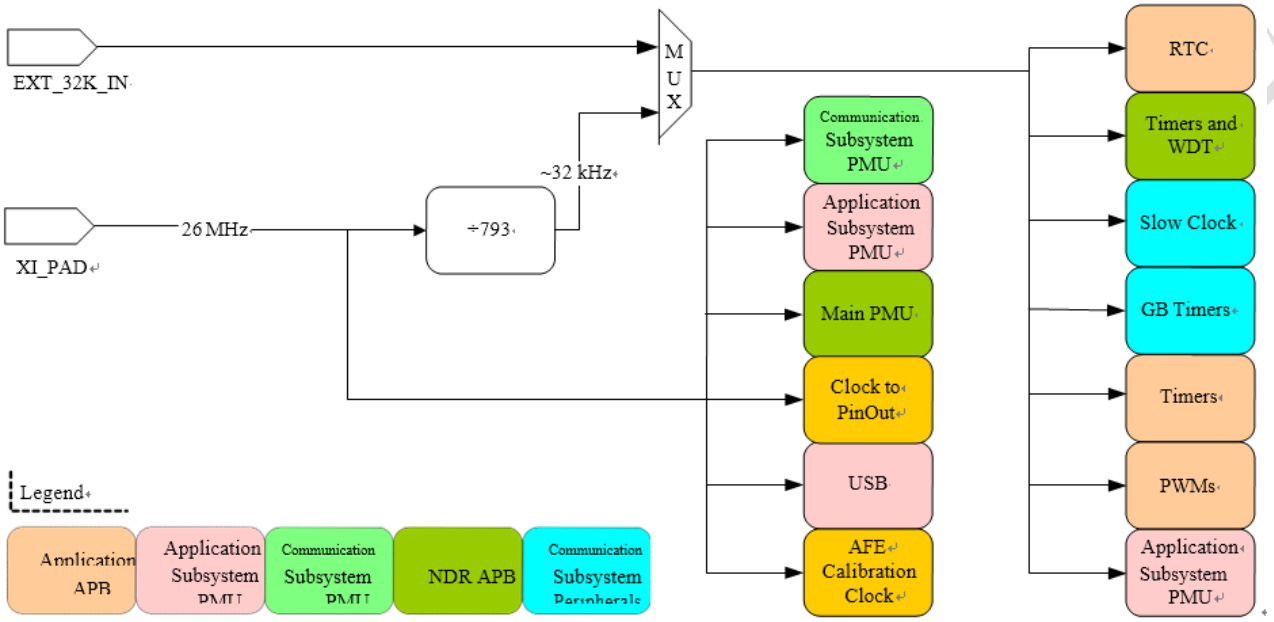


ASR CONFIDENTIAL

9.4.1 Input Clock Sources

As shown in Figure 35 and Figure 36, ASR1606 gets 2 input clock sources, VCTCXO (XI_PAD) and EXT_32K_IN.

Figure 36:Clock Sources for ASR1606



9.4.1.1 Analog Clock Input (XI_PAD Pin)

The analog clock (VCXO) input pin can accept a 26 MHz master clock signal from external clock sources.

The VCXO_EN output pin indicates that the ASR1606 requires a clock signal on the analog clock input pin. An external clock source must be applied when this pin is driven high. The VCXO_EN pin is driven high at reset and may be driven low during chip level Sleep mode with VCXO shutdown enabled in software configured registers. Software can configure the Sleep mode of the VCXO_EN pin using the VCTCXOSD bits in APCR, CPCR and DPCR of PMUs.

9.4.1.2 kHz Clock Input (EXT_32K_IN Pin)

The 32 kHz clock input signal is used by many of the logic modules responsible for reset and wakeup, as shown in Figure 36. The modules responsible for reset and wakeup use an always-on power source and the 32 kHz clock source to minimize the amount of power consumed during chip Sleep modes.

The 32 kHz clock used by logic is either a derived clock divided from the master 26 MHz clock or the externally sourced 32 kHz clock on this pin. The selection of the 32 kHz clock source is software controlled using the Slow Clock Select bit in the

ASR CONFIDENTIAL

Slow Clock Control Register of the Main PMU. The clock is sourced from the divided 26 MHz master clock input at reset. Software should switch to the external 32 kHz clock source after the external source is enabled and stable.

9.4.2 VCXO Output

The VCXO_OUT output pin is driven with the VCXO clock source if either of the following occurs:

- Input pin VCXO_REQx is high when relevant REQ_EN[x] is set to high in the VCTCXO SW Request Control Register (where 'x' is a number)

or

- software request (SW_REQ) is enabled in the VCTCXO SW Request Control Register

9.4.3 PLL Modules

There are 2 PLL available in ASR1606. The PLL modules get the master clock (26 MHz) and generate the highest frequency clocks.

9.4.3.1 PLL1

The internal PLL1 module is designed to generate 307, 350,409, 491, 614, and 819 MHz clock sources using the master input clock as reference. Run-time frequency changes in PLL1 output are available only for debugging purposes and should not be used by production systems.

The PLL1 module is enabled at reset and is shutdown only when the entire chip is in Sleep mode with VCXO shutdown enabled. The settings configured in the PLL and Oscillator Control Register of the Main PMU control how long before the PLL1 output clocks are considered stable when starting up after reset or Power mode shutdown.

9.4.3.2 PLL2

The internal PLL2 module is designed to be VCO programable, and default 2400Mhz VCO frequency. PLL2 output clk can be configured for CR5 cpu, psram, emmc and qspi.

ASR CONFIDENTIAL

9.4.4 Distribution

9.4.4.1 Overview

In PMUs, the highest frequency clocks are divided down to generate the variety of clock sources required by all the different clock domains inside the device. The primary clocks are:

- **Application Subsystem**–Gets the PLL 1 outputs. The Application Subsystem PMU generates the ARM Cortex-R5 processor clocks and the AXI bus and PSRAM clocks.
- **Communication Subsystem**–Driven by PLL 1 divided at the CPMU.
- **APB clock**–Fixed at 26 MHz. This clock is also used for modules using 26 MHz as a functional frequency.
- **USB clock**–26 MHz going to dedicated USB PLL on the USB PHY.

The master clock input is connected to two PLL modules inside the ASR1606 that generate the highest frequency clock signals. The highest frequency clock signals are then divided down to generate the variety of clock sources required by all the different clock domains inside the device.

9.4.4.2 ARM Cortex-R5 Subsystem Clocks

The ARM Cortex-R5 clocks have 4 PLL sources:

- pll1_614 (614 MHz clock from PLL1)
- pll1_819 (819 MHz clock from PLL1)

The following clock domain are available for the ARM Cortex-R5 subsystem.

- PCLK: Main CPU core clock
- ACLKM0: to main fabric AXI clock
- ACLKM1: to PSRAM controller AXI clock

The ARM Cortex-R5 subsystem has 2 AXI master ports:

- AXI-M0 connected to the main fabric
- AXI-M1 connected to the PSRAM controller

AXI-M0 clock is ACLKM0 and AXI-M1 clock is ACLKM1.They can run at different frequencies.

Gating of the ARM Cortex-R5 clocks is controlled by the Core Power Mode state machines in the Applications Subsystem PMU (APMU).

See Section 11.4.6 for further details about the core frequency change process.

9.4.4.3 ARM Cortex-R5 and DTC Clocks

The Modem ARM Cortex-R5 contain three clock domains. Their clock frequencies are individually configurable but all 3 clocks must be synchronous to each other. The 3 clocks are all divided from a common clock source selected in the Frequency Change Control Register of the Main PMU. The clock dividers are

ASR CONFIDENTIAL

configured in the AP Clock Control Registers of the Application Subsystem PMU.

The following clock domain are available for the ARM subsystem:

PCLK: Main CPU core clock

ACLKM0: L2 controller to CPSS AXI clock

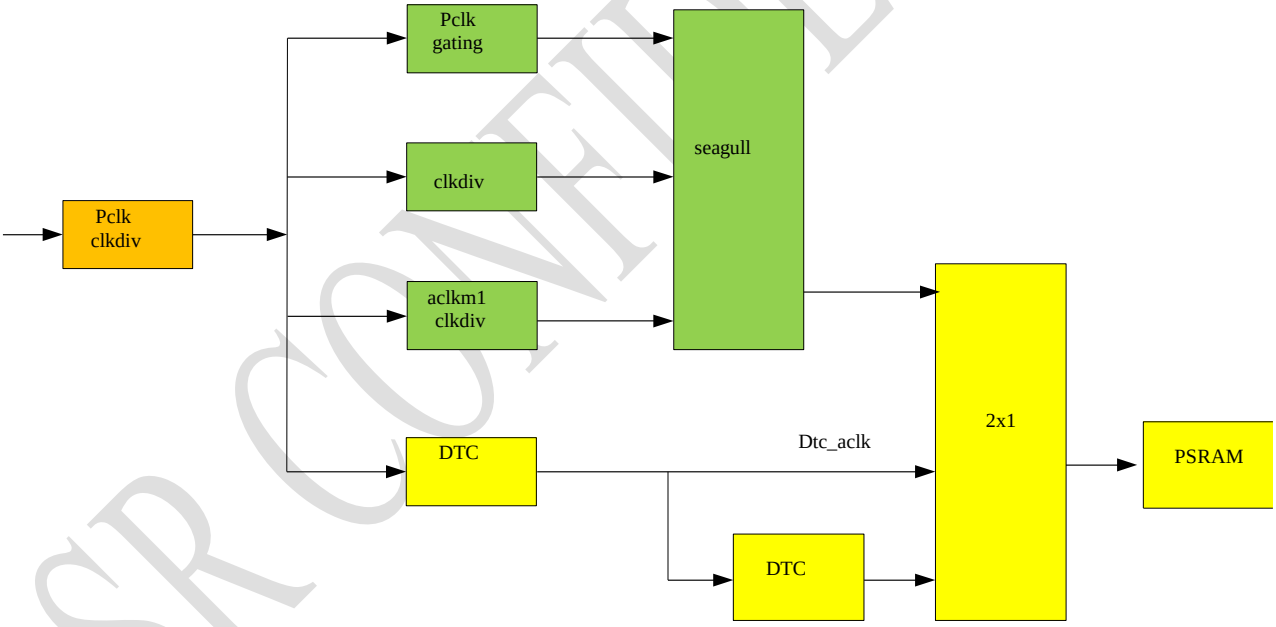
ACLKM1: L2 controller to PSRAM controller AXI clock The ARM Cortex-R5 subsystem has 2 AXI master ports:

AXI-M0 connected to the CPSS

AXI-M1 connected to the PSRAM controller

AXI-M0 clock is ACLKM0 and AXI-M1 clock is ACLKM1.They can run at different frequencies. DTC(clock is dtc_aclk) and seagull PSRAM interface (clock is ACLKM1) go through a fabric2x1 to PSRAM. Dtc_aclk and aclkm1 is synchronous to each other and the Fabric2x1 also work at dtc_aclk.

Figure 37:ARM Cortex-R5 Subsystem Clock



ASR CONFIDENTIAL

Copyright © 2022 ASR

9.4.4.4 Ceva-bx2 Core

The Ceva-bx2 Core in the Communication Subsystem contains a single clock domain (GB_CLK_CTRL). The frequency of this clock domain is set by the Clock Control Register in the Communication Subsystem PMU.

9.4.4.5 AXI Bus

The AXI fabric 1,2 AXI decoder in the Application Subsystem are contained in the same clock domain. The frequency of this clock domain is controlled by the Frequency Change Control Register in the Main PMU and the CP and AP Clock Control Registers in the Application Subsystem PMU. The clock divider setting sampled and used for an AXI bus frequency change are taken from the register used to issue the frequency change request via the BUS_FREQ_CHG_REQ bit.

See Section 11.4.6 for the steps required to change the AXI bus frequency. Gating of the AXI bus clock is controlled by the Application Subsystem PMU.

9.4.4.6 PSRAM Memory Controller

The PSRAM Memory Controller contains two clock domains: the external PSRAM interface clock and the internal logic clock. The external PSRAM interface clock is created by dividing the internal logic clock by two. The internal logic clock is defined as DCLK2X, and the external PSRAM interface clock is DCLK.

The internal logic clock is configured with the Frequency Change Control Register in the Main PMU and the CP and AP Clock Control Registers in the Application Subsystem PMU. These registers set the frequency of DCLK2X, making the external PSRAM interface clock (DCLK) one half the configured frequency. The clock divider setting sampled and used for a PSRAM Memory Controller frequency change are taken from the register used to issue the frequency change request via the PSRAM_ FREQ_CHG_REQ bit.

The PSRAM Memory Controller clock frequency (DCLK2X) is set to 156 MHz at reset.

See Section 10.4.6 for the steps required to change the PSRAM Memory Controller frequency. Gating of the PSRAM Memory Controller clock is controlled by the Application Subsystem PMU.

9.4.4.7 Applications Subsystem Peripheral Clocks

Each Application Subsystem APB peripheral has a register that controls its reset, functional clock selection, whether the functional clock is on or off, and whether the APB bus clock is on or off Table 23 describes the APB peripherals and these settings. In Table 23, the functional clock of each peripheral is abbreviated as

ASR CONFIDENTIAL

FNCLK, and the APB Peripheral Bus clock is abbreviated as APBCL

Table 23:Application APB Peripherals

Peripheral	Functional Clock Select Options APCRx[FNCLKSEL]	Software Reset RST	Clock Gating	
			FNCLK	APBCLK
Fuse Override	N/A	Yes	No	Yes
General Config Registers	N/A	Yes	No	Yes
GPIO	N/A - Always 26 MHz	Yes	Yes	Yes
Keypad	0x0 = 32.768 kHz 0x1 = 16 kHz 0x2 = 26 MHz	Yes	Yes	Yes
RTC	N/A - Always 32.768 kHz	Yes	No	Yes
SSP 0	0x0 = 6.5 MHZ 0x1 = 13 MHZ 0x2 = 26 MHZ 0x3 = 52 MHZ 0x4 = 3.25 MHZ 0x5 = 1.625 MHZ 0x6 = 0.8125MHZ 0x7 = 1 MHz	Yes	Yes	Yes
SSP 1	0x0 = 6.5 MHZ 0x1 = 13 MHZ 0x2 = 26 MHZ 0x3 = 52 MHZ 0x4 = 3.25 MHZ 0x5 = 1.625 MHZ 0x6 = 0.8125MHZ 0x7 = 1 MHz	Yes	Yes	Yes
SSP 2	0x0 = 6.5 MHZ 0x1 = 13 MHZ 0x2 = 26 MHZ 0x3 = 52 MHZ 0x4 = 3.25 MHZ 0x5 = 1.625 MHZ 0x6 = 0.8125MHZ 0x7 = 1 MHz	Yes	Yes	Yes
Application Timers 1	N/A - Always 13 MHz and 32.768 kHz	Yes	Yes	Yes

ASR CONFIDENTIAL

9.4.4.8 Communications Subsystem Peripheral Clocks

Each Communication Subsystem APB peripheral has a register that controls its reset, functional clock selection, whether the functional clock is on or off, and whether the APB bus clock is on or off. Table 24 describes the APB peripherals and these settings. In Table 24, the functional clock of each peripheral is abbreviated as FNCLK, and the APB Peripheral Bus clock is abbreviated as APBCLK.

Table 24:Communication APB Peripherals

Peripheral	Clock Select Options APCRx[FNCLKSEL]	Software Reset RST	Clock Gating	
			Functional FNCLK	APB Bus BACLK
Monitor Mux	N/A	No	No (FNCLK and BACLK are the same clock)	
E-Cipher	N/A - Always 26 MHz and 78 MHz	Yes	Yes	No
General Configuration Registers	N/A	No	No (FNCLK and BACLK are the same clock)	
GSM Slow Clock	N/A - Always low power 26 MHz and 32.768 kHz	Yes	Yes	No
IPC (CR5/Ceva BX2) and Resource IPC	N/A	Yes	No (FNCLK and BACLK are the same clock)	
TCU	N/A - Always low power 26 MHz	Yes	Yes	No
Timers	N/A - Always 13 MHz and 32.768 kHz	Yes	Yes	Yes
USIM 1/2	N/A - Always 48 MHz	Yes	Yes	Yes
AIRQ	N/A - Always low power 26 MHz	Yes	Yes	No

ASR CONFIDENTIAL

9.4.5 Clock Shutdown Conditions

Table 25 shows the conditions necessary to shut down the clocks to the primary clock domains.

Table 25:Primary Clock Domains Clock Shutdown Conditions

Clock Domain	"May be Idle" Decision Contributors	Notes
ARM Cortex-R5	ARM Cortex-R5	DTCM voting for clock generation shutdown
Ceva-BX2 core	Ceva-BX2 core	CEVA-BX2 DSPSD bit may disable CEVA-BX2 core clock generation gating
Internal SRAM	ARM Cortex-R5 and CEVA-BX2	
PSRAM		
DMA/Bridge		
AXI bus and PSRAM		Overall AXI domain, including all bus agents Overrides the programming of its subdomains
Any single APB peripheral, APB, or Functional Clock Domain		
Overall APB Clock Domain		Overrides specific domains programming
CEVA-BX2 (and all of its Subsystems)	ARM Cortex-R5 and CEVA-BX2 core	CEVA-BX2 DSPSD bit may disable CEVA-BX2 core clock generation shut down ARM Cortex-R5 or DSPSD bit may disable CEVA-BX2 Subsystem clock generation gating CEVA-BX2 core may enter Idle
Baseband logic – domain per PLL	ARM Cortex-R5 and CEVA-BX2 core	

Both the baseband logic and the Ceva-BX2 Subsystem are treated as sealed modules regarding the clock tree. This assumption is based on the existence of a dedicated PMU within each of these subsystems. The Main PMU enables/disables (gates) the various clocks provided to these subsystems.

ASR CONFIDENTIAL

9.4.6 Frequency Changes

The ASR1606 supports several combinations of frequency operating points of the internal clock domains. The combination of core, AXI bus, and PSRAM clock frequency settings is defined as a Product Point. Other peripheral clocks are independent of and not included in the Product Point definitions. Each stepping of the device has a defined set of supported product points. Switching between any two Supported Product Point is allowed when following the steps listed in the following subsections.

To obtain information on defined product points and associated power specs for the ASR1606 device, contact ASR.

9.5 Reset

9.5.1 System Reset Inputs Synchronization

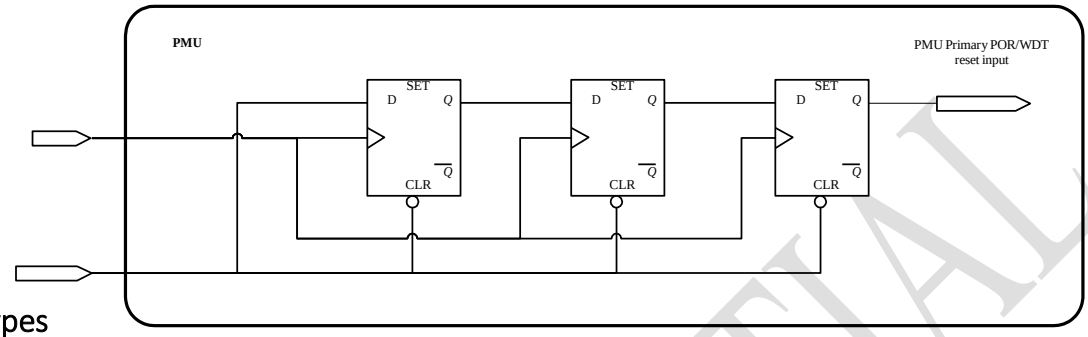
Power-on reset (POR) and watchdog reset (WDT) are driven asynchronously into the ASR1606 when asserted; however, their negation is synchronized at the entry point to the PMU.

The impact of the synchronization circuit is that the ASR1606 is not actually released from reset until the 32-kHz clock generated from the VCTCXO has begun toggling and the release is synchronous.

Figure 38 shows the synchronization circuit of which there are three such circuits in the PMU, one for each system-reset event.

ASR CONFIDENTIAL

Figure 38:POR and WDT Reset Synchronizers



9.5.2 Reset Types

The PMU supports three reset event types as follows:

- Power-On Reset (POR)–Sets off the power-up sequence of the ASR1606
- Watchdog (WDT) reset–Issued upon expiration of the Watchdog timer (WDT)
- Cores programmable hardware reset (HR)–Each core (ARM Cortex-R5 and Ceva-BX2) may drive hardware reset to some segments of the system

All reset events assert the receiving module hardware-reset ports asynchronously and typically negate them synchronously.

Specifically on powerup, the POR is driven to all recipients via combinational logic through the PMU as soon as the power supply stabilizes.

POR and WDT reset inputs to the PMU are negated synchronously.

ASR CONFIDENTIAL

Copyright © 2022 ASR

During POR and WDT reset, the SLAVE_RESET_OUT_N IO pin is asserted. This slave reset output also is forced asserted by setting APRR[6]. After reset, software must clear this bit to de-assert the slave reset output.

9.5.3 Reset Domains

All reset events are routed to the PMU, which drives a single hardware reset signal to each reset domain in the system based on the definition in Table 26. An output of reset excluding WDT reset is also available to enable product-specific logic, which is not reset by the WDT.

Table 26:System Reset Domains

Reset Type	System Reset		Programmable Hardware Reset		System Reset[b] Auto Negated by PMU
Reset Domains	Power On	WDT	ARM Cortex-R5	CevaBX2	
PMU – VCTCXO external request controller	+				Yes
PMU – all but VCTCXO controller, including sleep control and CEVA-BX2-related	+	+			Yes
GPIO[c]	+	+			Yes
ARM Cortex-R5 CPU	+	+	+		Yes
Ceva-BX2 core	+	+	+		Yes
APB peripherals - two reset domains per peripheral: APB and functional	+	+	+	+	Yes
Baseband logic	+	+	+	+	Yes
WDT – all but the reset generation logic	+	+	+[d]		No
WDT – reset generation logic	+		+[d]		No

a. Every joint cell marked with a "+" implies that its corresponding domains may only be collectively reset.

b. System reset is defined as power-on reset or WDT reset.

c. GPIOs that need not be reset by WDT are implemented outside the platform, on the product-specific logic, and should be connected to the pm_hrst_no_wdtn reset.

d. This bit can only be negated.

9.5.4 APB Peripheral Reset Sequence

The programmable hardware-reset sequence for any of the APB peripherals initiated by either cores is as follows:

1. Assert the peripheral functional portion reset.
2. If the peripheral functional clock is not already enabled, enable it.
3. Assert the peripheral APB portion reset (which is synchronously self-negating) and enable its APB clock if not already running (same address per peripheral).
4. Negate the peripheral functional portion reset (is synchronously negated).

This sequence is performed automatically for all APB peripherals during the

ASR CONFIDENTIAL

power-on reset sequence. Software facilitates this sequence only for reset during functional operation.

9.5.5 Reset Options Controlled by Fuse Settings

The reset options listed in this section are controlled by hardware fuses that are set by manufacturing equipment. Refer to Document MV-S301545-00 "ASR1606 Processor Fuse Application Note" for more details.

- Reset Vector Entry Point (High or Low Address)
- Core Clock Frequency Settings

ASR CONFIDENTIAL

10.JTAG

This chapter describes the JTAG features of the ASR1606.

- Section 10.1, Features
- Section 10.2, Operation
- Section 10.3, Board JTAG Design Note
- Section 10.4, Register Descriptions

JTAG provides a way to drive and sample the external pins of the device, regardless of the core state. Therefore, you can test both the device electrical connections to the circuit board and (in conjunction with other devices on the circuit board with a similar interface) the integrity of the circuit board connections between devices.

JTAG logic includes a Test-Access Port (TAP) Controller, TAP pins, an Instruction Register, and test data registers (TDRs). The registers include the Boundary Scan Register (BSR) to control the IO pins directly, a Bypass Register, a Device Identification (ID) Register, and data-specific registers, as shown in Figure 39. Data are shifted into all registers most significant bit (MSB) first serially. The JTAG interface is controlled through five dedicated TAP controller pins that interface to the TAP controller: PRI_TDI, PRI_TMS, PRI_TCK, PRI_TRSTn, and PRI_TDO.

10.1 Features

The JTAG interface has the following features and conformities:

- Provides access to IEEE STd. 1149.1 compatible registers such as IDCODE, BYPASS, EXTEST, etc. through the JTAG port.
- Support for hardware/software debug via the core TAP controller in Concatenation mode.

Refer to the IEEE Std. 1149.1 for an explanation of the terms used in this section and for a complete description of the TAP controller states.

10.2 Operation

This section describes the TAP controller, hardware connection, JTAG instruction set (Instruction Register), test-data registers, and data-specific register.

10.2.1 JTAG Select

The ASR1606 primary JTAG interface (PRI_TDI, PRI_TMS, PRI_TCK, and PRI_TDO) can be used for other functions as described in the pin-muxing documentation. This feature enables using these four pads as functional pads also in the final mobile phone design where JTAG is not required.

ASR CONFIDENTIAL

10.2.2 TAP Controller Reset

The JTAG interface includes a TAP controller state machine. To force the TAP controller into the test-logic-reset state (TLRS) at power on, the PRI_TRSTn pin can be asserted low, or the PRI_TMS pin can be held high for five PRI_TCK cycles. ASR recommends that PRI_TRSTn be driven from low to high (de-asserted) either before or while the hardware RESET_INn pin is de-asserted.

PRI_TRST_N should not be de-asserted after RESET_INn.

A clock on the PRI_TCK pin is not necessary to reset the device at power on. After power on, the TAP controller can be reset by either holding PRI_TMS high for five PRI_TCK cycles or asynchronously through PRI_TRSTn. Where JTAG is not used, ASR recommends that a reset IC be used to cause a reset on PRI_TRSTn at power on.

10.2.3 Instruction Register

The Instruction Register (IR) controls JTAG circuitry by selecting a specific Test Data Register (TDR) to be connected between PRI_TDI and PRI_TDO and controlling the operation affecting that data. These tasks are accomplished through a set of predefined instructions that can be either mandatory or optional as set forth in the IEEE Std. 1149.1a-1993, user defined, or private.

The IR is 9 bits wide in normal operation and 13 bits in concatenation mode. Concatenation mode allows access to the internal core 4-bit IR for use in core debug (refer to the core debug documentation for more details). To enable Concatenation mode, write 16'b0000000000001010 to the tap_ctl TDR. Any subsequent IR accesses must be 13 bits (the 9 MSB bits corresponding to the main TAP IR, the lower 4 bits for the core IR). To access only the core's TAP, ASR recommends shifting in all 1s into the main TAP, thus placing it in Bypass mode (that is, shift in 1FF followed by the 4 bits for the core IR). Refer to Table 27 for more information.

Writing all 0s into the tap_ctl TDR reverts the system to normal 9-bit IR mode.

Three mandatory public instructions are managed: extest, sample/preload, and bypass. Three optional public instructions are managed: idcode, clamp, and highz. Finally, one user-defined instruction is managed: tap_ctl, which is used to enable access to the core internal TAP controller via IR concatenation. The optional public instructions runbist, intest, and usercode are not supported. Table 27 describes each of these instructions.

ASR CONFIDENTIAL

Table 27:JTAG Instruction Description

Instruction /Requisite	Opcode	Description
extest IEEE 1149.1 Required	0x0AA	Initiates testing of external circuitry, typically board-level interconnects and external circuitry. **extest** connects the BSR between TDI and TDO in the Shift_DR state only. When **extest** is selected, all output signal pin values are driven by values shifted into the BSR and can change only on the falling-edge of TCK in the Update_DR state. Also, when **extest** is selected, all system input pin states must be loaded into the BSR on the rising-edge of TCK in the Capture_DR state. Values shifted into input latches in the BSR are never used by the internal logic of the processor.
clamp	0x0AB	Allows the state of the signals driven from the pins to be determined from the BSR while the **Bypass** Register is selected as the serial path between TDI and TDO. Signals driven from the component pins do not change while the **clamp** instruction is selected.
highz	0x0A9	Floats all three-stateable output and input pins. Also, when this instruction is active, the **Bypass** Register is connected between TDI and TDO. This register can be accessed through the JTAG Test-Access port throughout the device operation. Access to the **Bypass** Register can also be obtained with the **bypass** instructio**n.**
sample/ preload IEEE 1149.1 Required	0x0AC	Performs two functions: • When the TAP controller is in the Capture-DR state, the **sample** instruction occurs on the rising edge of TCK and provides a non-interfering snapshot of the component's normal operation. The instruction causes BSR cells associated with output pins to **sample** the value driven by or to the processor. Similarly, BSR cells associated with input pins **sample** the value driven on the inputs. • When the TAP controller is in the Update-DR state, the **preload** instruction occurs on the falling edge of TCK. It causes the transfer of data held in the boundary-scan cells to the slave register cells. Typically, the slave-latched data is then applied to the system outputs with the **extest** instruction.
idcode IEEE 1149.1 Optional	0x0FF	Used in conjunction with the Device Identification Register. It connects the ID Register between TDI and TDO in the Shift_DR state. When selected, **idcode** parallel-loads the hard-wired identification code (32 bits) on TDO into the ID Register on the rising edge of TCK in the Capture_DR state. **NOTE:** The ID Register is not altered by data being shifted in on TDI.
tap_ctl	0x098	Enables the concatenation of the Core's TAP controller IR onto the main TAP controller's IR chain by writing the following data into this 16-bit TDR: tap_ctl[15:0] = 0000000000001010
bypass IEEE 1149.1 Required	0x1FF	Selects the Bypass Register between TDI and TDO pins while in SHIFT_DR state, effectively bypassing the processor test logic. 0x0 is captured in the CAPTURE_DR state. While this instruction is in effect, all other test data registers have no effect on the operation of the system. Test data registers with both test and system functionality perform their system functions when this instruction is selected.

10.2.4 Test Data Registers

The test data registers include the Bypass Register, Boundary Scan Register (BSR), and the Device Identification (ID) Register.

10.2.4.1 Bypass Register

The single-bit Bypass Register is selected as the path between PRI_TDI and PRI_TDO so that the device can be bypassed during JTAG testing. Therefore, test data can move more rapidly to and from other components on a board that must perform JTAG test operations. When the bypass, highz, or clamp instruction is the current instruction in the IR, serial data is transferred from PRI_TDI to PRI_TDO

ASR CONFIDENTIAL

in the Shift-DR state with a delay of one PRI_ TCK cycle. A logic 0x0 is loaded from the parallel input of the Bypass Register in the Capture-DR state. There is no parallel output from the Bypass Register.

10.2.5 Boundary Scan Register (BSR)

The Boundary Scan Register (BSR) consists of a serially connected set of cells around the periphery of the device at the interface between the core logic and the system input/output pads. This register can be used to isolate the pins from the core logic and then drive or monitor the system pins. The connected BSR cells constitute a Shift Register.

For any instructions that use the BSR, the RESET_INn pin must always be de-asserted because the BSR logic powers down when either of these pins is asserted. No JTAG instructions using the BSR can be performed with these pins asserted.

The BSR is selected as the TDR to be connected between PRI_TDI and PRI_TDO only during the sample/preload and extest instructions. Values in the BSR are used but are not changed when the clamp instruction is executed.

In the normal (System) mode of operation, straight-through connections between the core logic and pins are maintained, and normal system operation is unaffected (the same is true when the sample/preload instruction is selected).

When EXTEST is the currently selected instruction, values can be applied to the output pins regardless of the actual values on the input pins and core logic outputs.

The BSR has indeterminate values after power on and after a JTAG reset (from asserting nTRST or entering the TLR state).

ASR CONFIDENTIAL

Note The high-speed interfaces (PSRAM, USB) are not part of the BSCAN chain.

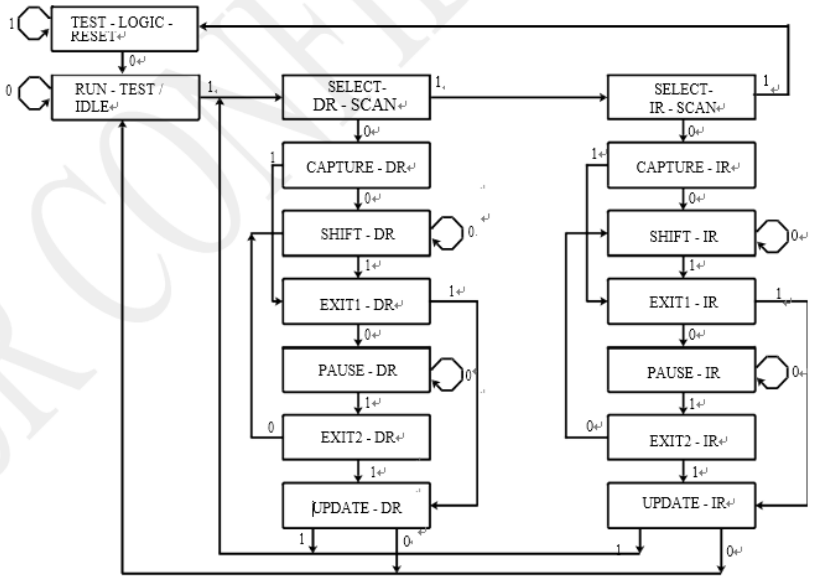
ASR CONFIDENTIAL

10.2.6 TAP Controller

The TAP Controller is a 16-state synchronous state machine that controls the sequence of JTAG operations. The TAP Controller can be controlled through a bus master. It changes state only in response to a rising edge on PRI_TCK or power on. The value of the Test-mode state (PRI_TMS) input signal at a rising edge of PRI_TCK controls the sequence of the state machine. The TAP Controller is reset automatically on reset-exit sequences through PRI_TRST_N. In addition, the TAP Controller can be reset by applying a high signal level on the PRI_TMS input for five PRI_TCK periods.

For information on the state transitions that occur in the TAP Controller, refer to IEEE 1149.1
Standard Test Access Port and Boundary-Scan Architecture document.

Figure 39:TAP Controller State Diagram



NOTE: ALL STATE TRANSITIONS ARE BASED ON THE VALUE OF TMS.

ASR CONFIDENTIAL

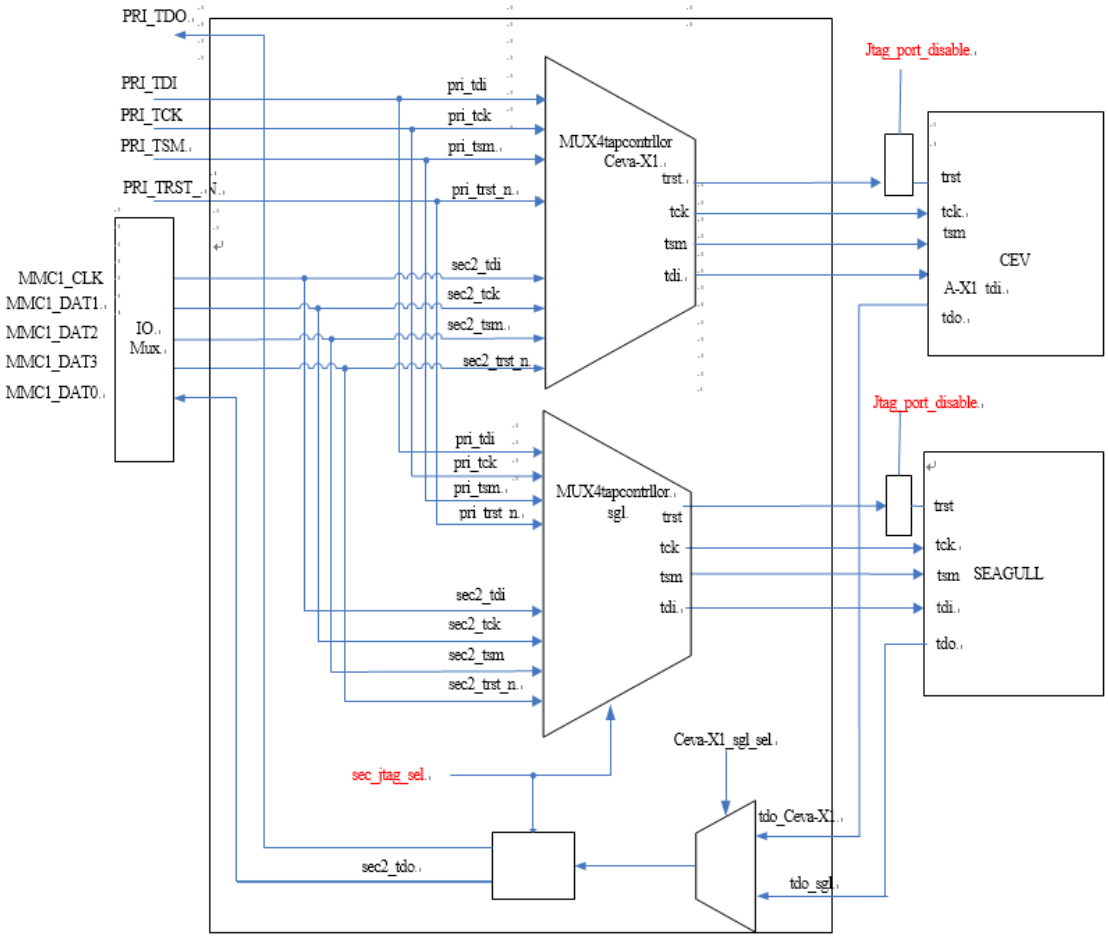
10.2.7 Secondary JTAG Controller

The JTAG_PORT_DISABLE fuse disables the JTAG by holding the TAP in reset.

The SEC_JTEG_SEL fuse connects all two TAP controller MUXs to the Primary JTAG pins when set to 1. The Primary JTAG interface can control all TAP interfaces at the same time.

The Secondary JTAG interfaces can only control a single TAP at a time. When the SEC_JTAG_SEL fuse is set to 0, the two Secondary JTAG pin sets are connected to the TAPs (Sec2 to CEVA-BX2/Seagull). The CEVA-BX2_SGL_SEL selects which of the CEVA-BX2 or Seagull TDO signals are routed to the output of Sec2.

Figure 40:Secondary JTAG Controller Connection



10.3 Board JTAG Design Note

The PMIC RESET_OUTN is an output signal. JTAG SRST resets the ASR1606 when JTAG connects to the ASR1606. If the PMIC RESET_OUTN, JTAG SRST, and ASR1606 RESET_INn are connected together, the ASR1606 RESET_INn signal may be about 0.5V instead of 0V when JTAG resets the ASR1606. This may make the JTAG connection to the ASR1606 unstable. ASR recommends adding one AND gate to stabilize this situation. The two inputs of the AND gate are the PMIC RESET_OUTN and JTAG SRST. The output of the AND gate is connected with the ASR1606 RESET_INn. This design makes the JTAG connection stable.

10.4 Register Descriptions

The JTAG logic has no memory-mapped associated registers.

ASR CONFIDENTIAL

11.DMA Controller

This section describes the ASR1606 on-board DMA Controller (DMAC).

- Section 11.1, Overview
- Section 11.2, Features
- Section 11.3, Operation
- Section 11.4, Register Descriptions

11.1 Overview

The Application Subsystem contains a Direct-Memory Access Controller (DMAC) that transfers data to and from memory in response to requests generated by peripheral devices or companion chips. The peripheral devices and companion chips do not directly supply addresses and commands to the Memory Controller. Instead, the states required to manage a data stream are maintained in 32 DMA channels, DMA[31:0], in the DMAC. Every DMA request from a peripheral device generates a memory-bus transaction. The processor can access the peripheral bus through the bridge portion the DMA thus bypassing the system DMA. Therefore, this document refers to DMA as the "DMA Bridge."

The DMAC supports flowthrough transfers. Figure 42 provides an overview of the DMAC. Table 29 provides a signal list.

11.2 Features

The DMAC provides the following features:

- Supports memory-to-memory, peripheral-to-memory, and memory-to-peripheral transfers in Flowthrough mode
- Supports Flowthrough mode for transfers between flash and PSRAM
- Employs a priority mechanism to process active channels (4 channels with outstanding DMA requests, at any given time)
- Allows each of the 32 channels to operate for Descriptor-fetch or no-Descriptor-fetch transfers (see Section DMA Descriptors)
- Supports special Descriptor modes (Descriptor comparison and Descriptor branching)
- Retrieves trailing bytes in the Receive peripheral-device buffers
- Supports programmable data-burst sizes (8, 16, or 32 bytes) and programmable peripheral device data widths (byte, half-word, or word)
- Supports up to 8191 bytes of data transfer per Descriptor. Larger transfers can be performed by chaining multiple Descriptors.
- Supports flow-control bits to process requests from peripheral devices. Requests are not processed unless the flow-control bit is set.

ASR CONFIDENTIAL

Table 28:DMA Support Matrix

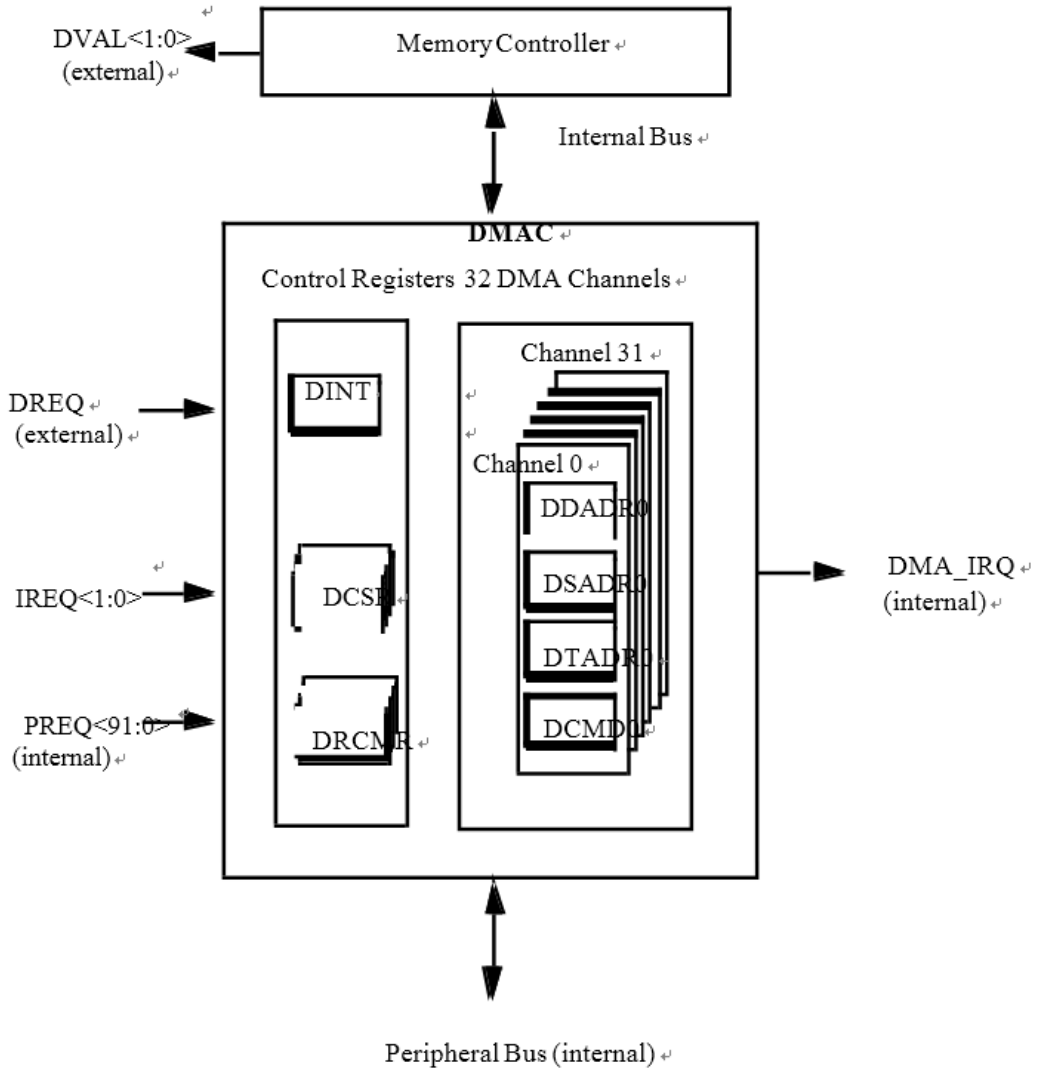
	Internal Memory	External Memory	Internal Peripheral	External Peripheral	Companion Chip
Internal Memory	flowthrough				
External Memory	flowthrough	flowthrough			
Internal Peripheral	flowthrough	flowthrough	—		
External Peripheral	flowthrough	flowthrough	—	—	

ASR CONFIDENTIAL

11.3 Operation

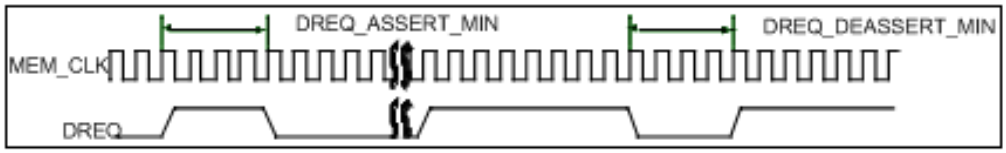
The DMAC can be configured to transfer data using flowthrough DMA. The DMAC has 32 configurable channels. See Figure 41.

Figure 41:DMAC Block Diagram



ASR

Figure 42:DREQ Timing Requirements



ASR CONFIDENTIAL

11.3.1 DMA Channels

Each one of the 32 DMA channels can be controlled by four 32-bit registers: DMA Descriptor Address Registers, DMA Source Address Registers 0-31, DMA Target Address Registers 0-31, and DMA Command Registers 0-31. Each channel can be configured to service any kind of transfer.
Each channel is serviced in increments of that device burst size and delivered in the granularity of the device port width. The burst size and port width for each device are programmed in the channel registers and are based on the device FIFO depth and bandwidth requirements. When multiple channels are actively executing, each channel is serviced with a burst of data. After each burst of data, the DMA Controller performs a context switch to another active channel. The DMA Controller performs context switches based on whether a channel is active, whether its target device is currently requesting service, and the channel priority.

11.3.1.1 DMA Channel-Priority Scheme

The DMA channel-priority scheme helps to ensure that peripherals are serviced according to their bandwidth requirements. Assign a higher priority to peripherals with higher bandwidth requirements and a lower priority to peripherals with lower bandwidth requirements. This action ensures that higher-bandwidth peripherals are serviced more often than lower-bandwidth peripherals.

The DMA channels are divided internally into four sets of eight channels each. The channels in each set use a round-robin priority. Set 0 has the highest priority and Set 3 has the lowest priority.
Program the modules that have the most severe latency requirements into set 0. Program
memory-to-memory moves and low-bandwidth peripherals to use Sets 2 and 3. Refer to Table 29 for details.

When all of the channels are running concurrently, set 0 is serviced four out of every eight consecutive channel-servicing instances, Set 1 is serviced twice, and Sets 2 and 3 are each serviced one time.

For example, if all of the channels request data transfers, the sets are prioritized in the following order: Set 0, Set 1, Set 0, Set 2, Set 0, Set 1, Set 0, Set 3. After eight channel-servicing instances, the pattern repeats. The channels in each set are given a round-robin priority.

ASR CONFIDENTIAL

Table 29:Channel Priority

Set	Channels	Priority	Number of Times Served
0	0, 1, 2, 3, 16, 17, 18, 19	Highest	4 / 8
1	4, 5, 6, 7, 20, 21, 22, 23	Higher than 2 and 3. Lower than 0.	2 / 8
2	8, 9, 10, 11, 24, 25, 26, 27	Higher than 3. Lower than 0 and 1.	1 / 8
3	12, 13, 14, 15, 28, 29, 30, 31	Lowest	1 / 8

11.3.1.2 Channel States

The following states apply to the DMA channels:

- Uninitialized—occurs after a reset. The <Stop interrupt> field in the DMA Channel Control/Status Registers 0-31 is set when uninitialized.
- Not running—occurs when either a valid Descriptor has been loaded into the DMA Descriptor Address Registers during a Descriptor-fetch transfer or valid DMA Source Address Registers 0-31, DMA Target Address Registers 0-31, and DMA Command Registers 0-31 have been programmed during a no-Descriptor-fetch transfer, but the corresponding run bit, <Run> field in the DMA Channel Control/Status Registers 0-31, is not set. For a no-Descriptor-fetch transfer,

ASR CONFIDENTIAL

the <Stop interrupt> is not cleared when the DMA Source Address Registers 0-31, DMA Target Address Registers 0-31, and DMA Command Registers 0-31 are programmed. For a Descriptor-fetch transfer, the <Stop interrupt> field is cleared when the DMA Controller updates the DMA Descriptor Address Registers.

- Running—For a Descriptor-fetch transfer, after programming DDADRx and setting the <Run> field, four words of Descriptors are fetched from the memory and the <Stop interrupt> field continues to be clear. For a no-Descriptor-fetch transfer, after programming the DMA Source Address Registers 0-31, DMA Target Address Registers 0-31 and DMA Command Registers 0-31 and setting the <Run> field, the corresponding channel clears the <Stop interrupt>, skips the "Descriptor-fetch, running" state and enters the "Wait for Request" or "Transfer Data" state (see Figure 44).
- Wait for request—occurs as the channel waits for a request before it starts to transfer data;
  <Stop interrupt> is clear.
- Transfer data—transferring data between the source and the target; <Stop interrupt> is clear.
- Channel error—the channel with the error remains in the Stopped state until software clears the error condition, re-initializes the channel, and sets the <Run> field and the <Bus error interrupt> field in the DMA Channel Control/Status Registers 0-31. For details, refer to the DMA Interrupt Register in the *ASR1606 Registers*.
- Stopped—the channel is stopped. The <Stop interrupt> field is set. For a no-Descriptor-fetch transfer, a stopped channel is re-initialized by updating the DMA Source Address Registers
  0-31, DMA Target Address Registers 0-31, and DMA Command Registers 0-31, then setting the
  <Run> field. For a Descriptor-fetch transfer, a stopped channel is re-initialized by updating the DMA Descriptor Address Registers and setting <Run>.

Table 30 summarizes the DMAC channel states.

ASR CONFIDENTIAL

Table 30:Channel States Based on Software Configuration

Descriptor Mode	Software Configuration	<Run>	<Stop interrupt>	Resulting Channel State
Descriptor-Fetch mode	Power-up	0	1	Uninitialized
	Write to DDADRx before DCSRx[RUN] is set (recommended flow)	0	0	Valid Descriptor, not running
	Set DCSRx[RUN] after writing to DDADRx **(recommended flow)**	1	0	Running
	Set DCSRx[RUN] before writing to DDADRx ***(not recommended)***	1	1	Invalid
	Write to DDADRx after DCSRx[RUN] is set ***(not recommended)***	1	0	Descriptor fetch, running.
	Stop running channel by clearing DCSRx[RUN] and DCSRx[MASKRUN]	0	0 -> 1	Channel, if not immediately, eventually switches to a Stopped state (identified by DCSRx[STOPINTR] toggling from low to high).
No-Descriptor-Fetch mode	Power-on	0	1	Uninitialized
	Write to DSADRx, DTADRx and DCMDx before DCSRx[RUN] is set **(recommended flow)**	0	1	Valid Descriptor, not running
	Set DCSRx[RUN] after configuring DSADRx, DTADRx, and DCMDx **(recommended flow)**	1	0	Running
	Set DCSRx[RUN] before configuring DSADRx, DTADRx, and DCMDx **(*not recommended*)**	1	0	Wait for Request, running. ***Channel uses current DSADRx, DTADRx and DCMDx for the transfer and may lead to unpredictable results.***
	Stop running channel, b y clearing DCSRx[RUN] and DCSRx[MASKRUN]	0	0 -> 1	Channel, if not immediately, eventually switches to a Stopped state (identified by DCSRx[STOPINTR] toggling from low to high)

ASR CONFIDENTIAL

11.3.2 DMA Descriptors

A DMA Descriptor is a four-word (32-bits per word) block, aligned on a 16-byte boundary in memory:

- Word [0] contains a value for the DMA Descriptor Address Register and a single flag bit (<Stop>).
- Word [1] contains a value for the DMA Source Address Register.
- Word [2] contains a value for the DMA Target Address Register.
- Word [3] contains a value for the DMA Command Register.

The DMAC can be operated in two distinct modes based on the DCSRx[NODESCFETCH] bit:

- DCSRx[NODESCFETCH] = 0 - Descriptor-fetch transfer
- DCSRx[NODESCFETCH] = 1 - No-Descriptor-fetch transfer

11.3.2.1 Descriptor-Fetch Transfer Operation

Descriptor-fetch transfers (<No-Descriptor Fetch> field in the DMA Channel Control/Status Registers 0-31 = 0) operate in the following manner.

Software must first clear the <Run> field and then clear the <No-Descriptor

Fetch> field. Software must write a valid Descriptor address to the DMA Descriptor Address Register and then set <Run>. Doing so in this order enables the DMAC to fetch the four-word Descriptor (if the memory is already set up with the Descriptor chain) from the memory that the DMA Descriptor Address Register indicates. The channel either waits for a request or starts the data transfer, as determined by the DCMDx[FLOW] source and target bits. After the channel transfers a number of bytes equal to the smaller of <Maximum burst size> and <Length of the transfer in bytes> fields in the DMA Command Registers 0-31, it either waits for the next request or continues with the data transfer until the <Length of the transfer in bytes> field reaches zero. The channel stops or continues with a new Descriptor fetch from the memory, as determined by the <Stop> field in the DMA Descriptor Address Registers. Figure 44 summarizes this operation.

If an error occurs during the Fetch operation, the channel enters the Stopped state and remains there unless the software clears the error condition, re-initializes the channel, and sets the <Run> field.

When a channel switches between a Descriptor-fetch transfer and a no-Descriptor-fetch transfer, it must be stopped before the mode switch.

For a Descriptor-fetch transfer, the software must load the DMA Descriptor Address Register and set the <Run> field. The channel-Descriptor fetch does not occur unless the <Run> field is set.

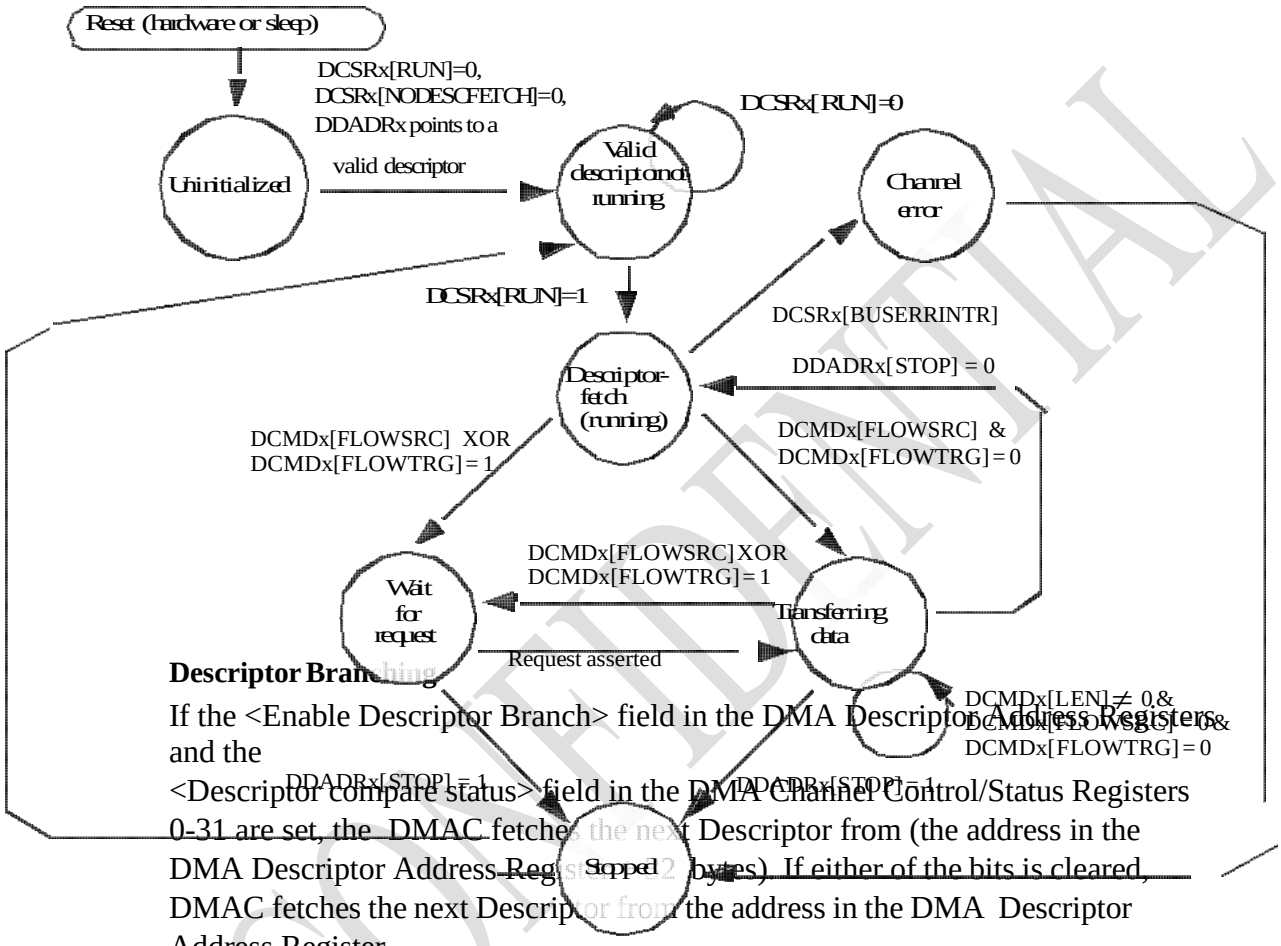
ASR CONFIDENTIAL

Although software loads the DMA Descriptor Address Register, the DMA Source Address Registers 0-31, DMA Target Address Registers 0-31, and DMA Command Registers 0-31 must be loaded indirectly from the DMA Descriptors. The DMA Descriptors pointed to by the DMA Descriptor Address Register are loaded into the registers of the associated DMA channel when a Write to the <Run> field switches the channel from stopped to running.

Bit [0] (<Stop>) of word [0] of a DMA Descriptor (the low bit of the DMA Descriptor Address Register field) is used to mark the special Descriptor, which resides at the end of a Descriptor list. The value of the stop bit does not affect the loading of the fields of a Descriptor into Channel registers in any way; however, if a Descriptor with the stop bit set is loaded into a Channel register, then the channel stops after completely transferring the data pertaining to that channel.

ASR CONFIDENTIAL

Figure 43:Descriptor-Fetch Transfer Channel State Diagram



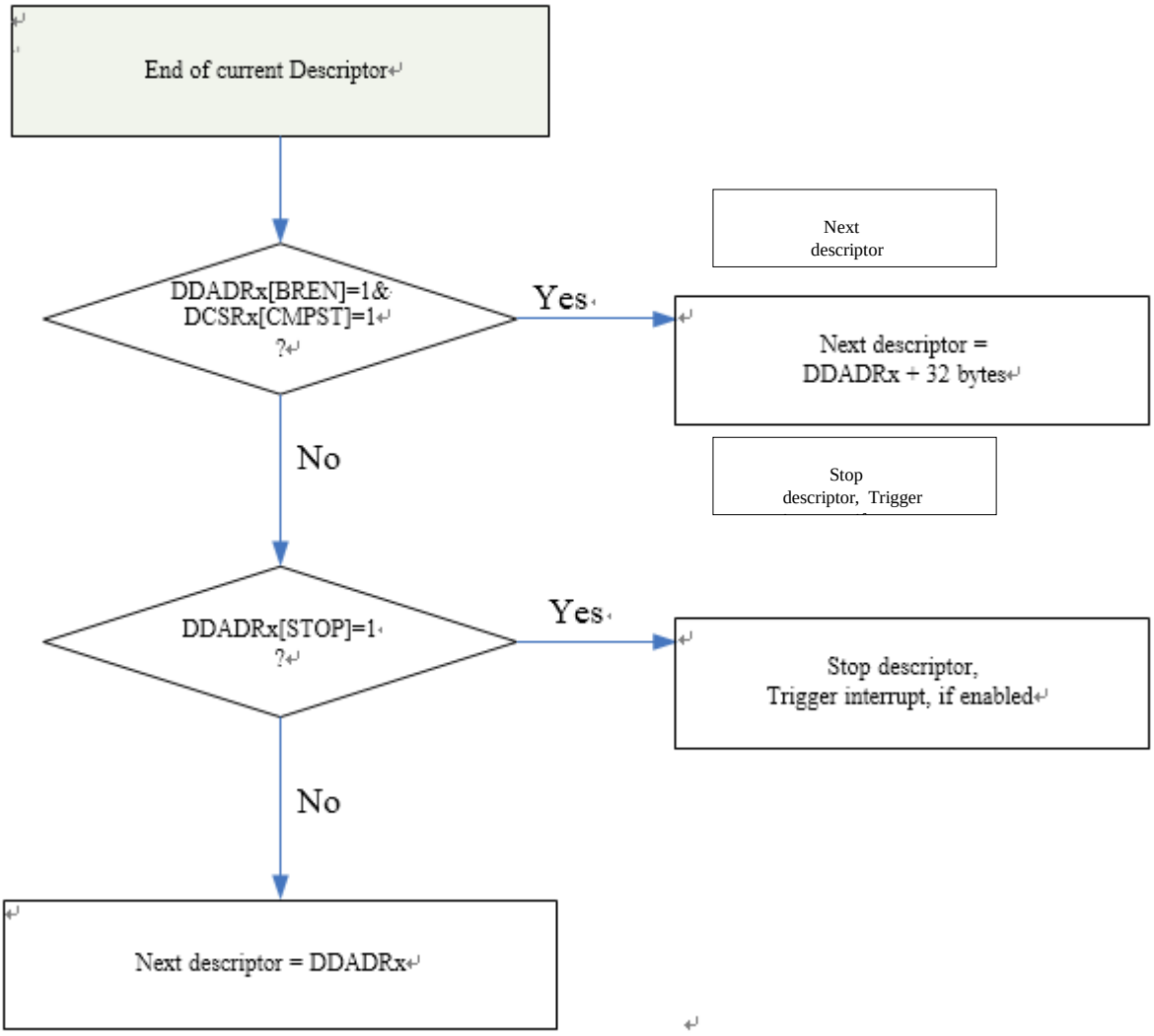
Descriptor Branching

If the <Enable Descriptor Branch> field in the DMA Descriptor Address Registers and the <Descriptor compare status> field in the DMA Channel Control/Status Registers 0-31 are set, the DMAC fetches the next Descriptor from (the address in the DMA Descriptor Address Register + 32 bytes). If either of the bits is cleared, DMAC fetches the next Descriptor from the address in the DMA Descriptor Address Register.

The <Enable Descriptor Branch> field is relevant only for Descriptor-fetch transfers (<No-Descriptor Fetch> field in the DMA Channel Control/Status Registers 0-31 is cleared).

ASR CONFIDENTIAL

Figure 44:Flow Chart for Descriptor Branching



Copyright © 2022 ASR

11.3.2.2 No-Descriptor-Fetch Transfer Operation

The typical no-Descriptor-fetch transfer (<No-Descriptor Fetch> = 1) operation follows.

The channel is in an uninitialized state after reset. Software must first clear the <Run> field and then set the <No-Descriptor Fetch> field. Software must write a valid source physical address to the DMA Source Address Registers 0-31, a target physical address to the DMA Target Address Registers 0-31, and a command to the DMA Command Registers 0-31. The DMA Descriptor Address Registers is reserved in this mode and must not be written. Next, the <Run> field must be set. A Descriptor fetch is not performed. Depending on the <Source flow control> and <Target flow control> fields in the DMA Command Registers 0-31, the channel either waits for a request or starts the data transfer. After transferring the number of bytes equal to the smaller of <Maximum burst size> and <Length of the transfer in bytes>, the channel either waits for the next request or continues with the data transfer until the <Length of the transfer in bytes> reaches zero, depending on the priority of the enabled DMA channels. The channel stops when <Length of the transfer in bytes> reaches zero. Figure 45 summarizes this operation.

Setting the <Stop interrupt enabled> field in the DMA Channel Control/Status Registers 0-31 to trigger a stop interrupt may cause the DMAC to trigger the stop interrupt prematurely, possibly even before the channel enters active-run mode. See the <Stop interrupt enabled> and <Stop interrupt> fields in the DMA Channel Control/Status Registers 0-31 for more information on the RUN condition.

ASR CONFIDENTIAL

To detect stoppage of a channel as the result of an end-of-receive (EOR) from a peripheral, software must validate the stoppage by checking the status of the <End of Receive Interrupt> field in the DMA Channel Control/Status Registers 0-31. If the EOR condition stopped the channel, then <End of Receive Interrupt> is set. See DMA Channel Control/Status Registers 0-31 for more information.

Detect normal channel stoppage by using the end interrupt (<End interrupt> field in the DMA Channel Control/Status Registers 0-31), not the stop interrupt (<Stop interrupt>). See DMA Channel Control/Status Registers 0-31 for more information.

If an error occurs during the Fetch operation, the channel enters the Stopped state and remains there unless software clears the error condition and sets the <Run> field.



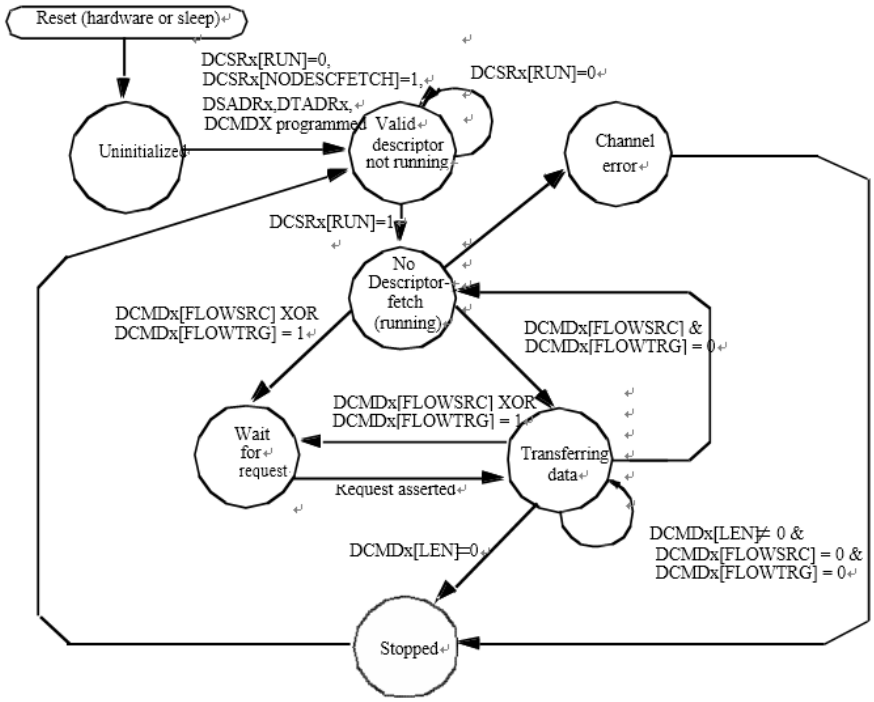
Note

When a channel switches between a Descriptor-fetch transfer and a no-Descriptor-fetch transfer, must be stopped before the switch occurs.

It is possible to use a few DMA channels for a Descriptor-fetch transfer and the remaining channels in a no-Descriptor-fetch transfer simultaneously. Do not program the DMA Descriptor Address Registers of the channels during a no-Descriptor-fetch transfer.

ASR CONFIDENTIAL

Figure 45:No-Descriptor-Fetch Transfer Channel State Diagram



ASR CONFIDENTIAL

11.3.3 Transferring Data

The on-chip peripherals connected to the DMA via the peripheral bus operate as flowthrough transfers (For details, refer to Servicing Internal Peripherals Using Flowthrough DMA Read Cycles,). Although the source or destination of a DMA transfer is usually a peripheral intended to be used as a source or sink of DMA data, the DMAC can transfer data to or from any memory location through memory-to-memory moves.

11.3.3.1 Servicing Internal Peripherals

The processor has two types of internal peripherals: the *peripheral-bus peripherals* and the *system- bus peripherals*. The DMAC provides DMA requests to the DMA Request to Channel Map Registers 0-63 and 64-99 that contain five bits of channel number for each of the possible DMA requests.
These possible peripheral requests are mapped to 32 available channels. If the on-chip peripheral address resides in the DMA Source Address Registers 0-31, the <Source flow control> field must be set to allow the processor to wait for the request before it initiates the transfer. If the on-chip peripheral address resides in the DMA Target Address Registers 0-31, then the <Target flow control> field must be set.

If <End interrupt enable> is set, a DMA interrupt is requested at the end of the last cycle associated with the byte that caused the <Length of the transfer in bytes> field to decrement to zero.

11.3.3.1.1 Servicing Internal Peripherals Using Flowthrough DMA Read Cycles

A flowthrough DMA Read begins when an on-chip peripheral sends a request to a channel in the DMAC while the channel is running. The number of bytes to be transferred is specified using the
<Length of the transfer in bytes>. The following process begins when the request is recognized:

1. The DMAC prompts the Memory Controller to read the required number of bytes addressed by DMA Source Address Registers 0-31 into a 32-byte buffer in the DMAC.
2. The DMAC transfers the data to the peripheral device addressed in the DMA Target Address Registers 0-31 [31:0]. The <WIDTH> field in the DMA Command Registers 0-31 specifies the width of the internal peripheral to which the transfer is being made.
3. At the end of the transfer, the DMA Source Address Registers 0-31 is increased and the
   <Length of the transfer in bytes> field is decreased by the smaller of <Length of the transfer in bytes> and <Maximum burst size>.

Use the following settings for the DMAC register bits for a flowthrough DMA Read from an internal peripheral:

ASR CONFIDENTIAL

<Source address> field in the DMA Source Address Registers 0-31 = memory address

<Target address> field in the DMA Target Address Registers 0-31 = internal peripheral address

<Source address increment> field in the DMA Command Registers 0-31 = 1

<Target address increment> in the DMA Command Registers 0-31 = 0

<Source flow control> field in the DMA Command Registers 0-31 = 0

<Target flow control> field in the DMA Command Registers 0-31 = 1

11.3.3.1.2 Servicing Internal Peripherals Using Flowthrough DMA Write Cycles

A flowthrough-DMA Write begins when an on-chip peripheral sends a request to a channel in the DMAC while the channel is running. The number of bytes to be transferred is specified using the
<Maximum burst size> field. When the request is recognized, the following process begins:

1. The DMAC processes the request by transferring the required number of bytes from the peripheral device addressed by DMA Source Address Registers 0-31 [31:0] into a DMAC buffer.
2. The DMAC transfers the data to the Memory Controller. The <WIDTH> field specifies the width of the internal peripheral from which the transfer is being made.

ASR CONFIDENTIAL

3. At the end of the transfer, DTADRx is increased and <Length of the transfer in bytes> is decreased by the smaller of <Length of the transfer in bytes> and <Maximum burst size>.

Use the following settings for the DMAC register bits for a flowthrough-DMA Write from an internal peripheral:

<Source address> = internal peripheral address

<Target address> = memory address

<Source address increment> = 0

<Target address increment> = 1

<Source flow control> = 1

<Target flow control>

= 0 Memory-to-

Memory Moves

Memory-to-memory moves do not involve request signals. For a memory-to-memory move, the processor writes to the <Run> field indicated by the channel that is configured to perform a memory-to-memory move. The <Source flow control> and <Target flow control> fields must be cleared by software as soon as the Descriptor is fetched. The transfer then begins.

If <End interrupt enable> is set, a DMA interrupt is requested at the end of the last cycle associated with the byte that caused <Length of the transfer in bytes> to decrement to zero.

Note Memory-to-memory moves can be performed from internal SRAM to external memory and vice versa.

11.3.3.1.3 Memory-to-Memory Moves: Flowthrough DMA Read/Write Cycles

A flowthrough DMA memory-to-memory Read or Write begins when the processor the DCSRx[RUN] bit. If the channel is in a Descriptor-fetch transfer, it fetches the four-word Descriptor. The <Source flow control> and <Target flow control> fields must be cleared for a memory-to-memory move. The channel begins to transfer data without waiting for a PREQ or DREQ assertion. The number of bytes to be transferred is specified using <Length of the transfer in bytes>. Processing begins as follows:

1. The DMAC prompts the Memory Controller to read the required number of bytes addressed by the DMA Source Address Registers 0-31 into a 32-byte

ASR CONFIDENTIAL

buffer in the DMAC.

2. The DMAC generates a Write cycle to the location addressed by the DMA Target Address Registers 0-31.
3. At the end of the transfer, DMA Source Address Registers 0-31 and DMA Target Address Registers 0-31 are increased and <Length of the transfer in bytes> is decreased by the smaller of <Length of the transfer in bytes> and <Maximum burst size>.

Use the following settings for the DMAC register bits for flowthrough memory-to-memory moves:

<Source address> = memory address

<Target address> = memory address

<Source address increment> = 1

<Target address increment> = 1

<Source flow control> = 0

<Target flow control> = 0

ASR CONFIDENTIAL

11.3.4 Programming Tips

This section provides information concerning software requirements, instruction ordering, and misaligned memory accesses.

11.3.4.1 Software Management Requirements

The information that must be maintained on a per-stream basis (for example, the memory address, the peripheral address, the transfer count, and the implied direction of data flow) is maintained in Descriptor registers in the DMAC. The Descriptor registers are loaded from memory locations specified by the software. Multiple DMA Descriptors can be chained together in a list to allow a DMA channel to transfer data to and from a number of separate locations. The Descriptor-based DMA design allows Descriptors to be added dynamically to an active DMA channel Descriptor chain, which is particularly useful in applications that involve network-transmit lists and network-receiver buffer-free lists.

Each demand for data that a peripheral generates is a memory data Read or Write. A peripheral must not request a DMA transfer unless it is prepared to read or write the full data block (8, 16, or 32 bytes) and can handle trailing bytes that may occur at the end of a DMA transfer.

11.3.4.2 Programmed I/O Operations

The processor can read and write the peripheral registers and FIFOs on the peripheral bus. The peripherals' internal registers must be accessed using word-access loads and stores. Internal register and FIFO space must be mapped as non-cacheable. Byte and half-word accesses to internal registers are not allowed. The FIFOs of some of the peripherals on the peripheral bus are accessed using byte, half-word, or word-access loads and stores. Refer to the individual peripheral sections for details.

11.3.4.3 Instruction Ordering

The DMAC executes programmed I/O instructions in the order specified by the software.

References to internal addresses generally complete more quickly than those issued to external addresses. This means that memory accesses can be sent in one order and completed in a different order.

The DMAC ensures that memory references made by a single DMA channel are presented to memory in order and that the Descriptor fetches occur between the data blocks. The order in which the accesses are completed cannot be guaranteed unless the channels refer to only one type of memory (either to the external memory or to the internal SRAM).

The channel references must not reference both internal and external memory in a DMA Descriptor chain for the following operations:

ASR CONFIDENTIAL

Copyright © 2022 ASR

Self-modifying DMA Descriptor chains.

Channels that write data blocks followed by status blocks while another channel (generally the processor) polls a field in the status block.

11.3.4.4 Misaligned Memory Accesses

The DMAC is a 64-bit device that can access memory on byte-aligned boundaries. The DMAC may encounter misaligned (not aligned to 64-bit boundary) addresses while it accesses memory.

Only the following type of data transfers may access misaligned addresses:

1. Memory-to-memory transfers.
2. Memory-to-peripheral transfers or peripheral-to-memory transfers. In this case, the peripheral addresses are 32-bit aligned.

Note All companion-chip-related transfers must use 64-bit aligned addresses for both source and target locations.

ASR CONFIDENTIAL

Addresses must be 64-bit aligned in the compare-Descriptor mode.

The DMAC employs channel-specific alignment buffers that hold either the leading or the lagging misaligned data. These buffers must be empty when the DMAC performs a context switch to service the next pending channel. When the DMAC completes a Descriptor transfer, it ensures that all of the data in the alignment buffers is properly flushed to its respective targets.

Because the DMAC encounters overhead while it works with misaligned data, restricting memory addresses to 8-byte boundaries can be helpful. Align the source and target addresses to 32-byte boundaries for optimal DMAC and Memory Controller performance.

By default, during data transfers the DMA Controller forces the least significant 3 bits of all external addresses to zeros and the least significant 2 bits of all peripheral addresses to zeros. To activate byte-aligned addressing, software must activate the Alignment Register. See DMA Alignment Register for details.

11.3.5 How DMA Handles Trailing Bytes

DMA normally transfers bytes equal to the transaction size specified by <Maximum burst size>. But if the Descriptor is reaching its end, the number of trailing bytes in the <Length of the transfer in bytes> field could be smaller than the transfer size. The DMA can transfer the exact number of trailing bytes if the <Source flow control> and <Target flow control> fields are both 0 or if it receives a corresponding request from the on-chip/off-chip peripheral or companion chip. The following cases are possible:

- **Memory-to-memory moves**—The DMA transfers bytes equal to the smaller of <Length of the transfer in bytes> or <Maximum burst size>.
- **Companion-chip-related transfers (flowthrough)**—The companion chip must assert the request to allow the DMA to handle the trailing bytes. If the request is asserted, the DMA transfers a number of bytes equal to the smaller of <Length of the transfer in bytes> and <Maximum burst size>.
- **Memory-to-on-chip-peripheral transfers**—Most of the on-chip peripherals send a request for trailing bytes. Refer to the appropriate section of this document for details of a specific peripheral's operation. The DMA transfers a number of bytes equal to the smaller of <Length of the transfer in bytes> and <Maximum burst size>.
- **On-chip-peripheral-to-memory transfers**—Special handshaking signals and interrupts are employed for transferring trailing bytes from an on-chip peripheral to memory. The conditions that use the handshaking signals and interrupts are explained below:
  - End of Packet (EOP)—the peripheral receives its last data sample from an

ASR CONFIDENTIAL

external codec and detects an EOP based on its Receive protocol. Any remaining data samples in the peripheral Receive FIFO are treated as trailing bytes. The peripheral can be programmed to initiate a DMA request, even if it has fewer bytes than its Receive trigger threshold. The DMA responds to this request and reads out the trailing bytes. CPU intervention is not required as long as the Descriptor chain has not ended.

Time Out (TO)—peripherals that do not support EOP protocols use a time-out mechanism to determine if they have received their last data sample. Refer to the peripheral sections for details of the time out implementation. Any remaining data samples in the peripheral Receive FIFO are treated as trailing bytes. The peripheral can be programmed to initiate a DMA request, even if it has fewer bytes than its Receive trigger threshold. The DMA responds to the DMA request and reads out the trailing bytes. CPU intervention is not required as long as the Descriptor chain has not ended.



Note When a peripheral signals either an EOP or a TO from an external device, the DMAC sets the end-of-receive (EOR) status bit in the corresponding channel Control Status register (DCSRx). See DMA Channel Control/Status Registers 0-31 for details.

- End-of-Descriptor chain (EOC)—indicates that a DMA channel is at the end of its last Descriptor. After the current transfer, <Length of the transfer in bytes> = 0 and <Stop> = 1. DMA signals the peripheral on an EOC and the peripheral interrupts the CPU to retrieve any trailing bytes. Refer to the individual peripheral sections for details on the processor interrupt implementation. EOC is the only trailing-bytes case that requires programmed I/O to retrieve data.
- Request-after-channel-stops (RAS)—status bit in the DMA Channel Control/Status Register 0-31. This bit is set when a peripheral asserts a DMA request after the channel to which the peripheral is mapped has stopped. See DMA Channel Control/Status Register 0-31 for details.

The following example illustrates the handling of various trailing bytes using EOR, EOC, and RAS. **Example**—the peripheral signals a DMA request to service trailing bytes in its Receive FIFO (RxFIFO). The current Descriptor <Length of the transfer in bytes> is equal to or greater than the trailing-bytes count.

1. The peripheral signals a Receive DMA request.
2. The DMAC responds and reads out all trailing bytes including the last byte.
3. The peripheral signals an EOR.
4. The DMAC transfers all trailing bytes to the channel target and then updates the <End of Receive Interrupt> field.

ASR CONFIDENTIAL

5. The DMA channel can be configured to stop, jump, or just wait for another request after receiving EOR, depending on the <Stop channel on EOR> and <Jump to the next Descriptor on EOR> fields in the DMA Channel Control/Status Registers 0-31. The <End of Receive Interrupt> field must be cleared before restarting a channel.
6. The <End of Receive Interrupt> field set indicates that all trailing bytes were read and transferred to the required target.
   - UART1, UART2, UART3, UART4
   - SSP1, SSP2, SSP3
   - USIM

11.3.6 Quick Reference to DMA Programming

Table 31 through Table 34 tabulate width, alignment, and address increment modes for various DMA data transfers.

ASR CONFIDENTIAL

Table 31:Configuration for Peripheral Bus Peripheral Related Data Transfers

Source	Target	Source Alignment (bytes)	Target Alignment (bytes)	<Source address increment> (binary)	<Target address increment> (binary)	<WIDTH> (binary)
Peripheral Bus Peripheral	Memory	4	1	0 or 1	1	01, 10 or 11
Memory	Peripheral Bus Peripheral	1	4	1	0 or 1	01, 10 or 11
Peripheral Bus Peripheral	Expansion memory FIFO	4	1	0 or 1	0	01, 10 or 11
Expansion memory FIFO	Peripheral Bus Peripheral	1	4	0	0 or 1	01, 10 or 11

NOTE: Memory refers to all types of memory explained in the Memory Controller chapter (including internal memory, external memory, variable-latency I/O memory, and expansion memory implemented in non-FIFO mode).

If a memory address is byte-aligned, then the DMA Alignment Register must be programmed. Refer to the *ASR1606 Registers* for details.

If <Source address increment> or <Target address increment> is set, then the DMAC increments the source or target address, after each transaction, by a number equal to the width of the peripheral bus peripheral. For example, if the DMAC is transferring 32 bytes of data from memory to a 4-byte wide peripheral bus peripheral, then the DMAC writes 4 bytes to the peripheral bus peripheral eight times and increments the target address by 4 bytes after each of the eight transactions.

Table 32:Configuration for Memory-to-Memory Data Transfers

Source	Target	Source Alignment (bytes)	Target Alignment (bytes)	<Source address increment> (binary)	<Target address increment> (binary)	<WIDTH> (binary)
Memory	Memory	1	1	1	1	00
Expansion memory FIFO	Memory	1	1	0	1	00
Memory	Expansion memory FIFO	1	1	1	0	00
Expansion memory FIFO	Expansion memory FIFO	1	1	0	0	00

NOTE: Memory refers to all types of memory explained in the Memory Controller chapter (including internal memory, external memory, variable-latency I/O memory, and expansion memory implemented in non-FIFO mode).

The DMA Alignment Register must be programmed if a memory address is byte aligned. Refer to the *ASR1606 Registers* for details.

ASR CONFIDENTIAL

Table 33:System Bus Peripheral Related Data Transfer Configuration

Source	Target	Source Alignment (bytes)	Target Alignment (bytes)	<Source address increment> (binary)	<Target address increment> (binary)	<WIDTH> (binary)
System Bus Peripheral	Memory	8	8	0 or 1	1	00
Memory	System Bus Peripheral	8	8	1	0 or 1	00
System Bus Peripheral	Expansion memory FIFO	8	8	0 or 1	0	00
Expansion memory FIFO	System Bus Peripheral	8	8	0	0 or 1	00

NOTE: Memory refers to all types of memory explained in the Memory Controller chapter (including internal memory, external memory, variable-latency I/O memory, and expansion memory implemented in non-FIFO mode).

If <Source address increment> or <Target address increment> is set, the DMAC increments the source or target address after each bursting transaction by a number equal to the transaction burst size (8, 16, or 32 bytes) or <Length of the transfer in bytes>. The latter is used if <Length of the transfer in bytes> is less than the burst size. For example, if the DMAC is transferring 48 bytes of data from memory to the system bus peripheral in bursts of 32 bytes, the DMAC increments the target address by 32 after the first burst and then by 16 after the second burst.

Table 34:Configuration for Companion Chip (CC) Related Data Transfers

Source	Target	Source Alignment (bytes)	Target Alignment (bytes)	<Source address increment> (binary)	<Target address increment> (binary)	<WIDTH> (binary)
CC or External Peripheral	Memory	8	8	0 or 1	1	00
Memory	CC or External Peripheral	8	8	1	0 or 1	00
CC or External Peripheral	Expansion memory FIFO	8	8	0 or 1	0	00
Expansion memory FIFO	CC or External Peripheral	8	8	0	0 or 1	00

NOTE: Memory refers to all types of memory explained in the Memory Controller chapter (including internal memory, external memory, variable-latency I/O memory, and expansion memory implemented in non-FIFO mode).

ASR CONFIDENTIAL

Any memory can be used for flowthrough Data-Transfer mode. The companion chip or external peripheral must be connected as a variable-latency I/O memory.

If <Source address increment> or <Target address increment> is set, then the DMAC increments the source or target address, after each bursting transaction, by a number equal to the transaction burst size (8, 16, or 32 bytes) or <Length of the transfer in bytes>. The latter is used if <Length of the transfer in bytes> is less than the burst size. For example, if the DMA is transferring 48 bytes of data from memory to the companion chip in bursts of 32 bytes, then the DMAC increments the target address by 32 after the first burst and then by 16 after the second burst.

Table 35 provides a quick reference for programming the DMAC for the on-chip peripherals.

ASR CONFIDENTIAL

Table 35:DMA Quick Reference for On-Chip Peripherals

Unit	Function	FIFO Address	Width (bytes)	DCMDx Width (binary)	Burst Size (bytes)	Source or Target	DRCMRx
Pin Mux DREQ 0							0x0100
Pin Mux DREQ 1							0x0104
UART 2	Receive	0xD401_7000	1 or 4	01 or 11	8, 16, 32, or trailing	Source	0x0110
	Transmit	0xD401_7000	1 or 4	01 or 11	8, 16, 32, or trailing	Target	0x0114
GSSP	Receive						0x0118
	Transmit						0x011C
USIM 1	Receive		1	01	8 or trailing	Source	0x0120
	Transmit		1	01	8 or trailing	Target	0x0124
USIM 2	Receive		1	01	8 or trailing	Source	0x0128
	Transmit		1	01	8 or trailing	Target	0x012C
APB E-Cipher							0x0130
UART 1	Receive	0xD401_8000	1 or 4	01 or 11	8, 16, 32, or trailing	Source	0x0154
	Transmit	0xD401_8000	1 or 4	01 or 11	8, 16, 32, or trailing	Target	0x0158
UART 3	Receive	0xD401_8000	1 or 4	01 or 11	8, 16, 32, or trailing	Source	0x015C
	Transmit	0xD401_8000	1 or 4	01 or 11	8, 16, 32, or trailing	Target	0x0160
SSP 1	Receive	0xD401_B010	1, 2, or 4	01, 10, or 11	8, 16, 32, or trailing	Source	0x01D0
	Transmit	0xD401_B010	1, 2, or 4	01, 10, or 11	8, 16, 32, or trailing	Target	0x01D4
SSP 3	Receive	0xD401_C010	1, 2, or 4	01, 10, or 11	8, 16, 32 or trailing	Source	0x01F0
	Transmit	0xD401_C010	1, 2, or 4	01, 10, or 11	8, 16, 32 or trailing	Target	0x01F4
GEU	Receive	0xD420_1028	16	10	16	Source	0x1110
	Transmit	0xD420_1058	16	10	16	Target	0x1114
Pin Mux DREQ 2							0x1128

ASR CONFIDENTIAL

Examples

Setting Up and Starting a Channel

The following code example shows how to set up and start a channel to transfer LEN words that start at the address in the DMA Source Address Register 0-31 to peripheral address in the DMA Target Address Register 0-31. In this example, the <Stop> field is set so that the DMA channel stops after it completely transfers LEN bytes of data associated with this Descriptor.

```
// build real
Descriptor
desc[0].ddadr =
STOP; desc[0].dsadr
= DSADR;
desc[0].dtadr =
DTADR; desc[0].dcmd
= DCMD;
// start the channel
DMANEXT[CHAN] =
&desc[0]; DRUN = 1;
```

Creating a Zero-Length Descriptor

The following code example shows how to initialize a Descriptor list for a channel that is running:

```
// Allocate a new Descriptor, and make it an End-
// Descriptor whose "ddadr" field points back
at itself newDesc = New Desc();
newDesc->ddadr = newDesc | STOP;
// make it a zero length
Descriptor newDesc->dcmd
= ZERO;
// Start the channel
DMANEXT[CHAN] =
newDesc; DRUN = 1;
```

ASR CONFIDENTIAL

The channel starts up, loads the Descriptor into its registers, detects that the transfer length is zero
and the stop bit is set, and stops the channel. No data transfer occurs in this case. To restart the channel, write to its DMA Descriptor Address Register, then set the <Run> field.

Initializing a Channel to Be Used by Direct DMA Master

The most effective way to move data between a peripheral device and memory is to use the built-in Descriptor-based DMA system. In most applications, a free DMA channel is readily available.

Some applications demand true direct-memory access. Each application has different requirements, so a Descriptor-based DMA may be best for some applications while a non-Descriptor-based DMA is best for others. For applications that cannot tolerate the time required to fetch a Descriptor before each DMA transfer, choose the non Descriptor-based DMA method. For applications that can tolerate it, a Descriptor-based DMA method can reduce the amount of CPU intervention.

By programming the Descriptor-based DMA system, it is possible to offer true direct-memory access to devices that require it.

ASR CONFIDENTIAL

In the following example, the companion chip requires the DMA to read four Descriptor words, program a channel as indicated by Descriptor, and continue the transfer based on the new Descriptor. Such a companion chip makes the following requirements:

1. When the companion chip asserts DREQ, fetch four Descriptor words from one of its ports.
2. Based on the information contained in the Descriptor, transfer data from the source address to the destination address without waiting for another request from the companion chip.
3. Transfer the number of bytes mentioned in <Length of the transfer in bytes>, then return to Step 1.

Such an external device can use a constant Descriptor in memory:

A channel that is used by a direct DMA master must be initialized in a special way. The following code example shows how to initialize such a channel:

```
struct {long
   ddadr;
   long dsadr;
   long dtadr;
   short len;
   short dcmd;
} desc[2];
desc[0].ddadr = &desc[1];
desc[0].dsadr = I_ADR +
I_DESC_OFFS; desc[0].dtadr
= &desc[1].dsadr;
desc[0].len = 8;
desc[0].dcmd = CMD_IncTrgAdr |
CMD_FlowThru; desc[1].ddadr =
&desc[0];
desc[1].dtadr = I_ADR +
I_DATA_OFFS; desc[1].dsadr = 0;
desc[1].len = 0;
desc[1].dcmd = 0;
```

When the external device has data to transfer, it makes a DMA request in the

ASR CONFIDENTIAL

standard way. The DMAC wakes up and reads four words from the device I_DESC_OFFS address. The DMAC transfers only four 32-bit words because the first Descriptor has an eight-byte count. The four words supplied by the external device are written over the DMA Source Address Registers 0-31, DMA Target Address Registers 0-31, and DMA Command Registers 0-31 of the next Descriptor. The DMAC then steps into the next (dynamically modified) Descriptor and uses the devices I_DATA_ OFFS address to process the requested transfer. When the data transfer is complete, the DMAC steps back to the first Descriptor and the process repeats.

Adding a Descriptor to End-of-Descriptor List (Channel Running)



Note The following example assumes that a Descriptor-fetch transfer is active.

DMA Descriptor lists are used as queues of full buffers for network transmitters and as queues of empty buffers for network receivers. Because each buffer can be small, on-the-fly manipulation of DMA Descriptor lists must be as efficient as possible.

ASR CONFIDENTIAL

To add a Descriptor to the end of a Descriptor list for a running channel:

1. Write <Run> = 0.
2. Wait for the channel to stop. The stop status is indicated in <Stop interrupt>.
3. Create the end Descriptor in the memory with the stop bit set.
4. Manipulate the DMA Descriptor Address Register of the last Descriptor of the current chain in the memory to ensure that the DMA Descriptor Address Register points to the newly created end Descriptor in step 3.
5. Create a new Descriptor with values in the DMA Descriptor Address Registers, DMA Source Address Registers 0-31, DMA Target Address Registers 0-31, and DMA Command Registers 0-31 that match those in the stopped DMA channel. The new Descriptor is the next Descriptor for this Descriptor list.
6. Examine the DMA channel registers and determine if the channel stopped in the last Descriptor of the chain. If it did, manipulate the DMA Descriptor Address Register of this Descriptor so that it points to the newly created end Descriptor.
7. Program the channel DMA Descriptor Address Register with the next Descriptor created in Step 5.
8. Set <Run> = 1.

Using Software Implementation of Full and Empty Bits

This example shows how to use the compare Descriptor and branch Descriptor modes to allow the software to implement full and empty bits. As shown in Figure 46, this implementation can be used to create a circular Descriptor chain to transfer a large amount of data. The full and empty bits are maintained in a memory table and are cleared prior to a DMA transfer. The DMAC uses the compare Descriptor mode to determine that the full and empty bits are cleared before it performs a data transfer. If either bit is set, the DMAC uses the branch Descriptor mode to branch to a Descriptor in which the <Stop> field is set, which terminates the transfer. If the full and empty bits are both cleared, the DMAC continues to perform a regular data transfer. After the DMAC completes the data transfer, it updates the full and empty memory table by setting the full and empty bits, which indicates a successful transfer.

ASR CONFIDENTIAL



Note

The setting of the <Alignment control for channel x> field in the DMA Alignment Register is relevant even when DMA Target Address Registers 0-31 contains immediate data rather than an address. If <Alignment control for channel x> is not set, then the lowest three bits of the fetched data are forced to 0 (as they would in an address case, forcing data to be 8-byte aligned). Thus, for any comparisons with <Alignment control for channel x> not set, the least significant 3 bits must always be 0 for a comparison with immediate data to work. This restriction is not present when <Alignment control for channel x> is set.

This example assumes the following memory addresses: SRC0, SRC1: source-address locations

TRG0, TRG1: target-address locations

FEUPDT: address pointing to data 0x0000_0003. Bit[1:0] = [Full:Empty].

In the example, the memory table contains the following full and empty bit information before the DMA transfers. Only the two least significant bits are valid. Bit[1:0] = [Full:Empty].

FETBL0: 0x0000_0000

FETBL1: 0x0000_0000

After successful DMA transfers, the same memory table contains the following information: FETBL0: 0x0000_0003

ASR CONFIDENTIAL

FETBL1: 0x0000_0003

The following code is a chain of Descriptors for full- and empty-bit implementation:

First Descriptor Set

First Descriptor

```
//Compare and Branch Descriptor modes enabled.
//No data transferred by this Descriptor.
//Source is indirectly addressed and target is directly
addressed
//On a successful compare of &FETBL0 with 0x0000,
// Descriptor chain branches to desc[1] + 4*32bits, i.e
desc[2].
//If Compare fails, then Descriptor chain jumps to desc[1].
//Desc[1] stops the channel as Full and Empty bits were
not both 0. desc[0].ddadr = &desc[1], BrEn = 1;
desc[0].dsadr = FETBL0;
desc[0].dtadr = 0x0000;
desc[0].dcmd = CmpEn=1, AddrMode = b01;
```

Second Descriptor

```
//Error setting Descriptor, which stops the channel as
//(Full:Empty) != 0b00
//No data transferred. Stop interrupt
triggered. desc[1].ddadr = (Stop = 1);
desc[1].dsadr =
ignored;
desc[1].dtadr =
ignored;
desc[1].dcmd =
Len=0;
```

Third Descriptor

```
//Data transferring
Descriptor
desc[2].ddadr =
&desc[3];
```

ASR CONFIDENTIAL

desc[2].dsadr = SRC0;

desc[2].dtadr = TRG0;

desc[2].dcmd = Len=4K

bytes; Fourth Descriptor

//FullEmpty table updating

Descriptor desc[3].ddadr =

&desc[4]; desc[3].dsadr =

FEUPDT;

desc[3].dtadr = FETBL0;

desc[3].dcmd = Len=4

bytes; **Second Descriptor**

Set

First Descriptor

//Compare and Branch Descriptor modes enabled.

//No data transferred by this Descriptor.

//Source is indirectly addressed and target is directly addressed

ASR CONFIDENTIAL

```
//On a successful compare of &FETBL1 with 0x0000,
// Descriptor chain branches to desc[5] + 4*32bits, i.e
desc[6].
//If Compare fails, then Descriptor chain jumps to desc[5].
//Desc[5] stops the channel as Full and Empty bits were
not both 0. desc[4].ddadr = &desc[5], BrEn = 1;
desc[4].dsadr = FETBL1;
desc[4].dtadr = 0x0000;
desc[4].dcmd = CmpEn=1, AddrMode = b01;
```

Second Descriptor

```
//Error setting Descriptor, which stops the channel as
//(Full:Empty) != 0b00
//No data transferred. Stop interrupt
triggered. desc[5].ddadr = (Stop = 1);
desc[5].dsadr =
ignored;
desc[5].dtadr =
ignored;
desc[5].dcmd =
Len=0;
```

Third Descriptor

```
//Data transferring
Descriptor
desc[6].ddadr =
&desc[7];
desc[6].dsadr = SRC1;
desc[6].dtadr = TRG1;
desc[6].dcmd = Len=4K
bytes;
```

Fourth Descriptor

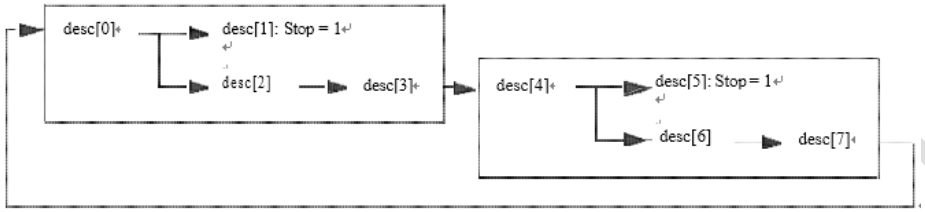
```
//FullEmpty table updating Descriptor
//Notice that the chain jumps back to desc[0].
//All ok, if software has loaded new data at the address
pointed to by
```

ASR CONFIDENTIAL

```
//desc[0] and has also
updated FETBL0
desc[7].ddadr = &desc[0];
desc[7].dsadr = FEUPDT;
desc[7].dtadr = FETBL1;
desc[7].dcmd = Len=4
bytes;
```

ASR CONFIDENTIAL

Figure 46:Descriptor Chain for Software Implementation of Full and Empty Bits



11.4 Register Descriptions

Refer to *ASR1606 Registers* for details on the DMA Controller registers.

ASR CONFIDENTIAL

12.USB 2.0 device

This chapter describes the USB 2.0 device features of the ASR1606.

- Section 12.1, Overview
- Section 12.2, Features
- Section 12.3, Block Diagrams
- Section 12.4, USB 2.0 device Operation
- Section 12.5, Register Descriptions
- Section 12.6, USB2 transmit FIFO usage note

12.1 Overview

The USB 2.0 device controller provides support for USB 2.0 device connections between ASR products and external USB 2.0 compliant hosts.

12.2 Features

- Device controller with integrated transceiver supports full USB2.0 functionality.
- High-speed and Full-speed functionality
- Software selectable port connection protocol
- Capability to respond as self or bus powered device and control to allow charging from bus.
- Processor accessible software configuration registers.
- Intelligent DMA controller is used to transfer data to and from system memory with configurable burst size.
- Configured by default to use bursting from the DMA controller with a maximum burst size of 64 bytes. This maximum size can be changed through SW to 128, 32 or 16 bytes.
- Device controller houses 16 IN and 16 OUT endpoints which act in the device function. EP 0 is always present as a control EP and may not be reconfigured.
- Device controller buffer: 11.5 KB transmit FIFO (1 KB FIFO for each first ten endpoints and 256B for each last six endpoints) and 2 KB receive FIFO

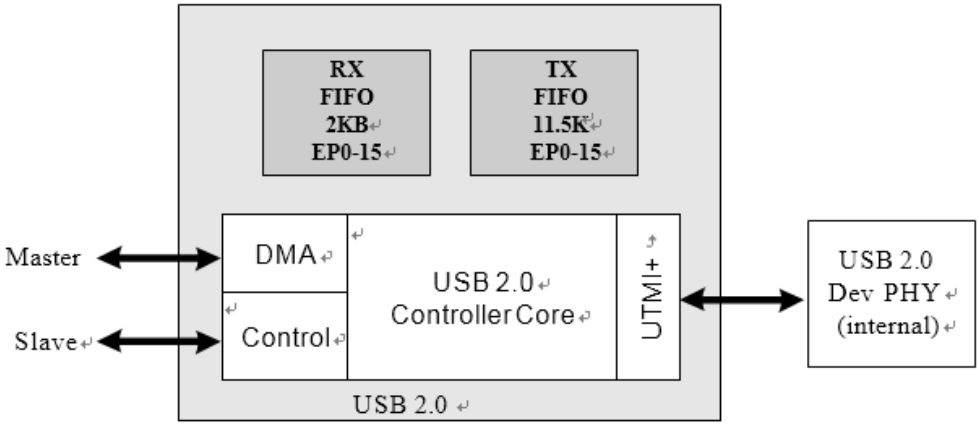
ASR CONFIDENTIAL

12.3 Block Diagrams

12.3.1 USB 2.0 Block Diagram

Figure 47 shows the overall USB 2.0 device controller unit including the USB 2.0 controller core, FIFO memory and interface connections.

Figure 47:USB 2.0 Block Diagram

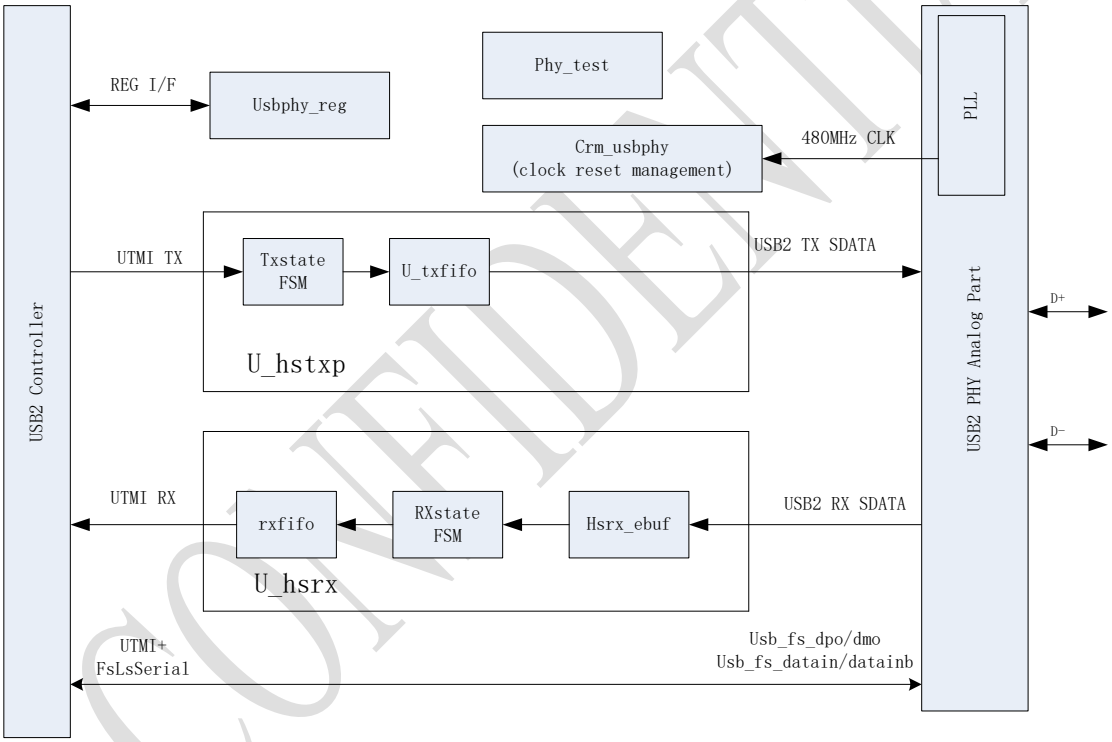


ASR CONFIDENTIAL

12.3.1.1 *USB 2.0 PHY Block Diagram*

Figure 48 shows the overall internal USB 2.0 PHY that is connected to the UTMI+ of the USB 2.0 controller. The block diagram does not show all power and ground pads required by the USB 2.0 PHY. Consult product pinout for more details.

Figure 48:USB 2.0 PHY Block Diagram



12.4 *USB 2.0 Operation*

12.4.1 Clock Control and Enables

The USB 2.0 Controller has separate clock enables for its system and USB clocks.

12.4.1.1 *System Clocks*

For the USB system clock, the USB AXI clock enable is located in the CPU Subsystem PMU:

- <USB AXI Clock Enable> field in the USB Clock/Reset Control Register is used for the USB 2.0 Controller
- Note: don't touch <USB dongle idle clock gating mask> field in the USB

ASR CONFIDENTIAL

Clock/Reset Control Register

12.4.1.2 Internal USB2 Device PHY

The internal USB2 Device PHY (transceiver) connects to the internal UTMI+ interface of the USB2 controller. The PHY contains a 480-MHz PLL that provides the 60-MHz UTMI clock required by the USB2 controller.

To enable the PHY for normal operation, perform the following steps:

10. SW shall enable USB AXI/AHB clock gating and release AXI/AHB reset(0xd428285c) before any configuration.
11. Wait for reset done. SW shall check reset bit of USBCMD (0xd4208140) to make sure usb controller is not under reset process before any configuration.
12. Wait for usb2 phy PLL ready. SW can monitor PLL status by checking USB2_reg01[0] (0xd4207004).
13. Release usb2 phy internal reset and enable clock gating

    Write 0x000060ef to USB2_reg01 (0xd4207004)

    Write 0x0000001c to USB2_reg0d (0xd4207034)

The PHY is ready for use.

12.4.1.3 FIFO Operation in the USB2 Controller

12.4.1.3.1 **Streaming Mode**

- ISO IN - Streaming mode
  - The Controller starts fetching data after the SW primes the Endpoint, and
  - fills the TX FIFO channel to the available space.
  - When the respective IN arrives from the Host, the Controller sends the data, fetching more data from system memory to TX FIFO as space becomes available.

ASR CONFIDENTIAL

- If the IN arrives and the packet is not fully in TX FIFO (packet smaller than TX FIFO), or if the TX FIFO is not yet full (packet larger than TX FIFO), the Device Controller will send a Zero-Length packet.
- Using an ISO packet size larger than the TX FIFO size is not a problem, as long as the system bus has enough bandwidth to backfill the TX buffer after the packet started being sent.

- ISO OUT - Streaming mode
  - When the ISO OUT comes and data is sent from Host, the Device Controller will begin storing it to the RX buffer.
  - The Device Controller will start sending data from RX FIFO to system memory as soon as a burst worth of data is available.
  - After all the ISO OUT data is received, and while the RX FIFO still has data inside, waiting to be transferred to system memory, the Device Controller is able to receive other packets.
  - If only 1 RX FIFO position is free, or full, the Device will BTO the OUT/DATA from Host.
  - If there is more than 1 position free, the Device will accept OUT/DATA from Host but if the FIFO is not read to system memory, an overflow will happen anyway, and the packet NAKed.
- Bulk IN - Streaming mode
  - When the Host sends the IN, if the Device hasn't primed the EP yet, it will NAK the IN.
  - After the EP is primed, the DMA engine in the Device Controller will start fetching data to the TX Buffer.
  - When the Protocol Engine part of the Controller is primed (after synchronization), the Device Controller will respond to an IN with data.
  - The Device Controller will keep on fetching data as space is available in the TX buffer (at least one burst worth of data of free space available).
  - If during a packet the system bus is unable to buffer all the data for that packet on time, the packet will be cut short due to underrun.
  - During a transfer, the Device Controller will NAK an IN if the TX buffer is empty, or more specifically, if no data was loaded for the next packet.
- Bulk OUT - Streaming mode
  - When the Bulk OUT comes and data is sent from Host, the Device Controller will begin storing it to the RX buffer.
  - The Device Controller will start sending data from RX FIFO to system memory as soon as a burst worth of data is available.
  - After all the Bulk OUT data is received, and while the RX FIFO still has data inside, waiting to be transferred to system memory, the device

ASR CONFIDENTIAL

Controller is able to receive other packets;

- If only 1 RX FIFO position is free, or full, the Device will BTO the OUT/DATA from Host.
- If there is more than 1 position free, the Device will accept OUT/DATA from Host but if the FIFO is not read to system memory, an overflow will happen anyway, and the packet NAKed.
- This behavior is the same as for ISO OUT.

Additional notes on TX FIFO buffering – IN Endpoints

- Initial priming:
  - When the priming is started on the DMA side, the Controller will load the leading data into the TX buffer, and only then will complete the priming operation (PE is primed). This pre-buffering is done for the whole first packet, or until the TX FIFO is full.

ASR CONFIDENTIAL

- Buffering after the first packet:
  - After the first packet, the 2nd packet in a dTD will be sent as long as at least one byte was loaded to the FIFO for the 2nd packet.
  - Once the FIFO is loaded with the first packet, the whole dTD (all the packets in one dTD) will be sent for every IN from the Host, and the system bus must keep back-filling the TX FIFO to keep up with the data being sent for the several packets.
- BTO or NAK from Host to a data packet:
  - If a packet is NAKed or BTOed by the Host, the Device will flush the TX buffer and remove the priming state.
  - Then it will go back and perform the buffering for the whole packet again.
  - If an IN comes from the host in the meantime, it will be NAKed.
  - Once the packet is fully loaded inside the TX buffer again, or the TX buffer is full, the prime state in the PE will be set to active.
  - Only then will the Device Controller respond to the Host IN token with the data packet.
- Underrun of the Device TX FIFO:
  - If a TX FIFO underrun occurs, the Device will clear the prime, and will flush the TX buffer, and then load all of the failing packet again to the TX FIFO.
  - While this is being done, the Device will NAK an IN from the Host.
  - Only when the packet is all in the FIFO again, or FIFO full, will the Device respond to the Host IN with a data packet.
- Buffering between dTDs
  - When the Host sends an ACK to the last data packet of the 1st dTD, the Device will disable the primed state.
  - This last data packet can either be max packet sized, a short packet or a Zero Length packet, depending on the transfer total length and the ZLT bit in the Queue Head. Either one will be ACKed by the Host, and the priming disabled after that.
  - The Device will then retire the dTD back to system memory, and load the new dTD.
  - In the meantime, any IN packet from the Host will be NAKed.
  - After loading the new dTD, the Device will start buffering the first packet into the TX buffer.
  - Once the packet is fully loaded inside the TX buffer again, or the TX buffer is full, the prime state in the PE will be set to active.
  - Only then will the Device Controller respond to the Host IN token with the data packet.
  - This behavior is similar to the initial priming of the 1st dTD.

ASR CONFIDENTIAL

12.4.1.3.2 Non-Streaming mode

- ISO IN - Non-Streaming mode
  - Same as streaming mode
- ISO OUT - Non-Streaming mode
  - Same as streaming mode
- Bulk IN - Non-Streaming mode
  - When the Host sends the IN, if the Device hasn't primed the EP yet, it will NAK the IN.
  - After the EP is primed, the DMA engine in the Device Controller will start fetching data to the TX Buffer.
  - The Device Controller will keep on fetching data as space is available in the TX buffer.
  - The Device Controller will only respond to the first IN with data when the entire packet is fetched to the TX FIFO or the TX FIFO is full. In the meantime, it will respond to the IN with NAK.
  - The following packets will only be sent if the next packet is fully in the FIFO, or if the FIFO is full again.
- Bulk OUT - Non-Streaming mode
  - When the Bulk OUT comes and data is sent from Host, the Device Controller will begin storing it to the RX buffer.
  - The Device Controller will start sending data from RX FIFO to system memory as soon as a burst worth of data is available.
  - The Device Controller responds with NYET to this first packet of a transfer, so the Host will send PINGs after that.
  - The Device Controller will send NAKs to the PINGs as long as data is still in the RX FIFO (RX FIFO not yet empty).
  - Only when the RX FIFO is empty again will the Device Controller ACK the PING, and the Host will send the next IN.

12.5 Register Descriptions

Refer to the *ASR1606 Registers* for details on the USB2 registers.

12.6 USB2 transmit FIFO usage note

Device controller implements 11.5 KB transmit FIFO. The default memory allocation is that 1 KB FIFO for each first ten endpoints and 256B for each last six endpoints. USB2 firmware/SW can change this FIFO memory allocation during initialization by programming RX Buffer Hardware Parameters Register.

Note : due to internal FIFO memory architecture, USB2 firmware/SW shall NOT make any endpoint's transmit FIFO cross the 8KB boundary. For example, 256B memory for the EP0, 1KB memory for the EP1~9 are not allowed because EP8 FIFO crosses the 8KB boundary.

ASR CONFIDENTIAL

13.Real-Time Clock (RTC)

The ASR1606 contains an RTC unit that provides a general-purpose, real-time reference clock for the system. This section describes:

- Section 13.1, Overview
- Section 13.2, Interrupts
- Section 13.3, Trim Procedure
- Section 13.4, Register Descriptions

The RTC unit contains a free running RTC Counter Register, which increments on each rising edge of a 1-Hz clock.

13.1 Overview

In addition to the RTC Counter Register, the RTC incorporates a 32-bit RTC Alarm Register. The RTC Alarm Register can be programmed with a value to be compared against the counter. On each rising edge of the 1-Hz clock, the counter is incremented and compared to the RTC Alarm Register. If the values match and the enable bit is set, the corresponding status bit in the RTC Status Register is set and an interrupt is generated. Another interruptible status bit which sets for every 1-Hz clock tick is available. If the enable bit is set, the corresponding status bit in the RTC Status Register is set and an interrupt is generated.

Each status bit can be cleared by writing a 1 to the RTC Status Register in the required bit position. The 1-Hz clock is generated internally in the RTC module by dividing down the 32.768 kHz crystal oscillator output.

The programmable divider logic for generating the 1-Hz clock allows users to "trim" the counter to adjust for inherent inaccuracies in the 32.768 kHz crystal frequency. This trimming mechanism allows for user adjustments to the RTC to an accuracy of ±5 seconds per month. The trimming procedure is described later in this section.

Not all registers in the RTC unit are reset by hardware reset; the RTC unit reset is software controlled only. For more details on RTC reset, refer to the Clock/Reset Control Register for RTC in the *ASR1606 Registers*.

13.2 Interrupts

The RTC has three interrupt outputs, which are used by the processor as follows:

- **Level alarm interrupt (rtc_slp_alarm)** - This interrupt is generated (if enabled) when a match occurs between the RTC Counter Register and the RTC Alarm Register. The interrupt is enabled by writing a 1 to the <RTC alarm interrupt enable> field in the RTC Status Register, and cleared by writing a 1 to the <RTC alarm detected> field in the RTC Status Register. Just

ASR CONFIDENTIAL

before enabling the interrupt, it must be cleared because it “remembers” past events (matches) even when not enabled. The interrupt is routed to the ICU. For more information, refer to Section 2.4, Interrupt Connectivity and Assignments.

- **Level 1-Hz interrupt (rtc_hzclk_int)** - This interrupt is generated (if enabled) every 1-Hz clock tick. The interrupt is enabled by writing a 1 to the <1-Hz interrupt enable> field in the RTC Status Register, and cleared by writing a 1 to the <1-Hz rising-edge detected> field in the RTC Status Register. The interrupt is routed to the ICU. For more information, refer to Section 2.4, Interrupt Connectivity and Assignments.
- **The sleep alarm interrupt goes to the ICU**. For more information, refer to Section 2.4, Interrupt Connectivity and Assignments.

ASR CONFIDENTIAL

13.3 Trim Procedure

The 1-Hz clock feeding the RTC is obtained by dividing the output of the oscillator multiplexor down approximately 32 kHz. The inherent inaccuracies of crystals, aggravated by varying capacitance of the board connections and other issues, cause the time base to be somewhat inaccurate, requiring a periodic adjustment in the 1 Hz clock period. The device, through the RTC Trim Register, allows for user adjustments to trim the 1 Hz time. When the trim circuitry is set to 0x00, the 1 Hz clock feeding the RTC maintains a high-level signal. 0x01 divides the output frequency of the oscillator multiplexor by two.

13.3.1 Oscillator Frequency Calibration

To generate the value to be entered into the RTC Trim Register, first measure the output frequency at the oscillator multiplexor, which would be approximately 32 kHz using an accurate time base, such as a frequency counter. The trim is accomplished by dividing the output of the oscillator by an integer value and then performing a fine-grain fractional adjustment by periodically deleting clocks from the stream feeding this integer divider.

13.3.2 RTTR Value Calculations

After the true oscillator frequency is known, it must be divided into integer and fractional portions. The integer portion of the value (minus one) is loaded into the <Clock divider count> field in the RTC Trim Register. This value is compared against a 16-bit counter clocked by the output of the oscillator multiplexor (approximately 32 kHz). The counter resets and generates a pulse when the two values are equal. This pulse constitutes the raw 1-Hz signal.

The fractional part of the adjustment is performed by periodically deleting clocks from the clock stream feeding the integer counter. The period, called the *trim interval*, is hard wired to be $(2^{10}-1)$ seconds (approximately 17 minutes). The number of clocks deleted, called the *trim delete value*, is a 10-bit programmable counter allowing from 0 to $(2^{10}-1)$ 32 kHz clocks to be deleted from the input clock stream once per trim interval. The <Trim delete count> field in the RTC Trim Register represents the number of clocks deleted per trim operation.

In summary, every $(2^{10}-1)$ seconds, the integer counter stops clocking for a period equal to the fractional error that has accumulated. If this counter is programmed to a zero, then no trim operations occur and the RTC is clocked with the raw 32-kHz clock. The relationship between the nominal 1-Hz clock frequency and the nominal 32-kHz clock (f1 and f32K, respectively) is illustrated in the following equation.

ASR CONFIDENTIAL

$$f1= \frac{(2^{10}-1)*(C<15..0>+1)-D<9..0>}{(2^{10}-1)*(C<15..0>+1)} * \frac{f32k}{(C<15..0>+1)}$$

13.3.2.1 Trim Example #1 – Measured Value Has No Fractional Component

In this example, the oscillator output is measured to be 36045.000 cycles/s (Hz). This output is exactly 3277 cycles over the nominal crystal frequency and has no fractional component. As such, only the integer trim function is needed (fractional trim is not required). Accordingly, the <Clock divider count> field in the RTC Trim Register is loaded with the binary equivalent of 36045-1 or 0x8CCC. The <Trim delete count> field remains at zero (power-up state) to disable fractional trimming. This trim exercise produces an error of zero in trimming.

ASR CONFIDENTIAL

13.3.2.2 Trim Example #2 – Measured Value Has a Fractional Component

This example is a more common case in that the measured oscillator frequency has a fractional component. If the oscillator output is measured at 32768.92 cycles/s (Hz), an integer trim is necessary so that the *average* number of cycles counted before generating one 1-Hz clock is
32768.92. Similar to the previous example, the integer <Clock divider count> field is loaded with the hexadecimal equivalent of 32768-1 or 0x7FFF (reset value).

Because the actual clock frequency is 0.92 cycles per second faster than the integer value, the 1-Hz clock generated by just the integer trimming is slightly *faster* than needed and must be slowed down. Accordingly, the fractional trim must be programmed to delete 0.92 cycles per second on average to bring the 1-Hz output frequency down to the proper value. Since the trimming procedure is performed only every 2^{10}-1=1023 seconds, the trim must be set to delete (.92*1023) = 941.16 clocks every 1023 seconds. The fractional component of this value cannot be trimmed out and constitutes the error in trimming, described below. Load the counter with the hexadecimal equivalent of 941 or 0x3AD.

This trim setting produces an error of 16 cycles per 1023 seconds. The error calculation yields (in parts-per-million (ppm)):

$$\text{Error} = \frac{0.16 \text{ cycles}}{1023 \text{ sec}} \text{X} \frac{1 \text{ sec}}{32768 \text{ cycles}} = 0.002 \text{ ppm}$$

13.3.2.3 Maximum Error Calculation Versus Real-Time Clock Accuracy

As seen from trim Example #2, the maximum possible error approaches 1 clock per (2^{10}-1) seconds. Calculating the ppm error for this scenario yields:

$$\text{Error (maximum)} = \frac{1 \text{ cycle}}{1023 \text{ sec}} \text{X} \frac{1 \text{ sec}}{32768 \text{ cycles}} = 0.03 \text{ ppm}$$

To maintain an accuracy of ±5 seconds per month, the required accuracy is calculated to be:

$$\text{Error} = \frac{5 \text{ sec}}{\text{month}} \text{X} \frac{1 \text{ month}}{2592000 \text{ sec}} = 1.9 \text{ ppm}$$

ASR CONFIDENTIAL

This calculation indicates that the accuracy of the trim mechanism is more than adequate to compensate for the static environmental and manufacturing variables, and still provides acceptable accuracy.

13.4 Register Descriptions

Refer to the *ASR1606* Registers for details on the RTC.

ASR CONFIDENTIAL

14. Timers

The ASR1606 has a triple timer plus a Watchdog timer. The 3, 32-bit general-purpose timers are based on a selectable functional clock (32 kHz, 13, 6.5, 3.25, and 1 MHz). This section provides information about the timer operation, related register addresses, and bit definitions.

- Section 14.1, Operation
- Section 14.2, Watchdog Timer Functionality
- Section 14.3, Timer Interrupts
- Section 14.4, Register Descriptions

14.1 Operation

The operating system timers and Watchdog timer module consist of 3, 32-bit Timer Clock Control Registers (TCCRn) up counters and 1, 16-bit WDT up counter. The module has 2 clock inputs:

- Selectable fast clock (13, 6.5, 3.25, and 1 MHz)
- Slow clock (32.768 kHz)

The fast clock is selectable through the <Functional Clock Select> field in the Clock/Reset Control Register for Timers (APBC_TIMERS_CLK_RST, refer to the *ASR1606 Registers* for more information). All 3 timers (Timer 0/1/2) operate at fast clock, slow clock of 32.768 kHz.

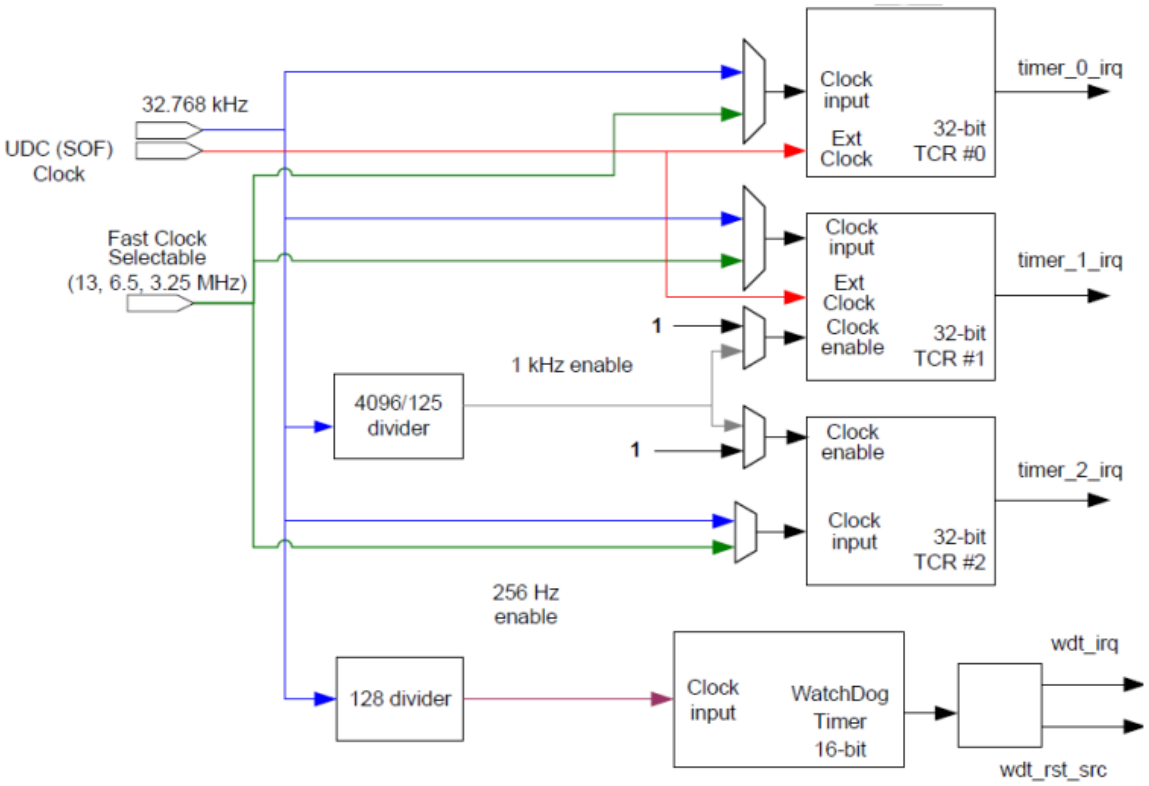
Each of the 3 Timer Count Registers (TCRn, where n is 0, 1, or 2) has 3, 32-bit Timer Match Registers (TMR_Tn_Mm) associated with it. When the value in the Operating System Count Register (TCRn) matches the value within any of the Match registers and the interrupt-enable bit is set, the corresponding bit in the Timer Status Register (TSR) is set. These bits are also routed to the Interrupt Controller where they are programmed to generate an interrupt. In addition, the 16-bit Watchdog timer operates using a timer module-derived clock of 256 Hz.

Reprogramming any Timer Control Register (TCCR, TPLVRn, TMR_Tn_Mm, TPLCRn, TCMRn, or TILRn) while the timer is operating is not guaranteed to be valid. To safely reprogram any of these registers, disable the timer, reprogram it, and re-enable it. The functionality of 1 enabled timer is not affected by the programming of another timer.

ASR CONFIDENTIAL

Copyright © 2022 ASR

Figure 49:Timers Block Diagram



14.2 Watchdog Timer Functionality

When the value in the Watchdog timer matches the value in TWMR and the TWMER[WME] bit is set, the wdt_rst_src# signal is asserted, creating a Watchdog-timer reset event in the system.

The Watchdog Reset mode is invoked when the software fails to prevent the Watchdog time out from occurring properly. Watchdog resets are generated when the software is not executing properly and has possibly destroyed data. During Watchdog Reset mode, most internal registers in the system are reset to their defined default values. An exception to this is the real-time clock.

To avoid WDT reset, the software must restart the WDT before it matches the match value. Set the TWCR[WCR] field to restart the WDT.

ASR



Note

The TWSR Register (and some other internal WDT flops), which determines whether a match event has occurred, is reset upon the assertion of power-on-reset or external-master reset. All other registers are reset upon the assertion of power-on-reset, external-master reset, or watchdog-timer reset.

ASR CONFIDENTIAL

When a WDT reset event occurs, the WDT generates a 4 msec-wide pulse on the wdt_rst_src# output. As a consequence, the system is held in a state of reset for this period of time. The system goes through a reset sequence when this reset signal is de-asserted.

Writing to all WDT registers is protected by 2 Access registers: TWFAR and TWSAR. Follow these steps to enable Writes to any WDT register:

1. Write the proper key value to the TWFAR register.
2. Write the proper key value to the TWSAR register.
3. Perform the write operation on the preferred WDT register. Once this write operation is complete, the WDT registers are re-locked. This sequence must be repeated to perform another write operation.

Note

After a reset occurs on Timer 2 Watchdog, the MPMU_APB clock must be re-enabled. This can be done by:

1. Writing 0x5 to the Clock/Reset Control Register for MPMU in the APB Clock.
2. Writing 0x3 to the Clock/Reset Control Register for MPMU.

14.3 Timer Interrupts

Note

There are 2 interrupts for each timer unit: Timer 0 has 1 interrupt, and Timer 1/2 share another interrupt. For more information on interrupts, refer to Table 3 .

14.4 Register Descriptions

Refer to the *ASR1606 Registers* for details on the Hardware Timers registers.

ASR CONFIDENTIAL

15.High-Speed TWSI Bus Interface Unit (I2C)

The two-wire serial interface (TWSI) bus is a true multi-master bus including collision detection and arbitration. For full details of TWSI bus operation, refer to the *TWSI Bus Specification*.

- Section 15.1, Overview
- Section 15.2, Features
- Section 15.3, Signal Descriptions
- Section 15.4, Operation
- Section 15.5, Register Descriptions

A separate TWSI module, referred to as the power TWSI module, is used to interface to the power management IC.

15.1 Overview

The TWSI bus interface unit allows the ASR1606 to serve as a master and slave device residing on the TWSI bus, which is a serial bus (developed by Phillips Corporation) consisting of a 2-pin interface. SDA is the data pin for input and output functions, and SCL is the clock pin for reference and control of the TWSI bus.

The TWSI bus allows the TWSI unit to interface to other TWSI peripherals and microcontrollers. The serial bus requires minimal hardware for an economical system to communicate status and control information between the ASR1606 and external devices.

The TWSI bus interface unit is a peripheral device that resides on the ASR1606 peripheral bus. Data is transmitted to and received from the TWSI bus via a buffered interface. Control and status information is relayed through a set of memory-mapped registers. Refer to the *TWSI Bus Specification* for complete details on TWSI bus operation.



Note The ASR1606 implementation of the TWSI unit does not support the hardware general call (see Section General Call Address for a description of the General Call Address), 10-bit slave addressing, or CBUS compatibility.

ASR CONFIDENTIAL

15.2 Features

The TWSI bus interface unit features are:

- TWSI unit is compliant to TWSI *Bus Specification Version 2.1* with the exception of support for the hardware general call (see Table 42 for a description of the General Call Address), 10-bit slave addressing, and CBUS compatibility.
- Multi-master and arbitration support
- Supports standard-mode operation up to 100 Kbps
- Supports fast-mode operation up to 400 Kbps
- Supports high-speed mode (HS mode) slave operation up to 3.4 Mbps (High-speed TWSI only)
- Supports high-speed mode (HS mode) master operation up to 3.3 Mbps (High-speed TWSI only)

Note TWSI operational frequencies during Master mode decrease due to pullup resistors on the bus. Therefore, SCL frequency is proportional to 1/R.

ASR CONFIDENTIAL

15.3 Signal Descriptions

Table 36 describes the TWSI bus signals, SDA and SCL. PWR_SCL and PWR_SDA are dedicated signals. Refer to Table 12 for more information.

Table 36:DMA Quick Reference for On-Chip Peripherals

Signal Name	Input/Output	Description
Two-Wire Serial Interface Signals		
CI2C_SDA	Bidirectional	TWSI serial data/address signal
CI2C_SCL	Bidirectional	TWSI serial clock line signal
Power Two-Wire Serial Interface Signals		
PWR_SDA	Bidirectional	TWSI serial data/address signal
PWR_SCL	Bidirectional	TWSI serial clock line signal

15.4 Operation

The *TWSI Bus Specification* defines a serial protocol for passing information between agents on the bus, using the 2-pin interface shown in Table 36: a serial data and address (SDA) line and a serial clock line (SCL). Each device on the TWSI bus is recognized by a unique 7-bit address and can operate as a transmitter or as a receiver in master or Slave mode. Table 37 defines the TWSI-bus terminology.

ASR CONFIDENTIAL

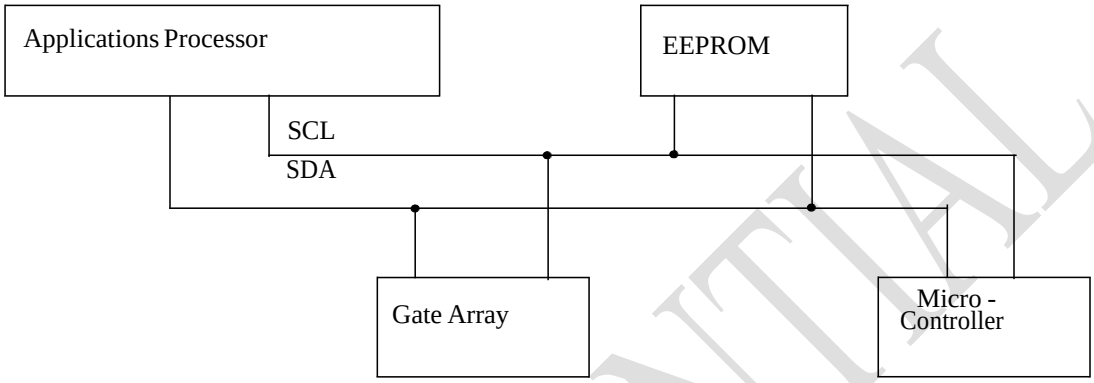
Table 37:TWSI Bus Definitions

TWSI Device	Definition
Transmitter	Sends data over the TWSI bus.
Receiver	Receives data over the TWSI bus.
Master	Initiates transfers, generates clock signals, and terminates transactions.
Slave	Device addressed by a master; it responds by transmitting or receiving data over the TWSI bus.
Multi-master	More than one master can attempt to control the bus at the same time without corrupting the message.
Arbitration	Ensures that only one master controls the bus when more than one master simultaneously tries to control the bus. This technique avoids message corruption.
Acknowledge	The receiver response to the master generated acknowledge clock pulse on SCL. The acknowledge can be either a positive-acknowledge (ACK) or a negative-acknowledge (NAK).
ACK	The condition on the TWSI bus where the master generates an acknowledge clock pulse and the receiver holds the SDA line low during the high period of the clock pulse.
NAK	The condition on the TWSI bus where the master generates an acknowledge clock pulse and the receiver holds the SDA line high during the high period of the clock pulse.

For example, the ASR1606 TWSI can act as a master on the bus to address an EEPROM as the slave to receive data (see Figure 50). When the TWSI addresses the EEPROM, it serves as a master transmitter and the EEPROM serves as a slave receiver. When the TWSI reads data, it serves as a master receiver and the EEPROM serves as a slave transmitter. Whether as a transmitter or receiver, the master generates the clock, initiates the transaction, and terminates the transaction.

ASR CONFIDENTIAL

Figure 50:TWSI Bus Configuration Example



The TWSI bus uses an open-drain wired-AND structure, which allows multiple devices to drive the bus lines and to communicate status about events such as arbitration, wait states, and error conditions. When a master drives the clock (SCL) line during a data transfer, it transfers a bit on every instance that the clock is high. When the slave is unable to accept or drive data at the rate requested by the master, the slave can hold SCL low between the high states to insert wait intervals. The master clock can be altered only by another master during arbitration or by a slow slave peripheral that keeps the clock line low.

The TWSI bus allows multiple masters, which means that more than 1 device can initiate data transfers at the same time. Bus arbitration resolves conflicts between masters. Two masters can drive the bus simultaneously, provided they drive identical data. A master loses the arbitration if it tries to drive SDA high while another master is driving SDA low. The SCL line is a synchronized combination of clocks generated by the masters using the wired-AND connection to the SCL line.

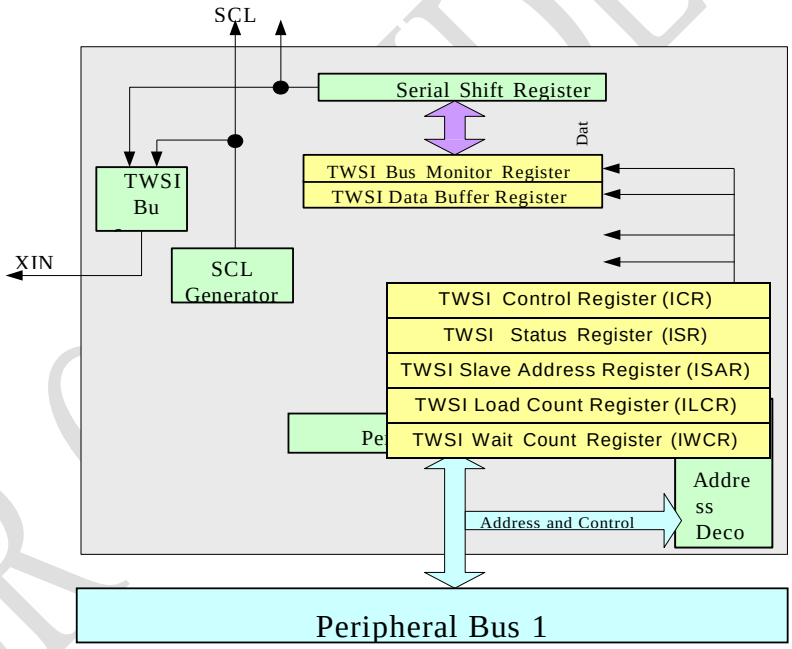
TWSI transactions are either initiated by the TWSI as a master or received by the TWSI as a slave. Both conditions can result in Reads, Writes, or both over the TWSI bus.

ASR CONFIDENTIAL

15.4.1 Operational Blocks

The TWSI unit resides on the ASR1606 peripheral bus. The ASR1606 interrupt mechanism can be used to notify the CPU that there is activity on the TWSI bus. Software polling of the TWSI Status register bits can be used instead of interrupts. The TWSI consists of the two-wire interface to the TWSI bus, an 8-bit buffer for passing data to and from the ASR1606, a set of Control and Status registers, and a Shift register for parallel/serial conversions (see Figure 51). With the new FIFO mode, a TX FIFO with 8 entries and a RX FIFO with 16 entries have been added. Also, the FIFOs' read and write pointers are R/W registers, so those registers can be cleared by software when the FIFOs need to be flushed after a critical interrupt (see Figure 52).

Figure 51:TWSI Bus Interface Unit Block Diagram



The TWSI initiates an interrupt to the ASR1606 when:

- A buffer is full
- A buffer is empty

ASR CONFIDENTIAL

- The TWSI slave address is detected
- Arbitration is lost
- A bus-error condition occurs

All interrupt conditions must be cleared explicitly by software. Refer to the TWSI Status Register in the *ASR1606 Registers* for details.

The TWSI Control, Status, and Data registers are located in the TWSI memory-mapped address space. The 8-bit TWSI Data Buffer Register (IDBR) is used to transmit and receive data to and from the TWSI bus using an internal Shift register that is not user accessible. The TWSI supports:

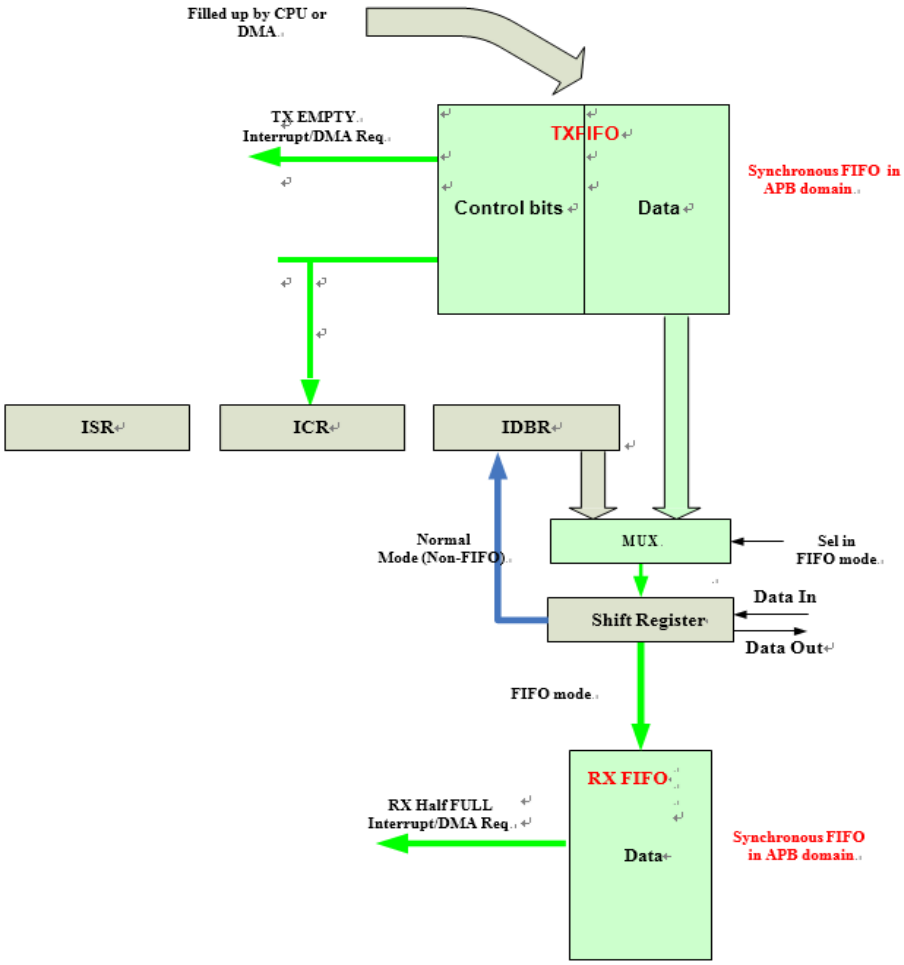
- high-speed mode slave operation up to 3.4 Mbps
- high-speed mode master operation up to 3.3 Mbps
- fast-mode operation up to 400 kbps
- standard-speed operation up to 100 kbps

See the *TWSI Bus Specification* for more information.

ASR CONFIDENTIAL

15.4.1.1 *FIFO Mode Block Diagram*

Figure 52:Tx and Rx FIFOs Shown in FIFO Mode



ASR
AL

In Figure 52, the green color blocks are new for FIFO mode. The Tx FIFO has control+data bits for both Read and Write transactions; whereas, Rx FIFO has data received from a Read transaction. In FIFO mode, the ASR1606 supports both PIO and DMA modes. So either an interrupt or a DMA request is made when the Tx FIFO is empty or the Rx FIFO is half full. There are new interrupt bits for this FIFO mode:

- Transaction is done
- Receive FIFO is half full
- Transmit FIFO is empty
- Receive FIFO is full
- Receive FIFO overrun happened

The control register has new bits to enable the new interrupts. It also has a TXBEGIN bit that must be set by software for a new transaction to start. FIFOEN bit has to be set for enabling FIFO mode.

The DMA_EN bit enables DMA mode.

15.4.2 TWSI Bus Interface Modes

The TWSI unit can accomplish a transfer in different operational modes. Table 38 summarizes the different modes.

Table 38:Modes of Operation

Mode	Description
Master-Transmit	• TWSI acts as a master • Used for transmit operations • TWSI sends the data • TWSI generates the clock • Slave device is in Slave-Receive mode
Master-Receive	• TWSI acts as a master • Used for receive operations • TWSI receives the data • TWSI generates the clock • Slave device is in Slave-Transmit mode
Slave-Transmit	• TWSI acts as a slave • Responds to a master Read operation • TWSI sends the data • Master device is in Master-Receive mode
Slave-Receive (default)	• TWSI acts as a slave • Responds to a master Write operation • TWSI receives the data • Master device is in Master-Transmit mode

While the TWSI is idle, it defaults to Slave-Receive mode, which allows the interface to monitor the bus and receive any slave addresses intended for the ASR1606 TWSI.

If the TWSI unit receives a slave address that matches a HS-mode master code

ASR CONFIDENTIAL

(Slave address equal to 00001XX, where X is either a 1 or a 0), and if the TWSI unit is enabled for high-speed mode (bit [16] of the TWSI Control Register is set to 0x1), the TWSI unit enters high-speed mode and remains in Slave-Receive mode.

When the TWSI receives an address that matches the 7-bit address in the TWSI Slave Address Register (ISAR) or the general-call address (see Section 16.4.15, General Call Address), the interface either remains in Slave-Receive mode or switches to Slave-Transmit mode. The Read/Write bit (R/nW) determines which mode the interface enters. The R/nW bit is the least significant bit (LSb) of the byte containing the slave address. If R/nW is clear, the master that

ASR CONFIDENTIAL

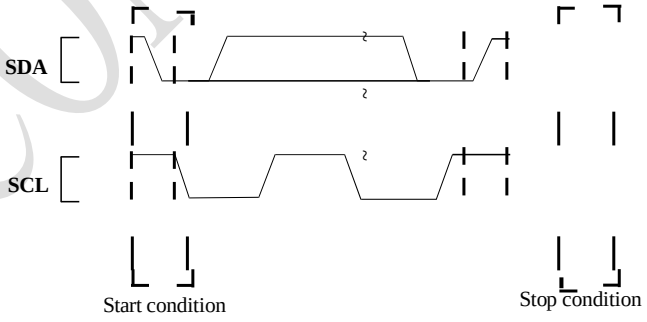
initiated the transaction intends to write data, and the TWSI remains in Slave-Receive mode. If the R/nW bit is set, the master that initiated the transaction intends to read data, and the TWSI switches to Slave-Transmit mode. Section 16.4.13, Slave Operations further defines slave operation.

When the ASR1606 initiates a Read or Write on the TWSI bus, it switches the interface from the default Slave-Receive mode to the Master-Transmit mode. If the transaction is a Write, the TWSI remains in Master-Transmit mode after the address transfer is completed. If the transaction is a Read, the TWSI transmits the slave address, then switches to Master-Receive mode.
Section 16.4.9, Master Operations further defines master operation.

15.4.3 Start and Stop Bus States

The TWSI bus specification defines a Start transaction used at the beginning of a transfer and a Stop transaction used at the end of a transfer. A Start condition occurs if a high-to-low transition occurs on the SDA line when SCL is high. A Stop condition occurs if a low-to-high transition occurs on the SDA line when SCL is high. Figure 53 shows the relationship between the SDA and SCL lines for Start and Stop.

Figure 53:SDA and SCL Signals During Start and Stop Conditions



The TWSI unit uses the ICR[START] and ICR[STOP] bits to:

- Initiate an additional byte transfer
- Initiate a Start condition on the TWSI bus
- Enable data chaining (repeated Start)
- Initiate a Stop condition on the TWSI bus

Table 39 defines the START and STOP bits in the ICR.

ASR CONFIDENTIAL

Table 39:START and STOP Bit Definitions

STOP Bit	START Bit	Condition	Notes
0	0	No Start or Stop	The TWSI sends a no Start or Stop condition when multiple data bytes are to be transferred.
0	1	Start condition and repeated Start	The TWSI sends a Start condition and transmits the IDBR eight-bit contents. The IDBR must contain the 7-bit slave address and the R/nW bit before a Start is initiated. For a repeated start, the IDBR contains the target slave address and the R/nW bit. This allows a master to make multiple transfers to different slaves without giving up the bus. The interface stays in Master-Transmit mode for Writes and switches to Master-Receive mode for Reads.
1	X	Stop condition	In Master-Transmit mode, the TWSI transmits the IDBR eight-bit contents and sends a Stop condition on the TWSI bus. In Master-Receive mode, ICR[ACKNAK] must be set, which defines a negative-Acknowledge (NAK) pulse (see Section 16.4.6, TWSI Acknowledge). The TWSI transmits the NAK pulse, places the received data byte into the IDBR, and sends a Stop condition on the TWSI bus.

ASR CONFIDENTIAL

15.4.3.1 Start Condition

The Start condition (ICR[START]=1, ICR[STOP]=0) initiates a master transaction or repeated Start. Before it sets ICR[START], software must load the target slave address and the R/nW bit in the IDBR (see TWSI Data Buffer Register). The Start and the IDBR contents are transmitted on the TWSI bus after ICR[TB] is set. The TWSI bus stays in Master-Transmit mode for Write requests and enters Master-Receive mode for Read requests. For a repeated Start, a change in Read or Write, or a change in the target slave address, the IDBR contains the updated target slave address and the R/nW bit. A repeated Start enables a master to make multiple transfers to different slaves without surrendering the bus.

The Start condition is not cleared by the TWSI. If the TWSI loses arbitration while initiating a Start, it may re-attempt the Start when the bus is freed. See Section 16.4.7, Arbitration for details on how the TWSI functions in those circumstances.

15.4.3.2 No Start or Stop Condition

The no Start or Stop condition (ICR[START]=0, ICR[STOP]=0) is used in Master-Transmit mode while the TWSI is transmitting multiple data bytes (see Figure 54). Software writes the data byte, and the TWSI sets ISR[ITE] and clears ICR[TB]. Software then writes a new byte to the IDBR and sets ICR[TB], which initiates the new byte transmission. This process continues until software sets ICR[START] or ICR[STOP]. ICR[START] and ICR[STOP] are not cleared automatically by the TWSI after the transmission of a Start, Stop, or repeated Start.

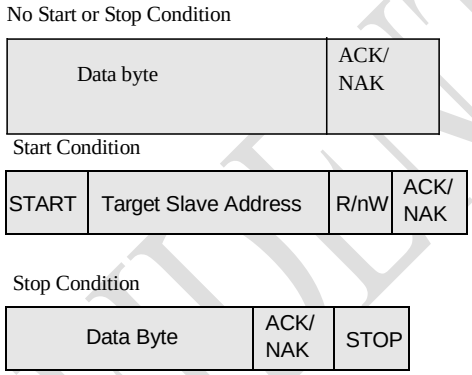
After each byte transfer, including the acknowledge pulse defined by the ICR[ACKNAK] control bit, the TWSI holds the SCL line low to insert wait states until ICR[TB] is set. This action notifies the TWSI to release the SCL line and allow the next information transfer to proceed.

ASR CONFIDENTIAL

15.4.3.3 *Stop Condition*

The Stop condition (ICR[START]=X, ICR[STOP]=1) terminates a data transfer. In Master-Transmit mode, ICR[STOP] and ICR[TB] must be set to initiate the last byte transfer (see Figure 54). In Master-Receive mode, the TWSI must set ICR[ACKNAK], ICR[STOP], and ICR[TB] to initiate the last transfer. Software must clear ICR[STOP] after the Stop condition is transmitted.

Figure 54:Start and Stop Conditions



15.4.4 Data Transfer Sequence

The TWSI unit transfers data in 1-byte increments and always follows this sequence:

1. Start
2. 7-bit slave address
3. R/nW bit
4. Acknowledge pulse
5. 8 bits of data
6. Acknowledge pulse
7. Repeat of steps 5 and 6 for the required number of bytes
8. Repeated Start (repeat step 1) or Stop

15.4.5 Data and Addressing Management

The TWSI Data Buffer register (IDBR) and the TWSI Slave Address register (ISAR) manage data and slave addressing. The IDBR (see TWSI Data Buffer Register) contains 1 byte of data or a 7-bit slave address and the R/nW bit. The ISAR contains the ASR1606 programmable slave address. The TWSI puts received data into the IDBR after a full byte is received and acknowledged. To transmit data, the CPU writes to the IDBR, and the TWSI passes the information to the serial bus when ICR[TB] is set. Refer to the TWSI Slave Address Register.

ASR CONFIDENTIAL

When the TWSI is in Master- or Slave-Transmit mode:

- Software writes data to the IDBR over the internal bus, which initiates a master transaction or sends the next data byte after ISR[ITE] is set.
- TWSI transmits data from the IDBR when ICR[TB] is set.

ASR CONFIDENTIAL

- When enabled with the ICR[ITEIE] bit set, an IDBR transmit-empty interrupt is signalled when a byte is transferred on the TWSI bus and the acknowledge cycle is complete.
- When the TWSI is ready to transfer the next byte before the CPU has written the IDBR and a Stop condition is not in place, the TWSI inserts wait states until the CPU writes a new value to the IDBR and sets ICR[TB].

Note In FIFO mode, software writes control+data information to the TX FIFO instead of the IDBR. FIFO mode is explained in Section 16.4.10, FIFO mode

When the TWSI is in master- or Slave-Receive mode:

- The ASR1606 reads IDBR data over the internal bus after the IDBR receive-full interrupt is signalled (if enabled with the ICR[DRFIE] bit set).
- TWSI transfers data from the Shift register to the IDBR after the acknowledge cycle completes.
- TWSI inserts wait states until the IDBR is read. See Section 16.4.6, TWSI Acknowledge for information about the acknowledge pulse in receive mode.
- After the CPU reads the IDBR, the TWSI unit updates the ICR[ACKNAK] and ICR[TB] bits, allowing the next byte transfer to proceed.

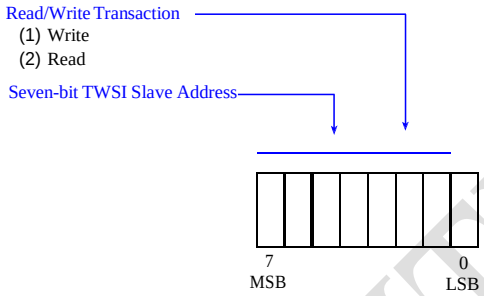
Note In FIFO mode, software reads from the RX FIFO instead of the IDBR. FIFO mode is explained in Section 16.4.10, FIFO mode.

15.4.5.1 Addressing a Slave Device

As a master device, the TWSI must compose and send the first byte of a transaction. This byte consists of the slave address for the intended device and a R/nW bit for transaction definition. This byte can also be a master code indicating the start of a high speed transaction. See Section 16.4.8, High Speed Mode (High Speed TWSI Only) for a description of HS-mode transfers. To address a slave device, write the slave address and the R/nW bit to the IDBR (see Figure 55).

ASR CONFIDENTIAL

Figure 55:Data Format of First Byte in Master Transaction



ASR CONFIDENTIAL

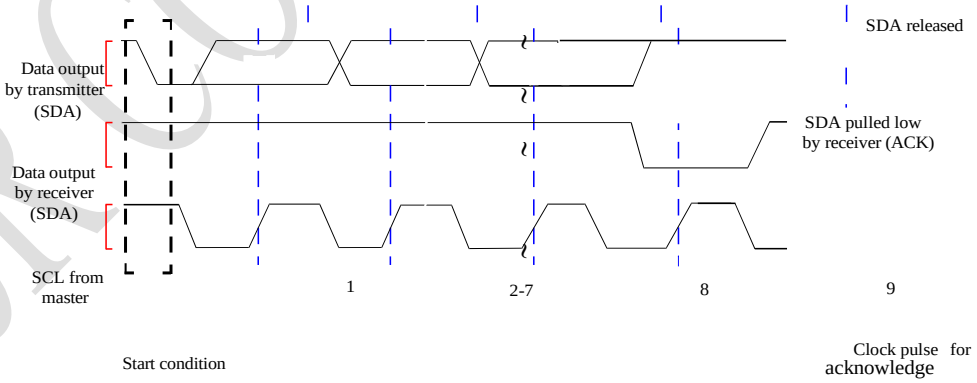
The first byte transmission must be followed by a positive-acknowledge (ACK) pulse from the addressed slave. When the transaction is a Write, the TWSI remains in Master-Transmit mode and the addressed slave device stays in Slave-Receive mode. When the transaction is a Read, the TWSI switches to Master-Receive mode immediately following the ACK, and the addressed slave device switches to Slave-Transmit mode. When a negative-acknowledge (NAK) is returned, the TWSI aborts the transaction by automatically sending a Stop and setting ISR[BED].

When the TWSI is enabled and idle, it remains in Slave-Receive mode and monitors the TWSI bus for a Start signal. When it detects a Start condition, the TWSI reads the first 7 bits and compares them to those in the ISAR and the general call address (0x00). When the bits match those in the ISAR register, the TWSI reads the eighth bit (R/nW bit) and transmits an ACK pulse. The TWSI either remains in Slave-Receive mode (R/nW = 0) or switches to Slave-Transmit mode (R/nW = 1). See Section 16.4.15, General Call Address for actions when a general call address is detected.

15.4.6 TWSI Acknowledge

Every TWSI byte transfer must be accompanied by an acknowledge pulse that the master or slave receiver must generate. The transmitter must release the SDA line for the receiver to transmit the acknowledge pulse (see Figure 56).

Figure 56:Acknowledge Pulse on the TWSI Bus



In Master-Transmit mode, if the target slave receiver cannot generate the positive-acknowledge (ACK) pulse, the SDA line remains high, which indicates a negative-acknowledge (NAK). The lack of an ACK causes the TWSI to set ISR[BED] and generate the associated interrupt when enabled. The TWSI automatically generates a Stop condition and aborts the transaction.

ASR CONFIDENTIAL

In Master-Receive mode, the TWSI sends a negative-acknowledge (NAK) pulse to signal the slave transmitter to stop sending data. The ICR[ACKNAK] bit controls the ACK/NAK pulse value driven onto the TWSI bus. As required by the TWSI bus protocol, ISR[BED] is not set for a Master-Receive mode NAK. The TWSI automatically transmits the ACK pulse after it receives each byte from the serial bus. Before the unit receives the last byte, software must set ICR[ACKNAK] to generate a NAK. The NAK pulse, sent after the last byte, signals that the last byte has been sent.

In Slave mode, the TWSI automatically acknowledges its own slave address, regardless of the ICR[ACKNAK] setting. In Slave-Receive mode, an ACK response automatically follows a data byte, regardless of the ICR[ACKNAK] setting. The TWSI unit sends the ACK value after receiving the eighth data bit of the byte.

ASR CONFIDENTIAL

In Slave-Transmit mode, receiving a NAK from the master indicates the last byte is transferred. The master then sends either a Stop or repeated Start. The ISR[UB] remains set until a Stop or repeated Start is received.

15.4.7 Arbitration

TWSI bus arbitration is required due to the multi-master capabilities of the TWSI bus. Arbitration is used when 2 or more masters simultaneously generate a Start condition within the minimum TWSI hold time of the Start condition.

Arbitration can continue for a long period of time. If the address field and the R/nW bit are the same, the arbitration moves to the data. Due to the wired-AND nature of the TWSI bus, no data is lost if both (or all) masters are outputting the same bus states. If the address, the R/nW bit, or the data are different, the master, which transitioned to the high state (master data is different from SDA), loses arbitration and shut off its data drivers. When losing arbitration, the TWSI bus interface unit shuts off the SDA or SCL drivers for the remainder of the byte transfer, set the arbitration loss detected bit (ISR[ALD]), then return to idle (Slave-Receive) mode.

In FIFO mode, software has to flush the FIFOs on arbitration loss. That can be done by clearing the Read and Write pointer registers for both the Transmit and Receive FIFOs.

- WFIFO_RPTR
- WFIFO_WPTR
- RFIFO_RPTR
- RFIFO_WPTR

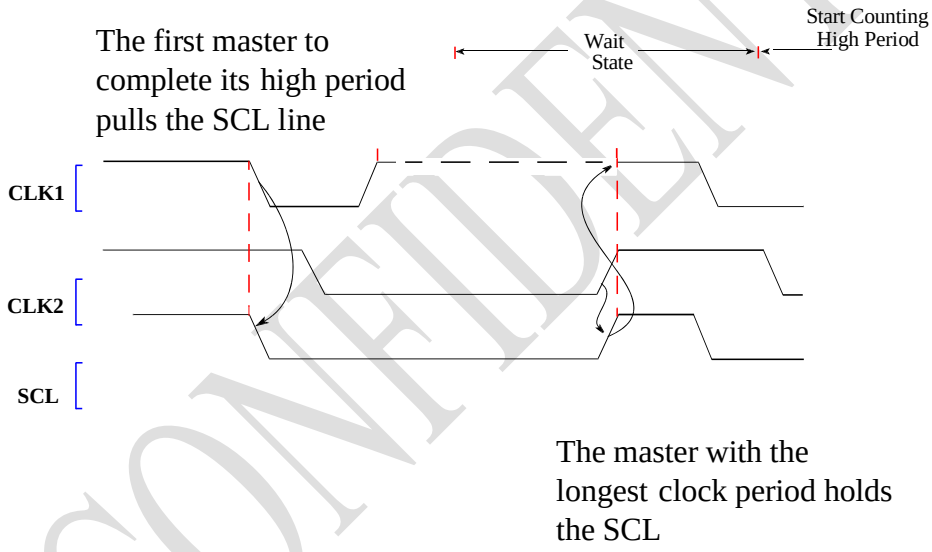
15.4.7.1 SCL Arbitration

Each master on the TWSI bus generates its own clock on the SCL line for data transfers. As a result, clocks with different frequencies may be connected to the SCL line. Because data is valid when a clock is in the high period, bit-by-bit arbitration requires a defined clock synchronization procedure.

ASR CONFIDENTIAL

Clock synchronization is through the wired-AND connection of the TWSIs to the SCL line. When a master clock changes from high to low, the master holds down the SCL line for its associated period (see Figure 57). A clock cannot switch from low to high if another master has not completed its period. The master with the longest low period holds down the SCL line. Masters with shorter periods are held in a high wait-state until the master with the longest period completes. After the master with the longest period completes, the SCL line changes to the high state and masters with the shorter periods continue the data cycle.

Figure 57:Clock Synchronization During Arbitration

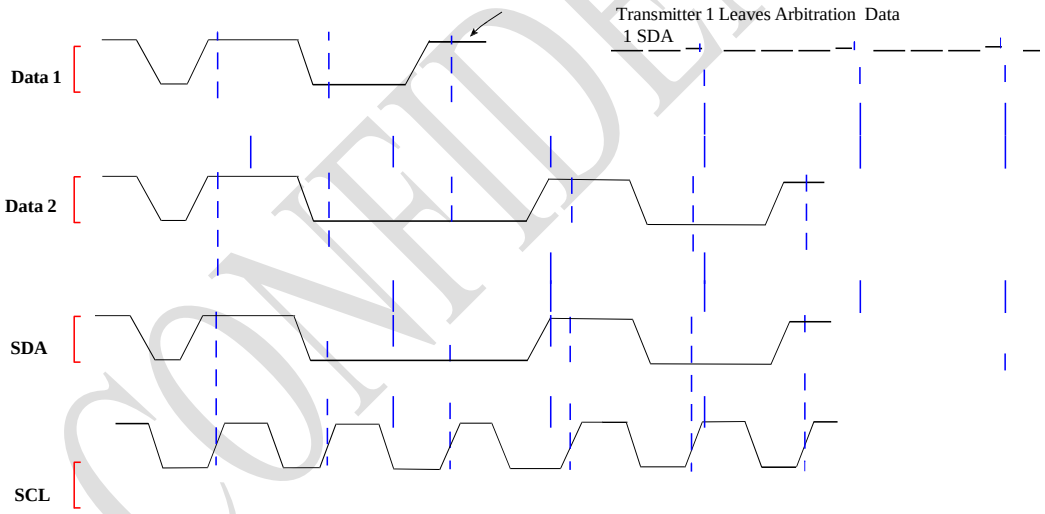


ASR CONFIDENTIAL

15.4.7.2 SDA Arbitration

Arbitration on the SDA line can continue for a long time because it starts with the address and R/nW bits and continues through the data bits. Figure 58 shows the arbitration procedure for 2 masters. More than 2 masters may be involved if more than 2 masters are connected to the bus. If the address bit and the R/nW are the same, the arbitration scheme considers the data. Because the TWSI bus has a wired-AND nature, a transfer does not lose data if multiple masters signal the same bus states. If the address and the R/nW bit or the data they contain are different, the master that sent the first high data bit loses arbitration and shuts off its data drivers. If the TWSI loses arbitration, it shuts off the SDA or SCL drivers for the rest of the byte transfer, sets ISR[ALD], and returns to Slave-Receive mode.

Figure 58:Arbitration Procedure for Two Masters



If the TWSI loses arbitration as the address bits are transferred and it is not addressed by the address bits, the TWSI re-sends the address when the TWSI bus becomes free. A re-send is possible because registers IDBR and ICR are not overwritten by the TWSI unit when arbitration is lost.

If the TWSI loses arbitration because another bus master addresses the ASR1606 TWSI unit as a slave device, the TWSI switches to Slave-Receive mode and overwrites the original data in register IDBR. Software can clear the Start and re-initiate the master transaction.

ASR CONFIDENTIAL

Caution Software must not allow the TWSI unit to write to its own slave address. This can cause the TWSI bus to enter an ***indeterminate*** state.

Boundary conditions exist for arbitration when an arbitration process is in progress and a repeated Start or Stop condition is transmitted on the TWSI bus.

To prevent errors, the TWSI unit, acting as a master, provides for the following sequences:

No arbitration occurs between a repeated Start condition and a data bit

ASR CONFIDENTIAL

No arbitration occurs between a data bit and a Stop condition

No arbitration occurs between a repeated Start condition and a Stop condition

These situations arise only when different masters write the same data to the same target slave simultaneously and arbitration is not resolved after the first data-byte transfer.



Note

Software must ensure that arbitration is resolved quickly. For example, software can ensure that masters send unique data by requiring that each master transmit its TWSI address as the first data byte of any transaction. When arbitration is resolved, the winning master sends a restart and begins a valid data transfer. The slave discards the master address and uses the other data.

15.4.8 High Speed Mode (High Speed TWSI Only)

The TWSI unit supports the HS-mode of operation which allows slave data rates up to 3.4 Mbps and master data rates up to 3.3 Mbps. HS-mode devices are backward compatible with Fast and Standard mode (F/S-mode) devices. When running in HS-mode, the same bus protocol and data format is maintained with the exception that clock synchronization and arbitration are not performed when in HS-mode since clock synchronization and bus arbitration are completed before HS-mode is entered.

HS-mode is entered when a master running in F/S-mode sends a master code and wins arbitration. At this point the master switches to HS-mode and generate TWSI transactions. HS-mode ends when a Stop condition is generated by the master.

The master codes are a set of reserved slave addresses that are used to indicate the start of a HS-mode transfer. The master codes always win arbitration over any other slave address. In the case of a multi-master, HS-mode system, each master is assigned a unique master code. This ensures that the clock synchronization and bus arbitration finishes in F/S-mode. Therefore, clock synchronization and bus arbitration is performed as described in Section 16.4.7, Arbitration.

There are 8 possible master codes of the form:

8’b0000_1xxx (x is 0 or 1)

Thus, a maximum of 8 masters are allowed in a multi-master HS-mode system.

15.4.8.1 Data Transfer in HS-mode

HS-mode is entered when a master running in F/S-mode sends and detects the following: send Start condition

ASR CONFIDENTIAL

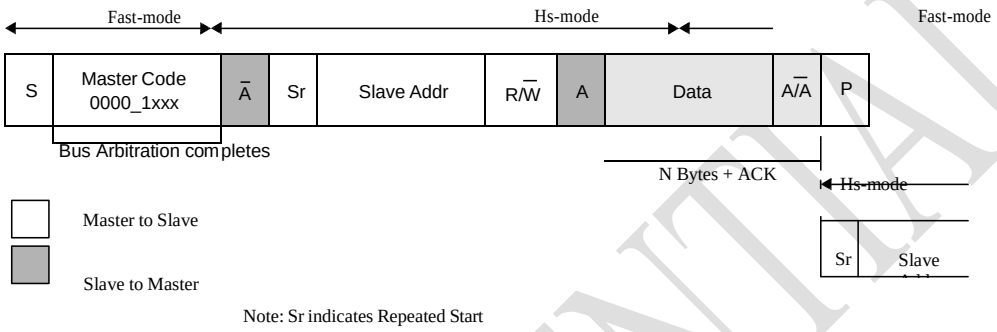
send Master Code (0x0000_1xxx) detect not-acknowledge bit (A)

During this sequence, clock sync and arbitration have completed in Fast-mode and 1 winning master remains. The master then switches to HS-mode and begins a bus transaction by issuing a repeated Start condition. Additional high speed data transfers can be linked by separating them with repeated start conditions. HS-mode ends when a Stop condition is sent. This sequence is shown in Figure 59.

To use the TWSI unit in HS-mode as either a master or a slave, the ICR[MODE] bits must be set to 0b10 or 0b11. When MODE = 0b10, all non-high speed transmits occur in Standard-mode. When MODE = 0b11, all non-high speed transmits occur in Fast-mode.

ASR CONFIDENTIAL

Figure 59:High Speed Mode Data Transfer



15.4.8.2 HS-mode Data Rate

According to *TWSI Bus Specification Version 2.1*, the maximum data rate supported in HS-mode is dependent on the capacitive load of each bus line. When the capacitive load of the bus lines is less than 100 pF, the maximum possible slave data rate is up to 3.4 Mbps, and the master data rate is up to 3.3 Mbps. At a higher capacitive load on the bus lines, the data rate decreases gradually. The capacitive load of each bus line must be less than 400 pF. The maximum possible data rate when the capacitive load is 400 pF up to half of the data rate with a 100 pF load (1.7 Mbps for slave and
1.65 Mbps for master). For capacitive loads between 100 pF and 400 pF, the maximum data rate is linearly interpolated.

15.4.9 Master Operations

When software initiates a Read or Write on the TWSI bus, the TWSI unit switches from the default Slave-Receive mode to Master-Transmit mode. The 7-bit slave address and the R/nW bit follow the Start pulse.

After the master receives an ACK, the TWSI enters 1 of 2 Master modes:

- Master-transmit—TWSI writes data
- Master-receive—TWSI reads data

When transmitting the master code, the master should receive a NAK and then enter HS-mode. The 7-bit slave address and the R/nW bit follow a repeated Start condition. The master receives an ACK and the TWSI unit enters 1 of 2 Master modes listed above.

ASR CONFIDENTIAL

The CPU writes to the ICR register to initiate a master transaction. Data is *read and written* from the TWSI unit through the memory-mapped registers. Table 40 describes the TWSI unit responsibilities as a master device.

Table 40:Master Transactions

TWSI Master Action	Mode of Operation	Definition
Generate clock output	Master-transmit Master-receive	• The master drives the SCL line. • ICR[SCLE] and ICR[IUE] must be set.
Write target slave address to IDBR	Master-transmit Master-receive	• The CPU writes to IDBR bits [7:1] before enabling a Start condition. • The first 7 bits are sent on the TWSI bus after Start. See Section 16.4.3, Start and Stop Bus States.
Write R/nW bit to IDBR	Master-transmit Master-receive	• CPU writes to least significant IDBR bit with R/nW control bit • If R/nW bit is low, master remains a Master-Transmitter. If high, master switches to a master receiver. See Section 16.4.5, Data and Addressing Management.
Signal Start condition	Master-transmit Master-receive	See Generate clock output action in this table. After the target slave address and R/nW bit are in the IDBR, • Software sets ICR[START]. • Software sets ICR[TB] to initiate the Start condition. See Section 16.4.3, Start and Stop Bus States.
Initiate first data byte transfer	Master-transmit Master-receive	• The CPU writes a data byte to the IDBR • The TWSI transmits the byte when ICR[TB] is set. • The TWSI clears ICR[TB] and sets ISR[ITE] when the transfer is complete.
Arbitrate for TWSI bus	Master-transmit Master-receive	If 2 or more masters signal a Start within the same clock period, arbitration must occur. • The TWSI arbitrates for as long as needed. Arbitration takes place during slave address and R/nW bit or data transmission and continues until all but one master loses the bus. No data is lost. • If the TWSI loses arbitration, it sets ISR[ALD] after the byte transfer is completed and switches to Slave-Receive mode. • If the TWSI loses arbitration as it attempts to send the target address byte, it attempts to resend the byte when the bus becomes free. Software must ensure that the boundary conditions described in Section 16.4, Operation do not occur.
Write one data byte to the IDBR	Master-transmit only	• Occurs when ISR[ITE] is set and ICR[TB] is clear. If the IDBR transmit-empty interrupt is enabled, the interrupt is generated. • The CPU writes 1 data byte to the IDBR, sets the appropriate START/STOP bit combination, and sets ICR[TB] to send the data. Eight bits are taken from the Shift register and written to the serial bus. The eight bits are followed by a Stop, if requested by ICR[STOP] being set.
Wait for Acknowledge from slave receiver	Master-transmit only	As a master transmitter, the ASR1606 TWSI generates the clock for the acknowledge pulse. The ASR1606 TWSI releases the SDA line to allow Slave-Receiver acknowledge transmission. See Section 16.4.6, TWSI Acknowledge.

ASR CONFIDENTIAL

Read one byte of TWSI data from the IDBR	Master-receive only	Eight bits are read from the serial bus, collected in the Shift register, then transferred to the IDBR after the ICR[ACKNAK] bit is read. • The CPU reads the IDBR when ISR[IRF] is set and ICR[TB] is clear. If the IDBR receive-full interrupt is enabled, it is signalled to the CPU. • When the IDBR is read, if ISR[ACKNAK] is clear (indicating ACK), the software must clear the ICR[ACKNAK] bit and set ICR[TB] to initiate the next byte Read. • If ISR[ACKNAK] is set (indicating NAK), ICR[TB] is clear, ICR[STOP] is set, and ISR[UB] is set, then the last data byte has been read into the IDBR, and the TWSI is sending the Stop. • If ISR[ACKNAK] is set (indicating NAK) and ICR[TB] is clear, but ICR[STOP] is clear, then the software has 2 options: 1. Set ICR[START], write a new target address to the IDBR, and set ICR[TB], which sends a repeated Start. 2. Set ICR[MA] and leave ICR[TB] clear, which sends a Stop only.
Transmit acknowledge to slave transmitter	Master-receive only	• As a master receiver, the ASR1606 TWSI generates the clock for the acknowledge pulse and drives the SDA line during the acknowledge cycle. • If the next data byte is to be the last transaction, the user software sets ICR[ACKNAK] for NAK generation. See Section 16.4.6, TWSI Acknowledge.
Generate a repeated Start to chain TWSI transactions	Master-transmit Master-receive	Data chaining takes place by using a repeated Start condition instead of a Stop condition. • The repeated Start is generated after the last data byte of a transaction has been transmitted on the TWSI bus, as described in Section 16.4.4, Data Transfer Sequence. • The software must write the next target slave address and the R/nW bit to the IDBR, sets ICR[START], and sets ICR[TB]. See Section 16.4.3, Start and Stop Bus States.
Generate a Stop	Master-transmit Master-receive	• A Stop is generated after the last data byte of a transaction has been transmitted on the TWSI bus, as described in Section 16.4.4, Data Transfer Sequence. • ICR[STOP] must be set in order to generate the Stop condition. See Section 16.4.3, Start and Stop Bus States.

When the software needs to read data, the TWSI unit transitions from Slave-Receive mode to Master-Transmit mode to transmit the start address. Immediately following the ACK pulse, the TWSI unit transitions to Master-Receive mode to wait for the reception of the Read data from the slave device (see Figure 60). It is also possible for multiple transactions to occur during a TWSI operation, such as transitioning from Master-Receive to Master-Transmit through a repeated start.

ASR CONFIDENTIAL

For example, switching from Master-Receive to Master-Transmit through a repeated start or data chaining (see Figure 61). Figure 62 shows the SDA and SCL wave forms for a complete data transfer.

Figure 60:Master Receiver Read from Slave Transmitter

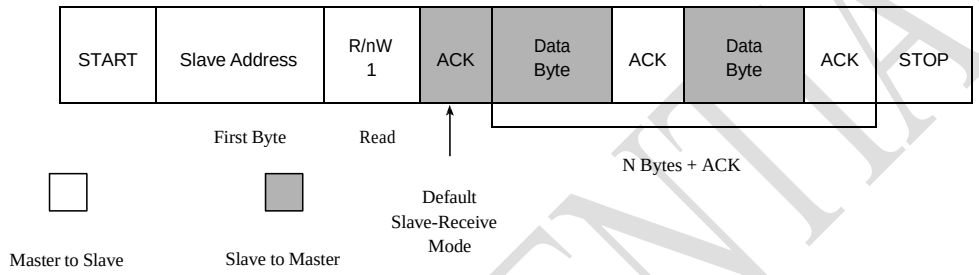


Figure 61:Master-Receiver Read from Slave-Transmitter / Repeated Start/Master-Transmitter Write to Slave-Receiver

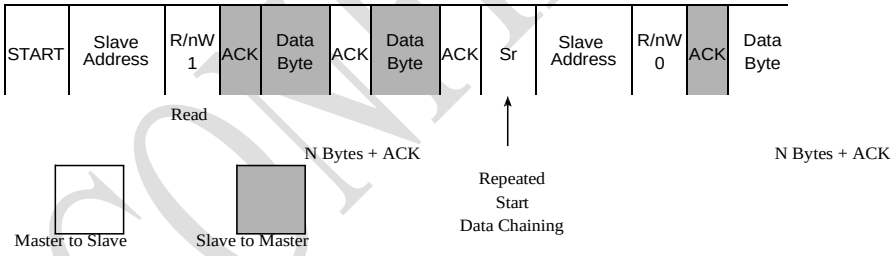
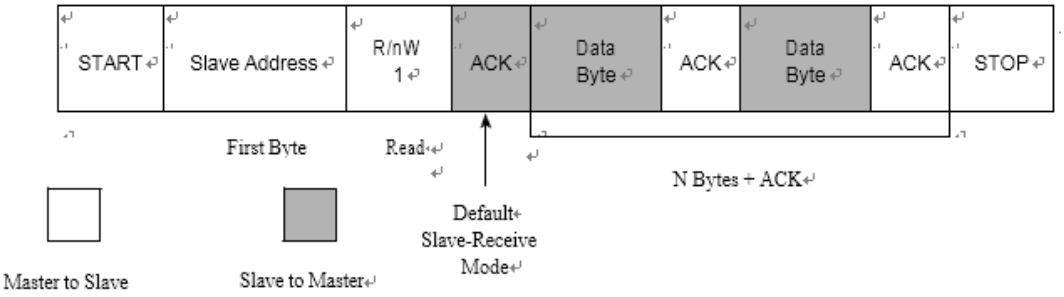


Figure 62:A Complete Data Transfer



ASR CONFIDENTIAL

15.4.10 FIFO mode

FIFO mode is an extension to the TWSI module. The main features of FIFO mode are:

- FIFOs are added on both transmit and receive sides.
  This helps reducing the number of IDBR empty/full interrupts. Instead of the core writing or reading to/from the IDBR 1 byte at a time, the FIFOs allow reading and writing multiple bytes without interrupting the core after each Byte.
- DMA mode is added.
  DMA mode allows improvement in long TWSI transactions (typically more than 8 bytes) where complete transaction can be programmed in DMA and allows reducing number of FIFO interrupts.



Note FIFO mode is completely backward compatible. It is allowed to disable FIFO mode and work in TWSI legacy mode by writing ICR[FIFO_EN]='0'.

Transmit FIFO has a width of 12 (4-bit control word + 8-bit data), and the number of entries is 8. Each entry has a data byte concatenated with the corresponding control word. The control word is nothing but the ICR[3:0] bits, which are required for each data byte. After a byte is transferred out, the next byte is copied from the Tx FIFO into the shift register, and the control word is copied into the ICR. This byte is now transferred, and it continues like that until the STOP bit is set.

Receive FIFO has a width of 8 (only data), and the number of entries is 16. Each entry stores the received byte. The control word for each Read byte concatenated with dummy data is put in the Transmit FIFO. When the receive FIFO is half full, an interrupt or DMA request is sent out and the FIFO entries have to be read out. Meanwhile, if new data bytes are read, they are stored in the remaining free entries in the FIFO.

In order to support the FIFO mode and fully utilize its capabilities, the following status and control bits were added:

- ICR[FIFO_EN] bit enables FIFO mode.
- ICR[TXBEGIN] bit used to start a transaction.
- New status bits in ISR have been added for FIFO mode interrupts and also a bit to flag Transaction done. ICR has the enable bits for all these interrupts.
- TXDONE interrupt generated at the end of each transaction (when STOP bit is send).

ASR CONFIDENTIAL

- ICR[DMA_EN] bit is used to enable/disable DMA mode and switch between DMA and PIO mode.

In DMA mode, all the FIFO related interrupts have to be disabled in ICR and ICR[DMA_EN] bit has to be set. That way only DMA requests are sent to the DMA and not any interrupts to the core.
Similarly in PIO mode, interrupts have to enabled and ICR[DMA_EN] bit cleared. So only interrupts to the core are generated when the Transmit FIFO is full or when the Receive FIFO is half full, full or overrun. ICR[TXDONE_IE] is for enabling Transaction Done Interrupt and needs to be set in both PIO and DMA modes. The core needs to get an interrupt after each transaction is done.

Note This FIFO mode should only be used when the TWSI is in Master mode. This FIFO mode is not for Slave mode.

ASR CONFIDENTIAL

15.4.11 TWSI Serial Clock Programming Guidelines

Before each of the TWSI modules has been initialized, set the clock:

1. Open and select the TWSI clock by setting the Clock/Reset Control Register for TWSI (APBC_ TWSI_CLK_RST).
2. Set the TWSI Load Count Register (TWSI_LCR) to the desired frequency.

15.4.12 Master Mode Programming Examples

15.4.12.1 *Initialize Unit*

Set the slave address in the ISAR.

Enable the preferred interrupts in the ICR. Do not enable the arbitration-loss-detected interrupt.

Set ICR[MODE] to the desired bus rate. For HS-mode bus rate, the master-code must be transmitted before HS-mode bus timing is enabled. Software should always set the HS mode bit (ICR[16]) as that bit makes the TWSI HS capable. This bit does not affect the non-HS transfers.

Set the ICR[IUE] and ICR[SCLE] bits to enable the TWSI and SCL.

15.4.12.2 *Write 1 Byte as a Master*

Load target slave address and R/nW bit in the IDBR. R/nW must be 0 for a Write.

Initiate the Write.

Set ICR[START], clear ICR[STOP], clear ICR[ALDIE], set ICR[TB]

When an IDBR transmit-empty interrupt occurs.

Read ISR register: ISR[ITE] = 1, ISR[UB] = 1, ISR[RWM] = 0

Write a 1 to the ISR[ITE] bit to clear interrupt.

Write a 1 to the ISR[ALD] bit if set.

If the master loses arbitration, it performs an address retry when the bus becomes free. The arbitration-loss-detected interrupt is disabled to allow the address retry.

Load data byte to be transferred in the IDBR.

Initiate the Write.

Clear ICR[START], set ICR[STOP], set ICR[ALDIE], set ICR[TB]

When an IDBR transmit-empty interrupt occurs (unit is sending Stop). Read ISR register: ISR[ITE] = 1, ISR[UB] = x, ISR[RWM] = 0

Write a 1 to the ISR[ITE] bit to clear the interrupt.

Clear ICR[STOP] bit.

15.4.12.3 *Read 1 Byte as a Master*

Load target slave address and R/nW bit in the IDBR. R/nW must be 1 for a Read.

ASR CONFIDENTIAL

Initiate the Write.

Set ICR[START], clear ICR[STOP], clear ICR[ALDIE], set ICR[TB]

When an IDBR transmit-empty interrupt occurs.

Read ISR register: ISR[ITE] = 1, ISR[UB] = 1, ISR[RWM] = 1

Write a 1 to the ISR[ITE] bit to clear the interrupt.

Initiate the Read.

Clear ICR[START], set ICR[STOP], set ICR[ALDIE], set ICR[ACKNAK], set ICR[TB]

When an IDBR receive-full interrupt occurs (unit is sending Stop).

Read ISR register: IDBR receive full (1), ISR[UB] = x, ISR[RWM] = 1, ACK/NAK bit (1)

Write a 1 to the ISR[IRF] bit to clear the interrupt.

Read IDBR data.

Clear ICR[STOP] and ICR[ACKNAK] bits

ASR CONFIDENTIAL

15.4.12.4 Write 2 Bytes and Repeated Start Read 1 Byte as a Master

Load target slave address and R/nW bit in the IDBR. R/nW must be 0 for a Write.

Initiate the Write.
Set ICR[START], clear ICR[STOP], clear ICR[ALDIE], set ICR[TB]

When an IDBR transmit-empty interrupt occurs.
Read ISR register: ISR[ITE] = 1, ISR[UB] = 1, ISR[RWM] = 0

Write a 1 to the ISR[ITE] bit to clear interrupt.

Load data byte to be transferred in the IDBR.

Initiate the Write.
Clear ICR[START], clear ICR[STOP], set ICR[ALDIE], set ICR[TB]

When an IDBR transmit-empty interrupt occurs.
Read ISR register: ISR[ITE] = 1, ISR[UB] = 1, ISR[RWM] = 0

Write a 1 to the ISR[ITE] bit to clear interrupt.

Repeat steps 5 to 8 one time.

Load target slave address and R/nW bit in the IDBR. R/nW must be 1 for a Read.

Send repeated Start as a master.
Set ICR[START], clear ICR[STOP], clear ICR[ALDIE], set ICR[TB]

When an IDBR transmit-empty interrupt occurs.
Read ISR register: ISR[ITE] = 1, ISR[UB] = 1, ISR[RWM] = 1

Write a 1 to the ISR[ITE] bit to clear interrupt.

Initiate the Read.
Clear ICR[START], set ICR[STOP], set ICR[ALDIE], set ICR[ACKNAK], set ICR[TB]

When an IDBR receive-full interrupt occurs (unit is sending Stop).
Read ISR register: ISR[IRF] = 1, ISR[UB] = x, ISR[RWM] = 1, ISR[ACKNAK] = 1.

Write a 1 to the ISR[IRF] bit to clear the interrupt.

Read IDBR data.

Clear ICR[STOP] and ICR[ACKNAK] bits

15.4.12.5 Read 2 Bytes as a Master—Send Stop Using the Abort

Load target slave address and R/nW bit in the IDBR. R/nW must be 1 for a Read.

Initiate the Write.
Set ICR[START], clear ICR[STOP], clear ICR[ALDIE], set ICR[TB]

When an IDBR transmit-empty interrupt occurs.
Read ISR register: ISR[ITE] = 1, ISR[UB] = 1, ISR[RWM] = 1

Write a 1 to the ISR[ITE] bit to clear interrupt.

ASR CONFIDENTIAL

Initiate the Read

Clear ICR[START], clear ICR[STOP], set ICR[ALDIE], clear ICR[ACKNAK], set ICR[TB]

When an IDBR receive-full interrupt occurs.

Read ISR register: ISR[IRF] = 1, ISR[UB] = 1, ISR[RWM] = 1, ACK/NAK bit (0)

Write a 1 to the ISR[IRF] bit to clear the interrupt.

Read IDBR data.

Clear ICR[STOP] and ICR[ACKNAK] bits

Initiate the Read.

Clear ICR[START], clear ICR[STOP], set ICR[ALDIE], set ICR[ACKNAK], set ICR[TB]

ICR[STOP] is not set because Stop or repeated Start is determined on the byte Read.

When an IDBR receive-full interrupt occurs.

Read ISR register: ISR[IRF] = 1, ISR[UB] = 1, ISR[RWM] = 1, ISR[ACKNAK] = 1

Write a 1 to the ISR[IRF] bit to clear the interrupt.

Read IDBR data.

ASR CONFIDENTIAL

Initiate Stop abort condition (Stop with no data transfer). Set ICR[MA].



Note

If a NAK is not sent in step 11, the next transaction must involve another data byte Read.

ASR CONFIDENTIAL

15.4.12.6 High-speed Mode: Write 1 Byte as a Master

Load master code in the IDBR.

Enable interrupts. Set ICR[ITEIE]

Enable high speed mode
Set ICR[MODE] = 0b10 or 0b11

Initiate the Write.
Set ICR[MODE], set ICR[START], clear ICR[STOP], set ICR[TB]

When an IDBR transmit-empty interrupt occurs.
Read ISR: IDBR transmit empty (1), unit busy (1), ACKNAK (1)

Write a 1 to the ISR[ITE] bit to clear interrupt.

Send a repeated start and slave address.
Load the slave address and the R/W bit in the IDBR.

Initiate the Write.
Set ICR[START], clear ICR[STOP], set ICR[TB]

Wait for the IDBR transmit empty interrupt.

Write a 1 to the ISR[ITE] bit to clear interrupt.

Load data byte to be transferred in the IDBR.

Initiate the Write.
Clear ICR[START], set ICR[STOP], set ICR[TB]

When an IDBR transmit-empty interrupt occurs (unit is sending Stop). Read ISR: IDBR transmit empty (1), unit busy (x), R/nW bit (0)

Write a 1 to the ISR[ITE] bit to clear the interrupt.

15.4.12.7 High-speed Mode: Read 1 Byte as a Master

Load master code in the IDBR.

Enable interrupts. Set ICR[ITEIE]

Initiate the Write.
Set ICR[MODE], set ICR[START], clear ICR[STOP], set ICR[TB]

When an IDBR transmit-empty interrupt occurs.
Read ISR: IDBR transmit empty (1), unit busy (1), ACKNAK (1)

Write a 1 to the ISR[ITE] bit to clear interrupt.

Send a repeated start and slave address.
Load the slave address and the R/W bit (1) in the IDBR.

ASR CONFIDENTIAL

Initiate the Write.
Set ICR[START], clear ICR[STOP], set ICR[TB]
Wait for the IDBR transmit empty interrupt.
Write a 1 to the ISR[ITE] bit to clear interrupt.

Initiate the Read.
Clear ICR[START], set ICR[STOP], set ICR[ACKNAK], set ICR[TB]
When an IDBR receive-full interrupt occurs (unit is sending Stop).
Read ISR: IDBR receive full (1), unit busy (x), R/nW bit (1), ACKNAK (1)
Write a 1 to the ISR[IRF] bit to clear the interrupt.
Read IDBR data.
Clear ICR[STOP] and ICR[ACKNAK] bits

15.4.12.8 FIFO mode: Write/Read n Bytes as a Master in PIO mode

Program TWSI slave address.
Write ICR[FIFOEN] to enable FIFO mode and enable the FIFO interrupts. An interrupt is generated here if the TX FIFO Empty interrupt is enabled.
Fill up the TX FIFO with control+data in proper format as shown in Figure 62 and Figure 63.
Write a 1 to the ISR[TXE] bit to clear the TX FIFO empty interrupt.
While the data transfer is happening, wait for the TX FIFO empty interrupt.
When TX FIFO empty interrupt is seen, refill the FIFO with more control+data and then write 1 to the ISR[TXE] bit to clear the TX FIFO empty interrupt.
Wait for TX DONE or TX empty again (depending on the transaction).
If TX DONE interrupt is received, program ICR for the next transaction and once the FIFO has the control+data, set the ICR[TXBEGIN] bit. This bit starts the next transaction.
If the RX FIFO becomes half full during this sequence, RX FIFO Half Full Interrupt is generated (If enabled).
Read the RX FIFO data and then write 1 to the ISR[RXF] bit to clear the RX FIFO Half Full interrupt.

15.4.12.9 FIFO mode: Write/Read n Bytes as a Master in DMA mode

Program the DMA descriptors and put the control+data in memory in the proper format as shown in Figure 63 and Figure 64.
Program TWSI slave address
Write 1 to the ICR[FIFOEN] bit to enable FIFO mode and the ICR[DMAEN] bit to enable DMA mode (disable TX FIFO empty & RX full interrupts, but

ASR CONFIDENTIAL

enable Transmit Done interrupt)

TX FIFO empty DMA request is generated immediately since the TX FIFO is empty when TWSI comes out of reset.

DMA services the request and fills up the TX FIFO with control+data.

The TWSI starts the transaction on the bus and when the TX FIFO becomes empty, a TX FIFO empty DMA request is generated.

Repeat #5 and #6 until a TX DONE interrupt is received.

When TX DONE is received (this means a STOP occurred), CPU must write to ISR and ICR registers to reconfigure them for the next transaction and also set the ICR[TXBEGIN] bit for the next transaction to start.

Transaction continues and when TX FIFO is empty, the TX FIFO empty DMA request is generated once again.

If the RX FIFO ever becomes half full during this sequence it sends a RX FIFO Half Full DMA request to the DMA.

DMA reads the RX FIFO contents.

After receiving a TXDONE interrupt, if there are any trailing bytes (Only for Read Transactions) it is up to the software to handle them. The hardware does not handle trailing bytes.

ASR CONFIDENTIAL

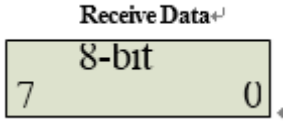
Figure 63:Tx FIFO Entry Format



Each entry in the Tx FIFO has the format as shown in Figure 63. The LSB 8 bits are for data and the MSB 4 bits are for control bits. For a Write transaction, Data consists of the Slave address followed by actual Write data.The control bits are nothing but ICR[3:0] bits.

For a Read transaction, as shown in Figure 64, Data consists of the Slave address followed by dummy data (actual Read data from the slave goes into the Rx FIFO). Again, the control bits are ICR[3:0] bits.

Figure 64:Rx FIFO Entry Format

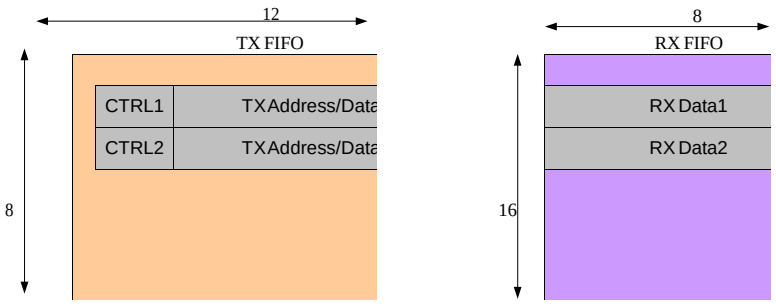


Each entry in the Rx FIFO is just like the IDBR register and has the format shown in Figure 64. Only 8-bit data received is stored in each entry.

15.4.12.10 FIFO Programming Examples

The Transmit and Receive FIFOs are as shown in Figure 65.

Figure 65:Transmit and Receive FIFOs



ASR CONFIDENTIAL

Each control word (CTRL) is 4 bits:

- [TB]
- [ACKNACK]
- [STOP]

ASR CONFIDENTIAL

and

- [START]

Each entry in the Tx FIFO has a control word concatenated with Address/Data byte. The control word has control information to send/receive that particular byte. Note that Transmit FIFO has 8 entries and RX FIFO has 16 entries. Interrupt/DMA requests are made when the Tx FIFO is empty or when Rx FIFO is half full.

Figure 66:Case 1: Write 10 Bytes



Programming model for Case 1

Core initially programs the ICR for FIFO mode. It then receives a Tx FIFO empty interrupt if in PIO mode or a DMA request if in DMA mode.

The core/DMA then writes 1 address byte + 7 data bytes (Byte1-Byte7) to the Tx FIFO (with each byte having a corresponding control word).

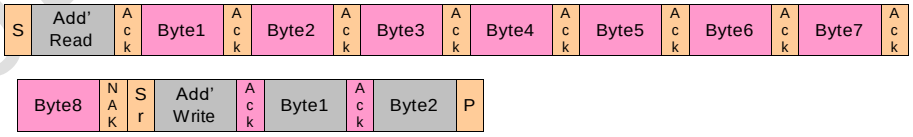
TWSI starts sending out the bytes and when the Tx FIFO is empty, it generates a Tx FIFO empty interrupt/DMA request.

The core/DMA then writes the last 3 bytes (Byte8-Byte10) to the Tx FIFO.

TWSI sends out the last 3 bytes, and when it sees that STOP has been sent out, it sets the ISR[TX_DONE] bit and generates an interrupt.

The core then cleans up the control and status registers (example: Clear ICR[STOP] bit, clear ISR[TX_DONE] bit) and starts the next transaction (set ICR[TX_BEGIN] bit).

Figure 67:Case 2: Read 8 Bytes Followed by Write 2 Bytes



Programming Model for Case 2

Core initially programs the ICR for FIFO mode. It then receives a Tx FIFO empty interrupt if in PIO mode or a DMA request if in DMA mode.

The core/DMA then writes 1 Addr + 7 Data Bytes to the TX FIFO (control

ASR CONFIDENTIAL

word + dummy data since it is a Read transaction).

After the address is sent out on the bus, for each control word, a Read byte is received and saved off in the Rx FIFO. Once the Tx FIFO is empty (mote that there are 7 bytes in the Rx FIFO by now), an interrupt/DMA request is made and the remaining bytes (1 data byte from Read transaction and 3 bytes from the next Write transaction) are loaded into the Tx FIFO.

After the 8 byte is received into the Rx FIFO, the Rx FIFO Half full interrupt/DMA request is set. This Read data now needs to be read out of the FIFO.

ASR CONFIDENTIAL

By now, the Read transaction is also done. But since there is NO stop bit after the Read and instead Repeated Start is used for the Write Transaction, the ISR[TX DONE] status bit is NOT set as it would have normally been set at the end of a transaction.

TWSI now starts the Write transaction by sending out the Address followed by the 2 Write bytes.

Once the Write transaction is done, ICR[TX_BEGIN] is automatically cleared and ISR[TX_ DONE] bit is set, which generates an interrupt to the core.

15.4.13 Slave Operations

Table 41 describes how the TWSI unit operates as a slave device.

Table 41:Slave Transactions

TWSI Slave Action	Mode of Operation	Definition
Slave-receive (default mode)	Slave-receive only	• The TWSI monitors all slave address transactions. • ICR[IUE] must be set. • The TWSI monitors bus for Start conditions. When a Start is detected, the interface reads the first 8 bits and compares the most significant 7 bits with the 7-bit ISAR and the general call address (0x00). If there is a match, the TWSI sends an ACK. • If the first 8 bits are zeros, this is a general call address. If ICR[GCD] is clear, both the ISR[GCAD] and ISR[SAD] are set. See Section 16.4.15, General Call Address. • If the eighth bit of the first byte (R/nW bit) is low, the TWSI stays in Slave-Receive mode, and ISR[SAD] is cleared. If R/nW bit is high, the TWSI unit switches to Slave-Transmit mode, and ISR[SAD] is set.
Set the slave-address-detected bit	Slave-receive Slave-transmit	• Indicates the interface has detected an TWSI operation that addresses the ASR1606 (this includes general call address). The ASR1606 can distinguish an ISAR match from a general call by reading ISR[GCAD]. • An interrupt is generated, if enabled, after the matching slave address is received and acknowledged.
Read one byte of TWSI data from the IDBR	Slave-receive only	• This operation occurs when ISR[IRF] is set and ICR[TB] is clear. If enabled, the IDBR receive-full interrupt is generated. • Eight bits are read from the serial bus into the shift register. When a full byte has been received and the ACK/NAK bit is completed, the byte is transferred from the Shift register to the IDBR. • Occurs when the IDBR receive full bit in the ISR is set and the transfer byte bit is clear. If enabled, the IDBR receive-full interrupt is signalled to the CPU. • Software reads one data byte from the IDBR. When the IDBR is read, the software must write the preferred ICR[ACKNAK] bit and sets ICR[TB]. This causes the TWSI to stop inserting wait states and let the master transmitter transmit the next chunk of information.
Transmit Acknowledge to master transmitter	Slave-receive only	• As a slave receiver, the TWSI pulls the SDA line low to generate the ACK pulse during the high SCL period. • ICR[ACKNAK] controls the acknowledge pulse that the TWSI drives. See Section 16.4.6, TWSI Acknowledge.

ASR CONFIDENTIAL

Write one byte of TWSI data to the IDBR	Slave-transmit only	• This operation occurs when ISR[ITE] is set and ICR[TB] is clear. If enabled, the IDBR transmit-empty interrupt is generated. • The software must write a data byte to IDBR and sets ICR[TB] to start the transfer.
Wait for Acknowledge from master receiver	Slave-transmit only	• As a slave transmitter, the TWSI releases the SDA line to allow the master receiver to pull the line low for the ACK. See Section 16.4.6, TWSI Acknowledge.

ASR CONFIDENTIAL

Figure 68 through Figure 70 are examples of TWSI transactions and show the relationships between master and slave devices.

Figure 68:Master Transmitter Write to Slave Receiver

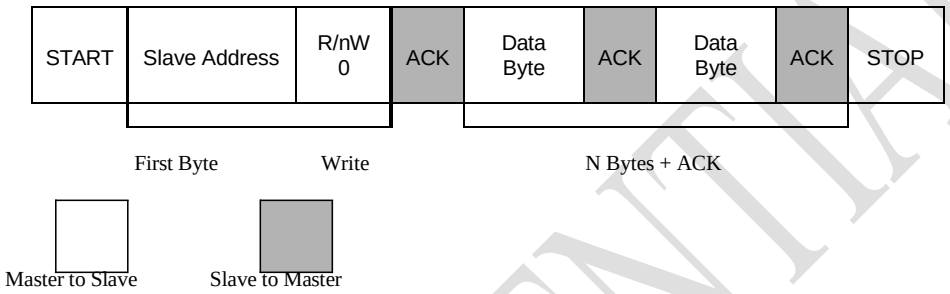


Figure 69:Master Receiver Read from Slave-Transmitter

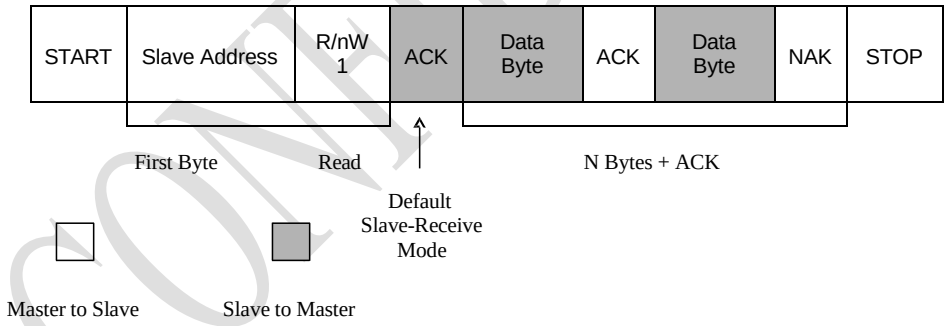
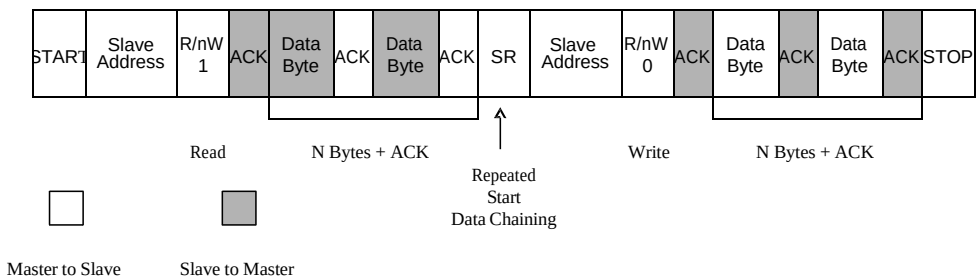


Figure 70:Master-Receiver Read to Slave-Transmitter, Repeated Start, Master-Transmitter Write to Slave Receiver



ASR CONFIDENTIAL

15.4.14 Slave Mode Programming Examples

15.4.14.1 *Initialize Unit*

Set the slave address in the ISAR.

Enable preferred interrupts in the ICR.

If the TWSI unit is a HS-mode slave, set ICR[MODE] = 0b10 or 0b11. Software should always set the HS mode bit (ICR[16]) as that bit makes the TWSI HS capable. This bit does not affect the non-HS transfers.

Set the ICR[IUE] bit to enable the TWSI.

15.4.14.2 *Transmit n Bytes as a Slave*

When a slave-address-detected interrupt occurs.
Read ISR register: ISR[SAD] = 1, ISR[UB] = 1, ISR[RWM] = 1, ISR[ACKNAK] = 0

Write a 1 to the ISR[SAD] bit to clear the interrupt.

Return from interrupt.

Load data byte to transfer in the IDBR.

Set ICR[TB] bit.

When an IDBR transmit-empty interrupt occurs.
Read ISR register: ISR[ITE] = 1, ISR[ACKNAK] = 0, ISR[RWM] = 0

Load data byte to transfer in the IDBR.

Set the ICR[TB] bit.

Write a 1 to the ISR[ITE] bit to clear interrupt.

Return from interrupt.

Repeat steps 6 to 10 for *n*-1 times. If, at any time, the slave does not have data, the TWSI keeps SCL low until data is available.

When a IDBR transmit-empty interrupt occurs.
Read ISR register: ISR[ITE] = 1, ISR[ACKNAK] = 1, ISR[RWM] = 0

Write a 1 to the ISR[ITE] bit to clear interrupt.

Return from interrupt

When a slave-Stop-detected interrupt occurs. Read ISR register: ISR[UB] = 0, ISR[SSD] = 1

Write a 1 to the ISR[SSD] bit to clear interrupt.

15.4.14.3 *Receive n Bytes as a Slave*

When a slave-address-detected interrupt occurs.
Read ISR register: ISR[SAD] = 1, ISR[UB] = 1, ISR[RWM] = 0

Write a 1 to the ISR[SAD] bit to clear the interrupt.

Return from interrupt.

Set ICR[TB] bit to initiate the transfer.

ASR CONFIDENTIAL

When an IDBR receive-full interrupt occurs.
Read ISR register: ISR[IRF] = 1, ISR[ACKNAK] = 0, ISR[RWM] = 0
Read IDBR to get the received byte.
Write a 1 to the ISR[IRF] bit to clear interrupt.
Return from interrupt.
Repeat steps 4 to 8 for *n*-1 times. Once the IDBR is full, the TWSI keeps SCL low until the data is read.
Set ICR[TB] bit to release TWSI bus and allow next transfer.

ASR CONFIDENTIAL

When a slave-stop-detected interrupt occurs. Read ISR register: ISR[UB] = 0, ISR[SSD] = 1

Write a 1 to the ISR[SSD] bit to clear interrupt.

15.4.14.4 High-speed Mode: Transmit 1 Byte as a Slave

Enable slave address detect interrupts. Set ICR[SADIE]

Enable high speed mode
Set ICR[MODE] = 0b10 or 0b11

A master sends a master code at the standard-mode or fast-mode data rate. Then, a slave address is sent at the high speed mode data rate.

Wait for the slave address detect interrupt.
Read the ISR: slave address detect (1), Unit Busy (1), R/nW bit (1), ACK/NAK (0).

Write a 1 to the ISR[SAD] bit to clear the interrupt.

Load the IDBR with the data to be written (read by the master).

Set the ICR[TB] bit.

When an IDBR transmit-empty interrupt occurs.
Read ISR: IDBR transmit empty (1), unit busy (1), ACKNAK (1), R/nW bit (0).

Write a 1 to the ISR[ITE] bit to clear interrupt.

Return from interrupt.

Wait for a slave stop detect interrupt.
Read ISR: slave stop detect (1), Unit Busy (0);

Write a 1 to the ISR[SSD] bit to clear interrupt.

15.4.14.5 High-speed Mode: Receive 1 Byte as a Slave

Enable slave address detect interrupts. Set ICR[SADIE]

Enable high speed mode
Set ICR[MODE] = 0b10 or 0b11

A master sends a master code at the standard-mode or fast-mode data rate. Then a slave address is sent at the high speed mode data rate.

Wait for the slave address detect interrupt.
Read the ISR: slave address detect (1), Unit Busy (1), R/nW bit (0), ACK/NAK (0).

Write a 1 to the ISR[SAD] bit to clear the interrupt.

Set the ICR[TB] bit to initiate transfer.

When an IDBR receive-full interrupt occurs.
Read ISR: IDBR receive full (1), unit busy (1), ACKNAK (0), R/nW bit (0).

ASR CONFIDENTIAL

Read IDBR to get the received byte.
Write a 1 to the ISR[IRF] bit to clear interrupt.
Return from interrupt.
Set ICR[TB] bit to release TWSI bus and allow the next transfer.
Wait for a slave stop detect interrupt.
Read ISR: slave stop detect (1), Unit Busy (0);
Write a 1 to the ISR[SSD] bit to clear interrupt.

ASR CONFIDENTIAL

15.4.15 General Call Address

Note Hardware general call is not supported in the ASR1606. (Refer to Section 16.1, Overview and Table 42 for more information.)

A general call address is a transaction with a slave address of 0x00. When a device requires the data from a general call address, it acknowledges the transaction and stays in Slave-Receive mode. Otherwise, the device ignores the general call address. The other bytes in a general call transaction are acknowledged by every device that uses it on the bus. Devices that do not use these bytes must not send an ACK. The meaning of a general call address is defined in the second byte sent by the master transmitter. Figure 71 shows a general call address transaction. The least significant bit (LSb) of the second byte, called B, defines the transaction. Table 42 shows the valid values and definitions when B = 0.

The TWSI unit supports sending and receiving general call address transfers on the TWSI bus. When software sends a general call message from the TWSI, software must set the ICR[GCD] bit to prevent the ASR1606 TWSI from responding as a slave.

Caution If the ICR[GCD] is not set when software sends a general call message from the TWSI, the TWSI bus enters an indeterminate state.

If the TWSI acts as a slave and receives a general call address while the ICR[GCD] bit is clear, it:

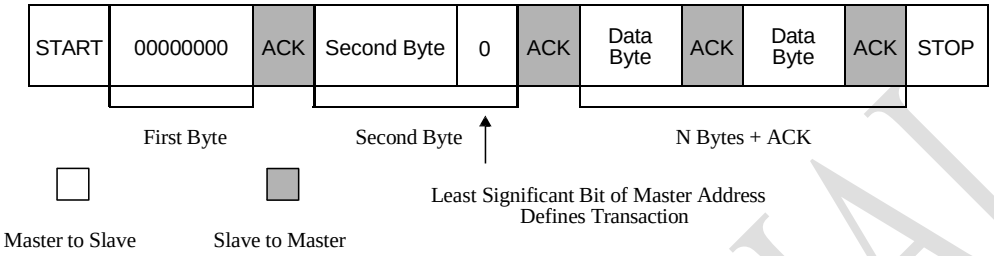
- Sets the ISR[GCAD] bit
- Sets the ISR[SAD] bit
- Interrupts the ASR1606 (if the interrupt is enabled)

If the TWSI receives a general call address and the ICR[GCD] bit is set, it ignores the general call address.

ASR CONFIDENTIAL

Copyright © 2022 ASR

Figure 71:General Call Address



ASR CONFIDENTIAL

Table 42:General Call Address Second Byte Definitions

Least Significant Bit of 2nd Byte (B)	2nd Byte Value	Definition
0	0x06	2-byte transaction in which the second byte tells the slave to reset and store this value in the programmable part of its address.
0	0x04	2-byte transaction in which the second byte tells the slave to store this value in the programmable part of its address. No reset.
0	0x00	Not allowed as a second byte

NOTE: Other values are not fixed and must be ignored.

Software must ensure that the TWSI is not busy before it asserts a reset. Software must also ensure that the TWSI bus is idle when the unit is enabled after reset. When directed to reset, the TWSI, except for ISAR, returns to the default reset condition. ISAR is not affected by a reset.

When B=1, the sequence is a hardware general call and is not supported by the TWSI. Refer to the
TWSI Bus Specification for information on hardware general calls.



Note TWSI 10-bit addresses and CBUS compatibility are not supported in the ASR1606.

15.4.16 Glitch Suppression Logic

The TWSI unit has built-in glitch-suppression logic. The glitch suppression logic is implemented differently depending on whether the TWSI unit is in F/S mode or High-speed mode.

For F/S mode, the glitch suppression specification is 50ns. Glitches are suppressed for a length of time given by: 4 * (1 / TWSI input clock frequency). For example, with a 32.8 MHz input clock frequency, glitches of 120 ns or less are suppressed.

For High-speed mode, the glitch suppression specification is 10 ns. Glitches are suppressed for a length of time given by: 1* (1 / TWSI input clock frequency). For example, with a 69.3 MHz internal input clock frequency, glitches of 14.5 ns or less are suppressed.

ASR CONFIDENTIAL

15.4.17 Reset Conditions

Software must ensure that the TWSI unit is not busy (ISR[UB] = 0) before it asserts a reset. Software must also ensure that the TWSI bus is idle (ISR[IBB] = 0) when the unit is enabled after reset. When directed to reset, the TWSI, except for ISAR, returns to the default reset condition. ISAR is not affected by a reset.

When the ICR[UR] bit is set, the TWSI resets but the associated TWSI MMRs remain intact. When resetting the TWSI with the ICR unit reset, use the following guidelines:

1. Set the reset bit in the ICR register and clear the remainder of the register.
2. Clear the ISR register.
3. Clear reset in the ICR.

15.5 Register Descriptions

Refer to the *ASR1606 Registers* for details on the TWSI Controller register.

ASR CONFIDENTIAL

16.SSP Serial Ports

This chapter describes the signal definitions and operation of the Synchronous Serial Protocol (SSP) ports, SSP0 and SSP2 in the Application Subsystem, and DSSP and GSSP in the Communication Subsystem.

- Section 16.1, Overview
- Section 16.2, Features
- Section 16.3, Signal Descriptions
- Section 16.4, Operation
- Section 16.5, Register Descriptions

SSP0 and SSP2 are identical in operation. The DSSP and GSSP ports are identical to the SSPx ports with the exception of some differences that are described in Section 10, Communication Subsystem Peripherals Functional Description. Refer to the *ASR1606 Registers* for the memory map locations for the registers for each SSP port.



Note

In the Application Subsystem SSP serial ports (SSPs 0 and 2), the following fields in the SSP Control Register 0 are Reserved:

- <Audio Clock Select>
- <52M[bps] Mode>
- <Network Clock Select>
- <Serial Clock Rate>
- <External Clock Select>

Since these fields are reserved in the Application Subsystem SSP serial ports, functionality of them as described in this section is not applicable. In addition, the SSP Audio Clock Divider Register and the SSP Audio Clock Dither Divider Register are also not applicable to the Application Subsystem SSP serial ports.

Refer to the *ASR1606 Registers* document for more information.



Note

Throughout this chapter there are many references to registers and their values. For more information on any of the SSPx registers, refer to the *ASR1606 Registers*.

ASR CONFIDENTIAL

16.1 *Overview*

The SSP port is a synchronous serial controller that can be connected to a variety of external Analog-to-Digital converters (ADC), audio and telecommunication codecs, and many other devices that use serial protocols for data transfer. The SSP Controllers directly support the following protocols:

- Texas Instruments* Synchronous Serial Protocol (SSP)
- Motorola* Serial Peripheral Interface (SPI)
- National Semiconductor* Microwire

ASR CONFIDENTIAL

- Programmable Serial Protocol (PSP)
  - Inter-IC Sound (I^2S) protocol is supported through the PSP mode.

The SSP ports operate as full-duplex devices for the SSP, SPI, and PSP protocols and as a half-duplex device for the Microwire protocol.

The SSP ports are configurable to operate in Master mode (the attached peripheral functions as a slave) or Slave mode (the attached peripheral functions as a master). The SSP ports support serial bit rates from 6.3 Kbps (minimum recommended speed) up to 52 Mbps. Serial data sample size can be set to 8, 16, 18, or 32 bits in length. A FIFO is provided for Transmit data and a second, independent FIFO is provided for Receive data. The two FIFOs are both 32 samples deep x 32 bits wide or both 64 samples deep x 16 bits wide.

The FIFOs can be loaded or emptied by the ASR1606 using programmed I/O (PIO) or by DMA burst transfers.

16.2 *Features*

The enhanced SSP port features are:

- Directly supports National Semiconductor* Microwire, Texas Instruments* Synchronous Serial Protocol (SSP), and Motorola* Serial Peripheral Interface (SPI).
- The I^2S protocol is supported by programming the PSP. Data sample sizes can be set to 8, 16, 18, or 32 bits
- One FIFO for Transmit data (TXFIFO) and a second, independent, FIFO for Receive data (RXFIFO); for Non-Packed Data mode, the two FIFOs are each 32 rows deep x 32 bits wide for a total of 32 samples
- FIFO Packed mode allows double depth FIFOs if the samples are 8 bits or 16 bits wide; for Packed Data mode, both FIFOs are 64 locations deep x 16 bits wide for a total of 64 samples
- 52 Mbps maximum serial bit-rate (see Note in SSP FIFO Access and FIFO Operation in Packed Mode regarding limitation)
- Master mode and Slave mode operation supported
- Receive-without-Transmit operation
- Network mode with up to eight time slots for PSP formats, and independent Transmit/Receive in any/all/none of the time slots
- Audio clock control to provide a 4x or 8x output clock to support most standard audio frequencies

ASR CONFIDENTIAL

16.3 *Signal Descriptions*

Table 43:ASR1606 External Signal Descriptions

Signal Name	Direction	Description
SSP 0 Signals		
SSP0_RXD	Input	Synchronous Serial Protocol Receive Data Serial data in. Sample length is selected by the <Extended Data Size Select> and <Data Size Select> fields in the SSP Control Register 0
SSP0_TXD	Output	Synchronous Serial Protocol Transmit Data Serial data out. Sample length is selected by the <Extended Data Size Select> and <Data Size Select> fields in the SSP Control Register 0
SSP0_CLK	I/O	Synchronous Serial Protocol Serial Clock Controls the timing of a serial transfer. SSPx_SCLK can be generated internally (Master mode) or taken from an external source (Slave mode) See the <SSP Serial Bit Rate Clock (SSPx_SCLKx) Direction> field in the SSP Control Register 1
SSP0_FRM	I/O	Synchronous Serial Protocol Serial Frame Indicator Indicates the beginning and the end of a serialized data sample. The SSPSFRM can be generated internally (Master mode) or taken from an external source (Slave mode). See the <SSP Frame (SSPx_FRM) Direction> field in the SSP Control Register 1
SSP 2 Signals		
SSP2_RXD	Input	Synchronous Serial Protocol Receive Data Serial data in. Sample length is selected by the <Extended Data Size Select> and <Data Size Select> fields in the SSP Control Register 0.
SSP2_TXD	Output	Synchronous Serial Protocol Transmit Data Serial data out. Sample length is selected by the <Extended Data Size Select> and <Data Size Select> fields in the SSP Control Register 0
SSP2_SCLK	I/O	Synchronous Serial Protocol Serial Clock Controls the timing of a serial transfer. SSPx_SCLK can be generated internally (Master mode) or taken from an external source (Slave mode) See the <SSP Serial Bit Rate Clock (SSPx_SCLK) Direction> field in the SSP Control Register 1
SSP2_FRM	I/O	Synchronous Serial Protocol Serial Frame Indicator Indicates the beginning and the end of a serialized data sample. The SSPx_FRM can be generated internally (Master mode) or taken from an external source (Slave mode). See the <SSP Frame (SSPx_FRM) Direction> field in the SSP Control Register 1
Communication Subsystem SSP Ports		
DSSP Signals		
DSSP0_FRM	I/O	**Synchronous Serial Port Frame 0** The serial frame sync can be configured as an output (Master mode operation) or an input (Slave mode operation). **NOTE:** DSSP0: RXTXEN
DSSP0_RX_TX	I/O	**Synchronous Serial Port Transmit/Receive Data 0** Serial data driven out or in synchronously with the bit clock **NOTE:** DSSP0: RXTXDATA
DSSP0_SCLK	I/O	**Synchronous Serial Port Clock 0** The serial bit clock can be configured as an output (Master mode operation) or an input (Slave mode operation).

ASR CONFIDENTIAL

DSSP0_TX	O	**Synchronous Serial Port Transmit Data 0** Serial data driven out synchronously with the bit clock
DSSP0_RX	I	**Synchronous Serial Port Receive Data 0** Serial data driven in synchronously with the bit clock
GSSP Signals		
GSSP_RXD	I	**Synchronous Serial Port Receive Data** Serial data latched using the bit clock
Signal Name	**Direction**	**Description**
GSSP_SCLK	I/O	**Synchronous Serial Port Clock** The serial bit clock can be configured as an output (Master mode operation) or an input (Slave mode operation).
GSSP_SFRM	I/O	**Synchronous Serial Port Frame** The serial frame sync can be configured as an output (Master mode operation) or an input (Slave mode operation).
GSSP_TXD	O	**Synchronous Serial Port Transmit Data** Serial data driven out synchronously with the bit clock

16.4 *Operation*

Serial data is transferred between the ASR1606 and an external peripheral through FIFOs in the SSPx port. Data transfers between an SSPx port and memory are initiated by the ASR1606 using programmed I/O (PIO) or by DMA bursts. Separate Transmit and Receive FIFOs and serial data paths permit simultaneous transfers in both directions to and from the external peripheral, depending on the protocols chosen.

PIO can transfer data between:

- The ASR1606 and the FIFO Data Register for the TXFIFO
- The ASR1606 and the FIFO Data Register for the RXFIFO
- The ASR1606 and the SSP Control Registers 1 and 2 or SSP

Status Register DMA bursts can transfer data between:

- The memory and the FIFO Data Register for the TXFIFO
- The memory and the FIFO Data Register for the RXFIFO

Data written to the FIFO Data Register by either the CPU or DMA is automatically transferred to the Transmit FIFO. When reading the FIFO Data Register by either the CPU or DMA, the “oldest” data in the Receive FIFO is automatically transferred to the FIFO Data Register.

ASR CONFIDENTIAL

16.4.1 SSP FIFO Access

The data is accessed through the TXFIFO and RXFIFO. An ASR1606 access takes the form of PIO, transferring one FIFO entry per access (see FIFO Operation in Packed Mode). The ASR1606 accesses are normally triggered by an interrupt caused by an SSP Status Register event (see *ASR1606 Registers*) and must always be 32-bits wide. The ASR1606 writes to the TXFIFO are 32-bits wide, but the serializing logic ignores all bits beyond the programmed FIFO data size (see SSP_SSCR0[EDSS] and SSP_SSCR0[DSS] fields in the *ASR1606 Registers*). The ASR1606 Reads from the RXFIFO are also 32-bits wide, but the data that is received by the RXD interface signal is written, with zeroes inserted in the MSBs down to the programmed data size, into the RXFIFO.

The TXFIFO and RXFIFO can also be accessed by DMA bursts, which must be 8, 16, or 32 bytes in length, and must transfer one FIFO entry per access. When the SSP_SSCR0[EDSS] bit is set, the SSPx port must be configured as a 32-bit peripheral. The DMA_CMDX[WIDTH] field must be programmed to at least the same data size programmed into this SSP_SSCR0[EDSS] and SSP_ SSCR0[DSS] fields (see the *ASR1606 Registers*.)

The TXFIFO and RXFIFO are each seen as one 32-bit location by the ASR1606. For data transmission, the SSPx port takes the data from the TXFIFO, serializes it, and transmits it via the SSPTXDx serial interface signal to the external peripheral. Data received from the external peripheral via the SSPRXDx interface signal is converted to parallel words and written into the RXFIFO.

ASR CONFIDENTIAL

A programmable FIFO trigger threshold, when exceeded, generates an interrupt or DMA service request that, if enabled, signals the ASR1606 or DMA, respectively, to empty the RXFIFO or to refill the TXFIFO.

The TXFIFO and RXFIFO are differentiated by whether the access is a Read or a Write transfer. Reads from the FIFO Data Register automatically target the RXFIFO. Writes to the FIFO Data Register automatically target the TXFIFO. From a memory-map perspective, the TXFIFO and the RXFIFO are at the same address. Each FIFO is 32 rows deep x 32 bits wide for a total of 32 data samples. Each sample can be 8, 16, 18, or 32 bits in length.

FIFO Operation in Packed Mode

When the TXFIFO and RXFIFO are operating in packed mode, each FIFO is 64 rows deep x 16-bits wide for a total of 64 data samples. For packed mode, each sample can be 8 or 16 bits in length.
When the data is serialized and transmitted, Bits 15 to 0 are transmitted first, followed by Bits 31 to 16.

When the TXFIFO and RXFIFO are operating in packed mode, they may best be thought of as a single entry of 32 bits holding two 8- or 16-bit samples. Thus, the ASR1606 or the DMA should write and read 32 bits of data at a time where each Write or Read transfers two samples. The entire FIFO width (32 bits) must be read/written in this mode. The SSPx port does not support writing two separate 16-bit samples in this mode. Calculate the thresholds based on the number of 32-bit Writes or Reads, not the number of 16-bit or less values.



Note

At serial bit rates approaching 13 MHz for continuous data transfers, the DMA might not be able to access the RXFIFO or TXFIFO fast enough to avoid overflow or underflow, respectively. Using packed mode improves performance.

16.4.2 Trailing Bytes in RXFIFO

When the number of samples in the RXFIFO is less than its trigger threshold level and no additional data is received, the remaining bytes are called *RXFIFO trailing bytes*. RXFIFO trailing bytes can be handled by either the ASR1606 or by DMA, as indicated by the Trailing Byte field in the SSP Control Register 1, SSP_SSPCR1[TRAIL] (see the *ASR1606 Registers*). RXFIFO trailing bytes are identified by means of a time-out mechanism and the existence of data within the RXFIFO after timeout.

ASR CONFIDENTIAL

Note

When FIFO packed mode is used, the DMA cannot be used to handle the RXFIFO trailing bytes. The RXFIFO trailing bytes must be handled by the ASR1606.

16.4.2.1 Timeout

A timeout condition exists when the RXFIFO has been idle for a period of time defined by the value programmed within the SSP_SSTO[Timeout Value] field, see the *ASR1606 Registers*. When a timeout occurs, the Receiver Time-out Interrupt bit, SSP_SSCR1[TINT], is set to 1, and if the Receiver Time-out Interrupt Enable bit, SSP_SSCR1[TINTE], is set, a timeout interrupt signals the ASR1606 that a timeout condition has occurred. The timeout timer is reset after a new data sample is received into the RXFIFO. Once the SSP_SSCR1[TINT] bit is set, it must be cleared by writing 0x1 to the SSP_SSCR1[TINT] bit. Clearing it also causes the timeout interrupt, if enabled, to be de-asserted.

ASR CONFIDENTIAL

16.4.2.2 *Peripheral Trailing Byte Interrupt*

It is possible for the DMA to reach the end of its Descriptor chain while removing RXFIFO data. When this happens, the ASR1606 must take over because the DMA can no longer service the SSPx port until a new chain is linked. When the DMA has reached the end of its Descriptor chain with data in the RXFIFO, the SSPx port:

1. Sets the Peripheral Trailing Byte Interrupt bit, SSP_SSSR[PINT].
2. Asserts the SSPx port interrupt to signal to the ASR1606 that a peripheral trailing-byte interrupt condition has occurred (if the interrupt is enabled by setting the Peripheral Trailing Byte Interrupt Enable bit, SSP_SSCR1[PINTE]).
3. Sets the End Of Chain, SSP_SSSR[EOC], field in the SSP Status Register. If more data is received after the SSP_SSSR[EOC] field was set (and it remains set), then the SSP_ SSSR[PINT] field is set. The SSP_SSSR[EOC] field must be cleared by writing 0x1 to it.
4. Once the SSP_SSSR[PINT] field is set, the ASR1606 must clear the bit by writing 0x1 to it. Clearing it also de-asserts the SSP interrupt if it has been enabled (SSP_SSCR1[PINTE] = 1).

The remaining bytes must then be removed with the PIO method by the ASR1606 as described in Removing FIFO Trailing Bytes or by reprogramming a new Descriptor chain and restarting the DMA. Programmers need to be aware of this possibility. Refer to the DMA chapter for details on Descriptor programming and End-of-Chain (EOC) events.

16.4.2.3 *Removing FIFO Trailing Bytes*

When the Trailing Byte, SSP_SSCR1[TRAIL], bit is cleared, trailing bytes left in the RXFIFO are handled by the ASR1606 programmed I/O method. This is the default method.

If a timeout occurs, the ASR1606 is only interrupted by a timeout interrupt if it has been enabled by setting the Receiver Time-out Interrupt Enable SSP_SSCR1[TINTE] field. To read out the trailing bytes from the RXFIFO, software should wait for the timeout interrupt and then read all trailing bytes as indicated by the Odd Sample Status, SSP_SSSR[OSS], Receive FIFO Level, SSP_SSSR[RFL], and Receive FIFO Not Empty, SSP_SSSR[RNE] fields in the SSP Status Register. To remove trailing bytes using PIO, enable the timeout interrupt by setting the SSP_SSCR1[TINTE] field.

ASR CONFIDENTIAL

Note

If FIFO Packed mode is enabled (SSP_SSRC0[FPCKE]=1), trailing bytes must be removed using programmed I/O. If the SSP_SSSR[OSS] field in SSP Status Register is set, then the last FIFO line only contains one sample.

When the Trailing Byte SSP_SSCR1[TRAIL] bit is set, trailing bytes left in the RXFIFO are handled by the DMA controller.

A DMA service request is issued automatically after the SSP_SSCR1[TRAIL] field is set and a timeout occurs, SSP_SSSR[TINT]=1. The DMA empties the RXFIFO unless the DMA reaches the end of its Descriptor chain (see Peripheral Trailing Byte Interrupt). When handling trailing bytes using the DMA, if a timeout occurs and the RXFIFO is empty (SSP_SSSR[RNE]=0), an end-of-receive (EOR) is sent to the DMA. Refer to Chapter Section 13, DMA Controller for details regarding EOR.

If a DMA EOC occurs (SSP_SSSR[EOC]=1) at the time that the last sample is read out of the RXFIFO (the DMA Descriptor chain was just exactly long enough) and the timeout counter is still running (that is, a timeout has not occurred and SSP_SSTO[TIMEOUT] is non-zero), then, when the timeout does occur, the SSPx port generates a DMA request. When this occurs, re-initialize the DMA registers and re-enable the channel for the SSPx port to send its EOR to the DMA controller. For more information on the SSP Status Register, see the *ASR1606 Registers*.

ASR CONFIDENTIAL

Note

When the SSPx port is running in Network mode and the FIFO Packing Enable SSP_ SSPSCR0[FPCKE] bit is set, use the ASR1606 to handle trailing bytes in the RXFIFO with the SSP_SSCR1[TRAIL] bit cleared and the SSP_SSCR1[PINTE] bit set. After the Peripheral Trailing Byte Interrupt SSP_SSSR[PINT]=1 occurs, the interrupt service routine must clear the SSP_SSCR0[MOD] bit and wait for SSP_SSTSS[NMBSY] bit to go low before removing any extra or trailing samples from the RXFIFO, which can be discarded.

When the SSP_SSCR0[MOD] bit is set, the SSPx port continues transceiving data even after the TXFIFO is empty and until the SSP_SSCR0[MOD] bit is cleared. Since the DMA does not have a way to clear the SSP_SSCR0[MOD] bit, it is possible that extra samples are received in the RXFIFO. Since software must clear the SSP_ SSCR0[MOD] bit, the ASR1606 must also handle the trailing bytes.

16.4.3 Data Formats

This section describes the types of formats used to transfer serial data between the ASR1606 and external peripherals.

16.4.3.1 Serial Data Formats for Transfer to/from Peripherals

Four interface signals for each SSPx port transfer data between the ASR1606 and external peripherals. Although serial-data formats exist, each has the same basic structure, and in all cases, the interface signals used are:

- SSPx_SCLK↓Defines the bit rate at which serial data is driven onto and sampled from the port
- SSPx_FRM↓Defines the boundaries of a basic data "unit" which is comprised of multiple serial bits
- SSPTXDx↓The serial datapath for transmitted data from the SSPx port to the peripheral
- SSPRXDx↓The serial datapath for received data from peripheral to the SSPx port

A data frame can contain 8, 16, 18, or 32 bits (see SSP_SSCR0[EDSS] and SSP_SSCR0[DSS] fields in the *ASR1606 Registers*. Serial data is transmitted with the MSb first. The formats directly supported are the Motorola* SPI, Texas Instruments* SSP, and the National Semiconductor* Microwire. The I²S protocol is supported by programming the PSP format.

The SSPx_FRM function and use varies between each format. SSPx_FRM is programmable in direction, delay, polarity, and width. Both Master and Slave modes are supported.

ASR CONFIDENTIAL

- SPI format: SSPx_FRM functions as a chip select to enable the external device (target of the transfer) and is held active-low during the data transfer. During continuous transfers, the SSPx_FRM signal can be either held low or pulsed depending upon the value of the Motorola* SPI SSPx_SCLK phase setting, SSP_SSCR1[SPH], field in the SSP Control Register 1. Master and Slave modes are supported. SPI is a full-duplex format.
- SSP format: SSPx_FRM is pulsed high for one (serial) data period at the start of each frame.
  Master and Slave modes are supported. SSP is a full-duplex format.
- Microwire format: SSPx_FRM functions as a chip select to enable the external device (target of the transfer) and is held active-low during the data transfer. Slave mode is not supported.
  SSPx_FRM is also held low during continuous transfers. This is a half-duplex protocol.
- PSP format (I^2S): SSPx_FRM is programmable in direction, delay, polarity, and width. Master and Slave modes are supported. PSP can be programmed to be either full- or half-duplex format.

The SSPx_SCLK function and use varies between each format:

ASR CONFIDENTIAL

- SPI format: Programmers choose which edge of SSPx_SCLK to use for switching Transmit data and for sampling Receive data. In addition, moving the phase of SSPx_SCLK can be user-initiated, shifting its active state one-half cycle earlier or later at the start and end of a frame. Master and Slave modes are supported, and in both, the SSPx_SCLK only toggles during active transfers (does not run continuously).
- SSP format: Data sources switch Transmit data on the rising edge of SSPx_SCLK and sample Receive data on the falling edge. Master and Slave modes are supported. When driven by the SSPx port, the SSPx_SCLK only toggles during active transfers (not continuously) unless the SSP_SSCR1[SCFR], SSP_SSCR1[ECRA], or SSP_SSCR1[ECRB] functions are used. (See Section 17.3, Signal Descriptions for a description of each of the SSP signals.) When the SSPx_SCLK is driven by another device, it is allowed to be either continuous or only driven during transfers.
- Microwire format: Both data sources switch Transmit data (change to the next bit) on the falling edge of SSPx_SCLK and sample Receive data on the rising edge. Slave mode is not supported.
- PSP format (I^2S): Programmers choose which edge of SSPx_SCLK to use for switching Transmit data and for sampling Receive data. In addition, programmers can control the Idle state for SSPx_SCLK and the number of active clocks that precede and follow the data transmission. Master and Slave modes are supported. When driven by the SSPx port, the SSPx_SCLK toggles only during active transfers, not continuously, unless the SSP_ SSCR1[SCFR], SSP_SSCR1[ECRA], or SSP_SSCR1[ECRB] functions are used. When the SSPx_SCLK is driven by another device, it is allowed to be either continuous or driven only during transfers, but certain restrictions on PSP parameters apply (see Programmable Serial Protocol (PSP) Format).

Microwire uses a half-duplex, master-slave messaging protocol. At the start of a frame, the controller transmits a one-or two-byte control message to the peripheral; no data is sent by the peripheral. The peripheral interprets the message and if the message is a read request, the peripheral, responds with the requested data, one clock after the last bit of the request message. Return data—part of the same frame—can be from eight to 16-bits in length. The total frame length is 13 to 33 bits. SSPx_ SCLK is active during the entire frame.

Normally, if the serial clock (SSPx_SCLK) is driven by the SSPx port, it toggles only while an active data transfer is underway. However, there are several conditions that may cause the clock to run continuously. If the Receive-without-Transmit mode is enabled by setting the Receive Without Transmit, SSP_SSCR1[RWOT], field *and* the frame format is not Microwire then the SSPx_SCLK toggles regardless of whether Transmit data exists within the Transmit FIFO. The SSPx_SCLK also toggles continuously if the SSPx port is in Network mode, or if the SSP_SSCR1[ECRA] or SSP_ SSCR1[ECRB] bits are

ASR CONFIDENTIAL

enabled. At other times, SSPx_SCLK is held in an inactive or idle state, as defined by the specified protocol under which it operates.

16.4.3.2 TI-SSP Format Details

When outgoing data in the SSP Controller is ready to transmit, SSPx_FRM asserts for one clock period. On the following clock, data to be transmitted is driven on SSPTXDx one bit at a time, with the MSb first. For Receive data, the peripheral similarly drives data on the SSPRXDx pin. Word length can be 8, 16, 18, or 32 bits. All output transitions occur on the rising edge of SSPx_SCLK while data sampling occurs on the falling edge. The SSPTXDx signal either retains the value of the last bit sent (bit 0) (see Figure 72 and Figure 73) or goes to a high impedance (see Figure 82 and Figure 83) state at the end of the transfer. If the SSPx port is disabled or reset, SSPTXDx is forced to zero (unless the TXD Three-State Enable, SSP_SSCR1[TTE], bit is set, in which case it goes into a high impedance state).

ASR CONFIDENTIAL

Figure 72 shows the TI Synchronous Serial Protocol for a single transmitted frame. Figure 73 shows the TI Synchronous Serial Protocol for when back-to-back frames are transmitted. Once the Transmit FIFO contains data, SSPx_FRM is pulsed high for one SSPx_SCLK period and the value to be transmitted is transferred from the Transmit FIFO to the Transmit Logic Serial Shift register. On the next rising edge of SSPx_SCLK, the most significant bit of the eight to 32-bit data frame is shifted to the SSPTXDx pin. Likewise, the MSB of the received data is shifted onto the SSPRXDx pin by the off-chip serial slave device. Both the SSP port and the off-chip serial slave device then latch each data bit into the serial shifter on the falling edge of each SSPx_SCLK. The received data is transferred from the serial shifter to the Receive FIFO on the first rising edge of SSPx_SCLK after the last bit has been latched.

For back-to-back transfers, the start of one frame immediately follows the completion of the previous. The MSb of one transfer immediately follows the LSb of the preceding with no “dead” time between them.

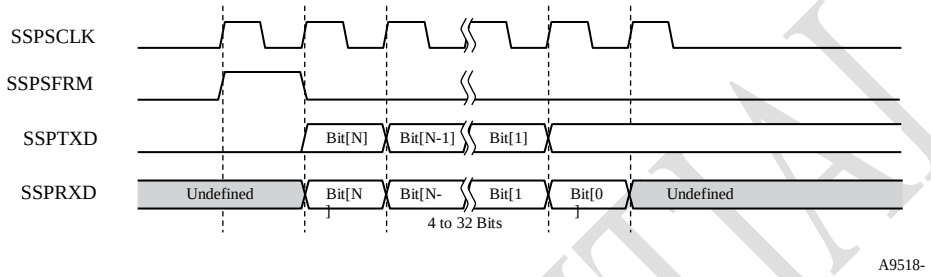
When the enhanced SSPx port is a master to the frame synch (SSPx_FRM) and a slave to the clock (SSPx_SCLK), then at least three extra clocks (SSPx_SCLK) are needed at the beginning and end of each block of transfers to synchronize control signals from the ARM* peripheral bus (APB) clock domain into the SSP clock domain (a block of transfers is a group of back-to-back continuous transfers).

Note

When configured as either master or slave to SSPx_SCLK or SSPx_FRM, the SSP port continues to drive SSPTXDx until the last bit of data is sent (the LSB) or the SSPTXDx line becomes high impedance (see Figure 81 and Figure 82). If SSP_SSCR0[SSE] is cleared, the SSPTXDx line goes low. SSP_SSPSP[EDTS] has no effect when in SSP mode. SSPRXDx is undefined before the MSB is sent and after the LSB is sent.
SSPRXDx must not float.

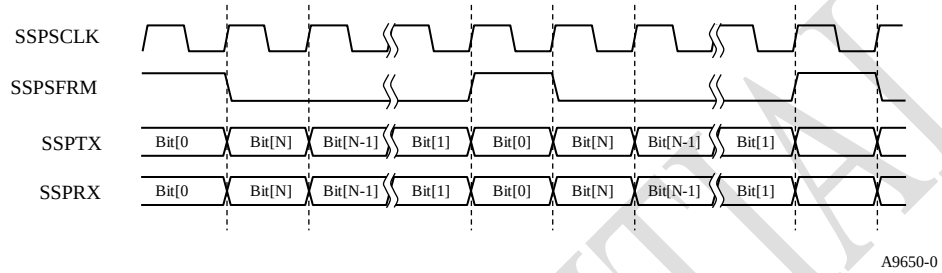
ASR CONFIDENTIAL

Figure 72:Texas Instruments* Synchronous Serial Frame Protocol (Single Transfers)



ASR CONFIDENTIAL

Figure 73:Texas Instruments* Synchronous Serial Frame Protocol (Multiple Transfers)



16.4.3.3 Motorola SPI Format Details*

The SPI format has four possible sub-modes depending on the SSPx_SCLK edges selected for driving data and sampling received data and on the selection of the phase mode of SSPx_SCLK (see Serial Clock Phase (SPH) for a complete description of each sub-mode).

Note

The following description applies only when SPH = 0 and SPO = 0. Other combinations of SPH and SPO result in different polarities and timings (see Figure 75 and Figure 76).

When the SSP port is disabled or in idle mode, SSPx_SCLK and SSPTXDx are low and SSPx_FRM is high. When Transmit data is ready to be sent, SSPx_FRM goes low (one clock period before the first rising edge of SSPx_SCLK) and stays low for the remainder of the frame. The most significant bit of the serial data is driven onto SSPTXDx one half-cycle later. Halfway into the first bit period, SSPx_SCLK asserts high and continues toggling for the remaining data bits. Data transitions on the falling edge of SSPx_SCLK and is sampled on the rising edge of SSPx_SCLK. 8, 16, 18, or 32 bits can be transferred per frame.

With the assertion of SSPx_FRM, Receive data is driven simultaneously from the peripheral on SSPRXDx, MSb first. Data transitions on SSPx_SCLK falling edges and is sampled by the controller on SSPx_SCLK rising edges. At the end of the frame, SSPx_FRM is de-asserted high one clock period (one half clock cycle after the last falling edge of SSPx_SCLK) after the last bit has been latched at its destination and the completed incoming word is shifted into the “incoming” FIFO. The peripheral can drive SSPRXDx to a high-impedance state after sending the last bit of the frame.

SSPTXDx retains the last value transmitted when the controller goes into Idle mode, unless the enhanced SSPx port is disabled or reset (which forces

ASR CONFIDENTIAL

SSPTXDx to zero).

For back-to-back transfers, start and completion are like those of a single transfer, but SSPx_FRM does not de-assert between words. Both transmitter and receiver are configured for the word length and internally track the start and end of frames. There are no “dead” bits; the LSb of one frame is followed immediately by the MSb of the next.

When in Motorola* SPI format, the enhanced SSPx port can be either a master or a slave device, but the clock and frame direction must be the same. For example, the SSP Serial Bit Rate Clock Direction, SSP_SSCR1[SCLKDIR], and the SSP Frame Direction, SSP_SSCR1[SFRMDIR], fields must either both be set or cleared.

ASR CONFIDENTIAL

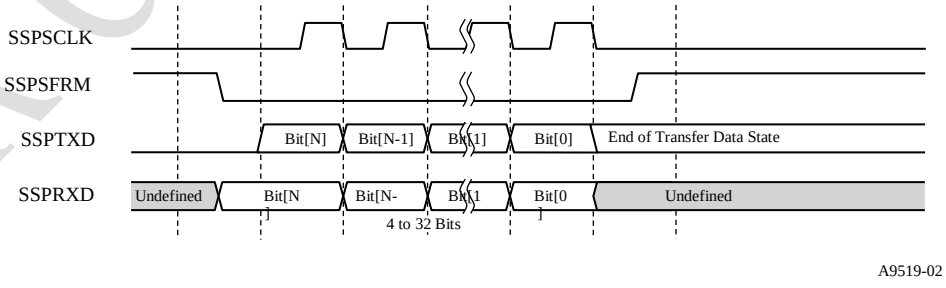
When in Motorola* SPI format, if the SSP port is the master and SSP_SSPSP[ETDS] is cleared, the end-of-transfer data state for SSPTXDx is low. If the SSP port is the master and SSP_ SSPSP[ETDS] is set, the end-of-transfer data state for SSPTXDx remains at the last bit transmitted (LSB). If the SSP port is the slave, then the SSP_SSPSP[ETDS] is undefined. SSPRXDx is undefined before the frame is active and after the LSB is received. SSPRXD must not float. When the SSP port is configured as a master and SSP_SSCR1[TTE] is set, SSP_SSPSP[ETDS] is ignored and SSPTXDx becomes high impedance between active transfers (see Figure 84).

Note The input clock to the SSPx port must not be active when SSPx_FRM is de-asserted.

Note When the SSP port is slave to clock and frame, SSP_SSCR1[SCFR] must be set.

Figure 74 shows one of the four possible configurations for the Motorola* SPI frame protocol for a single transmitted frame. Figure 75 shows when back-to-back frames are transmitted for the Motorola* SPI frame protocol.

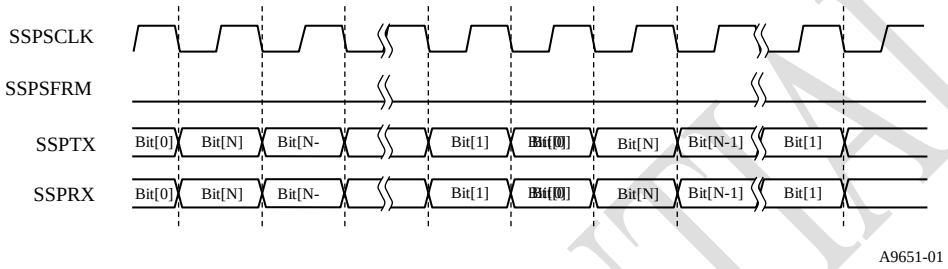
Figure 74:Motorola* SPI Frame Protocol (Single Transfers)



Note The phase and polarity of SSPx_SCLK can be configured for four different modes. This example shows just one of those modes (SSP_SSCR1[SPO] and SSP_SSCR1[SPH] cleared). Other settings for SPO and SPH result in different polarities and timing.

ASR CONFIDENTIAL

Figure 75:Motorola* SPI Frame Protocol (Multiple Transfers)



Serial Clock Phase (SPH)

The phase relationship between SSPx_SCLK and SSPx_FRM when the Motorola* SPI protocol is selected is controlled by SSP_SSCR1[SPH].

The combination of the SSP_SSCR1[SPO] and SSP_SSCR1[SPH] settings determine when SSPx_ SCLK is active during the assertion of SSPx_FRM and which SSPx_SCLK edge transmits and receives data on SSPTXDx and SSPRXDx.

When SPH is cleared, SSPx_SCLK remains in its inactive (idle) state (as determined by SSP_ SSCR1[SPO]) for one full cycle after SSPx_FRM is asserted low at the beginning of a frame. SSPx_ SCLK continues to toggle for the rest of the frame. It is then held in its inactive state for one-half of an SSPx_SCLK period before SSPx_FRM is de-asserted high at the end of the frame.

When SPH is set, SSPx_SCLK remains in its inactive or idle state (as determined by SSP_ SSCR1[SPO]) for one-half cycle after SSPx_FRM is asserted low at the beginning of a frame. SSPx_SCLK continues to toggle for the remainder of the frame and is then held in its inactive state for one full SSPx_SCLK period before SSPx_FRM is de-asserted high at the end of the frame.

When programming SSP_SSCR1[SPO] and SSCR1[SPH] to the same value (both set or both cleared), Transmit data is driven on the falling edge of SSPx_SCLK and Receive data is latched on the rising edge of SSPx_SCLK. When programming SSP_SSCR1[SPO] and SSP_SSCR1[SPH] to opposite values (one set and the other cleared), Transmit data is driven on the rising edge of SSPx_ SCLK and Receive data is latched on the falling edge of SSPx_SCLK.

ASR CONFIDENTIAL

Note SSP_SSCR1[SPH] is ignored for all data frame formats except for the Motorola* SPI protocol.

Figure 76 and Figure 77 show the timing for all four programming combinations of SSP_ SSCR1[SPO] and SSP_SSCR1[SPH]. SSP_SSCR1[SPO] inverts the polarity of SSPx_SCLK and SSP_SSCR1[SPH] determines the phase relationship between SSPx_SCLK and SSPx_FRM, shifting SSPx_SCLK one-half phase to the left or right during the assertion of SSPx_FRM.

ASR CONFIDENTIAL

Figure 76:Motorola* SPI Frame Protocols for SPO and SPH Programming (SPH Set)

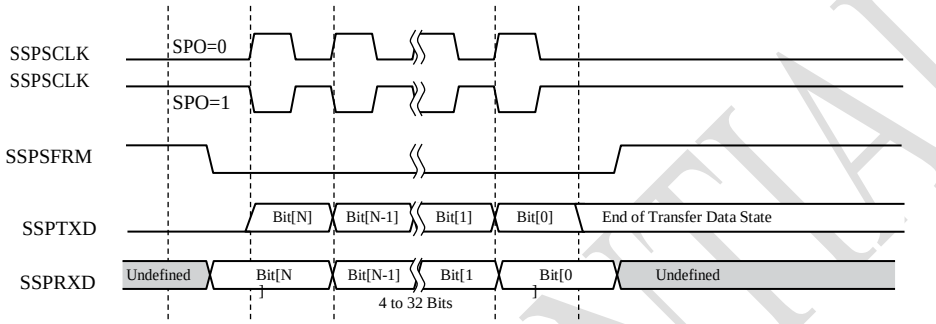
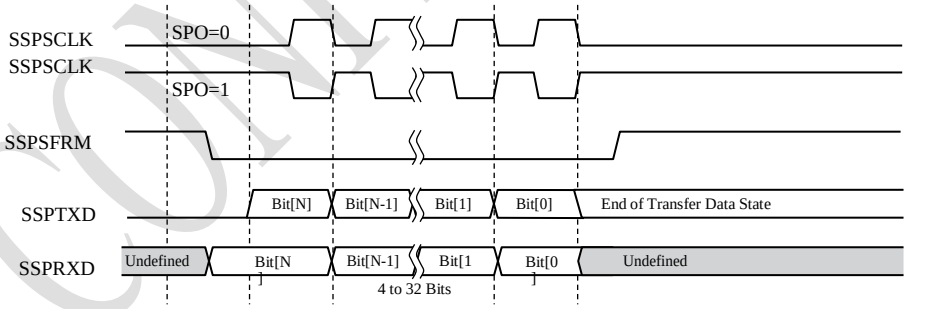


Figure 77:Motorola* SPI Frame Protocols for SPO and SPH Programming (SPH Cleared)



ASR CONFIDENTIAL

16.4.3.4 Microwire Protocol

The Microwire protocol is similar to SPI, except transmission is half-duplex instead of full-duplex and it uses master-slave message passing. While in the Idle state or when the SSP port is disabled, SSPx_SCLK and SSPTXDx are low and SSPx_FRM is high.

Each serial transmission begins with SSPx_FRM asserting low, followed by an eight or 16-bit command word sent from the controller to the peripheral on SSPTXDx. The command word data size is selected by the Microwire Transmit data size, SSP_SSCR1[MWDS], bit. SSPRXDx is controlled by the peripheral and remains in a high-impedance state. SSPx_SCLK asserts high (rising edge) midway into the command's most significant bit and continues toggling at the configured SSPx_SCLK bit rate.

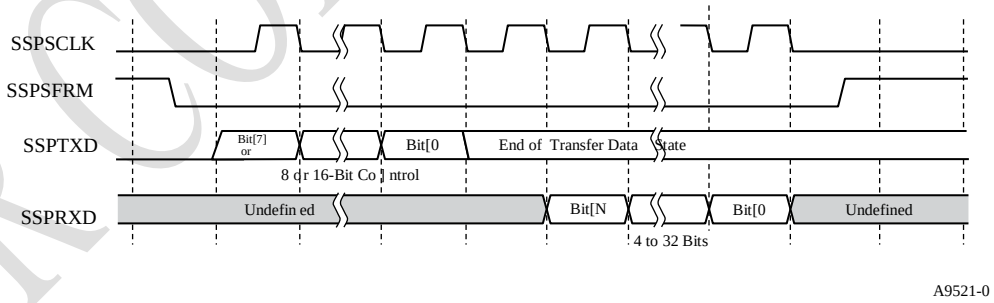
ASR CONFIDENTIAL

One bit period after the last command bit, the peripheral returns the serial-data-requested most significant bit first on SSPRXDx. Data transitions on the falling edge of SSPx_SCLK and is sampled on the rising edge. The last falling edge of SSPx_SCLK coincides with the end of the last data bit on SSPRXDx and SSPx_SCLK remains low after that. If it is the only word or the last word of the transfer. SSPx_FRM de-asserts high one-half clock period later.

The start and end of a series of back-to-back transfers are like those of a single transfer; however, SSPx_FRM remains asserted (low) throughout the transfer. The end of a data word on SSPRXDx is followed immediately by the start of the next command byte on SSPTXDx with no dead time. If SSP_ SSCR1[TTE] is set (selecting SSPTXDx to be high impedance between active transfers), then a three-wire Microwire mode can be supported where SSPTXDx and SSPRXDx are externally tied together (shorted).

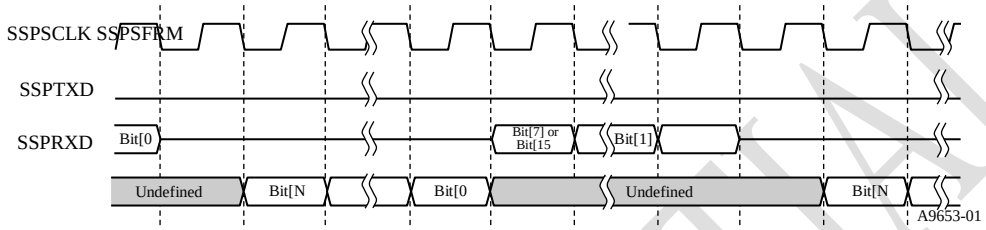
When using the Microwire protocol, the SSP port can function only as a master (frame and clock are outputs). Therefore, both SSP_SSCR1[SCLKDIR] and SSP_SSCR0[SFRMDIR] must both be cleared. Figure 78 shows the National Semiconductor* Microwire frame protocol with 8- or 16-bit command words for a single transmitted frame. Figure 79 shows the National Semiconductor* Microwire frame protocol with eight-bit command words when back-to-back frames are transmitted.

Figure 78:National Semiconductor* Microwire Frame Protocol (Single Transfer)



ASR CONFIDENTIAL

Figure 79:National Semiconductor* Microwire Frame Protocol (Multiple Transfers)



ASR CONFIDENTIAL

Note

(1) Microwire format is normally used with SSP_SSPSP[ETDS] cleared so that the end-of-transfer data state for SSPTXDx is low. If SSP_SSPSP[ETDS] is set then SSPTXDx remains at the level of the last transmitted bit (LSB). When SSP_ SSCR1[TTE] is set, SSPSPx[ETDS] is ignored and SSPTXDx becomes high impedance between active transfers (see Figure 85).

(2) The state of SSPRXDx is undefined before the MSB and after the LSB is transmitted. This pin must not float.

ASR CONFIDENTIAL

16.4.3.5 *Programmable Serial Protocol (PSP) Format*

The PSP format defines programmable parameters that determine the transfer timings between data samples.

Four serial clock modes are defined in the Serial Bit-rate Clock Mode, SSP_SSPSP[SCMODE], field in the SSP Programmable Serial Protocol Register. These modes select the SSPx_SCLK rising and falling edges for driving data, sampling received data, and the SSPx_SCLK idle state.

As shown in Table 44, the Idle and Disable modes of the SSPTXDx, SSPx_SCLK, and SSPx_FRM interface signals are programmable using the following fields in the SSP Programmable Serial Protocol Register: End Of Transfer Data State (SSP_SSPSP[ETDS]), Serial Frame Polarity (SSP_ SSPSP[SFRMP]), and Serial Bit-rate Clock Mode (SSP_SSPSP[SCMODE]). When Transmit data is ready, SSPx_SCLK remains in its Idle state for the number of serial clock (SSPx_SCLK) periods programmed into the Start Delay (SSP_SSPSP[STRTDLY]) field in the SSP Programmable Serial Protocol Register.

SSPx_SCLK then starts toggling. SSPTXDx remains in the idle state for the number of serial clock periods programmed into the Dummy Start (SSP_SSPSP[DMYSTRT]) field in the SSP Programmable Serial Protocol Register. SSPx_FRM is asserted after the number of half serial clock periods programmed into the Serial Frame Delay (SSP_SSPSP[SFRMDLY]) field. SSPx_FRM remains asserted for the number of serial clock periods programmed into the Serial Frame Width (SSP_SSPSP[SFRMWDTH]) field in the SSP Programmable Serial Protocol Register, then SSPDFRMx de-asserts.

Serial data of 8, 16, 18, or 32 bits can be transferred per frame by setting the SSP_SSCR0[EDSS] and SSP_SSCR0[DSS] fields to the preferred data size select. Once the last bit (LSb) is transferred, SSPx_SCLK continues toggling for the number of serial clock periods programmed into the Dummy Stop (SSP_SSPSP[DMYSTOP]) field. Depending on the value programmed into the End Of Transfer Data State (SSP_SSPSP[EDTS]) field when the SSPx port goes into Idle mode, SSPTXDx either retains the last bit-value transmitted or is forced to 0 unless the SSPx port is disabled or reset, which forces SSPTXDx to 0.

With the assertion of SSPx_FRM, Receive data is driven simultaneously from the peripheral onto SSPRXDx, MSb first. Data transitions on the SSPx_SCLK edge based on the serial-clock mode that is selected (SSP_SSPSP[SCMODE]) and is sampled by the SSPx port on the opposite clock edge. When the SSPx port is a master to SSPx_FRM and a slave to SSPx_SCLK, at least three extra SSPx_SCLKs are needed at the beginning and end of each block of transfers to synchronize control signals from the APB clock domain into the SSP clock domain (a block of transfers is a group of back-to-back continuous transfers).

ASR CONFIDENTIAL

In general, because of the programmable nature of the PSP protocol, this protocol can be used to achieve a variety of serial protocols. For example: some DigRF protocol timing can be achieved by programming these values: Start Delay (SPP_SSPSP[STRTDLY]) = 0, Dummy Start (SPP_ SSPSP[DMYSTRT]) = 0, Dummy Stop (SPP_SSPSP[DMYSTOP]) = 0, and Serial Frame Delay (SPP_SSPSP[SFRMDLY]) = 0. The SSPx port should be configured as a clock slave (SSP Serial Bit Rate Clock Direction (SSP_SSCR1[SCLKDIR]) = 1) and frame master (SSP Frame Direction (SSP_ SSCR1[SFRMDIR]) = 0). Also, the Frame Sync Relative Timing Bit (SSP_SSPSP[FSRT]) field in the SSP Programmable Serial Protocol Register must be set for continuous transfers, and the Serial Frame Width (SSP_SSPSP[SFRMWDTH]) field should be equal to the data sample size.

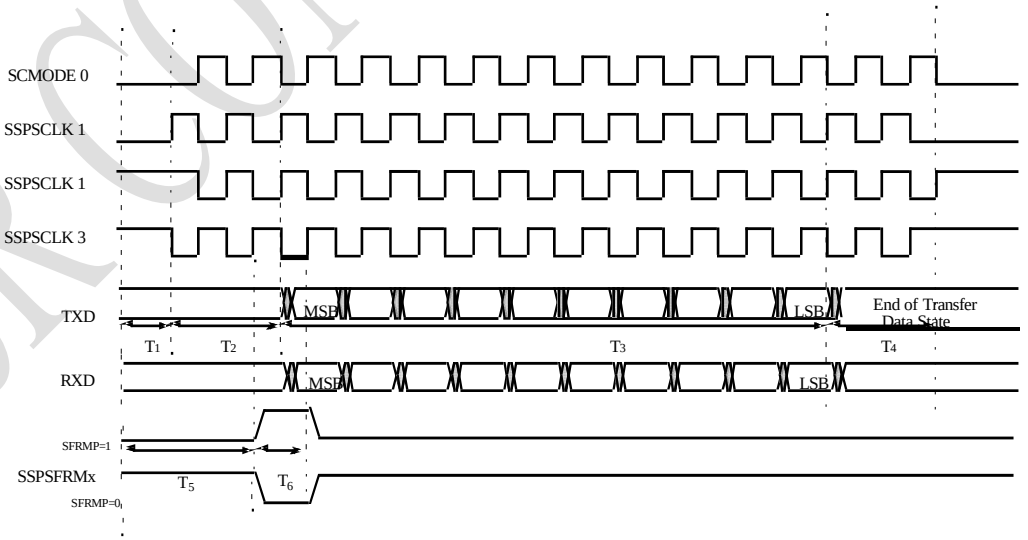
Note

The PSP mode should not be configured as:

sfrmdly/2 + sfrmwidth = dmystrt + dmystop + frame_size & dmystop = 0

This is an obscure case and the PSP mode gives an error if it is configured in this way.

Figure 80:Programmable Serial Protocol Format



Note: If SSPx port is the master of SSPSCLKx (output) and SSPSP_x[ETDS=0],the end-of-transfer-data state for the SSPTXD line is 0. If the SSP is the master of the clock, and the SSPSP[ETDS] bit is set, then the SSPTXD line remains at the last bit transmitted (LSB). If the SSPx port is a slave to SSPSCLK (input), and modes 1 or 3 are used, then the End of Transfer data state (ETDS) can only change from the LSB if more SSPSCLKs are sent to the SSPx port.

ASR CONFIDENTIAL

Figure 81:Programmable Protocol Format (Consecutive Transfers)

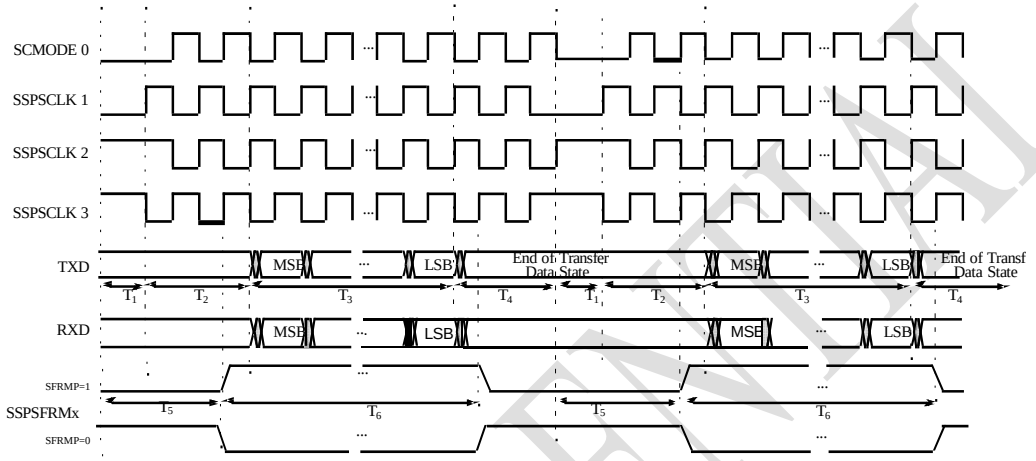


Table 44:Programmable Protocol Parameters

Symbol	Definition	Range	Units
-	Serial clock mode (SSP_SSPSP[SCMODE])	(Drive, Sample, SSPx_ SCLK Idle) 0 = Fall, rise, low 1 = Rise, fall, low 2 = Rise, fall, high 3 = Fall, rise, high	-
-	Serial frame polarity (SSP_SSPSP[SFRMP])	High or low	-
T_1	Start delay (SSP_SSPSP[STRTDLY])	0 to 7	Clock period
T_2	Dummy start (SSP_SSPSP[EDMYSTRT] + SSP_ SSPSP[DMYSTRT])	0 to 15	Clock period
T_3	Data size (SSP_SSCR0[EDSS] and SSP_ SSCR0[DSS])	4 to 32	Clock period
T_4	Dummy stop (SSP_SSPSP[EDMYSTOP] + SSP_ SSPSP[DMYSTOP])	0 to 31	Clock period
T_5	SSPSFRM delay (SSP_ SSPSP[SFRMDLY])	0 to 127	Half-clock period
T_6	SSPSFRM width (SSP_ SSPSP[SFRMWDTH])	1 to 63	Clock period
-	End of transfer data state (SSP_ SSPSP[ETDS])	Low or bit 0	-

ASR CONFIDENTIAL

The SSPx_FRM delay (T_5) must not extend beyond the end of T_4. The SSPx_FRM width (T_6) must be asserted for at least one SSPx_SCLK period and should be de-asserted before the end of T_4 (for example, in terms of time, not bit values

($T_5 + T_6$) <= ($T_1 + T_2 + T_3 + T_4$), 1<= T_6 < ($T_2 + T_3 + T_4$), and ($T_5 + T_6$) >= (T_1 + 1) to ensure that SSPx_FRM is asserted for at least two edges of SSPx_SCLK). Program T_1 to 0b0 when SSPx_SCLK is enabled by any of the SSP_SSCR1[SCFR], SSP_SSCR1[ECRA], or SSP_ SSCR1[ECRB] fields in the SSP Control Register 1. While the PSP can be programmed to generate the assertion of SSPx_FRM during the middle of the data transfer (for example, after the MSB has been sent), the SSPx port is unable to Receive data in frame-Slave mode (SSP_SSPSP[SFRMDIR] is set, if the assertion of the frame is not before the MSB is sent (for example, T_5 <= T_2 if the SSP_ SSCR1[SFRMDIR] bit is set). Transmit data transitions from the end-of-transfer-data state (SSP_ SSPSP[ETDS]) to the next MSB data value upon assertion of the internal version of SSPx_FRM. Program the SSP_SSPSP[STRTDLY] field to 0x00 whenever SSPx_SCLK or SSPx_FRM is configured as an input (for example, SSP_SSCR1[SCLKDIR] and SSP_SSCR1[SFRMDIR] are cleared.

Note

When the SSPx port is slave to the frame, the sum of T_1+T_2+T_3+T_4 can be less than the actual time from the beginning of the current frame to the beginning of the next frame. For example, when the rate of SSPx_SCLK is 13 MHz and the data sample size is 16-bits, the beginning of the frame can occur at a rate of 8 kHz.

16.4.4 High Impedance on SSPTXDx

The SSP supports placing the SSPTXDx into high impedance during idle time instead of driving SSPTXDx as controlled by the TXD Three-State Enable (SSP_SSCR1[TTE]) and TXD Three-state Enable On Last Phase (SSP_SSCR1[TTELP]) fields in the SSP Control Register 1. The SSP_ SSCR1[TTE] enables a high-impedance state on SSPTXDx. The SSP_SSCR1[TTELP] determines on which SSPx_SCLK phase SSPTXDx becomes high impedance.

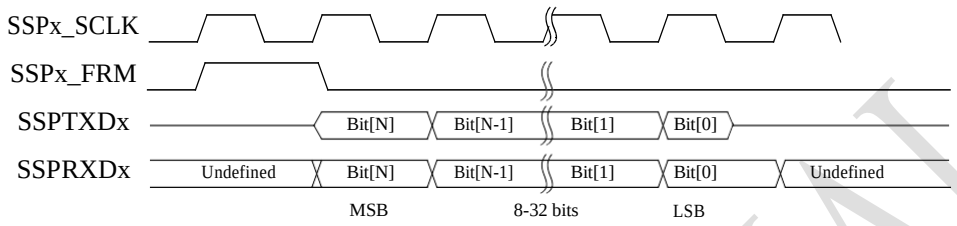
16.4.4.1 TI SSP Format

When SSP_SSCR1[TTE] =0, the enhanced SSPx port behaves as described in PSP Format,

When SSP_SSCR1[TTE] is set, SSPTXDx behaves as shown in Figure 82 or Figure 83, depending on the value in the SSP_SSCR1[TTELP] bit.

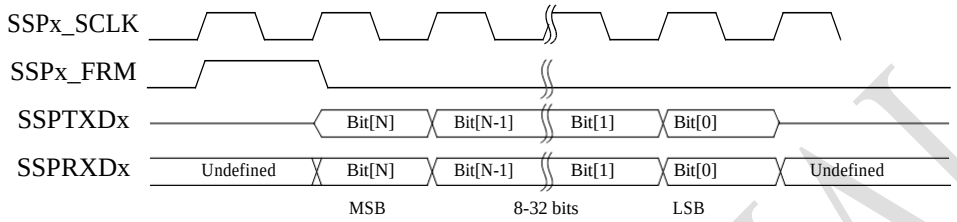
ASR CONFIDENTIAL

Figure 82:TI SSP with SSP_SSCR1[TTE] = 1 and SSP_SSCR1[TTELP]] = 0



ASR CONFIDENTIAL

Figure 83:TI SSP with SSP_SSCR1[TTE] = 1 and SSP_SSCR1[TTELP] = 1

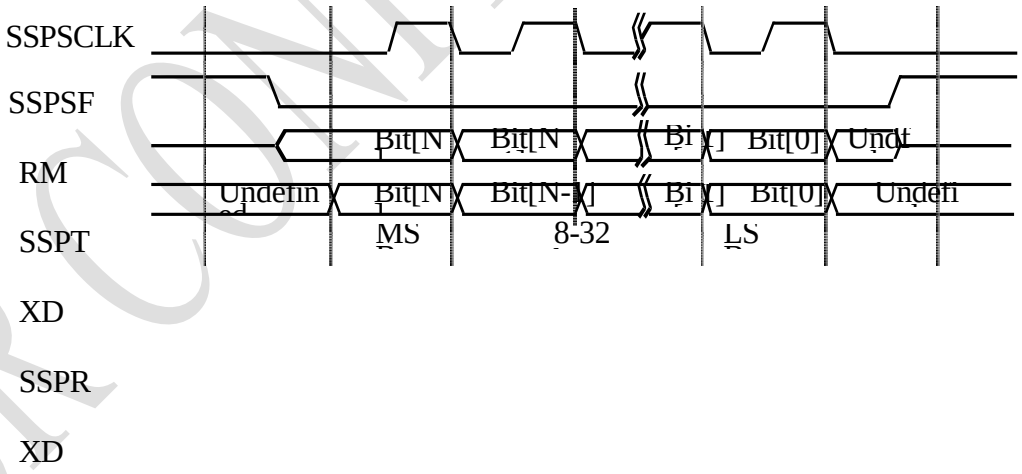


16.4.4.2 Motorola SPI Format*

When SSP_SSCR1[TTE] is cleared, refer to Motorola* SPI Format Details.

For Motorola* SPI format, SSP_SSCR1[TTELP] must be cleared. If SSP_SSCR1[TTE] = 1, see Figure 84 for SSPTXDx functionality. For Motorola* SPI format, SSPTXDx goes to a high-impedance state whenever SSPx_FRM is not asserted.

Figure 84:Motorola* SPI with <TXD Three-State Enable> = 1 and <TXD Three-state Enable On Last Phase> = 0



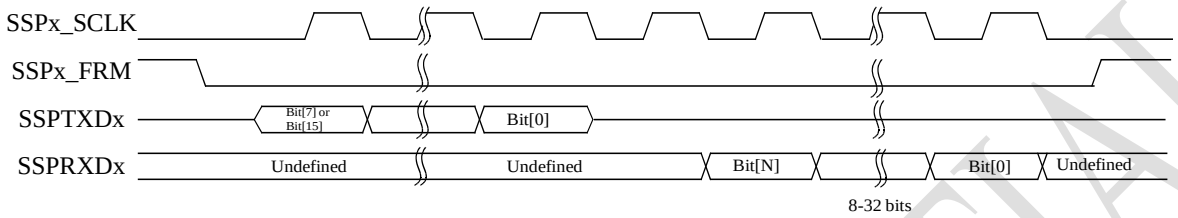
16.4.4.3 National Microwire Format

When SSP_SSCR1[TTE] is cleared, the SSP port behaves as described in Microwire Protocol.

When SSP_SSCR1[TTE] is set, SSPTXDx behaves as in PSP Format. SSP_SSCR1[TTELP] must be cleared for National Microwire format, and SSPTXDx becomes high impedance whenever SSPx_FRM is inactive (and after the LSB is sent on SSPTXDx).

ASR CONFIDENTIAL

Figure 85:National Microwire with SSP_SSCR1[TTE] = 1 and SSP_SSCR1[TTELP] = 0

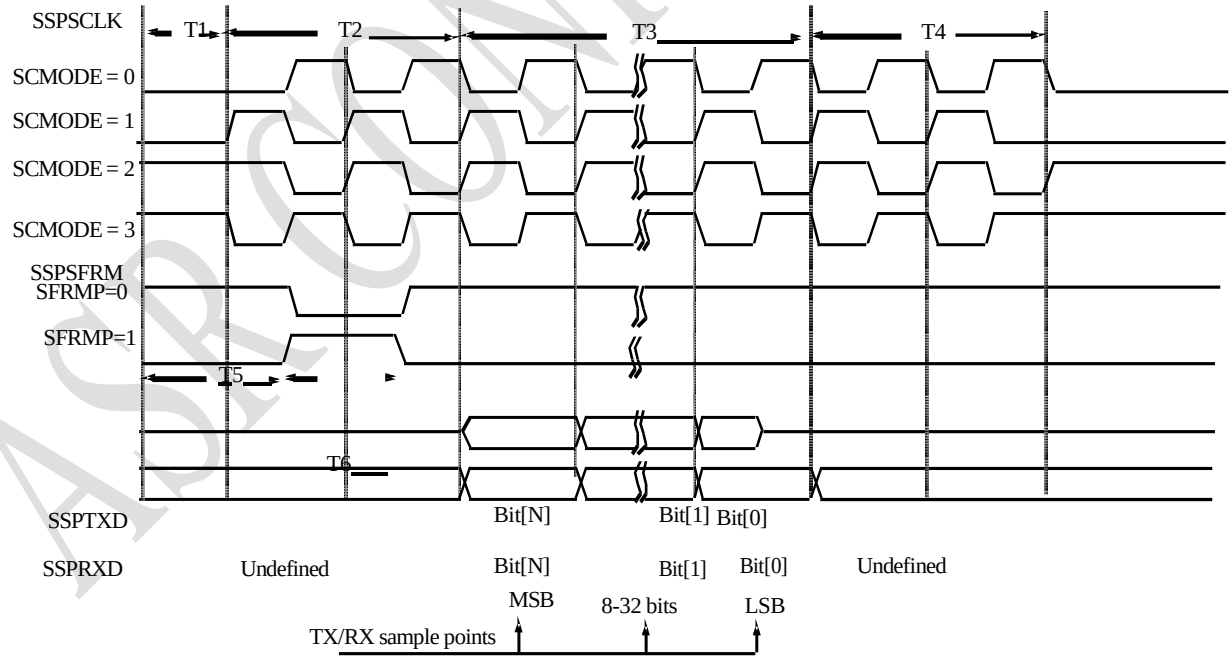


16.4.4.4 PSP Format

For PSP functionality, clear SSP_SSCR1[TTE] (refer to Programmable Serial Protocol (PSP) Format).

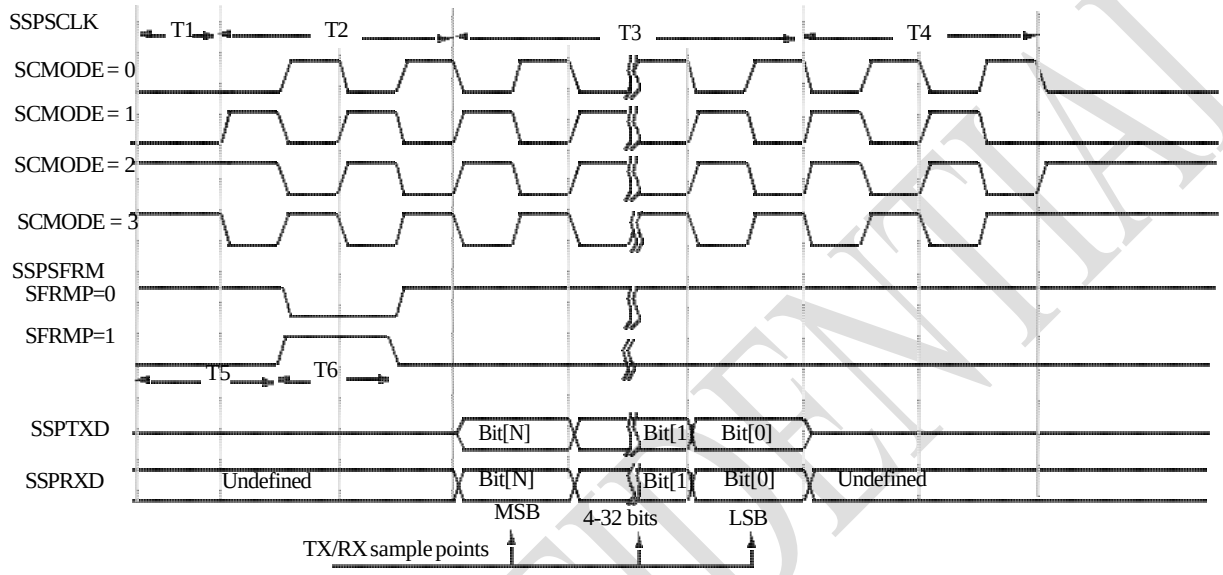
For SSPTXDx functionality when SSP_SSCR1[TTE] = 1, see Figure 86 and Figure 87.

Figure 86:PSP Format with SSP_SSCR1[TTE] = 1, SSP_SSCR1[TTELP] = 0, and SSP_ SSCR1[SFRMDIR] = 1



ASR CONFIDENTIAL

Figure 87:PSP Format with SSP_SSCR1[TTE] = 1, and either SSP_SSCR1[TTELP] = 1, or SSP_ SSCR1[SFRMDIR] = 0



16.4.5 Network Mode Operation

The SSP_SSCR0[MOD] bit selects between Normal and Network modes. Normal mode (MOD 0x0) is used when using the Texas Instruments* Synchronous Serial Protocol (SSP) and the Motorola* Serial Peripheral Interface (SPI). Network mode (MOD = 0x1) is used for emulating the I²S protocol.
Software should set MOD only when using the PSP format. If the SSPx port is a master of the clock and SSP_SSCR1[SCLKDIR] is cleared, then setting MOD causes the SSPx_SCLK to run continuously.

When in Network mode, only one SSPx_FRM is sent (Master mode) or received (Slave mode) for the number of time slots programmed into the SSP_SSCR0[FRDC] field. When beginning in Network mode, while the SSPx port is a master to the SSPx_FRM interface signal, the first SSPx_ FRM signal does not occur until after data is in the TXFIFO. After assertion of the first SSPx_FRM signal, if the SSP is a master to SSPx_FRM, subsequent SSPx_FRM signals continue to assert regardless of whether data resides in the TXFIFO. Therefore, the transmit underrun bit, SSP_ SSSR[TUR], is set to 0b1 if there is no data in the TXFIFO and the SSPx port is programmed to drive SSPTXDx data in the current time slot, even if the SSPx port is master to SSPx_FRM. When using PSP format in Network mode, the parameters SFRMDLY, STRTDLY, DMYSTOP, DMYSTRT must all be 0b0. The other parameters SFRMP, SCMODE, FSRT, SFRMWDTH are programmable.

When the SSPx port is a master to the SSPx_FRM signal and a need arises to

ASR CONFIDENTIAL

exit from Network mode, software should:

- Clear the SSP_SSCR0[MOD] bit. SSP_SSCR0[SSE] does not need to change.
- Wait until SSP_SSTSS[NMBSY] is cleared.
- Disable the SSPx port by clearing SSP_SSCR0[SSE].
- Before exiting Network mode, verify the TXFIFO is empty (SSP_SSSR[TFL]=0b0000 and SSP_ SSSR[TNF]=0b1.)

ASR CONFIDENTIAL

- If data remains in the TXFIFO after the Network mode is exited, a non-Network mode frame is sent.

Due to synchronization delay between the internal bus and the SSPx port clock domain, one extra frame may be transmitted after software clears the SSP_SSCR0[MOD] bit. The SSPx port continues to drive SSPx_SCLK (if SSP_SSCR1[SCLKDIR] is cleared) and SSPx_FRM (if SSP_ SSCR1[SFRMDIR] is cleared) until the end of the last valid time slot.

If the SSPx port is a slave to both SSPx_SCLK (SSP_SSCR1[SCLKDIR] set) and SSPx_FRM (SSP_SSCR1[SFMRDIR] set), the SSP_SSTSS[NMBSY] bit remains asserted until the SSP_ SSCR0[MOD] bit is cleared or until one SSPx_SCLK after the end of the last valid time slot.

Note

When operating in Slave mode (SSP_SSCR1[SCLKDIR] = SSP_SSCR1[SFRMDIR] =
1) the external codec must provide the correct number of bits as determined by the frame width (SSP_SSPSP[SFRMWDTH]) and number of time slots (SSP_ SSCR0[FRDC]). When the number of bits read in during a Frame cycle do not match the number expected, a Bit Count Error occurs (SSP_SSSR[BCE]) and the contents in the FIFO cannot be guaranteed. Software must be used to handle any error conditions when they occur.

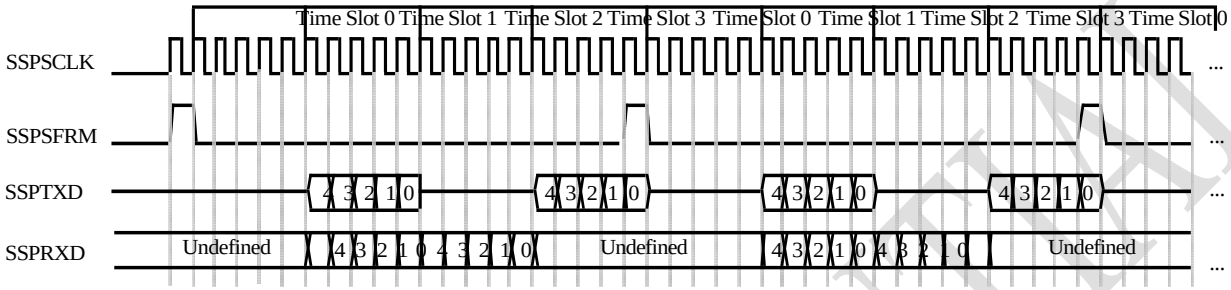
Note

In the next example, a data format of 5 is used. However, a data format of 5 is not a supported data size for the SSP Controller. A data size of 5 was used only to reduce the size of the diagram.

Figure 88 shows an example diagram using Network mode with a PSP data format with a data size of 5 bits (SSP_SSCR0[EDSS,DSS]=0b0,0b0101). In this example the SSPx Controller is a master of the SSPx_SCLK and SSPx_FRM. The SSPx Controller has been programmed for 4 time slots (SSP_SSCR0[FRDC] = 0x3). The SSP Controller is programmed to receive data in time slots 1 and 2 (SSP_SSRSA[RTSA] = 0x6) and transmit data in time slots 1 and 3 (SSP_SSTSA[TTSA] = 0xA).

ASR CONFIDENTIAL

Figure 88:Network Mode (Example Using 4 Time Slots)



NOTE: A data format of 5 is not a supported data size for the SSP Controller. A data size of 5 was used only to reduce the size of the diagram.

ASR CONFIDENTIAL

16.4.5.1 Network Mode Registers

The register bits and fields that must be programmed for the Network Mode are SSP_SSCR0[MOD], SSP_SSCR0[FRDC], SSP_SSPSP[FSRT], SSP_SSPSP[SFRMWDTH], SSP_SSPSP[SCMODE], SSP_SSPSP[SFRMP], and SSP_SSPSP[SFRMDLY]. The definitions of each of these bits and fields can be found in Appendix A.

16.4.6 Parallel Data Formats for FIFO Storage

All ASR1606 and DMA accesses transfer one FIFO entry per access. Data in the FIFOs is either stored with one 32-bit value per data sample (in non-packed or sample > 16 bits) or in a 16-bit value in packed mode when the data is 4 or 16 bits. Within each 32- or 16-bit field, the stored data sample is right-justified, with the LSb of the word in bit 0. In the Receive FIFO, unused bits are packed as zeroes above the MSb. In the Transmit FIFO, unused "don't-care" bits are above the MSb. For example, DMA and ASR1606 accesses do not have to write to the unused bit locations. Logic in the SSP automatically formats data in the Transmit FIFO so that the sample is properly transmitted on SSPTXDx in the selected frame format.

16.4.7 FIFO Operation

This section describes the operation of Transmit and Receive FIFOs.

16.4.7.1 *ASR1606-Initiated Data Transfers*

Two separate and independent FIFOs are present for transmitting (TXFIFO to peripheral) and receiving (RXFIFO from peripheral) serial data. The FIFOs are filled or emptied by programmed I/O or DMA bursts.

16.4.7.2 *Using Programmed I/O Data Transfers*

FIFO filling and emptying can be performed by the ASR1606 in response to an interrupt from the FIFO logic. Each FIFO has a programmable FIFO trigger threshold that triggers an interrupt. When the number of entries in the RXFIFO exceeds the RXFIFO Trigger Threshold (SSP_SSCR1[RFT]) field in the SSP Control Register 1, an interrupt is generated (if enabled) that signals the ASR1606 to empty the RXFIFO. When the number of entries in the TXFIFO is less than or equal to the TXFIFO Trigger Threshold (SSP_SSCR1[TFT]) field in the SSP Control Register 1 plus 1, an interrupt is generated (if enabled) that signals the ASR1606 to refill the TXFIFO.

The SSP Status Register can be polled to determine how many samples are in a FIFO and whether the FIFO is full or empty. Software is responsible for ensuring that the proper RXFIFO Trigger Threshold and TXFIFO Trigger Threshold values are chosen to prevent Receive FIFO Overrun and Transmit FIFO Underrun

ASR CONFIDENTIAL

(in the SSP Status Register) error conditions.

16.4.7.3 Using DMA Data Transfers

The DMA controller can also be programmed to transfer data to and from the FIFOs. To prevent overruns of the TXFIFO or underruns of the RXFIFO when using the DMA, be careful when setting the FIFO trigger threshold levels by setting the correct DMA burst sizes. TXFIFO overruns and RXFIFO underruns are silent errors: There is no indication of the overrun or underrun condition other than missing data at the receiving end of the link. The DMA burst size must be smaller than the trigger threshold.

The programming model for using DMA is:

- Program the total number of transmit/receive byte lengths, burst sizes, and peripheral width.
- When not using the FIFO packed mode, program the Width (DMA_DCMDx[WIDTH]) field in the DMA Command Registers 0-31 to 0x1 for FIFO data sizes of 8 bits, 0x2 for FIFO data sizes of 16 bits, and 0x3 for FIFO data sizes of more than 16 bits. When not using packed mode, the

ASR CONFIDENTIAL

SSP stores one data sample per FIFO location where each FIFO has 16 locations. For example, the DMA burst size (DMA_DCMDx[Size]) must not exceed 16 bytes when DMA_ DCMDx[WIDTH] is set to 0x1 (byte wide).

- When using FIFO packed mode (see FIFO Operation in Packed Mode), program the DMA_DCMDx[WIDTH] field to 0x3. When using packed mode, the SSP stores two data samples per FIFO location where each FIFO has 16 locations. Therefore, the DMA burst size (Maximum burst size DMA_DCMDx[SIZE] field in the DMA Command Registers 0-31) must not exceed 16 bytes when DMA_DCMDx[WIDTH] is set to 0x3 (more than 16 bits wide).
- Because the SSP is not flow-controlled and has only 16 location FIFOs, software must program the TXFIFO threshold (SSP_SSCR1[TFT]) field, RXFIFO threshold (SSP_SSCR1[RFT]) field, and the DMA burst size to ensure that a TXFIFO overrun or RXFIFO underrun does not occur. Software must also ensure that the SSP DMA requests are properly prioritized in the system to prevent overruns and underruns.
- Program the preferred values into the SSP Control Registers 0 and 1
- Enable the SSP by setting the Synchronous Serial Port Enable (SSP_SSCR0[SSE]) field in the SSP Control Register 0.
- Set the run bits in the DMA Command Register.
- The DMA waits for either the TXFIFO or RXFIFO service request.
- If the receive byte length is not an even multiple of the transfer burst size, a trailing-byte condition may occur as described in Section 17.4.1, Trailing Bytes in RXFIFO.

In full-duplex formats where the SSP always receives the same number of data samples that it transmits, the DMA channel should be set up to transmit and receive the same number of bytes.

Table 45:SSP_SSCR1[TFT] and SSP_SSCR1[RFT] Values with DMA Burst Sizes

SSP Sample Size (bits)	**DMA Width (DMA_ DCMDx[WIDTH])**	**Burst Size** DMA_ DCMDx[SIZE]	**TXFIFO Trigger Threshold** (SSP_SSCR1[TFT])	**RXFIFO Trigger Threshold** (SSP_SSCR1[RFT])
8	1 byte WIDTH = 0x1	8	7	8
16, 18, or 32	4 bytes WIDTH = 0x3	16	3	4
16, 18, or 32	4 bytes WIDTH = 0x3	32	7	8

The DMA transmit-burst size is limited because the smallest TFT value is zero,

ASR CONFIDENTIAL

which equates to one sample left in the TXFIFO. Software must program the TXFIFO threshold and the DMA burst size (DMA_DCMDx[SIZE]) such that an overflow of the TXFIFO does not occur. Software must program the RXFIFO threshold and the DMA burst size such that an underflow of the RX FIFO does not occur.

Note

When the FIFO Packing Enable (SSP_SSCR0[FPCKE]) field in the SSP Control Register 0 is set to 0x1, the SSP_SSCR1[TFT] and SSP_SSCR1[RFT] fields represent twice the number of FIFO entries as when the packing enable bit is 0x0. So, when in packed mode, the maximum allowed DMA burst size (8 or 16) in Table 53 could be doubled.

ASR CONFIDENTIAL

16.4.8 Baud-Rate Generation

When the SSP is configured as the master (output) of SSPx_SCLK as determined by clearing the SSP_SSCR1[SCLKDIR] field, the baud rate (or serial bit-rate clock SSPx_SCLK) is obtained by selecting from one of the four fixed clock sources (6.5 MHz, 13 MHz, 26 MHz, or 52 MHz) from the Application PMU. A variable clock source is available from the output of two dithering dividers external to the SSP unit. Clock selection is achieved by writing to the Functional Clock Select (APBC_SSPx_CLK_RST[FNCLKSEL]) field of the SSPx Clock/Reset Control Register.

The dithering dividers are primarily intended for generating flexible clock frequencies based on the audio sampling rates. The read-write Audio Clock Divider Register (SSP_SSACD) and Audio Clock Dither Divider Register (SSP_SSACDD) are used to determine the output the SSPx_SCLK and I2S_ SYSCLK (respectively) frequency of the dithering-frequency dividers. These registers are identical and have a 15-bit numerator field (NUM) and a 12-bit denominator field (DEN) that are used to generate the dithered audio clock frequency according to the formula in equations.

SSPx_SCLK = 312MHz / (SSP_SSACD[ASYSCLK_NUM] / (MPMU_ASYSDR[ASYSCLK_DENOM]
+ 1))

SSPx_SCLK = ASYSCLK / (MPMU_SSPDR[SSPx_SCLK_NUM) / (MPMU_SSPDR[SSPx_SCLK_ DENOM] + 1))

Output of dithering divider 1 (SSPSYSCLK) = 312 MHz x (ASYSDR[DEN]/ASYSDR[NUM])

Output of dithering divider 2 (fixed SSPx_SCLK) = Output of dithering divider 1 x (ASSPDR[DEN]/ASSPDR[NUM])

When the source clock is to be changed, software must:

- Disable the SSPx port by writing 0x0 to SSP_SSCR0[SSE].
- Disable the SSP port internal clock by clearing the appropriate bit in the Clock Enable Register.
- Write 0x1 to the appropriate bits in the Main PMU ACGR register to enable functional clocks to the SSP units.
- If applicable, set appropriate NUM and DEN values for the dithering-frequency dividers.
- Enable APB clock to the SSP unit by programming the APBC_SSPx_CLK_RST register.
- Set the Synchronous Serial Port Enable (SSP_SSCR0[SSE]) bit to re-enable

the SSPx port. Whenever the baud rate is to be changed, software must:

- Disable the SSPx port by clearing the Synchronous Serial Port Enable

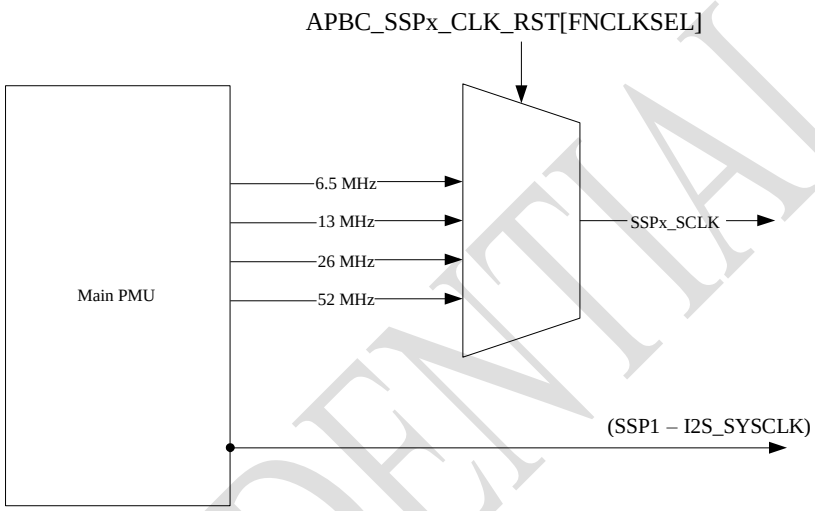
ASR CONFIDENTIAL

(SSP_SSCR0[SSE]) bit.

- Set the Synchronous Serial Port Enable (SSP_SSCR0[SSE]) bit to re-enable the SSPx port.
- Wait two SSPx_SCLK cycles before writing new data to the TXFIFO.

ASR CONFIDENTIAL

Figure 89:SSP Baud Rate Generation



32-Bit I^{2}S Emulation Using SSP

To emulate 32-bit I²S mode using the SSP Controller, follow the configurations below to support either normal or “MSB-justified” mode.

“Normal” Mode

The following bit fields must be configured:

- SSP_SSCR0[EDSS] = 0b1 (32-bit data)
- SSP_SSCR0[FRF] = 0b11 (PSP format)
- SSP_SSCR0[DSS] = 0b1111 (32-bit data)

The example below shows the use of the recommended settings:

- SSP_SSCR0 = 0x001000BF
- SSP_SSCR1 = 0x203C3C03 (SSP_SSCR1[SCLKDIR] and SSP_SSCR1[SFRMDIR] must be cleared, all other bit fields settings are optional.)
- SSP_SSPSP = 0x02100000 (all bit fields must be cleared except: FSRT = 1 and SFRMWDTH = 16, DMYSTART=0,1)

“MSB-justified” Mode

All configurations in the example for “normal” mode above remain the same for “MSB-justified” mode, with the exception for SSPSP.

SSP_SSPSP = 0x00100000 (all bit fields must be cleared except: SFRMWDTH = 16, DMYSTART

ASR CONFIDENTIAL

=0,1)

16.5 *Register Descriptions*

Refer to *ASR1606 Registers* for details on the SSP registers.

ASR CONFIDENTIAL

17.SDIO

- Section 17.1, Overview
- Section 17.2, Features
- Section 17.3, Signal Descriptions
- Section 17.4, Operation

17.1 *Overview*

The ASR1606 supports a SDIO device interface that conforms to the industry standard SDIO Full-Speed card specification and allows a host controller using the SDIO bus protocol to access the Wireless SoC device.

The ASR1606 acts as the device on the SDIO bus. The host unit can access registers of the SDIO interface directly and can access shared memory in the device through the use of BARs and a DMA engine.

17.2 *Feature*

The SDIO device interface main features include:

- On-chip memory used for CIS
- Supports SPI, 1-bit SDIO, and 4-bit SDIO transfer modes at the full clock range of 0 to 50 MHz
- Special interrupt register for information exchange
- Allows card to interrupt host

17.3 *Signal Descriptions*

Table 46 shows the signal mapping between the ASR1606 device and the SDIO specification.

ASR CONFIDENTIAL

Table 46 SDIO Interface Signal Description

ASR1606PinName	SignalName	Type	Description
GPIO[26]	CLK	I/O	SDIO 1-bit mode: Clock SDIO SPI mode: Clock
GPIO[25]	CMD	I/O	SDIO 1-bit mode: Command line SDIO SPI mode: Data input
GPIO[20]	DAT3	I/O	SDIO 4-bit mode: Data line bit [3] SDIO 1-bit mode: Not used SDIO SPI mode: Chip select (active low)
GPIO[21]	DAT2	I/O	SDIO 4-bit mode: Data line bit [2] or Read Wait (optional) SDIO 1-bit mode: Read Wait (optional) SDIO SPII mode: Reserved
GPIO[22]	DAT1	I/O	SDIO 4-bit mode: Data line bit [1] SDIO 1-bit mode: Interrupt SDIO SPI mode: Interrupt
GPIO[24]	DAT0	I/O	SDIO 4-bit mode: Data line bit [0] SDIO 1-bit mode: Data line SDIO SPI mode: Data output

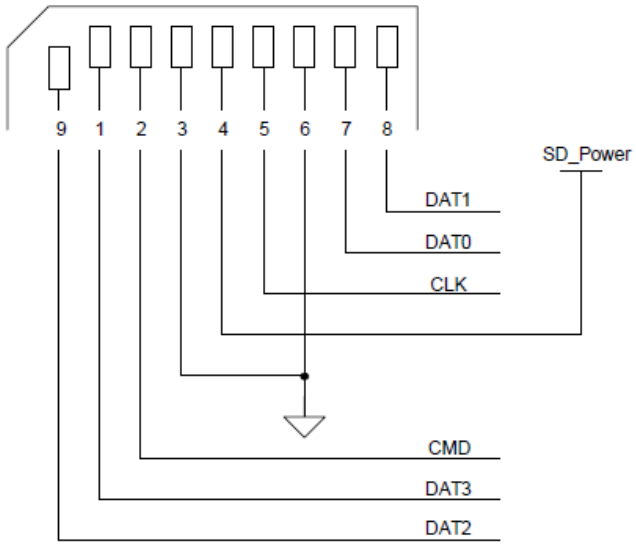
Copyright © 2022 ASR

17.4 *Operation*

17.4.1 SDIO Interface Functional Description

17.4.1.1 SDIO Connection/Function

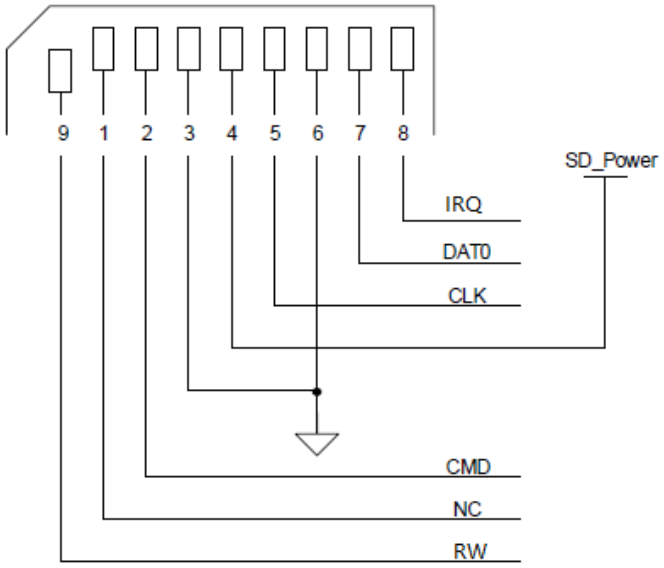
Figure 90 SDIO Physical connection - 4bit mode1



1. In 4-bit SDIO mode, data is transferred on all 4 data pins (DAT[3:0]), and the interrupt pin is not available for exclusive use as it is utilized as a data transfer line. Thus, if the interrupt function is required, a special timing is required to provide interrupts.

ASR CONFIDENTIAL

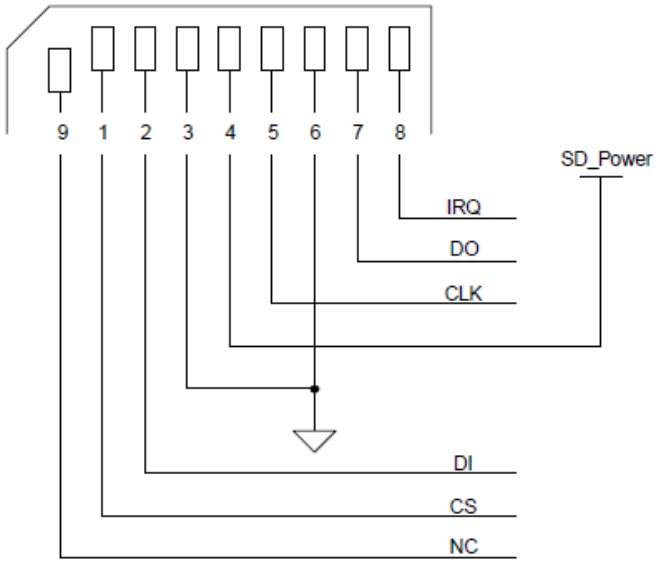
Figure 91 SDIO physical connection - 1bit mode1



1. In 1-bit SDIO mode, data is transferred on the DAT[0] pin only. Pin 8, which is undefined for memory, is used as the interrupt pin.

ASR CONFIDENTIAL

Figure 92 SDIO Physical Connection – SPI mode1



1. Pin 8, which is undefined for memory, is used as the interrupt pin in SPI mode.

Table 47 SDIO Electrical Function Definition - 1-bit

Pin	Chip Pin Name	SDIO 1-bit Mode	
1	20	N/C	Not used
2	25	CMD	Command line
3	N/A	VSS1	Ground
4	N/A	VDD	Supply voltage
5	26	CLK	Clock
6	N/A	VSS2	Ground
7	24	DATA	Data line
8	22	IRQ	Interrupt
9	21	RW	Read Wait (optional)

Table 48 SDIO Electrical Function Definition - SPI

Pin	Chip Pin Name	SPI Mode	
1	20	CS	Card Select
2	25	DI	Data input
3	N/A	VSS1	Ground
4	N/A	VDD	Supply voltage
5	26	SCLK	Clock
6	N/A	VSS2	Ground
7	24	DO	Data output
8	22	IRQ	Interrupt
9	21	NC	Not used

Table 49 SDIO Electrical Function Definition - 4-bit

Pin	Chip Pin Name	SDIO 4-bit Mode	
1	20	CD/DAT[3]	Data line 3
2	25	CMD	Command line
3	N/A	VSS1	Ground
4	N/A	VDD	Supply voltage
5	26	CLK	Clock
6	N/A	VSS2	Ground
7	24	DAT[0]	Data line 0

8	22	DAT[1]	Data line 1 or Interrupt (optional)
9	21	DAT[2]	Data line 2 or Read Wait (optional)

ASR CONFIDENTIAL

17.4.1.2 SDIO Command List

All mandatory SDIO commands are supported.

Table 50 SD Mode, SDIO Commands

Command	Command Name	Description
CMD0	GO_IDLE_STATE	Used to change from SDIO to SPI mode
CMD3	SEND_RELATIVE_ADDR	SDIO Host asks for RCA
CMD5	IO_SEND_OP_COND	SDIO Host asks for and sets operation voltage
CMD7	SELECT/DESELECT_CARD	Sets SDIO target device to command state or back to standby
CMD15	GO_INACTIVE_STATE	Sets SDIO target device to inactive state
CMD52	IO_RW_DIRECT	Used to read/write host register and CIS table
CMD53	IO_RW_EXTENDED	Used to read/write data from/to SQU memory

Table 51 SPI Mode, SDIO Commands

Signal Name	Type	Description
CMD0	GO_IDLE_STATE	Used to change from SDIO to SPI mode
CMD5	IO_SEND_OP_COND	Used in initialization state
CMD52	IO_RW_DIRECT	Used to read/write host register and CIS table
CMD53	IO_RW_EXTENDED	Used to read/write data from/to SQU memory
CMD58	CRC_ON_OFF	Enable/disable CRC (SPI only)

Copyright © 2022 ASR

17.4.1.3 Power on Reset

The power-up transaction sequences for SDIO and SPI modes are as follows:

SDIO Mode Power-Up Transaction Sequence

1. CMD5 (arg=0)
2. SDIO Card response with OCR
3. CMD5 from host to set operation voltage using OCR
4. SDIO Card response with IORDY = 1 and MP = 0 (not SDIO memory, not combo card)
5. CMD3 and CMD7 to set in one active mode
6. CMD15 to put the card inactive

SPI Mode Power-Up Transaction Sequence

1. CMD0 + CS = LOW
2. CMD5 (arg =0)
3. SDIO Card response with OCR
4. CMD5 from host to set operation voltage using OCR
5. CMD15 to put the card inactive

17.4.1.4 Operation Sequence

Table 52 lists the registers used to program the operation sequence. See the SDIO Registers section of the Host Interface Registers document for register programming information.

Table 52 SDIO Registers

Register/Offset
Host Transfer Status (HOST_RESTART), Offset 0x128
Card to Host Event (C2H_INTEVENT), Offset 0x130

SDIO Host Reads CIS Table Sequence

1. Check Card to Host Event (C2H_INTEVENT), Offset 0x130[2]. It is set by Card after CIS table is initialized.

2. HOST reads CIS Table using Function 0 address 8030_807F and function 1 address 8080_80FF.

SDIO Host Downloads Packet

1. Card sets Card to Host Event (C2H_INTEVENT), Offset 0x130[0].

ASR CONFIDENTIAL

2. Host Polls Dnld_Card_Rdy and IO_Ready.

3. Host starts CMD53 block mode using function 1 with IO port address. CMD53 write clears

Dnld_Card_Rdy.

4. Host sends the data in terms of predefined blocks. If BUSY, host delays the next block.

5. After CMD53 write completes, an interrupt CardInt is sent to firmware.

6. Firmware checks Host Transfer Status (HOST_RESTART), Offset 0x128[2] and Host Transfer

Status (HOST_RESTART), Offset 0x128[0] registers. If Dnld_CRC_Err = 1, this packet has CRC error. If Dnld_Restart = 1, firmware ignores this packet.

7. Back to the first step.

SDIO Host Uploads Packet

1. Card sets Card to Host Event (C2H_INTEVENT), Offset 0x130[1].

2. Upld_Card_Rdy triggers an interrupt to SDIO HOST in the interrupt period.

3. SDIO Host read to clear INT or write 0 to clear the interrupt.

4. SDIO Host checks Upld_Card_Rdy and IO_Ready.

5. Host starts CMD53 read using function1 with IO address with infinite block number or defined block number. CMD53 clears Upld_Card_Rdy bit.

6. After Host receives all data, HOST writes Abort using CMD52.

7. This terminate read operation. Card gets an interrupt with abort and with packet read, is

complete.

8. Firmware reads interrupt, clears interrupt.

9. Firmware reads Host Transfer Status (HOST_RESTART), Offset 0x128[1]. If it is set by Host,

Firmware prepares to reissue this packet.

ASR CONFIDENTIAL

10. Back to the first step.

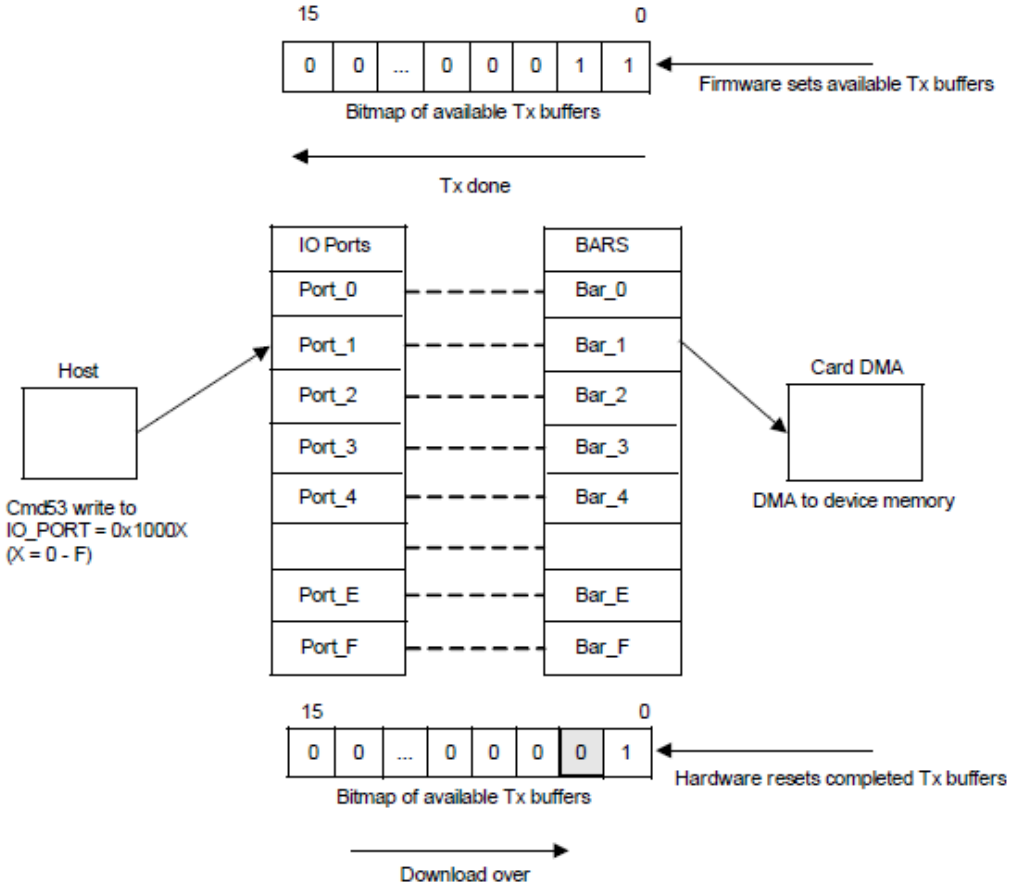
The SDIO Host checks IO_READY before starting a new CMD53. The SDIO target device can take a new SDIO host command only after the SDIO target device internal state machine is set back to IO_Ready state. Otherwise, the new CMD53 is ignored.

17.4.2 Multiple I/O Ports

Each I/O port is mapped to a base address (BAR). There are 16 ports per direction (Tx/Rx). The availability of each port is indicated through a bitmap register in which the firmware sets the bits (before setting Tx done or Upload ready). The hardware clears the bits upon completion of the corresponding Cmd53. Figure 93 shows a sample dataflow for Tx (Rx is imilar).

ASR CONFIDENTIAL

Figure 93 Multi-port dataflow



17.4.2.1 Tx Dataflow Sequence

1. Firmware programs the buffer address to the Tx BAR registers.

Bar_0_reg = dst_addr_bar0

Bar_1_reg = dst_addr_bar1

2. Firmware sets the corresponding IO port Tx bitmap = 0x3.

3. Firmware sets Tx download ready.

4. Driver receives Tx download ready interrupt and reads Tx bitmap (Cmd53).

5. Driver downloads a packet through Cmd53 write to Port_0.

ASR CONFIDENTIAL

(Driver can optionally download the second packet to Port_1).

6. Firmware receives Tx download over after completion of Cmd53 to Port_0.

7. Firmware reads Tx bitmap to get the buffer with download data.

8. Firmware sets the Tx bitmap again when the Tx buffer is free (after MAC Tx).

Setting Tx download ready is delayed until the firmware has multiple buffers free and the driver has some buffers for future downloads. This eliminates the need to interrupt the driver after each Cmd53 write.

17.4.2.2 Rx Dataflow Sequence

1. Firmware programs the buffer address to the Rx BAR registers.

Bar_0_reg = src_addr_bar0 Len_0_reg = 0x430

Bar_1_reg = src_addr_bar1 Len_1_reg = 0x64

2. Firmware sets the corresponding IO port Rx bitmap = 0x3.

3. Firmware sets Rx upload ready.

4. Driver receives Rx upload ready interrupt and reads Rx bitmap and length registers(Cmd53).

5. Driver uploads a packet through Cmd53 read from Port_0.

(Driver can continue to upload the second packet from Port_1).

6. Firmware receives Rx upload over after completion of Cmd53 to Port_0.

7. Firmware reads Rx bitmap to determine which buffer was uploaded.

8. Firmware sets the Rx bitmap again when the Rx packet is available.

Setting Rx upload ready is delayed until the firmware has multiple packets and the driver has some buffers pending for uploads. This eliminates the need to interrupt the driver after each Cmd53 read.

ASR CONFIDENTIAL

17.4.3 Host Interface Specifications

The SDIO electrical specifications are identical for the 1-bit SDIO, 4-bit SDIO, and SPI modes

Figure 94 SDIO Protocol Timing Diagram - Normal Mode

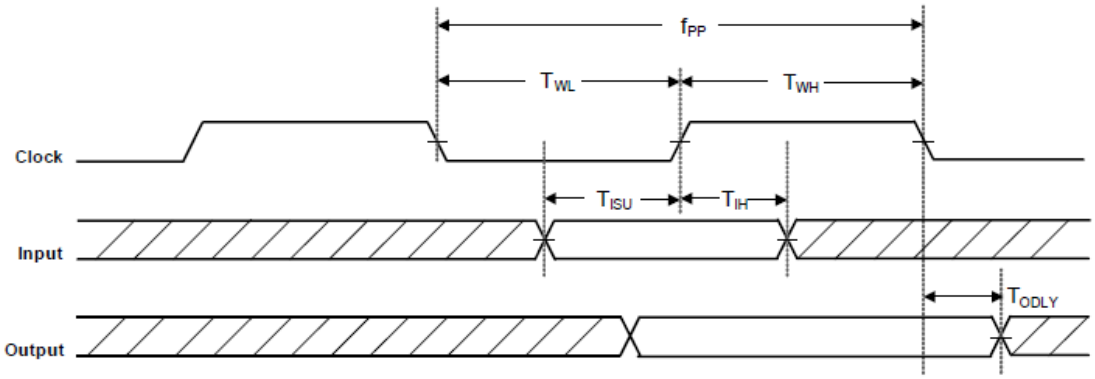
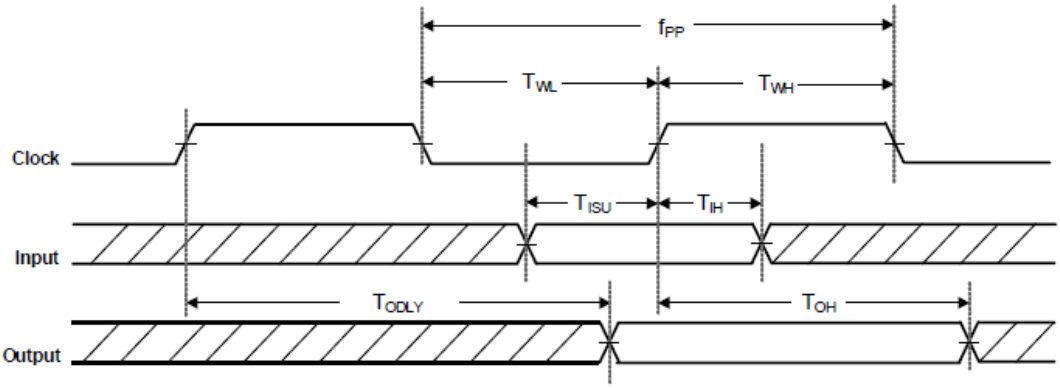


Figure 95 SDIO Protocol Timing Diagram – High Speed Mode



Copyright © 2022 ASR

Table 53 SDIO Timing Data1

Symbol	Parameter	Condition	Min	Typ	Max	Units
fPP	Clock Frequency	Normal	0	--	25	MHz
		High Speed	0	--	50	MHz
TWL	Clock Low Time	Normal	10	--	--	ns
		High Speed	7	--	--	ns
TWH	Clock High Time	Normal	10	--	--	ns
		High Speed	7	--	--	ns
TISU	Input Setup Time	Normal	5	--	--	ns
		High Speed	6	--	--	ns
TIH	Input Hold Time	Normal	5	--	--	ns
		High Speed	2	--	--	ns
TODLY	Output Delay Time	--	0	--	14	ns
TOH	Output Hold Time	High Speed	2.5	--	--	ns

Note: 1. The SDIO-SPI CS signal timing is identical to all other SDIO inputs.

2. Over full range of values specified in the Recommended Operating Conditions unless otherwise specified.

ASR

18.UART Controllers

This section describes the universal asynchronous receiver/transmitter (UART) serial ports.

- Section 18.1, Key Features
- Section 18.2, Overview
- Section 18.3, Signal Descriptions
- Section 18.4, Operation
- Section 18.5, Register Descriptions

18.1 *Key Features*

The serial ports are controlled via direct-memory access (DMA) or programmed I/O. The UARTs share the following features:

- Functionally compatible with the 16550A and 16750
- Ability to add or delete standard asynchronous communications bits (start, stop, and parity) in the serial data
- Independently controlled Transmit, Receive, line status, and data-set interrupts
- Modem control functions (CTSn and RTSn on UART1, UART2.)
- Auto-flow capability controls data I/O without generating interrupts:
  - RTSn (output) controlled by UART Receive FIFO
  - CTSn (input) from modem controls UART transmitter
- Programmable serial interface:
  - 7-bit or 8-bit characters
  - Even, odd, or no parity detection
  - 1 stop-bit generation
  - Baud-rate generation up to 3.6 Mbps for the 2 Fast UARTs
  - False start-bit detection
- 64-byte Transmit FIFO
- 64-byte Receive FIFO
- Complete status-reporting capability
- Ability to generate and detect line breaks
- Internal diagnostic capabilities that include:
  - Loopback controls for communications link fault isolation
  - Break, parity, and framing-error simulation
- Fully prioritized interrupt system controls
- Separate DMA requests for Transmit and Receive data services
- Serial infrared asynchronous interface that conforms to the Infrared Data Association (IrDA) specification

ASR CONFIDENTIAL

18.1.1 Compatibility with 16550A and 16750

The UARTs are functionally compatible with the 16550A and 16750 industry standards. Each UART supports most of the 16550A and 16750 functions as well as the following features:

- DMA requests for Transmit and Receive data services
- Serial infrared asynchronous interface
- Non-Return to Zero (NRZ) encoding/decoding function
- 64 byte Transmit/Receive FIFO buffers
- Programmable Receive FIFO trigger threshold
- Auto baud-rate detection
- Auto flow

18.2 *Overview*

The ASR1606 has 4 UARTs (UART 1, UART 2, UART 3 and UART 4). The UARTs use the same programming model.

Each port contains a UART and a slow serial infrared Transmit encoder and Receive decoder that conform to the IrDA serial infrared specification.[1]

Each UART performs serial-to-parallel conversion on data characters received from a peripheral device or a modem and parallel-to-serial conversion on data characters received from the ASR1606.

Software can read a complete UART status for the Line Status Register. Status information includes the type and condition of transfer operations and error conditions (parity, overrun, framing, or break interrupt) associated with the UART.

Each serial port operates in either FIFO or non-FIFO mode. In FIFO mode, a 64-byte Transmit FIFO holds data from the ASR1606 until it is transmitted on the serial link; a 64-byte Receive FIFO buffers data from the serial link until it is read by the ASR1606. In non-FIFO mode, the Transmit and Receive FIFOs are bypassed, and the Transmit Holding Register and Receive Buffer Register are used instead.

Each UART includes a programmable baud-rate generator that can divide the input clock by any value from 1 to (2^16 – 1), which produces a 16X clock that can be used to drive the internal Transmit and Receive logic. Software can program interrupts to meet its requirements, which minimizes the number of computations required to handle the communications link. Each UART operates in an environment that is either controlled by software and can be polled or is interrupt driven.

All 4 UARTs support the 16550A and 16750[2] functions, but support slightly different features as described in the following sections.

The supported baud rates of each UART are shown in Table 54.

ASR CONFIDENTIAL

Table 54:Supported Baud Rates

UART	Support Baud Rates									
	9600	19.2	38.4	57.6	115.2	230	460	921	1.8	3.6
	Hz				kHz				MHz	
1	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
2	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
3	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
4	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

1. Infrared Data Association, *Serial Infrared Physical Layer Link Specification*, October 17, 1995, Version 1.1
2. The 16550A was originally produced by National Semiconductor Inc. The 16750 is produced as the TL16C750 by Texas Instruments.

ASR CONFIDENTIAL

Copyright © 2022 ASR

All four UARTs support the following modem control signals:CTSn, RTSn, Rx, Tx, DSRn, DTRn, RIn, and DCO.

18.3 *Signal Descriptions*

Table 55 lists and describes each external signal that is connected to a UART module and how these pins function as modem control lines. The pins transmit digital CMOS-level signals and are connected to the ASR1606 through GPIOs. Refer to Multi-Function pin registers for details on the alternate-function pin configuration.

Table 55:UART Signal Descriptions

Name	Type	Description
RXD	Input	**Serial Input** Serial data input to the Receive Shift register. In Infrared mode, it is connected to the infrared receiver input.
TXD	Output	**Serial Output** Serial data output to the communications-link peripheral, modem, or data set. The TXD signal is set to the logic 1 state upon a reset operation. It is connected to the output of the infrared transmitter in Infrared modeAuto-flow mode.
CTSn	Input	**Clear to Send** When asserted, indicates that the modem or data set is ready to exchange data. The CTSn signal is a modem status input, and its condition can be tested by reading the <CTS> field in the Modem Status Register. The <CTS> field is the complement of the CTSn signal. The <Delta Clear to Send> field in the Modem Status Register indicates whether the CTSn input has changed state since the last time the Modem Status Register was read. CTSn has no effect on the transmitter. When the <CTS> field changes state and the modem-status interrupt is enabled, an interrupt is generated. **Non-Auto-flow mode**: When not in Auto-flow mode, the <CTS> field indicates the state of CTSn. The <Delta Clear to Send> field indicates whether the CTSn input has changed state since the previous reading of MSR. CTSn has no effect on the transmitter. The user can program the UART to interrupt the ASR1606 when DCTS changes state. Software can then stall the outgoing data stream by starving the Transmit FIFO or disabling the UART with the Interrupt Enable Register. **NOTE:** If UART transmission is stalled by disabling the UART, no Modem Status Register interrupt is received when CTSn re-asserts because disabling the UART also disables interrupts. To get around this issue, use either auto-CTS in Auto-flow mode or program the CTSn GPIO pin to interrupt. **Auto-flow mode**: In this mode, the UART Transmit circuity checks the state of CTSn before transmitting each byte. No data is transmitted when CTSn is high.

ASR CONFIDENTIAL

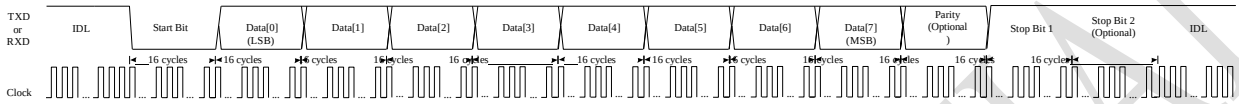
Name	Type	Description
DSRn	Input	**Data Set Ready** When asserted, indicates that the modem or data set is ready to establish a communications link with a UART. The DSRn signal is a modem-status input and its condition can be tested by reading the <Data Set Ready> field in the Modem Status Register, which is the complement of DSRn. The <Delta Data Set Ready> field in the Modem Status Register indicates whether the DSRn input has changed state since the Modem Status Register was last read. When the <Data Set Ready> changes state, an interrupt is generated if the modem-status interrupt is enabled.
DCDn	Input	**Data Carrier Detect** When asserted, indicates that the data carrier has been detected by the modem or data set. The DCDn signal is a modem-status input and its condition can be tested by reading the <Data Carrier Detect> field in the Modem Status Register, which is the complement of the DCDn signal. The <Delta Data Carrier Detect> field in the Modem Status Register indicates whether the DCDn input has changed state since the previous reading of the Modem Status Register. DCDn has no effect on the receiver. An interrupt is generated when the <Data Carrier Detect> field changes state and the modem-status interrupt is enabled.
RIn	Input	**Ring Indicator** When asserted, indicates that the modem or data set has received a telephone ringing signal. The RIn signal is a modem-status input and its condition can be tested by reading the <Ring Indicator> field in the Modem Status Register, which is the complement of the RIn signal. The <Trailing Edge Ring Indicator> field in the Modem Status Register indicates whether the RIn input has changed from low to high since the Modem Status Register was last read. An interrupt is generated when the RI bit of the Modem Status Register changes from a high to low state and the modem-status interrupt is enabled.
DTRn	Output	**Data Terminal Ready** When asserted, signals the modem or the data set that the UART is ready to establish a communications link. To assert the DTRn output (active low), set the <Data Terminal Ready> field in the Modem Control Register, which is the complement of the output signal. A reset operation de-asserts this signal (high). Loop-mode operation holds DTRn de-asserted.
RTSn	Output	**Request To Send** When asserted, signals the modem or the data set that the UART is ready to exchange data. To assert the RTSn output (active low), set the <Request to Send> field in the Modem Control Register, which is the complement of the output signal. A reset operation de-asserts this signal (high). Loop-mode operation holds RTSn de-asserted. **Non-Auto-flow mode**: To assert the RTSn output (active low), set <Request to Send>. **Auto-flow mode**: RTSn is asserted automatically by the auto-flow circuitry when the Receive buffer exceeds its programmed trigger threshold. It is de-asserted when enough bytes are removed from the buffer to lower the data level back to the trigger threshold.

ASR CONFIDENTIAL

18.4 *Operation*

Figure 96 shows the format of a UART data frame.

Figure 96:Example UART Data Frame



The Receive-data sample-counter frequency is 16 times the value of the bit frequency. The 16X clock is created by the baud-rate generator. Each bit is sampled three times in the middle. Shaded bits in Figure 96 are optional and can be programmed by software.

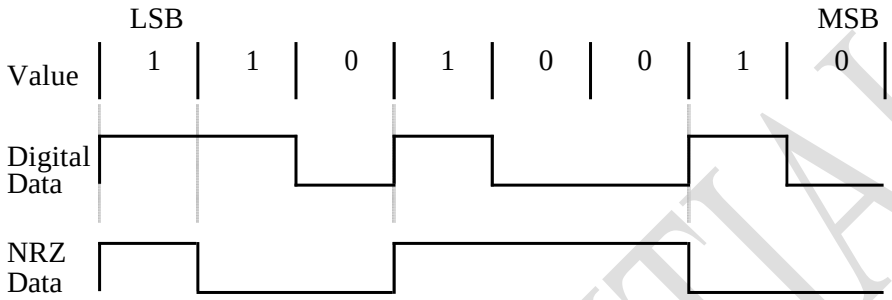
Each data frame is between 9 and 11 bits long, depending on the size of the data programmed, whether parity is enabled A data frame begins by transmitting a start bit that is represented by a high-to-low transition. The start bit is followed by 8 bits of data that begin with the Least Significant bit (LSb). The data bits are followed by an optional parity bit. The parity bit is set if: even parity is enabled and the data byte has an odd number of ones or if odd parity is enabled and the data byte has an even number of ones. The data frame ends with 1 stop bit. The stop bit represented by 1 successive bit period of logic one.

Each UART has 2 FIFOs: 1 Transmit and 1 Receive. The Transmit FIFO is 64 bytes deep and 8 bits wide. The Receive FIFO is 64 bytes deep and 11 bits wide. Three bits are used for tracking errors.

The UART can use NRZ coding to represent individual bit values. To enable NRZ coding, set the <NRZ Coding Enable> field in the Interrupt Enable Register. A bit value of 1 is represented by a line transition, and 0 is represented by no line transition. Figure 97 shows the data byte 0b0100_1011 in NRZ coding. The LSb in the byte is transmitted first.

ASR CONFIDENTIAL

Figure 97:Example NRZ Bit Encoding — 0b0100_1011



ASR CONFIDENTIAL

18.4.1 Reset

The UARTs are disabled on reset. To enable a UART, software must program the Multi-function Pin registers, then set the <UART Unit Enable> field in the Interrupt Enable Register. When the UART is enabled, the receiver waits for a frame start bit, and the transmitter sends data if it is available in the Transmit Holding Register. Transmit data can be written to the Transmit Holding Register before the UART unit is enabled. In FIFO mode, data is transmitted from the FIFO to the Transmit Holding Register before it goes to the pin.

When the UART unit is disabled, the transmitter or receiver finishes the current byte and stops transmitting or receiving more data. Data in the FIFO is not cleared and transmission resumes when the UART is enabled.

18.4.2 FIFO Operation

Each UART has a Transmit FIFO and a Receive FIFO, with each FIFO holding 64 characters of data. There are 2 methods for moving data into and out of the FIFOs:

DMA

Program I/O

In DMA mode, interrupts are used to control the data flow; whereas, in Program I/O mode, polling is used.



Note

The end-of-chain interrupt does not occur in polled mode.

In DMA mode, software must set the DMA stop interrupt on the last Descriptor in the chain to avoid errors.

18.4.2.1 *FIFO Interrupt Mode Operation*



Note

The <OUT2 Signal Control> field in the Modem Control Register is a global UART interrupt enable bit and must be set to enable UART interrupts.

ASR CONFIDENTIAL

Receive Interrupt

For a Receive interrupt to occur, the Receive FIFO and Receive interrupts must be enabled. The <Interrupt Source Encoded> field in the Interrupt Identification Register changes to show that Receive data is available when the FIFO reaches its trigger threshold. The <Interrupt Source Encoded> field changes to show the next waiting interrupt when the FIFO drops below the trigger threshold. A change in the <Interrupt Source Encoded> field triggers an interrupt to the core.
Software reads the <Interrupt Source Encoded> field to determine the cause of the interrupt.

The Receive-line-status interrupt (Interrupt Identification Register = 0xC6) has the highest priority; the received-data-available interrupt (Interrupt Identification Register = 0xC4) is lower. The line-status interrupt occurs only when the character at the front of the FIFO has errors.

The <Data Ready> field in the Line Status Register is set when a character is transferred from the Shift register to the Receive FIFO. <Data Ready> is cleared when the FIFO is empty.

ASR CONFIDENTIAL

18.4.2.1.1 Character Timeout Interrupt

A character timeout interrupt occurs when the Receive FIFO and Receive timeout interrupt are enabled and all of the following conditions exist:

- At least 1 character is in the FIFO.
- The most recently received character was received more than 4 continuous character times ago.
- The most recent FIFO read was performed more than 4 continuous character times ago.

After the ASR1606 reads 1 character from the Receive FIFO or a new start bit is received, the timeout interrupt is cleared, and the timeout is reset. If a timeout interrupt has not occurred, the timeout is reset when a new character is received or the ASR1606 reads the Receive FIFO.

18.4.2.1.2 Transmit Interrupt

Transmit interrupts can occur only when the Transmit FIFO and Transmit interrupt are enabled. The Transmit data-request interrupt occurs when the Transmit FIFO is at least half empty. The interrupt is cleared when the Transmit Holding Register is written or the Interrupt Identification Register is read.

18.4.2.1.3 Removing Trailing Bytes

The ASR1606 must remove trailing bytes when not in DMA mode or when DMA mode bit (<Trailing Bytes> field in the FIFO Control Register) is not set. The presence of trailing bytes is signaled by the assertion of a character timeout interrupt. When servicing a character timeout interrupt, the ASR1606 uses the following procedure:

1. Read the Line Status Register and check for errors.
2. Disable the receiver timeout interrupt via <Receiver Time-out Interrupt Enable> field in the Interrupt Enable Register.
3. Read data from the UART FIFO.
4. Read the Line Status Register, check for errors, and LOOP back to step 3. If the <Data Ready> field is SET, go to step 5.
5. No more data in FIFO: Re-enable RTO interrupt via the <Modem Interrupt Enable> field in the Interrupt Enable Register.
6. Done

18.4.2.2 *FIFO Polled Mode Operation*

When the FIFOs are enabled, clearing both the <DMA Requests Enable> field and bits [4:0] in the Interrupt Enable Register places the port in FIFO polled operating mode. The receiver and the transmitter are controlled separately. Either one or both can be in polled mode. In polled mode, software checks receiver and transmitter status via the Line Status Register. The ASR1606 polls the following

ASR CONFIDENTIAL

bits for Receive and Transmit data service:

- Receive Data Service—the ASR1606 checks the <Data Ready> field, which is set when 1 or more bytes remain in the Receive FIFO or Receive Buffer Register.
- Transmit Data Service—the ASR1606 checks the <Transmit Data Request> field in the Line Status Register, which is set when the transmitter needs data.

The ASR1606 can also check the <Transmitter Empty> field in the Line Status Register, which is set when the Transmit FIFO.

18.4.2.3 FIFO DMA Mode Operation

The UART has 2 DMA requests: 1 for Transmit data service and 1 for Receive data service. DMA requests are generated in FIFO mode only. The requests are activated by setting the <DMA Requests Enable> field in the Interrupt Enable Register.

- Data Transmitter Data Service—when <DMA Requests Enable> is set, if the Transmit FIFO is absolutely less than half full, the Transmit-DMA request is generated. The DMA Controller

ASR CONFIDENTIAL

(DMAC) then writes data to the FIFO. For each DMA request, the DMAC can send 8, 16, or 32 bytes of data to the FIFO. The UART FIFO accepts partial-word or full-word transfers of 1, 2, 3, or 4 consecutive bytes from the DMAC or Program I/O. The actual number of bytes to be transmitted is programmed in the DMAC.

- Data Receiver Data Service—when <DMA Requests Enable> is set, the Receive-DMA request is generated when the Receive FIFO reaches its trigger threshold with no errors in its entries. The DMAC then reads data from the FIFO. For each DMA request, the DMAC can read 8, 16, or 32 bytes of data from the FIFO. When in 32-bit peripheral bus mode, the DMAC always attempts to read 4 bytes of data per transfer. Where less than 4 bytes are being transferred, the valid bytes are indicated by a data-valid bus shared between the UART and the DMAC. The UART can send 1, 2, 3, or 4 bytes of data per bus transaction. The actual number of bytes to be read is programmed in the DMAC along with the bus width.

18.4.2.4 DMA Receive Programming Errors

If the DMA channel stops prematurely due to the end of a Descriptor chain or other error, the ASR1606 must be notified since the DMAC can no longer service the UARTs FIFOs. If this occurs, the ASR1606 must correct the situation by programming another Descriptor or by servicing the FIFOs via interrupt or polling mode, as described above. There are 2 methods for notifying the ASR1606 of a stopped DMA channel:

- Program the DMAC to interrupt on the event of a stopped channel by setting DCSR[StopIrqEn].
- For the Receive channel, the UART interrupts with an end-of-Descriptor chain (EOC) interrupt if <Trailing Bytes> is set, such that the UART makes a DMA request to remove trailing bytes (see Removing Trailing Bytes In DMA Mode). Using the UART interrupt for the Receive channel is preferable to the DMA DCSR interrupt because extra logic exists to ensure that the UART EOC interrupt asserts only when necessary. For example, a UART EOC interrupt does not assert if the UART has completed the reception of its message (indicated by the character timeout timer) and the Receive FIFO is empty. The <DMA End of Descriptor Chain> field in the Interrupt Identification Register interrupt does not assert if <Trailing Bytes> is cleared.

18.4.2.5 DMA Error Handling

If an error occurs while in DMA mode, the Receive-DMA requests are disabled and the error interrupt, <Interrupt Source Encoded>, is generated.

The ASR1606 must now read out the error bytes through Programmed Input/Output (PIO). After all errors have been removed from the FIFO, the Receive DMA requests are once again enabled by the UART.

ASR CONFIDENTIAL

If an error occurs when the Receive FIFO trigger threshold has been reached such that a Receive DMA request is set, software must wait for the DMA to finish the transfer before reading out the error bytes through PIO. Otherwise, FIFO underflow could occur.

Note Ensure that the DMAC has completed the previous Receive DMA requests before the error-interrupt handler begins to clear the errors from the FIFO. Otherwise, FIFO underflow could occur.

18.4.2.6 Removing Trailing Bytes In DMA Mode

When the number of entries in the Receive FIFO is less than its trigger threshold and no additional data is received, the remaining bytes are called trailing bytes. Set <Trailing Bytes> to program the UART to make a DMA request to remove the trailing bytes. Setting <Trailing Bytes> also enables the <DMA End of Descriptor Chain> interrupt, as described in Removing Trailing Bytes In DMA Mode.

ASR CONFIDENTIAL

A request is issued automatically for the remaining number of bytes left in the Receive buffer when the DMAC is removing trailing bytes. The DMAC then empties the contents of the Receive buffer unless the DMA reaches the end of its Descriptor chain. If the DMA reaches the end of the Descriptor chain while removing trailing bytes, the ASR1606 is forced to take over because the DMAC can no longer service the UART request until a new chain is linked. In this situation, the UART sets <DMA End of Descriptor Chain> if data exists in the Receive FIFO, and if <Receiver Time-out Interrupt Enable> is set, it also sets the <Time Out Detected> field in the Interrupt Identification Register. The remaining bytes must then be removed using Processor I/O mode as described in FIFO Interrupt Mode Operation..

18.4.2.7 False EOR Due to Character Time-out Expiration

It is possible for a false EOR to be asserted by the UART in the middle of receiving a message if a pause in the remote data transmissions is long enough to cause the timeout counter to expire. This situation causes an EOR to be sent to the DMAC if in DMA mode. If this situation occurs, the EOR is applied to the last byte of data in the FIFO when the DMA responds to the EOR request. The EOR is not applied to the last byte in the FIFO at the time of the character timeout. Therefore, if remote transmission resumes before the DMA responds to the EOR request, the EOR flag is applied to the new data that entered the FIFO and not to the last byte in the FIFO at the time of the character timeout.

18.4.2.8 EOR Must be Serviced Prior to Transmission of New Message

A caveat to this behavior could be encountered under legitimate EOR situations: for example, if Message A ends with 3 bytes in the FIFO, an EOR request is made to the DMAC to remove these bytes. If transmission of a new Message B resumes before the DMAC responds to the EOR request of Message A, the EOR could be applied to the first byte of Message B if this byte is written into the FIFO before the DMAC responds to message A's EOR request. Although this situation could occur, it would be considered a programming error because the higher communication protocol must prevent Message B transmission until the local receiver acknowledges the receipt of Message A. The exception to this scenario would be if enough new bytes enter the FIFO to push the FIFO level to its programmed data threshold. If this situation occurs, the request is treated as a normal service request and no EOR flag is asserted to the DMAC.

18.4.3 Auto-Flow Control

Auto-flow control uses the Clear to Send (CTSn) and Request to Send (RTSn) signals to automatically control the flow of data between the UART and external modem. When auto-flow is enabled, the remote device is not allowed to send data unless the UART asserts (that is, sets to 0) RTSn. If the UART de-asserts

ASR CONFIDENTIAL

(that is, sets to 1) RTSn while the remote device is sending data, the remote device is allowed to send 1 additional byte after RTSn is de-asserted. An overflow could occur if the remote device violates this rule. Likewise, the UART is not allowed to transmit data unless the remote device asserts CTSn (that is, sets to 0). ASR recommends using this feature because it increases system efficiency and eliminates the possibility of a Receive-FIFO-overflow error due to long interrupt latency.

Auto-flow mode can be used in 2 ways:

- full auto-flow, automating both CTSn

and RTSn and

- half auto-flow, automating only CTSn.

Set the <Request to Send> and <Auto-flow Control Enable> fields in the Modem Control Register to enable full auto-flow. Set <Auto-flow Control Enable> and clear <Request to Send> to enable auto-CTSn-only mode.

ASR CONFIDENTIAL

18.4.3.1 RTSn (UART Output)

When in full Auto-flow mode, RTSn is asserted (0) when the UART FIFO is ready to receive data from the remote transmitter. This scenario occurs when the amount of data in the Receive FIFO is below the programmable trigger threshold value. RTSn is de-asserted (set to 1) when the amount of data in the Receive FIFO reaches the programmable trigger threshold. It is asserted again when enough bytes are removed from the FIFO to lower the data level below the trigger threshold.

18.4.3.2 CTSn (UART Input)

When in full- or half-Auto-flow mode, CTSn is asserted (set to 0) by the remote receiver when the receiver is ready to receive data from the UART. The UART checks CTSn before sending the next byte of data and does not transmit the byte until CTSn is low. The transmitter completes this byte if CTSn goes high while the transfer of a byte is in progress.

If UART transmission is stalled by disabling the UART, none of the interrupts in the Modem Status Register indicate an interrupt when CTSn re-asserts because disabling the UART also disables interrupts. ASR recommends using auto-CTS in Auto-flow mode.

Note Auto-flow mode can be used only in conjunction with FIFO mode.

18.4.4 Auto-Baud-Rate Detection

Each UART supports auto-baud-rate detection. When enabled, the UART counts the number of clock cycles within the start-bit pulse. This number is then written into the Auto-Baud Count Register (as described in the *ASR1606 Registers*) and is used to calculate the baud rate. When the Auto-Baud Count Register is written, an auto-baud-lock interrupt is generated (if enabled), and the UART automatically programs the Divisor Latch Registers with the appropriate baud rate. If preferred, the ASR1606 can read the Auto-Baud Count Register and use this information to program the Divisor Latch Low Byte Register and Divisor Latch High Byte Register with a baud rate calculated by the ASR1606. After the baud rate has been programmed, the ASR1606 verifies that the predetermined characters (usually **AT** or **at**) are being received correctly.

If the UART is to program the Divisor Latch Registers, software can use either of 2 methods for auto-baud calculation: table-based and formula-based. The method is selected via the <ABT> field in the Auto-Baud Control Register. The baud

ASR CONFIDENTIAL

rates that are seen in most commercial electronics, which are referred to as "common," include:

- **Formula method**—any baud rate shown in Table 56 can be programmed by the UART. This method works well for higher baud rates, but it could fail below 28.8 kbps if the remote transmitter's actual baud rate differs by more than 1 percent of its target.
- **Table method**—is more immune to such errors, because the table rejects uncommon baud rates and rounds to the common ones. The table method allows any baud rate defined by the formula in Section 18.4.7, Programmable Baud-Rate Generator above 28.8 kbps. Below 28.8 kbps, the only baud rates that can be programmed by the UART are 19200, 14400, 9600, 4800, 1200, and 300 baud.

When the baud rate is detected, the auto-baud circuitry disables itself by clearing the <ABE> field in the Auto-Baud Count Register. To re-enable auto-baud detection, set the <ABE> field again.

ASR CONFIDENTIAL

Note Changing the baud rate is not permitted when actively transmitting or receiving data. Auto-baud-rate detection is not supported in IrDA (serial infrared) mode.

See the Auto-Baud Control Register as described in the *ASR1606 Registers* for more information on auto-baud.

18.4.5 32-Bit Peripheral Bus

Each UART supports an 8- (default) or 32-bit peripheral bus. If a 32-bit bus is preferred, set the <32-Bit Peripheral Bus> field in the FIFO Control Register. The bytes are written in Little Endian format (7:0) with byte 3 (the most recent byte) starting at bit [31], byte 2 starting at bit [23], and so on.

8-bit mode—only the least significant byte contains valid data on the peripheral bus. The upper 24 bits are ignored.

32-bit mode—the UART can read or write partial words of 1, 2, 3, or 4 continuous bytes from the peripheral bus. The method in which the valid bytes of data are determined differs depending on whether the transaction is being handled by the DMAC or PIO.

DMA—the DMAC can read or write 1, 2, 3, or 4 continuous bytes per word. The number of valid bytes available per word is determined internally between the DMAC and the UART.

PIO—the ASR1606 is restricted to reading or writing 1, 2, or 4 bytes per word. When reading, the ASR1606 must read the Receive FIFO Occupancy Register to retrieve the number of bytes available in the Receive buffer. If the number of bytes available is 4 or greater, the ASR1606 can request any number of bytes per word (except 3). If the number is less than 4, software must request the proper number of bytes. When 3 bytes are remaining, software must request either 2 bytes followed by 1 byte or 1 byte followed by 2 bytes. The UART retrieves unusable data for the non-valid bytes if the ASR1606 reads more than the number of bytes available in the Receive buffer. The Receive FIFO counters do not increment.

ASR CONFIDENTIAL

Note The Receive and Transmit FIFOs must be enabled when in 32-bit mode.

18.4.6 Serial Infrared Asynchronous Interface

The slow Serial Infrared (SIR) interface is used with all 4 UARTs to support two-way wireless communication that uses infrared transmission. The SIR interface provides a Transmit encoder and Receive decoder to support a physical link that conforms to the IrDA Serial Infrared Specification.

The SIR interface does not contain the actual IR LED driver or the Receive amplifier. The I/O pins attached to the SIR interface have only digital CMOS-level signals. SIR supports two-way communication, but full-duplex communication is not possible because reflections from the Transmit LED enter the receiver. SIR supports frequencies up to 115.2 kbps. The baud divisor must be 8 or more because the input clock is 14.857 MHz.

ASR CONFIDENTIAL

Note

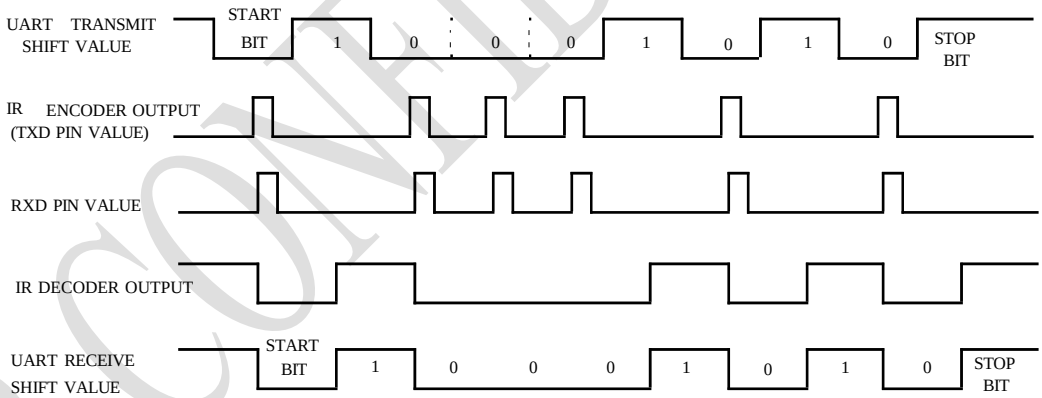
This SIR interface should be turned on and off only while the UART module is completely idle and disabled. On-the-fly changes are likely to produce problems and are not recommended.

The <Transmit Pulse Width Select> field in the Infrared Selection Register must be set to 1.

18.4.6.1 Operation

The SIR modulation technique works with 7- or 8-bit characters with an optional parity bit. The data is preceded by a zero-value start bit and ends with 1 stop bit. The encoding scheme sends a 1.6-µs pulse 3/16 of a bit wide in the middle of every zero-value bit and sends no pulses for bits with a value of 1. The pulse for each zero-value bit must occur, even for consecutive bits with no edge between them. Figure 98 shows an example of Transmit/Receive operation.

Figure 98:IR Transmit and Receive Example



The top line in Figure 98 shows an asynchronous transmission as it is sent from the UART. The second line shows the pulses generated by the IR encoder at the TXD pin. A 1.6 µs pulse is generated in the middle of the START bit and any data bit that is a zero. The third line shows the values received at the RXD input pin. The fourth line shows the Receive-decoder output. The Receive decoder drives the Receive data line low when it detects a pulse. The bottom line shows how the UART receiver interprets the decoder action. This last line is the same as the first, but it is shifted half a bit period. ASR recommends setting the <Transmit Pulse Width Select> field in the Infrared Selection Register for the Transmit pulse width. When the <Transmit Pulse Width Select> is clear, each zero bit has a pulse width of 3/16 of a bit time. The <Transmit Pulse Width Select> field

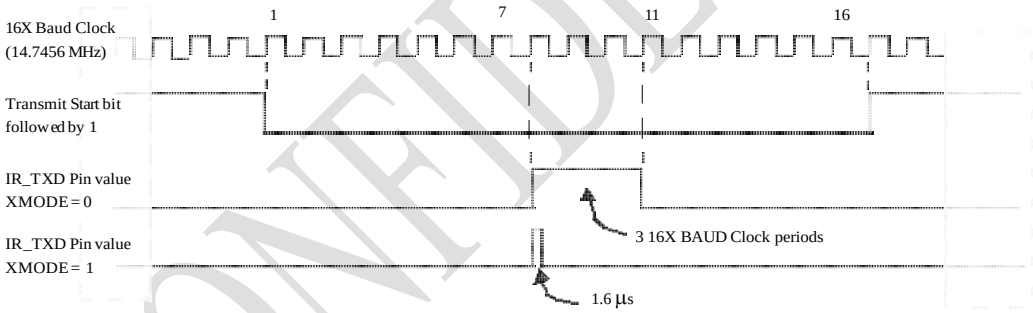
ASR CONFIDENTIAL

must be set to 0x1 so a pulse of 1.6 µs is generated in the middle of each zero bit. The shorter infrared pulse generated when the <Transmit Pulse Width Select> field is set reduces the LED power consumption. At 2400 bps, the LED is normally on for 78 µs for each zero bit that is transmitted. When the <Transmit Pulse Width Select> field is set, the LED is on for only 1.6 µs. Figure 98 shows an example of XMODE operation.

Note

<Transmit Pulse Width Select>=1 toggles only the Transmit pulse width. It does not affect the Receive pulse width, which is always set to XMODE = 1.

Figure 99:XMODE Example



To prevent transmitter LED reflection feedback to the receiver, disable the IR Receive decoder when the IR Transmit encoder transmits data, and disable the IR Transmit encoder when the IR Receive decoder receives data. RCVEIR and XMITIR must not be set at the same time.

Note

This restriction specifically is to ensure no feedback; no filtering is performed to remove this feedback data.

18.4.7 Programmable Baud-Rate Generator

Each UART contains a programmable baud-rate generator that can take a fixed-input clock and divide it down to generate the preferred baud rate. The baud rate is calculated by taking the 14.7456 MHz fixed-input clock (in regular mode (<High Speed UART Enable> field in the Interrupt Enable Register = 0)) or the 59.429 MHz clock (in high speed mode (<High Speed UART

ASR CONFIDENTIAL

Enable> = 1)) and dividing it by the Divisor Latch Low Register. For high speed mode, a divisor of 1 or 2 is required.

The baud-rate generator output frequency is 16 times the baud rate. Two 8-bit Divisor Latch Registers (Divisor Latch Low Register and Divisor Latch High Register as described in the *ASR1606 Registers*) store the divisor in a 16-bit binary format. Load these divisor latches during initialization to ensure that the baud-rate generator operates properly. The 16X clock stops if each Divisor Latch register is loaded with 0x0.

ASR CONFIDENTIAL

The RS-232 standard requires a clock frequency of 14.7456 MHz; however, the processor clock divisors allow only a frequency of 14.857 MHz, which results in a slightly higher baud rate value compared to the standard.

The baud rate of the data shifted into or out of a UART is given by the formula:
In high speed mode, (<High Speed UART Enable> = 1)

Note Only divisors of 1 and 2 are valid in high-speed mode.

In regular mode, (<High Speed UART Enable> = 0)

$$\textbf{BaudRate} = \frac{14.857\ \text{MHz}}{(16 \times \textbf{Divisor})}$$

For example: if the divisor is 24 (in regular mode), the baud rate is 38400 bps.

Note Changing the baud rate is not permitted when actively transmitting or receiving data.

Refer to Table 56 for a list of recommended baud rates based on divisor values (Divisor Latch High Byte Register:Divisor Latch Low Byte Register).

ASR CONFIDENTIAL

Table 56:Recommended Baud Rates

Required Baud Rate	Divisor	14.857 MHz Actual Baud Rate	59.429 MHz Actual Baud Rate
9600	96	9673	—
19200	48	19345	—
38400	24	38690	—
57600	16	58036	—
115200	8	116071	—
230400	4	232143	—
460800	2	464286	—
921600	1	928571	—
1842000	2	—	1857143
3686400	1	—	3714286

The divisor reset value is 0x0002. Changing the baud rate (writing to registers Divisor Latch Low Byte Register and Divisor Latch High Byte Register) is not permitted while actively transmitting or receiving data.

ASR CONFIDENTIAL

Copyright © 2022 ASR
Jan.24, 2022

18.5 *Register Descriptions*

Refer to *ASR1606 Registers* for details on the UART Controller registers.

ASR CONFIDENTIAL

19.General-Purpose I/O Unit

This section describes the operation of the General-Purpose Input/Output (GPIO) unit of the ASR1606.

- Section 19.1, Overview
- Section 19.2, Function Description
- Section 19.3, Software Guide
- Section 19.4, Register Descriptions

19.1 Overview

The ASR1606 provides GPIO ports for use in generating and capturing application-specific input and output. All ports are brought out of the device via the alternate function muxing. GPIO unit is in charge of GPIO ports control and status check. When programmed as an input, a GPIO port can also serve as an interrupt source. At the assertion of all resets, all GPIO ports are configured as inputs and remain inputs until they are configured either by the boot process or by user software.

The general features of the GPIOs include the following:

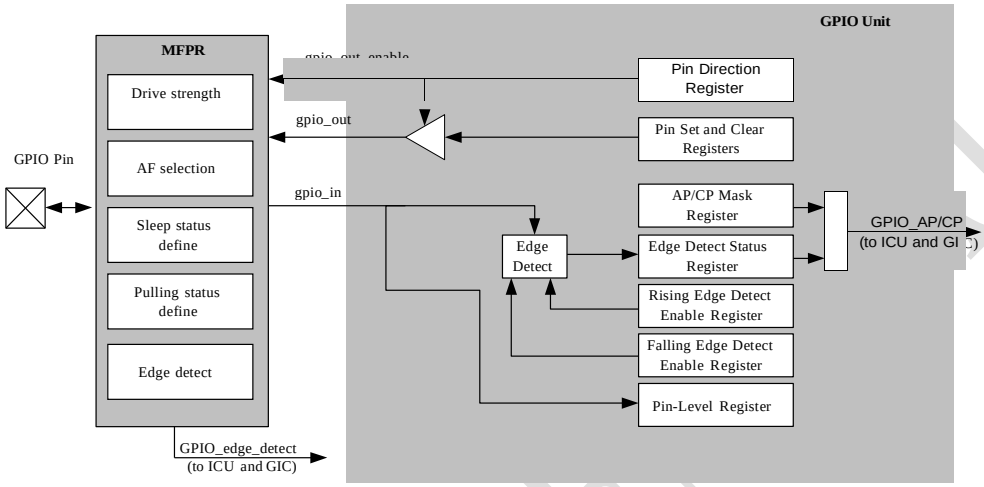
- As inputs, they can be programmed to generate an interrupt from a rising edge, a falling edge, or both.
- As outputs, they can be cleared or set individually.
- As inputs, the values can be read individually.

19.2 Function Description

Figure 100 indicates GPIO ports and GPIO unit connection.

ASR CONFIDENTIAL

Figure 100:General-Purpose I/O Block Diagram



By virtue of GPIO unit, software is able to set and check the status of a GPIO port. Please note that there are 3 interrupts relevant to GPIOs as shown in Table 57.

ASR CONFIDENTIAL

Table 57:GPIO Interrupts

ICU Inputs	Signal Name	Notes
Int Req [55]	GPIO_CP	Generated in GPIO unit
Int Req [50]	GPIO_edge_detect	Generated in MFPR, wake-up source
Int Req [49]	GPIO_AP	Generated in GPIO unit

All of the interrupts may be generated when GPIO input edge is detected. However, they are generated in different modules. GPIO_edge_detect is generated in MFPR which is in always on domain and can be used as wake-up source. The other 2 interrupts are generated in GPIO unit which is shut down when chip is in sleep mode. As a result, GPIO_AP and GPIO_CP can not be used as wake-up sources. Furthermore, GPIO_AP and GPIO_CP have their corresponding bit masks which allows software to mask a bit or not, while GPIO_edge_detect do not have its bit mask.

19.3 Software Guide

The GPIO logic is best thought of as a logical function only that gives software the ability to configure and examine the value of a port, but does not control the selection of which logical function is connected to the actual physical pins.

For a GPIO port multiplexed on a particular multi-function I/O pin to have effect, 2 conditions must be met:

- The GPIO alternate function (sometimes referred to as a GPIO_REG alternate function) must be selected by using the MFPRx registers in the pad configuration registers.
- The GPIO unit registers must be configured correctly.

It is important to grasp the operation of both the GPIO control logic and the pin control logic to understand the operation of the GPIOs in the ASR1606. For detailed information regarding configuring the pins as GPIOs, refer to the GPIO registers in the *ASR1606 Registers*.

The direction of the GPIO ports is controlled by writing to the GPIO Pin Direction Register. When programmed as an output, the port can be set by writing to the GPIO Pin Output Set Register and cleared by writing to the GPIO Pin Output Clear Register. The set and clear registers can be written regardless of whether the port is configured as an input or an output. If a port is configured as an input, the programmed output state takes effect when the port is reconfigured as an output.

The value of each GPIO port can be read through the GPIO Pin-Level Register. This register can be read at any time and can confirm the port state for both input and output configurations. In addition, each GPIO port can be programmed to detect a rising and/or falling edge through the GPIO
Rising-Edge Detect Enable Register and GPIO Falling-Edge Detect Enable Register. The state of the edge-detect can be read through the GPIO Edge Detect Status Register. These edge-detects can be programmed to generate interrupts.

ASR CONFIDENTIAL

Note

The GPIO port does not control the Pullup and Pulldown state of an I/O pin even when the GPIO function is selected for this pin. Refer to Pad Configuration registers for information on how to control the Pullup and Pulldown states.

The GPIO information described in this section applies ONLY to the GPIO alternate function. Still, it is possible for a system to use the GPIO functions internally and not actually connect to a physical pin.

ASR CONFIDENTIAL

19.4 Register Descriptions

Refer to *ASR1606 Registers* for details on the General Purpose I/O Unit registers.

ASR CONFIDENTIAL

20.Audio codec

This chapter describes the audio codec of the ASR1606.

- Section 20.1, Overview
- Section 20.2, Features
- Section 20.3, Block Diagrams
- Section 20.4, Audio Front-End Performance
- Section 20.5, Control & Interface
- Section 20.6, Audio codec Operation
- Section 20.7,Register Descriptions

20.1Overview

The audio codec include audio DAC/ADC,audio DFE ,8KB audio buffer and other device such as ADMA ,I2S. As play back, the audio data is prepared by DSP or external DMA and stored to audio buffer ,then send out though DAC path by the internal ADMA. When recording ,the audio data is sampled and received though ADC adc path ,the received data is also stored in audio buf and then moved by DSP or external DMA .

20.2Features

- Integrated High quality audio codec and audio front-end

  ADC: 90dB SNR@20~20kHz

  DAC: 95dB SNR@20~20kHz

  (Class-AB): THD<-85dB@32-ohm loading
- Single MICs input
- Audio content sampling rates: 8kHz to 48kHz
- Quad vocoders for adaptive multi-rate (AMR)
- Noise suppression and echo cancellation
- Digital side tone generator with programmable gain
- Voice power amplifier with programmable gain

20.3Block Diagrams

Figure 101 shows the block diagram of audio codec.

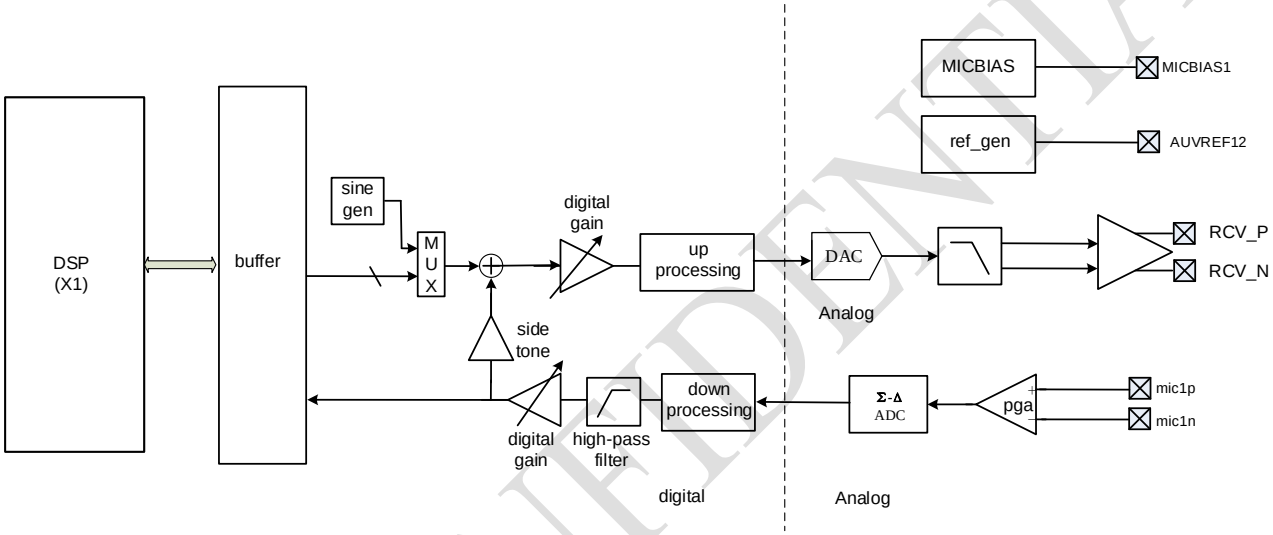
- Input Paths: the audio codec has a mono 16bit audio ADC, it supports 1

ASR CONFIDENTIAL

analog MICs input.

- Output path: the audio codec has a mono audio DAC with over 100dB dynamic range. It support a mono differential RCV output(RCV_P/_N), which can directly drive 32-ohm loading or support external Class-D amplifier.
- Digital gain control and side tone generator.

Figure 101: Audio Codec Block Diagram



20.4 Audio Front-End Performance

Table 58: Analog Microphone input to ADC performance

The following applies unless otherwise noted: AVDD_AUD=1.8V, T=25℃.

parameter	condition	Min	Typ	Max	Units
System Clock			6.144		MHz
Dynamic Range	Measured as THD+N at 60dB below full scale, 0dB PGA gain,20~20KHz		91		dB
Peak-SNDR	-4dBFS output, at 0dB gain		86		
THD	-1dBFS output, at 0dB gain		80		dB
PSRR	From AVDD18_AUD,Fin=217Hz,to digital output		80		dB
PGA+ADC gain options	GAIN= (-6dB, -2.5dB, 0dB, 3.5dB, 6dB, 8.5dB, 12dB, 14.5dB, 18dB, 20.5dB)		12		dB

Table 59: General Audio Rx Path Feature Summary

ASR CONFIDENTIAL

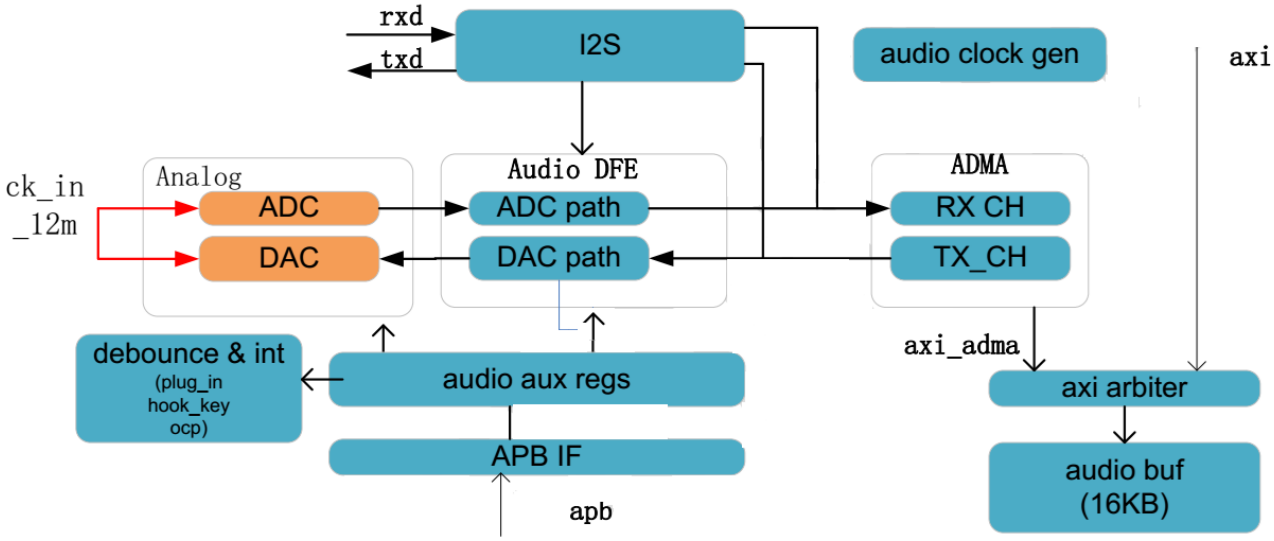
The following applies unless otherwise noted: AVDD_AUD=1.8V, T=25℃.

Feature	Details	DR(Typ)	THD+N(Typ)	Max POWER(1%THD)
DAC	RL=10KΩ	101dBA	-96dB(@vout -2dBv)	1.59Vp
Class-AB	Mono,32Ω, differential	100dBA	-90dB(0.00316%) (@20mW output)	37mW

20.5Control & Interface

the audio aux regs controls the audio subsytem such as ADC/ADC configuration , the clock reset control, the interrupt control and indicates the related status of the subsystem.The aux register are read or write through the apb bus.There is an audio buffer inside to store the audio data temporarily. the data is sent or received buy DMA. There are two data path for both tx and rx, one is though audio DFE ,the other is though I2S. the debounce & int model is responsible for the analog signal debounce and some interrupt detection such as plug in/out , hook key detection.

Figure 102: Audio Codec interface Diagram



20.6Audio Codec Operation

20.6.1 clocks and resets

To initial the audio codec , software must enable the audio subsytem clock by open the 12.28Mhz clock and 245.6Mhz clock from system PLL. For the clocks

Copyright © 2022 ASR

inside the audio subsystem,software should enable the clock of audio DFE, ADMA ,I2S.

20.6.2 interrupt and low power

There are several interrupts that software need care, such as aud_plug_int, hook_key_int, ocp_int, adma_tx_int ,adma_rx_int. these interrupt can both route to CR5 core and DSP.

The audio codec and keep working when the chip under lowpower mode. when the chip under low mode ,the audio interrupt can wake up the CR5 or DSP for event handling or audio data processing.

20.6.3 Program tips

Software can allocate pingpong buffer for audio play back and record.For example, the adma fetching data from ping buffer, and the dsp is storing data to pong buf at the same time.when all the data in ping buf are fetched and the data in pong buffer are ready(the dsp storing much faster than adma fetch), the adma switches to pong buffer and dsp switches to ping buffer.by using the pingpong buffer ,the audio tx processing will not be suspended when the preparing the data.

when initializing the code ,it is recommended that set the analog ADC/DAC related registeres first and then set the DFE registered and enable the ADMA at last.

20.7Register Descriptions

Refer to *ASR1606 Registers* for details on the audio control /adma /i2s registers.

ASR CONFIDENTIAL

21.SD Host Controller

The SD Host Controller is a hardware block that acts as a host of the SD/MMC bus to transfer data between SD memory, and MMC, SDIO, and CE-ATA cards and the internal bus master. This section describes the following information:

- Section 21.1, Features
- Section 21.2, Signal Descriptions
- Section 21.3, SD/MMC Bus Protocol Description
- Section 21.4, Special Bus Transactions
- Section 21.5, Card Detection
- Section 21.6, SPI Mode
- Section 21.7, MMC Mode
- Section 21.8, CE-ATA Mode
- Section 21.9, Register Descriptions

One side of this block interfaces with an internal bus. The internal interface access can be programmed in 3 modes:

- **ADMA mode.** This block acts as an internal master that accesses the DMA descriptor list and performs the complete DMA according to the descriptor list (scatter gather list).
- **SDMA mode.** This block acts as an internal master which performs data transfer as programmed and generates an interrupt after completion of each data transfer.
- **PIO mode.** The CPU accesses the data through the internal bus. This interface is most likely used for debugging purposes.

The SD Host Controller block diagram is shown in Figure 103.

21.1Features

- SD 3.0 and MMC/eMMC 4.5.1 specification compliant with support for 3 sockets
- Supports PIO mode and SDMA mode data transfer
- Supports ADMA1 and ADMA 2 (32-bit addressing) data transfer
- Supports 1-bit/4-bit SD memory and SDIO
- Supports 1-bit/4-bit MMC and CE-ATA cards
- SPI mode supported for both SD and MMC cards
- Following modes are supported:
  - Default Speed (DS)–up to 25 MHz, 3.3V signaling
  - High Speed (HS)–up to 50 MHz, 3.3V signaling
  - HS200–up to 200 MHz for eMMC device, 1.8V signaling

ASR CONFIDENTIAL

  - SDR12–SDR up to 25 MHz, 1.8V signaling
  - SDR25–SDR up to 50 MHz, 1.8V signaling
  - SDR50–SDR up to 100 MHz, 1.8V signaling
  - SDR104–SDR up to 208 MHz, 1.8V signaling
  - DDR50–DDR up to 50 MHz, 1.8V signaling
- Supports read-wait control in SDIO cards
- Supports suspend resume in SDIO cards
- Hardware generation/checking of CRC on all command and data transaction on the card bus

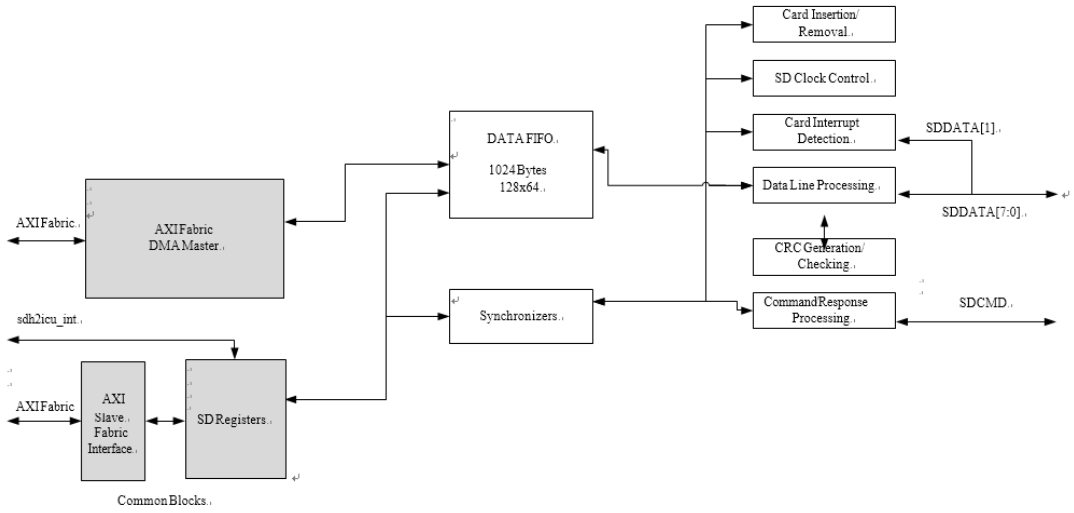
ASR CONFIDENTIAL

- Card insertion/removal detection for sockets 1

Refer to the AXI Fabric Specification for more information on terminology or concepts described below.

- SD AXI master always generates an INCR (AWBURST/ARBURST = 0x1) burst and 8-bytes burst size (AWSIZE/ARSIZE = 0x3).
- SD AXI master bursts are normal, non-secure data accesses (AWPROT/ARPROT = 0x2).
- SD AXI master never generates atomic accesses (AWLOCK/ARLOCK = 0x0).
- SD AXI slave does not support "WRAP" burst type.
- SD AXI slave does not support exclusive, protected, or atomic accesses. The response signaling is always "OKAY" (RRESP/BRESP = 0x0) or "DECERR" (RRESP/BRESP = 0x2). The "DECERR" response is signalled if an address is accessed where no registers exist.

Figure 103:SD Host Block Diagram



21.2Signal Descriptions

Table 60 defines the SD signals used in port 1 of the SD Host Controller.

Table 60:SD Host Signal Descriptions

Signal	Type	Description
MMCx_DAT[3]	I/O	MultiMediaCard Data 3 MMC and SD: Bi-directional line for Read and Write data MultiMediaCard Chip Select 1 SPI: Chip select 1

Copyright © 2022 ASR

MMCx_DAT[2]	I/O	MultiMediaCard Data 2 MMC and SD: Bi-directional line for Read and Write data MultiMediaCard Chip Select 0 SPI: Chip select 0
MMCx_DAT[1]	I/O	MultiMediaCard Data 1 MMC and SD: Bi-directional line for Read and Write data SDIO: Signals an interrupt condition to the controller
MMCx_DAT[0]	I/O	MultiMediaCard Data 0 MMC and SD: Bi-directional line for Read and Write data SPI: Input for response token and read data
MMCx_CMD	I/O	MultiMediaCard Command MMC and SD: Bi-directional line for command and response tokens SPI: Output for command and write data
MMCx_CLK	I/O	MultiMediaCard and SD Card Bus Clock
MMC1_WP	I	Write Protect
MMC1_CD	I	Card Detect

21.3SD/MMC Bus Protocol Description

Communication over the SD bus is based on commands and data bit streams that are initiated by a start bit and terminated by a stop bit.

- Command: A command is a token that starts an operation. It is sent from the host to the card(s) and is transferred serially on the SDCMD line (1 bit).
- Command Response: A command response is a token that is sent from an addressed card to the host as an answer to a previously received host command. It is transferred serially on the SDCMD line since the SDCMD line is a bidirectional signal.
- Data: There are 4 data lines. Data can be transferred from the card to the host or vice versa. Data is transferred via Data line, and they are bidirectional signals. In 1-bit mode, SDDATA[0] is used, and the other is in Z-state.

Figure 104, Figure 105, and Figure 106 show basic operations of the SD cards.

ASR CONFIDENTIAL

Figure 104: “No Response” and “No Data” Operation

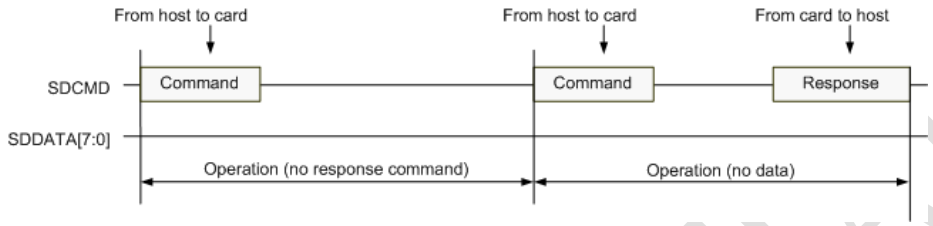


Figure 105:Multiple Block Read Operation

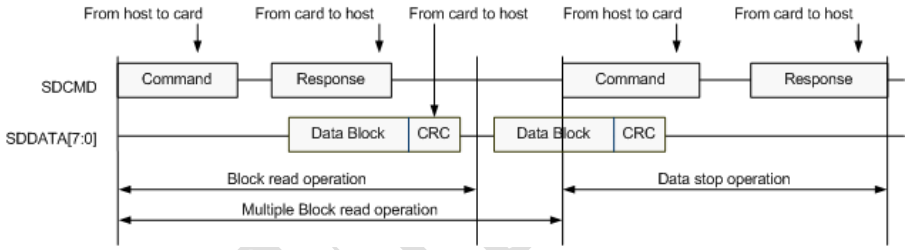
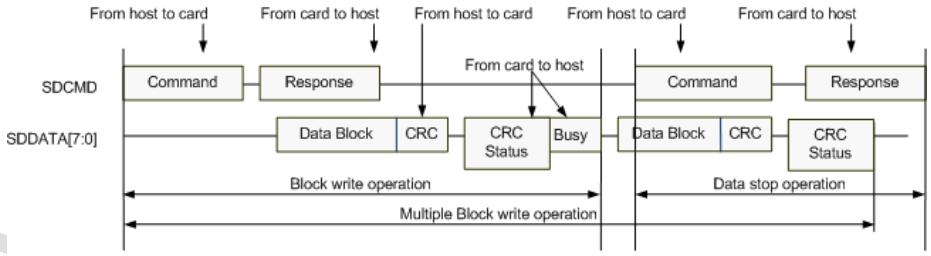


Figure 106:Multiple Block Write with Card Busy Operation



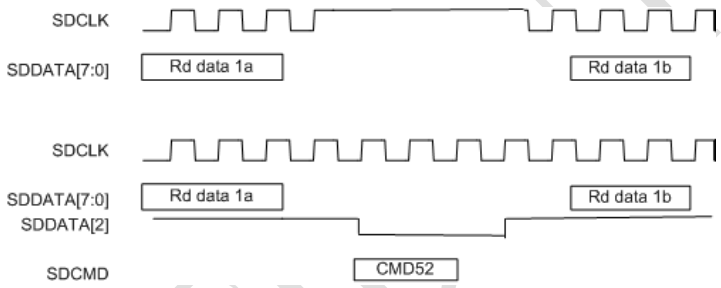
ASR CONFIDENTIAL

21.4Special Bus Transactions

21.4.1 Read Wait Command

The host usually stops the clock to stop the Read data output from the card whenever the host cannot accept any more data. During the clock stop, the host capabilities are limited as it cannot issue commands during a Read. A Read Wait command allows the host to stop the Read data while the clock is still on. A Read Wait command is issued after the data block end.

Figure 107:Read Wait Controlled by Stopping Clock



ASR CONFIDENTIAL

21.4.2 Packet Format

Figure 108, Figure 109, Figure 110, and Figure 111 show the formats of commands from the host and responses from cards and data.

Figure 108:Command Token Format

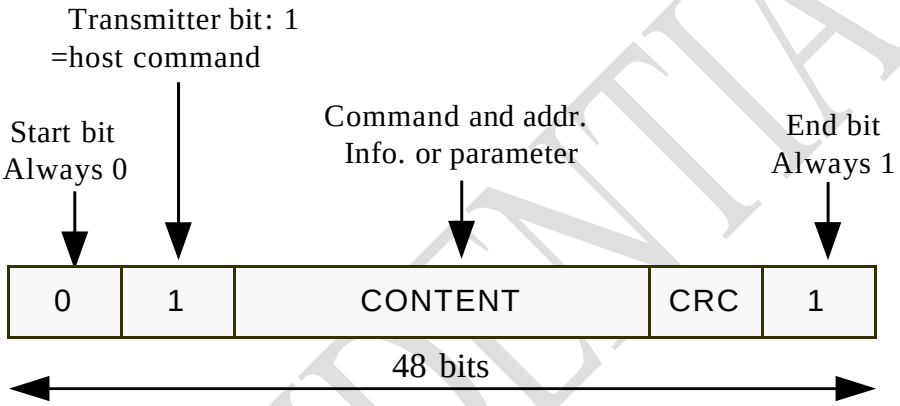
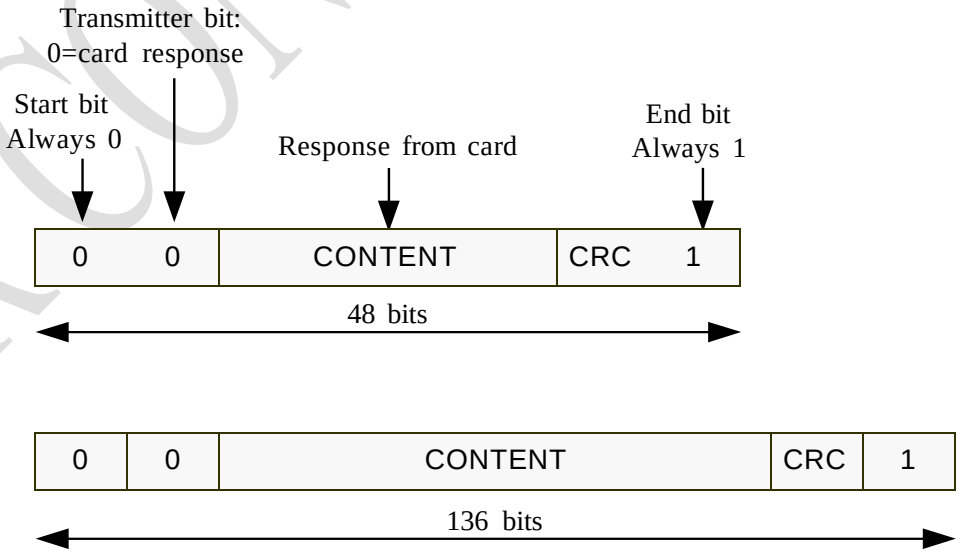


Figure 109:Response Token Format



ASR CONFIDENTIAL

Figure 110:Data Packet Format, Standard Bus (Only SDDATA[0] Used)

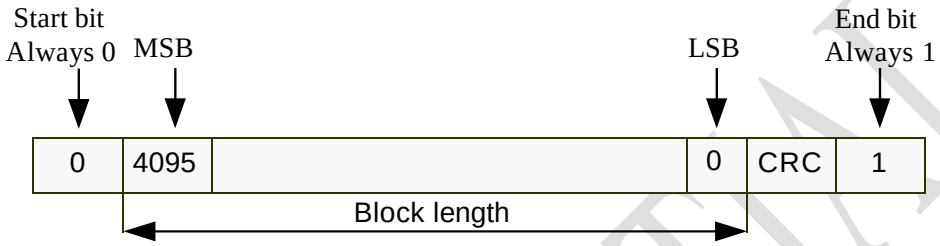
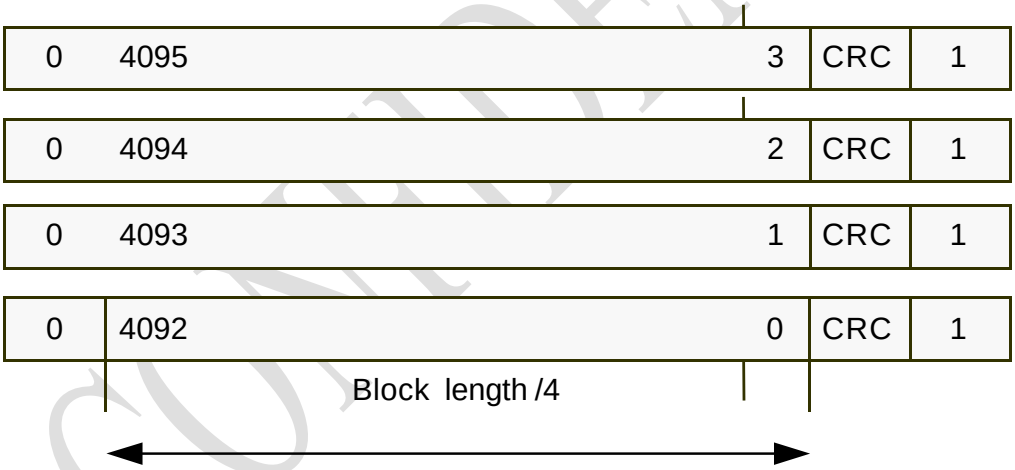


Figure 111:Data Packet Format, Wide Bus (All Four Data Lines Used)



21.4.3 Sequences of Host and Card Interaction

SD cards are connected to the host with a dedicated interface, such as its own SDCLK, SDCMD, or SDDATA[3:0] lines.

The host and SD cards go through 2 phases after power on reset, software reset, or if a new card is plugged in:

- Card Identification Phase: The host looks for new cards on the bus. While in this phase, the host resets all the cards that are in card identification mode. Any card already identified will not be reset. Then the host sends the command to validate operation voltage range, identifies cards and asks them to publish the Relative Card Address (RCA). This operation is done to each card separately on its own SDCMD line in the case of SD memory,. All data communication in the Card Identification phase uses the SDCMD line only.

ASR CONFIDENTIAL

- Data Transfer Phase: The host enters this phase after identifying all the cards on the bus. In this phase, the host is ready to transfer data.

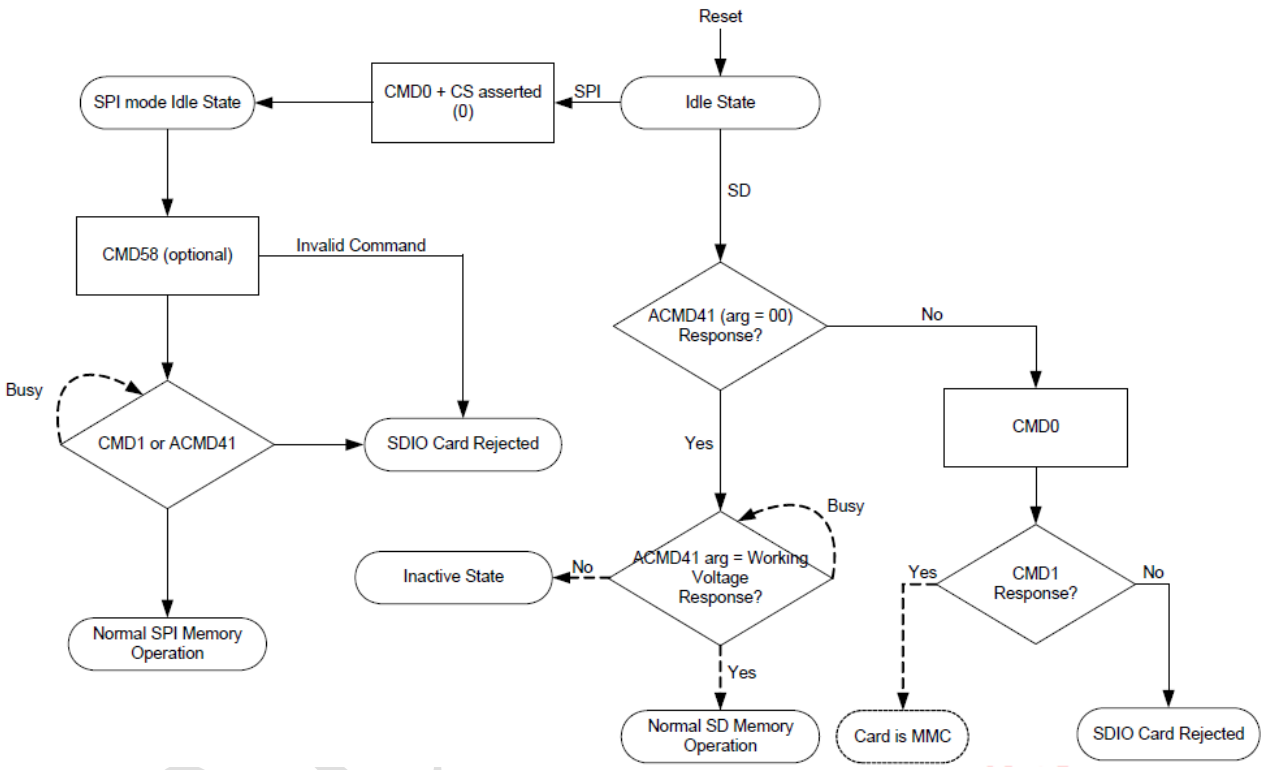
After the host driver sets up the Host Controller, it starts the transfer by writing to the Command Register, which is written last.

ASR CONFIDENTIAL

For R1b commands, Command Inhibit will stay set as long as the busy signal is held low. CMD_ COMPLETE will then be set when Command Inhibit changes from 1 to 0.

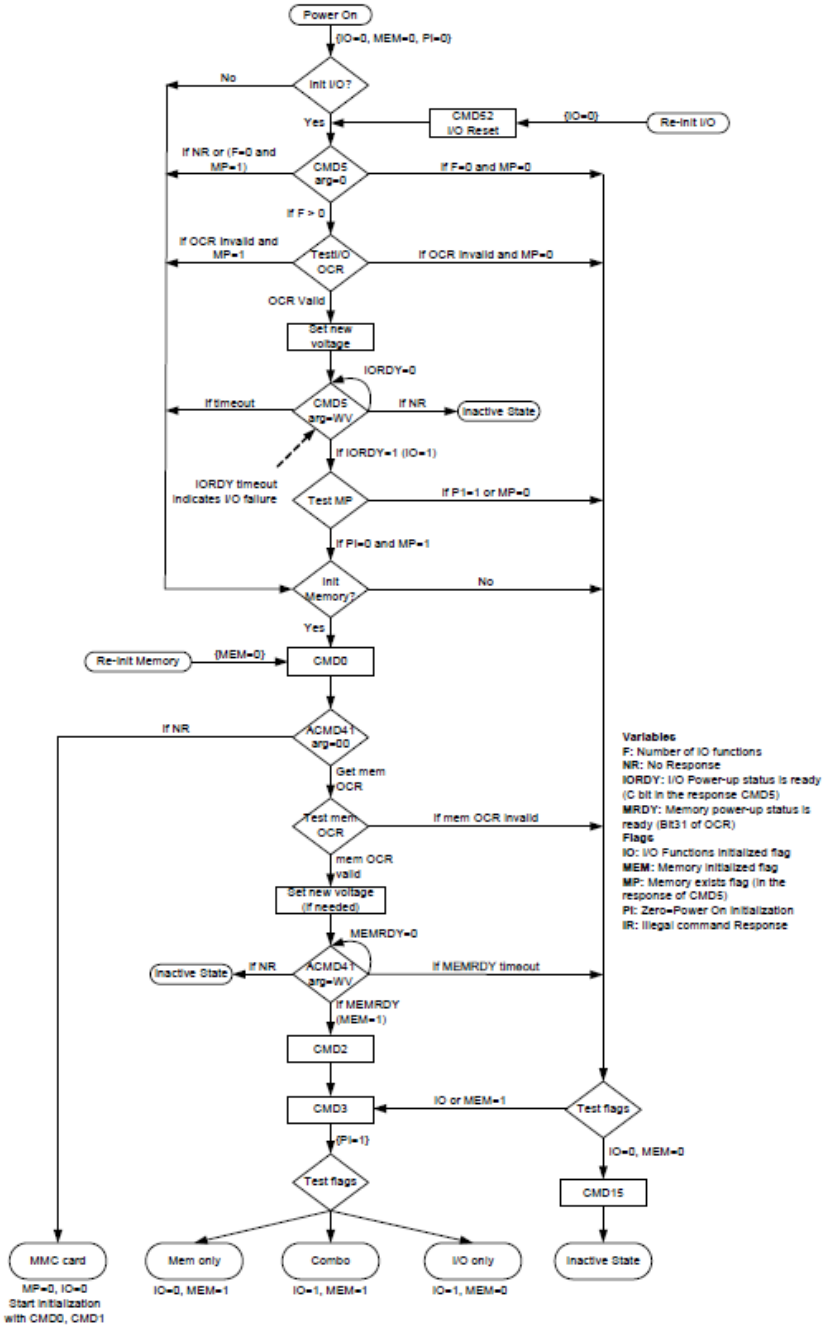
The host initialization flow charts are shown in Figure 112, Figure 113, and Figure 114.

Figure 112:Host Initialization Flow Chart–SDIO Response to non-I/O Aware Initialization



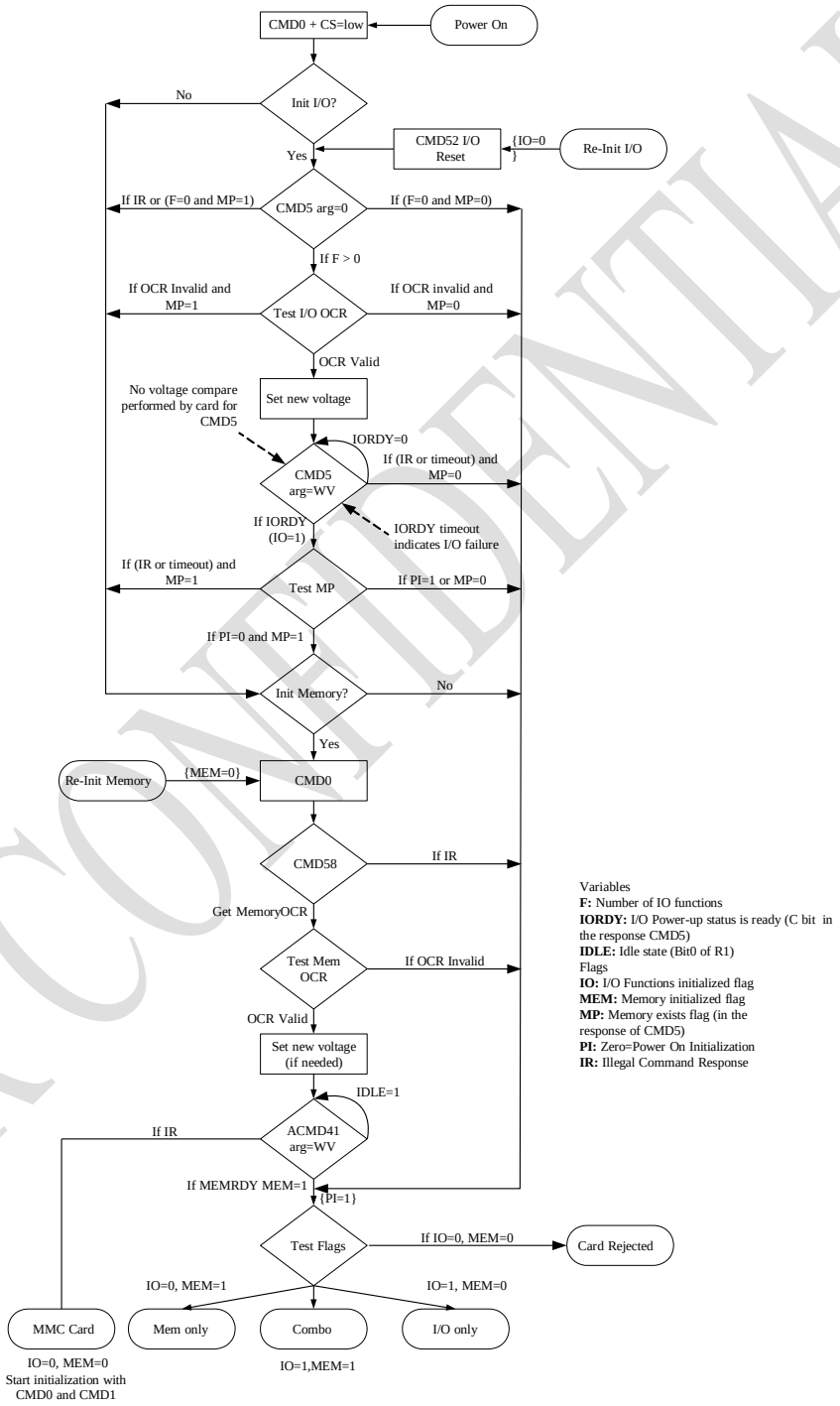
ASR CONFIDENTIAL

Figure 113:Host Initialization Flow Chart–Card Initialization Flow in SD Mode



Copyright © 2022 ASR

Figure 114:Host Initialization Flow Chart–Card Initialization in SPI Mode



ASR CONFIDENTIAL

Note For more information on SD, refer to the SD Specifications Part A2 SD Host Controller Simplified Specification Version 2.00, February 8, 2007.

ASR CONFIDENTIAL

21.5Card Detection

The ASR1606 design supports card detection (insertion/removal) based on the card-detect switch level on the SD socket. The card detection features can be accessed though the Normal Interrupt Status Enable bits. If enabled, an interrupt is generated when a logic change is detected on the card-detect switch input. This logic change is debounced before generating an interrupt.

21.6SPI Mode

SPI mode is enabled by setting the <spi_en> field in the SPI Mode Register (0xE4). The rest of the flow is similar to the SD flow except for error interrupts. If an error occurs in the SPI mode then the
<spi_err> field in the Error Interrupt Status Register. The cause of this error can be seen in the
<spi_err_token> field in the SPI Mode Register.

21.7MMC Mode

This mode is enabled by setting the <mmc_card> field in the CE-ATA Register 2 (0xEA). The rest of the data flow is similar to the regular SD mode. If 8-bit data width setting is required, the <mmc_ width> field in the CE-ATA Register 2 should be set to 0x1. This would override the <Data_width> field in the Host Control Register.

21.8CE-ATA Mode

This mode is enabled by setting the <mmc_card> and <ceata_card> field in the CE-ATA Register 2 (0xEA). The command completion checking, disabling are all in this CE-ATA Register 1 (0xE8) and CE-ATA Register 2 (0xEA). The rest of the data flow is similar to a MMC card. If a command completion timeout occurs, then the <cpl_timeout_err> field in the Error Interrupt Status Register is set to 0x1.

21.9Register Descriptions

Refer to the *ASR1606 Registers* for details on the SD Host Controller registers.

ASR CONFIDENTIAL

22.Resistive Touch Screen Controller

This section describes the following:

- Section 22.1, Introduction
- Section 22.2, Feature List
- Section 22.3, Overview
- Section 22.4, Function description

22.1 Introduction

TSC is Resistive Touch Screen Controller which can detect press/rise action on 4-wire resistive touch screen and collect position coordinates of the touch point.

A 4-wrie touch screen has 4 pins: XL, XR, YD and YU. XL and XR are the pins of X direction. YD and YU are the pins of Y direction. TSC can apply a voltage to one direction and measure the voltage of the other direction with the on-chip ADC

22.2 Feature List

- Supports press/rise detection and can generate corresponding interrupts.
- Supports automatic measurement and collection of X/Y coordinates.
- Supports configuration of dot count collected each time in succession.

Copyright © 2022 ASR

22.3 Overview

Figure 115 TSC block diagram

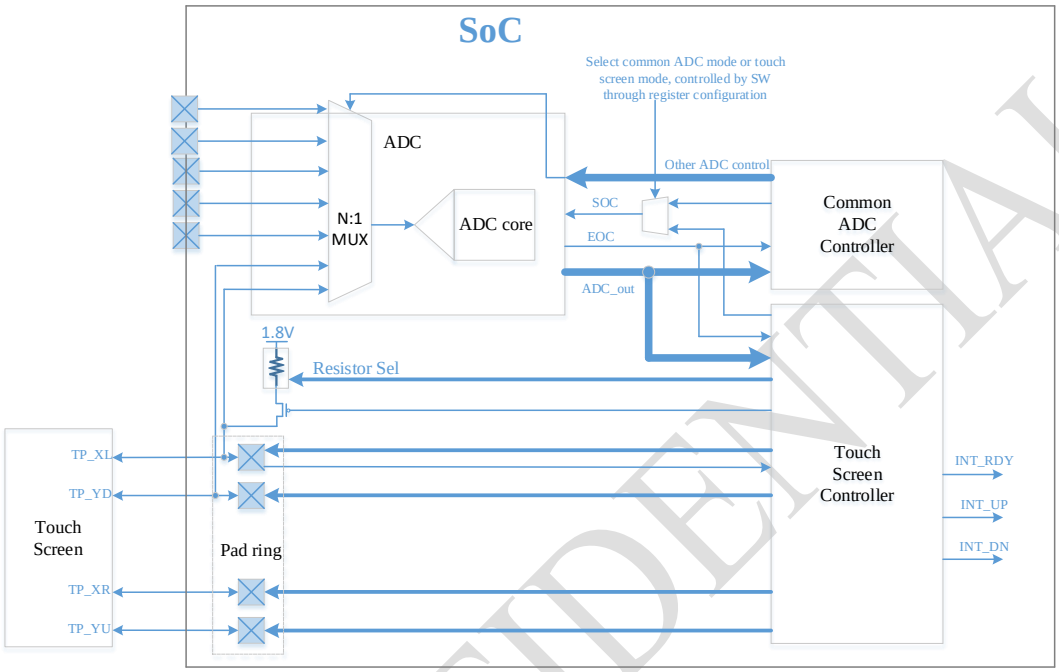


Figure115 is the block diagram of TSC in SoC. As shown in the figure, SoC has four GPIOs (XL, XR, YD and YU) dedicated for 4-wire touch screen. If touch screen is not used in a board-level system, these four pins can also be used as common GPIOs through MFPR configuration. SoC's on-chip ADC can be controlled by either Common ADC Controller or TSC. Their control signals are muxed and the selection signals are controlled by a register bit. On-chip ADC has 7 input channels. 5 (inaux1~5) of them are off-chip inputs and 2 (inaux6~7) are on-chip inputs. inaux6~7 can be used to measure the voltage of XL and YD and this procedure is controlled by TSC. Then the voltage value will be collected by TSC and SW can transform them to position coordinates. When press/rise action is detected or voltage values are ready in TSC, corresponding interrupt signals are generated to inform CPU to handle these events.

22.4 Function description

22.4.1 Register Description

Table 61: TSC Control Register

TSCR	bits	R/W	Description	Default value
VOL_APLY_DLY	[31:24]		Clock cycles of voltage applied to touch screen before the start of conversion of ADC	0
XL_PU_RES_SEL	[23:21]		resistance value selection of pull-up resistor for XL	0
YD_CHL_SEL	[20:16]	rw	ADC input channel selection for YD	0
XL_CHL_SEL	[15:11]	rw	ADC input channel selection for XL	0

ASR CONFIDENTIAL

	[10]		Reserved	
XL_PULL_UP_EN	[9]	rw	XL pull-up resistor enable 0 = disable 1 = enable	0
INT_RDY_EN	[8]	rw	Dot collection completion interrupt enable 0 = disable 1 = enable	0
INT_UP_EN	[7]	rw	Rise-up action detection interrupt enable 0 = disable 1 = enable	0
INT_DN_EN	[6]	rw	Press-down action detection interrupt enable 0 = disable 1 = enable	0
DOT_CNT	[5:3]	rw	Dot count of collection in succession Dot count = DOT_CNT + 1	000
TSC_MODE	[2:1]	rw	TSC work mode 00 = NOP 01 = wait for press-down 10 = wait for rise-up 11 = automatic measurement and collection of X/Y coordinates（writing this value will trigger automatic measurement） *note：mode switch is controlled by SW*	00
TSC_EN	[0]	rw	TSC enable 0 = on-chip ADC is controlled by Common ADC Controller completely 1 = on-chip ADC's conversion procedure is controlled by TSC	0

Table 62: TSC Status Register

TSSR	bits	R/W	Description	Default value
	[31:9]		Reserved	
INT_RDY_STA	[8]	rw	Dot collection completion interrupt status 0 = no interrupt 1 = interrupt happens W1C；W0 has no effect	0
INT_UP_STA	[7]	rw	Rise-up interrupt status 0 = no interrupt 1 = interrupt happens W1C；W0 has no effect	0
INT_DN_STA	[6]	rw	Press-down interrupt status 0 = no interrupt 1 = interrupt happens W1C；W0 has no effect	0
	[5:1]		Reserved	-
UPDOWN	[0]	ro	status in "wait for press-down" or "wait for rise-up"	1

Copyright © 2022 ASR

			modes 0 = press-down 1 = rise-up	

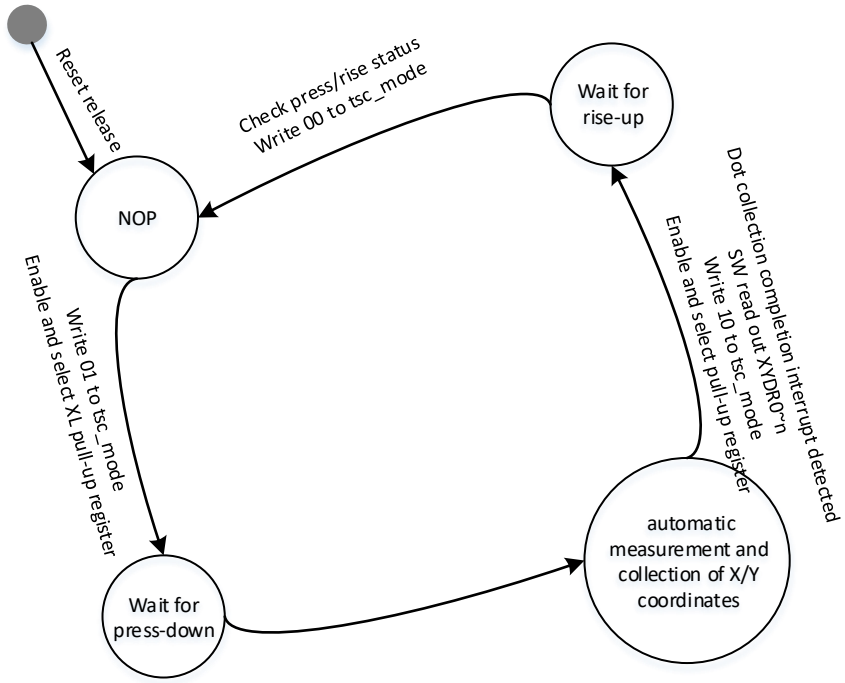
Table 63: X/Y Voltage Data Register0~7(XYDR0~7)

XYDR	bits	R/W	Description	Default value
Y_DAT	[31:16]	ro	Y voltage value(ADC output)	-
X_DAT	[15:0]	ro	Y voltage value(ADC output)	-

ASR CONFIDENTIAL

22.4.2 SW State Machine Design

Figure 116 SW State Machine Design



Copyright © 2022 ASR

23.RF Transceiver

This chapter describes the RF transceiver of the ASR1606.

- Section 23.1, Functional Description
- Section 23.2, Electrical Specfications

23.1 Functional Description

23.1.1 Overview

The ASR1606 RF transceiver is composed of two main functional sections: LTE transmitter section and LTE receiver section. Each of these sections is described below.

The ASR1606 supports the operating bands shown in Table 64.

Table 64: LTE Operating Bands

E-UTRA Operating Band	Uplink (UL) Operating Band BS Receive UE Transmit	Downlink (DL) Operating Band BS Transmit UE Receive	Duplex Mode
	F_{UL_low} to F_{UL_high}	F_{DL_low} to F_{DL_high}	
1	1920MHz to 1980MHz	2110MHz to 2170MHz	FDD
2	1850MHz to 1910 MHz	1930MHz to 1990MHz	FDD
3	1710MHz to 1785MHz	1805MHz to 1880MHz	FDD
4	1710MHz to 1755MHz	2110MHz to 2155MHz	FDD
5	824MHz to 849MHz	869MHz to 894MHz	FDD
6	830MHz to 840MHz	875MHz to 885MHz	FDD
7	2500MHz to 2570MHz	2620MHz to 2690MHz	FDD
8	880MHz to 915MHz	925MHz to 960MHz	FDD
9	1749.9MHz to 1784.9MHz	1844.9MHz to 1879.9MHz	FDD
10	1710MHz to 1770MHz	2110MHz to 2170MHz	FDD
11	1427.9MHz to 1447.9MHz	1475.9MHz to 1495.9MHz	FDD
12	699MHz to 716MHz	729MHz to 746MHz	FDD
13	777MHz to 787MHz	746MHz to 756MHz	FDD
14	788MHz to 798MHz	758MHz to 768MHz	FDD
17	704MHz to 716MHz	734MHz to 746MHz	FDD
18	815MHz to 830MHz	860MHz to 875MHz	FDD
19	830MHz to 845MHz	875MHz to 890MHz	FDD
20	832MHz to 862MHz	791MHz to 821MHz	FDD
21	1447.9MHz to 1462.9MHz	1495.9MHz to 1510.9MHz	FDD
23	2000MHz to 2020MHZ	2180MHz to 2200MHz	FDD
24	1626.5MHz to 1660.5MHZ	1525MHz to 1559MHz	FDD
25	1850MHz to 1915MHz	1930MHz to 1995MHz	FDD

ASR CONFIDENTIAL

26	814MHz to 849MHz	859MHz to 894MHz	FDD
27	807MHz to 824MHz	852MHz to 869MHz	FDD
28	703MHz to 748MHz	758MHz to 803MHz	FDD
29	N/A	717MHz to 728MHz	N.A.
31	452.5MHz to 457.5MHz	462.5MHz to 467.5MHz	FDD
34	2010MHz to 2025MHz	2010MHz to 2025MHz	TDD
38	2570MHz to 2620MHz	2570MHz to 2620MHz	TDD
39	1880MHz to 1920MHz	1880MHz to 1920MHz	TDD
40	2300MHz to 2400MHz	2300MHz to 2400MHz	TDD
41	2496MHz to 2690MHz	2496MHz to 2690MHz	TDD
66	1710MHz to 1780MHz	2110MHz to 2200MHz	FDD
72	451MHz to 456MHz	461MHz to 466MHz	FDD

Copyright © 2022 ASR

Table 65: RF Port Mapping and Band Configurations

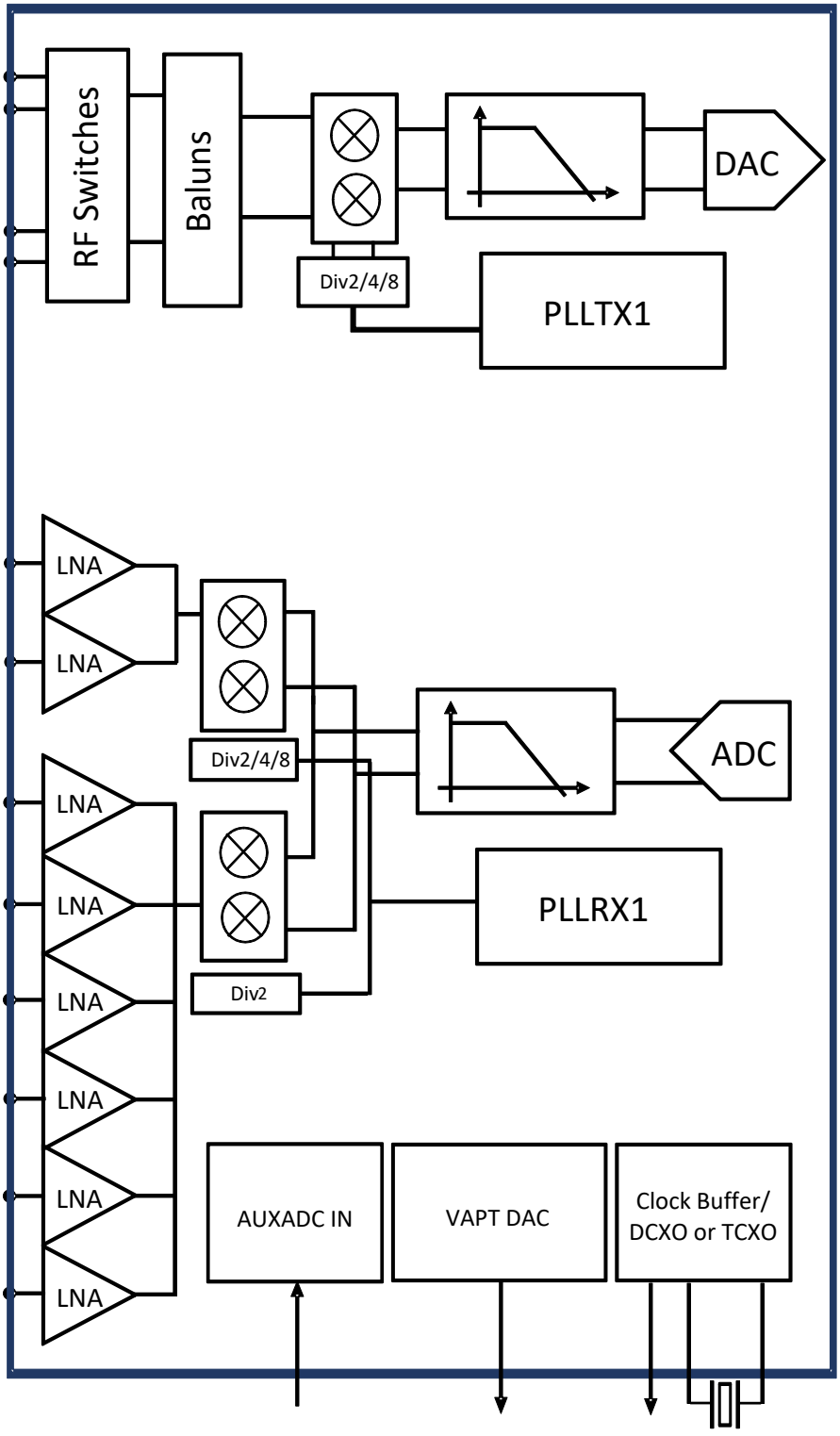
RX PORT	Pin	F_Low [MHz]	F_High[MHz]
LNA1_MHB	W1	1805	2690
LNA2_MHB	Y2	1805	2690
LNA3_MHB	W3	1805	2690
LNA4_MHB	Y4	1805	2690
LNA5_MHB	W5	1805	2690
LNA6_MHB	Y7	1805	2690
LNA7_LMB	Y6	460	1511*
LNA8_LMB	Y8	460	1511*

TX PORT	Pin	F_Low [MHz]	F_High[MHz]
TXLB1	W12	450	915
TXLB2	Y12	450	915
TXMHB1	Y14	1427	2690
TXMHB2	Y13	1427	2690

Note : * LMB LNAs may be used for TDD bands up to 2100MHz

ASR CONFIDENTIAL

Figure 117: R F Block Diagram



Copyright © 2022 ASR

23.1.2 LTE Transmitter

LTE TX mode uses direct IQ modulation technique. There are four TX RF output ports, 2 MHB and 2 LB.

23.1.2.1 Baseband

IQ samples are received from TX DFE

23.1.2.2 Variable Gain I/Q RF Upconverter

The RF upconverter converts the incoming I/Q signal to the desired RF output signal. The upconverter has programmable output level for output power control. VCO and LO generation blocks supply the LO signal to the upconverter.

The transmit path of the ASR1606 includes four single ended variable RF output ports to control the power level to the PA input. At any time one of the four outputs can be turned ON.

The output level is variable yielding over 64dB of dynamic range.

ASR CONFIDENTIAL

23.1.3 LTE Receiver Section

The ASR1606 includes a single receive path with 8 LNA inputs. RX path include low-noise RF amplification, down conversion, analog baseband block, high dynamic A/D conversion, gain control, and digital filtering.

The receive path consists of 6 MHB, 2 LMB Low Noise Amplifiers (LNAs). The receive paths are 50Ω single ended with external matching.

23.1.4 Transmit and Receive Synthesizers

The ASR1606 provides fully integrated transmit and receive VCOs that are phase locked with fully programmable synthesizers. These VCOs have capacitor arrays for coarse tuning and varactor diodes for fine tuning.

The output of the LO blocks drive the LO port of the TX RF up converters and RX Mixers.

Separate synthesizers are used for the transmitter sections and receiver.

Copyright © 2022 ASR

23.1.5 Reference Clocks

The ASR1606 can support an external TCXO. It can also support an internal oscillator with an external crystal. 26MHz operation is supported. There is an auxiliary clock output (OCLK1) with separate enable controls.

23.1.6 Power Supply

The ASR1606 RF operates on 1.8V supply domain.

Table 66: Logic Levels

Symbol	Parameter	Conditions	Min	Typ	Max	Units
	1.8V DC-DC Supply Voltage		1.75	1.8	1.85	V
	1.8V DC-DC Ripple				20	mV_{P-P}
	VDD18_TCXOIN LDO Noise	10Hz-100KHz			50	µVRMS
	Operating Ambient Temperature		-40		+85	°C
Logic Output Voltage Range						
VOH	Output Voltage	High = -500µA	0.8*VDD		VDD	V
VOL		Low = 500µA	0		0.2*VDD	
Logic Input Voltage Range						
VIH	Input Voltage	High	0.7*VDD		VDD+0.3	V
VIL		Low	-0.3		0.48	

Table 67: R F Power Supply Pin Details

Pin Name	Pin number	Power domain	Typ
VDD18_RX_BBLO	U8	RX Baseband, LO	1.8V
VDD18_PLLTX	U11	TX LO	1.8V
VDD18_RF_DIG	V15	RF Digital	1.8V
VDD18_RX_LNA	V4	RX LNA	1.8V
VDD18_TCXOIN	U17	Clock LDO Input	1.8V
VDD18_TX_ANA	V14	TX Analog	1.8V
VDDTCXO_OUT	U16	TCXO Supply Output	1.8V

Copyright © 2022 ASR

23.2 Electrical Specifications

23.2.1 Absolute Maximum Ratings

Table 68: Absolute Maximum Ratings[1]

Note: Stresses above those listed in Absolute Maximum Ratings may cause permanent device failure. Functionality at or above these limits is not implied. Exposure to absolute maximum ratings for extended period may affect device reliability.

Parameter	Range	Units
V_{DD} @ 1.8V	-0.3 to +2.2	V
Input voltage	GND to V_{CC}	V
Power dissipation	400 (MAX)	mW
Storage Temperature Range	-50 to +125	°C
Lead Temperature (soldering, 10s)	300	°C

1. Exceeding the absolute maximum ratings may damage the device.

23.2.2 Recommended Operating Conditions

Table 69: Recommended Operating Conditions[1]

Parameter	Min	Typ	Max	Units
Supply voltage	1.75	1.8	1.85	V
Operating ambient temperature*	-20		+70	°C

1. This device is not guaranteed to function outside the specified operating range.

23.2.3 Electrical Characteristics

Table 70: Electrical Characteristics

The following applies unless otherwise noted: V_{CC} = 1.8V, T_A = 25°C.

Symbol	Parameter	Conditions	Min	Typ	Max	Units
General Receive Specifications						
Z_{IN}	Input impedance (all bands)	With external matching components		50		Ω
NF	Gain state Cascaded noise figure at LNA input (all bands)	High gain state		2.5	3	dB
IIP2	2nd Order Input Intercept Point: (@ f_{RX} - Duplex Spacing and f_{RX} - Duplex Spacing +1)	Maximum gain, 2-tone test (2-tone level -27dBm), All bands	+46	+52		dBm
IIP3	3rd Order Input Intercept Point: (@ f_{RX} - Duplex Spacing+1 and f_{RX} - Duplex Spacing/2)	Maximum gain, 2-tone test, All bands	-10			dBm
	3rd Order Input Intercept Point: (@ f_{RX} +3.5MHz and f_{RX} + 5.9MHz)	All bands	-10			dBm
	3rd Order Input Intercept Point: Inband Blocker (@ f_{RX} +10MHz and f_{RX} + 20.1MHz)		-14			dBm
LTE Transmit, RF Output Specifications						
P_{OUTMAX}	Maximum output power	MPR=0	+4			dBm
P_{OUTMIN}	Minimum output power	LTE			-60	dBm
P_{OUT_N} @ RX	Output noise floor, corresponding receive band: (General)	P_{OUT} > 0dBm		-159	-155	dBc/Hz
	Output noise floor, corresponding receive band: (Bands 12,13,14,17,20)	P_{OUT} > 0dBm			-150	

Copyright © 2022 ASR

Table 71: Electrical Characteristics

The following applies unless otherwise noted: V_{CC} = 1.8V, T_A = 25°C.

Symbol	Parameter	Conditions	Min	Typ	Max	Units
	LTE mode E-UTRA ACLR1 (ΔfC=6BW) -all bands -all applicable ch. BWs UTRA ACLR1 (ΔfC = (BW/2 + 2.5MHz)) -all bands -all applicable ch. BWs	MPR=0; P_{OUT} ≤ 4dBm MPR=1; P_{OUT} ≤ 3dBm			-39	dBc
		MPR=0; P_{OUT} ≤ 4dBm MPR=1; P_{OUT} ≤ 3dBm			-42	dBc
	UTRA ACLR2 (ΔfC = (BW/2 + 7.5MHz)) -all bands -all applicable ch. BWs	MPR=0; P_{OUT} ≤ 4dBm MPR=1; P_{OUT} ≤ 3dBm			-43	dBc
	Carrier Leakage (with calibration)	-10dBm ≤ P_{OUT} ≤ +6dBm			-36	dBc
		-50dBm ≤ P_{OUT} ≤ -10dBm			-30	
		-60dBm ≤ P_{OUT} ≤ -50dBm			-25	
		-70dBm ≤ P_{OUT} ≤ -60dBm			-15	
	Image Rejection	P_{OUT} > -40dBm	-30	-35		
	LTE In Band Emission	Non allocated RBs, P_{OUT} > -			-33	
EVM	Error vector magnitude	P_{OUT} = -35dBm to P_{OUTMAX}		3	6	%rms
	Equalizer spectrum flatness (LTE)		-2		+2	dB
	Harmonic suppression ($2f_{TX}$, $3f_{TX}$, $4f_{TX}$)	P_{OUT} = P_{OUTMIN} to P_{OUTMAX}	12			dB

Table 72: Electrical Characteristics (Three-Wire Serial Bus Interface)

The following applies unless otherwise noted: V_{CC} = 1.8V, T_A = 25°C.

Symbol	Parameter	Conditions	Min	Typ	Max	Units
Three-Wire Serial Bus						
V_{IH}	Output voltage	High	1.5			V
V_{IL}		Low			0.5	
	Rise/fall time				3	ms

Table 73: Electrical Characteristics (Reference Clocks - All Modes)

The following applies unless otherwise noted: V_{CC} = 1.8V, T_A = 25°C.

Symbol	Parameter	Conditions	Min	Typ	Max	Units
Reference Clock Input						
	Reference clock input voltage (pseudo-sinusoidal)		0.5	1	1.5	V
	Duty cycle		45	50	55	%
	Reference clock input impedance		10			kΩ
	Reference clock input capacitance			1		pF
Reference Oscillator Output Buffers (26MHz)						
	Reference clock output voltage	Z_L = 10pF || 30kΩ	700	1100	1500	mVp-p
	Duty cycle		45	50	55	%
	Fall/rise time	10% to 90% of signal swing, Z_L = 10pF || 30kΩ	4		8	ns

Copyright © 2022 ASR

Table 74: Electrical Characteristics (APT DAC)

The following applies unless otherwise noted: V_{CC} = 1.8V, T_A = 25°C.

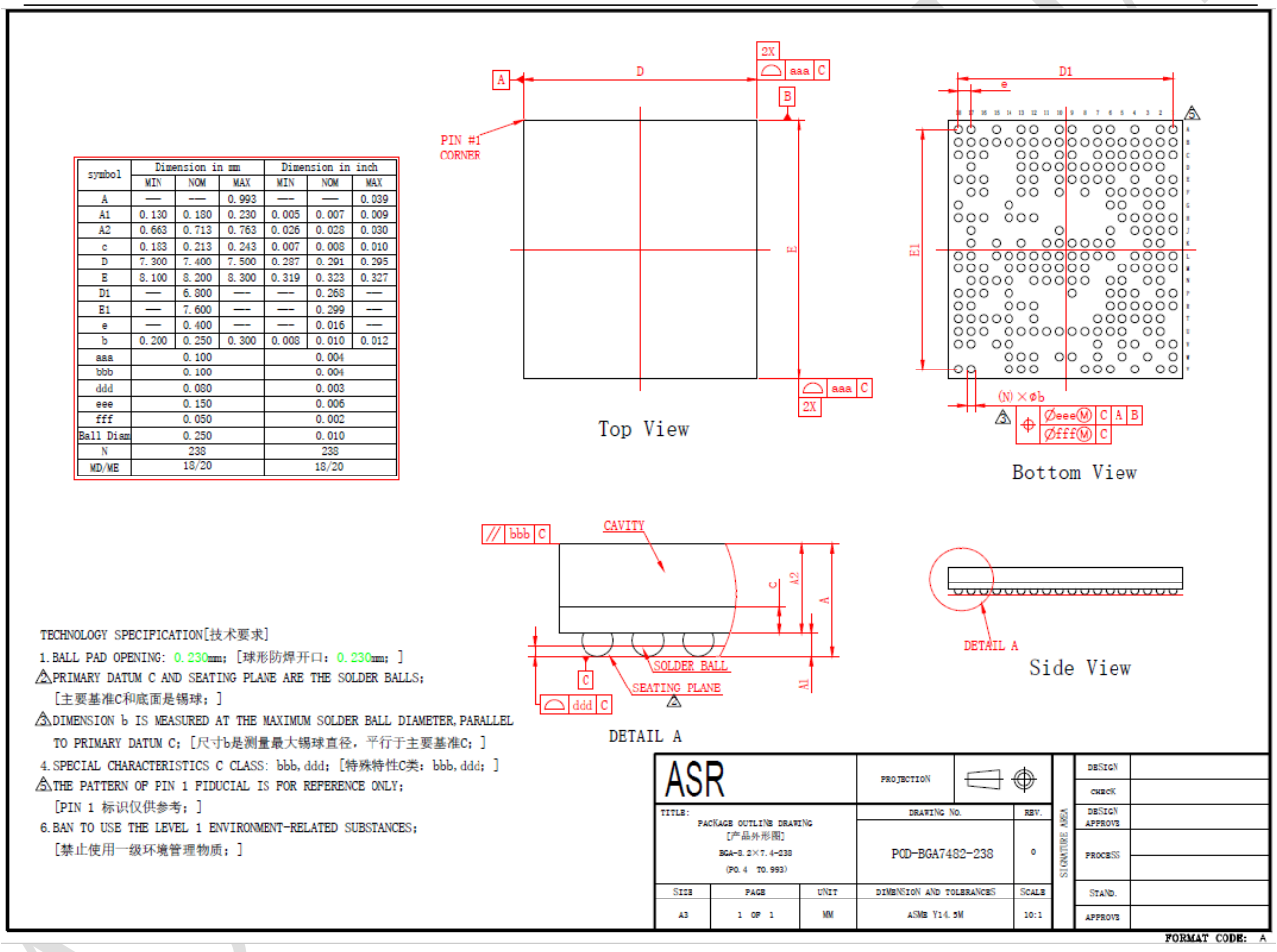
Symbol	Parameter	Conditions	Min	Typ	Max	Units
	Output Voltage		0.1		1.5	V
	Load Capacitance	Linearity requirements apply to GND			10	pF
	Load Resistance		10			kΩ

Copyright © 2022 ASR

24. Mechanical Drawings

This section provides the mechanical specifications for the ASR1606 Communications Processor.

Figure 118:238-pin 8.2X7.4mm BGA Package



symbol	Dimension in mm			Dimension in inch		
	MIN	NOM	MAX	MIN	NOM	MAX
A	——	——	0.993	——	——	0.039
A1	0.130	0.180	0.230	0.005	0.007	0.009
A2	0.663	0.713	0.763	0.026	0.028	0.030
c	0.183	0.213	0.243	0.007	0.008	0.010
D	7.300	7.400	7.500	0.287	0.291	0.295
E	8.100	8.200	8.300	0.319	0.323	0.327
D1	——	6.800	——	——	0.268	——
E1	——	7.600	——	——	0.299	——
e	——	0.400	——	——	0.016	——
b	0.200	0.250	0.300	0.008	0.010	0.012
aaa	0.100			0.004		
bbb	0.100			0.004		
ddd	0.080			0.003		
eee	0.150			0.006		
fff	0.050			0.002		
Ball Diam	0.250			0.010		
N	238			238		
MD/ME	18/20			18/20		

Copyright © 2022 ASR

25.Part Order Numbering/Package Marking

25.1 *Part Order Numbering*

The current part order numbering scheme is the same as the package marking. Details see package marking.

Table 75:ASR1606 Part Order Options

Package Type	Part Order Number	
238-pin BGA	1606S(8 MB pSRAM +8 MB QSPI flash) 1606C(4 MB pSRAM +4 MB QSPI flash)	

Table 76:ASR1606 Work Temperature

Commercial Grade

Symbol	Parameter	Condition	Min	Typ	Max	Units
T_{case}	Case Operating Temperature	Top.center of package	-20		70	° C

Industrial Grade

Symbol	Parameter	Condition	Min	Typ	Max	Units
T_{case}	Case Operating Temperature	Top.center of package	-40		85	° C

25.2 *Package Marking*

Figure 119 shows a sample Commercial package marking and pin 1 location for the ASR1606.

Figure 119:Commercial Package Marking and Pin 1 Location



ASR CONFIDENTIAL

Marking Content		
Item	**Content**	**Description**
Line 1	Logo	ASR logo
Line 2	ASR1606	Part number
Line 3	XXXXXX-XXXXXXX	Lot Number - Product code
Line 4	XXXXXXXXXX-	Assemble lot no. For Industrial Grade,the last letter is 'I'.
Line 5	YYWW	Date code
Line 6	Pin A1 dot	Pin A1 dot

Note:

The above drawing is not drawn to scale. Location of markings is approximate.

ASR CONFIDENTIAL

A Acronyms and Abbreviations

3GPP	Third Generation Partnership Project
A/D	analog-to-digital
ACK	positive-acknowledge
ACS	Automatic clock stop
ADC	Analog-to-Digital converters
AIRQ	ARM Interrupt Request
ALU	Arithmetic Logic Unit
APMU	Application Subsystem PMU
ATR	answer-to-reset
BA	Bank Address
BBIC	Baseband IC
BE	Byte Enable
BGT	Block Guard Time
BIU	Bus Interface Unit
BWT	Block Waiting Time
CA	Column Address
CBC	Cipher Block Chaining
CPMU	Communication PMU
CTI	Cross Trigger Interface
CWT	Character Waiting Time
DCU	Data Cache Unit
DLL	Delay-Locked Loop
DMA	direct-memory access
DMAC	Direct-Memory Access Controller
DMC	Dynamic Memory Controller
DPD	Deep Powerdown
DPU	Data Processing Unit
DSP	Digital Signal Processor

ASR CONFIDENTIAL

ECB	Electronic CodeBook
ECC	Error Checking and Correction
EIC	External Interrupt Controller
EOC	End-of-Descriptor chain
EOP	End-of-Program
EOP	End of Packet
EOR	end-of-receive
ETM	Embedded Trace Macrocell
FFS	Flash File System
FIQ	Fast Interrupt Request
FS	Full/Low speed
GEU	Generic Encryption Unit
GPIO	General Purpose Input/Output
GSM	Global System for Mobile Communications
HR	hardware reset
HS	High-speed
HSDPA	High Speed Downlink Packet Access
HSL	High Speed Logger
I^2S	Inter-IC Sound
ICU	Instruction Cache Unit
ICU	Interrupt Controller
IPC	Inter Processor Communication
IrDA	Infrared Data Association
IRQ	Interrupt Request
LSU	Load/Store Unit
LTE	Long Term Evolution
MCB	Memory Controller Bus
MFPR	Multi-Function Pin Register
MIRQ	Modem DSP Interrupt Request
MPMU	Main PMU
MPU	Memory Protection Unit
NAK	negative-acknowledge

ASR CONFIDENTIAL

NRZ	Non-Return to Zero
OTP	One-Time-Programmable
PFU	Prefetch Unit
PIO	programmed I/O
PLE	Preload Engine
PMU	Power Management Unit
PN	Pseudo Number
POR	Power-on reset
PPS	parameter selection
PSP	Programmable Serial Protocol
PSRAM	Pseudo RAM
RA	Row Address
RAS	Request-after-channel-stops
RCA	Relative Card Address
RFIC	Radio Frequency IC
RIPC	Resource IPC
RTC	Real-Time Clock
SCL	Serial Clock Line
SDA	serial data and address
SIMD	Single Instruction Multiple Data
SIR	Serial Infrared
SMC	Static Memory Controller
SPH	Serial Clock Phase
SPH	Single-Port Host
SPI	Serial Peripheral Interface
SS	SuperSpeed
SSP	Synchronous Serial Protocol
STB	Store Buffer
TAP	Test-Access Port
TCU	Timing Control Unit
TLRS	test-logic-reset state

ASR CONFIDENTIAL

TO	Time Out
TWSI	Two-Wire Serial Interface
USIM	Universal Subscriber Identity Module
VIPT	Physically Tagged
VLIO	Variable Latency Input/Output
WDT	Watchdog Timer
WWT	work waiting time
XIP	eXecute In Place

ASR CONFIDENTIAL

B Revision History

Table 77:Revision History

Revision	Date	Description
Rev.V1.0	Jan.28, 2022	Initial Release.

ASR CONFIDENTIAL

THIS PAGE INTENTIONALLY LEFT

ASR CONFIDENTIAL